

llm-d Official Docs

Exported from the official `llm-d/llm-d` repository `docs/` tree.

Source website: <https://llm-d.ai/docs/>

Note: llm-d does not appear to publish an official prebuilt PDF, so this file is a static export of the official docs content.

Contents

1. The fastest path to state-of-the-art LLM inference on any accelerator [getting-started/README.mdx](#)
2. Quickstart [getting-started/quickstart.md](#)
3. llm-d Accelerators [getting-started/accelerators.md](#)
4. Well-Lit Paths [well-lit-paths/README.md](#)
5. Core Capability Building Blocks [well-lit-paths/foundations/README.md](#)
6. Optimized Baseline [well-lit-paths/foundations/optimized-baseline.md](#)
7. Workloads [well-lit-paths/workloads/README.md](#)
8. Agentic Serving [well-lit-paths/workloads/agentic-serving.md](#)
9. Predicted Latency-Based Scheduling [well-lit-paths/foundations/predicted-latency.md](#)
10. Multimodal Workload Serving [well-lit-paths/workloads/multimodal-serving.md](#)
11. Precise Prefix Cache Routing [well-lit-paths/foundations/precise-prefix-cache-routing.md](#)
12. Tiered Prefix Cache [well-lit-paths/foundations/tiered-prefix-cache.md](#)
13. [Experimental] Enable P2P Prefix Cache Sharing [well-lit-paths/foundations/enable-p2p-prefix-cache-sharing.md](#)
14. P/D Disaggregation [well-lit-paths/foundations/pd-disaggregation.md](#)
15. Multi-Node Wide Expert Parallelism [well-lit-paths/foundations/wide-expert-parallelism.md](#)
16. Flow Control [well-lit-paths/foundations/flow-control.md](#)
17. Workload Autoscaling [well-lit-paths/foundations/workload-autoscaling.md](#)
18. Multi-Model Routing [well-lit-paths/foundations/multi-model-routing.md](#)
19. Fast Model Actuation [well-lit-paths/foundations/fast-model-actuation.md](#)
20. Batch Serving [well-lit-paths/workloads/batch-serving/README.md](#)
21. Asynchronous Processing [well-lit-paths/workloads/batch-serving/asynchronous-processing.md](#)
22. Batch Gateway [well-lit-paths/workloads/batch-serving/batch-gateway.md](#)
23. Architecture [architecture/README.md](#)
24. InferencePool [architecture/core/inferencepool.md](#)
25. Latency Predictor [architecture/advanced/latency-predictor.md](#)
26. Model Servers [architecture/core/model-servers.md](#)
27. Autoscaling [architecture/advanced/autoscaling/README.md](#)
28. KEDA with EPP Metrics [architecture/advanced/autoscaling/keda-epp.md](#)
29. Batch Inference [architecture/advanced/batch/README.md](#)
30. Batch Gateway Architecture [architecture/advanced/batch/batch-gateway.md](#)

31. KV Cache Management [architecture/advanced/kv-management/README.md](#)
32. Prefix-Cache Aware Routing [architecture/advanced/kv-management/prefix-cache-aware-routing.md](#)
33. Llm-d Router [architecture/core/router/README.md](#)
34. Proxy [architecture/core/router/proxy.md](#)
35. SLO-Aware Autoscaling with KEDA — the control law [architecture/advanced/autoscaling/slo-aware-keda.md](#)
36. Async Processor Architecture [architecture/advanced/batch/async-processor.md](#)
37. KV-Cache Indexer [architecture/advanced/kv-management/kv-indexer.md](#)
38. Workload Variant Autoscaler (WVA) — Deprecated [architecture/advanced/autoscaling/wva.md](#)
39. KV-Cache Offloading [architecture/advanced/kv-management/kv-offloader.md](#)
40. Endpoint Picker (EPP) [architecture/core/router/epp/README.md](#)
41. Request Handler [architecture/core/router/epp/request-handling.md](#)
42. Flow Control [architecture/core/router/epp/flow-control.md](#)
43. Request Scheduler [architecture/core/router/epp/scheduling.md](#)
44. Data Layer [architecture/core/router/epp/datalayer.md](#)
45. Configuration [architecture/core/router/epp/configuration.md](#)
46. Operational Excellence [operations/README.md](#)
47. vLLM Model-Aware Readiness Probes [operations/readiness-probes.md](#)
48. Router Operations Guide [operations/router.md](#)
49. Llm-d Async Processor Operations Guide [operations/async-processor.md](#)
50. Graceful Shutdown & Request Draining [operations/graceful-shutdown.md](#)
51. Disaggregated Serving Operations [operations/disaggregation/README.md](#)
52. Disaggregated Serving: Operations (vLLM) [operations/disaggregation/vllm.md](#)
53. Observability [operations/observability/README.md](#)
54. Metrics [operations/observability/metrics.md](#)
55. Rollouts [operations/rollouts/README.md](#)
56. Serve External APIs [operations/serve-external-apis/README.md](#)
57. LiteLLM for Hosting Llm-d Inference Stack and External APIs [operations/serve-external-apis/litellm.md](#)
58. Disaggregated Serving: Operations (SGLang) [operations/disaggregation/sglang.md](#)
59. Distributed Tracing [operations/observability/tracing.md](#)
60. Kong AI Gateway for Routing Llm-d Inference Stack and External APIs [operations/serve-external-apis/kong.md](#)
61. PromQL Query Reference [operations/observability/promql.md](#)
62. Observability Setup [operations/observability/setup.md](#)
63. Alerting [operations/observability/alerting.md](#)
64. Infrastructure & Environments [infrastructure/README.md](#)
65. Multi-Node Serving Orchestration [infrastructure/multi-node.md](#)
66. No-Kubernetes Deployment [infrastructure/no-kubernetes-deployment.md](#)
67. Gateway Guides [infrastructure/gateway/README.md](#)
68. Istio [infrastructure/gateway/istio.md](#)
69. GKE [infrastructure/gateway/gke.md](#)
70. Agentgateway [infrastructure/gateway/agentgateway.md](#)

71. Envoy AI Gateway [infrastructure/gateway/envoy-ai-gateway.md](#)
72. Installing Gateway API CRDs [infrastructure/gateway/install-crds.md](#)
73. API Reference [api-reference/README.md](#)
74. InferencePool [api-reference/inferencepool.md](#)
75. InferenceObjective [api-reference/inferenceobjective.md](#)
76. InferenceModelRewrite [api-reference/inferencemodelrewrite.md](#)
77. EndpointPickerConfig (EPP Configuration) [api-reference/endpointpickerconfig.md](#)
78. EPP HTTP APIs Reference [api-reference/epp-http-apis.md](#)
79. EPP gRPC APIs Reference [api-reference/epp-grpc-apis.md](#)
80. EPP HTTP Headers Reference [api-reference/epp-http-headers.md](#)
81. Glossary [api-reference/glossary.md](#)
82. Artifacts [api-reference/artifacts.md](#)
83. Disaggregated Serving [architecture/advanced/disaggregation/README.md](#)
84. Inference Payload Processor (IPP) [architecture/advanced/inference-payload-processing/README.md](#)
85. Kustomize Dry-Run CI [ci/kustomize-dry-run.md](#)
86. Llm-d Infrastructure Providers [infrastructure/providers/README.md](#)
87. Deploying Llm-d on Azure Kubernetes Service [infrastructure/providers/aks/README.md](#)
88. Llm-d on DigitalOcean Kubernetes Service (DOKS) [infrastructure/providers/digitalocean/README.md](#)
89. Llm-d on Google Kubernetes Engine (GKE) [infrastructure/providers/gke/README.md](#)
90. TPU Dynamic Slicing on GKE [infrastructure/providers/gke/dynamic-slicing/README.md](#)
91. Llm-d on minikube [infrastructure/providers/minikube/README.md](#)
92. Llm-d on OpenShift [infrastructure/providers/openshift/README.md](#)
93. Llm-d on Red Hat OpenShift on AWS [infrastructure/providers/openshift-aws/README.md](#)
94. Remote Direct Memory Access (RDMA) and Networking Configuration [infrastructure/rdma/README.md](#)

1 [getting-started/README.mdx](#) official page

The fastest path to state-of-the-art LLM inference on any accelerator

Llm-d is an open-source inference serving stack for Kubernetes. It runs your model server of choice—vLLM and SGLang, and more—across your cluster, turning single-node engines into production-grade distributed inference on the infrastructure you already run. Get state-of-the-art performance for leading open models—on NVIDIA, AMD, and custom accelerators.

Built for agentic pipelines, LLMs, multimodal models, and high-throughput serving. Completely engine- and hardware-agnostic.

Get started [Explore the performance data](#)

Well-Lit Paths

In addition to the software components, llm-d provides Well-Lit Paths — tested, benchmarked deployment recipes for common production patterns. These paths are starting points designed to be adapted for your models, hardware, and traffic patterns to support agentic, multimodal, and batch workloads.

Each path includes:

- Deployable Helm charts and Kustomize manifests
- Key configuration knobs for performance tuning
- Sample workloads and benchmarks against baseline setups
- Monitoring and observability configuration

See all Well-Lit Paths

Quickstart

This guide provides a simplified, end-to-end walkthrough for deploying an **Optimized Baseline** configuration using llm-d. This setup reduces tail latency and increases throughput through load-aware and prefix-cache aware balancing.

For this quickstart, we will use the **Standalone Mode** deployment, which is the easiest way to get started with llm-d.

Prerequisites

- Installed proper client tools (kubectl, helm).
- Checkout llm-d repo:

```
bash export branch="main" git clone https://github.com/llm-d/llm-d.git && cd llm-d && git checkout ${branch}
```

- Set the following environment variables:

```
bash export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel)) source ${REPO_ROOT}/guides/env.sh export
GUIDE_NAME="quickstart" export NAMESPACE=llm-d-quickstart
```

- Install the Gateway API Inference Extension CRDs:

```
bash # GAIE_URL is automatically calculated from GAIE_VERSION at ${REPO_ROOT}/guides/env.sh kubectl apply -f
https://github.com/kubernetes-sigs/gateway-api-inference-extension/${GAIE_URL}/v1-manifests.yaml
```

- Create a target namespace for the installation:

```
bash kubectl create namespace ${NAMESPACE} --dry-run=client -o yaml | kubectl apply -f -
```

- Create the llm-d-hf-token secret in your target namespace with the key HF_TOKEN matching a valid HuggingFace token to pull models.

```
bash export HF_TOKEN=<your HuggingFace token> kubectl create secret generic llm-d-hf-token \ --from-
literal="HF_TOKEN=${HF_TOKEN}" \ --namespace "${NAMESPACE}" \ --dry-run=client -o yaml | kubectl apply -f -
```

Installation Instructions

1. Deploy the llm-d Router (Standalone Mode)

The llm-d Router provides the intelligent load balancing. In Standalone Mode, it includes a built-in proxy (Envoy).

```
helm install ${GUIDE_NAME} \
  ${ROUTER_STANDALONE_CHART} \
  -f guides/recipes/router/base.values.yaml \
  -f guides/optimized-baseline/router/optimized-baseline.values.yaml \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

2. Deploy the Model Server

Deploy the default model server (vLLM running on NVIDIA GPUs). This will deploy 8 replicas of Qwen/Qwen3-32B by default.

```
kubectl apply -n ${NAMESPACE} -k guides/optimized-baseline/modelserver/gpu/vllm/base/
```

[!TIP] If you are using different hardware (AMD, Intel, TPU, or CPU), you can find alternative configurations in the `guides/optimized-baseline/modelserver/` directory.

Wait for all model server pods to be running and ready before proceeding:

```
bash ${REPO_ROOT}/guides/scripts/wait-for-pods-ready.sh -l llm-d.ai/guide=optimized-baseline -n ${NAMESPACE}
```

[!NOTE] Model server pods may take several minutes to start as they need to download the model weights.

Verification

1. Get the IP of the Proxy

Retrieve the ClusterIP of the llm-d Router service:

```
export IP=$(kubectl get service ${GUIDE_NAME}-epp -n ${NAMESPACE} -o jsonpath='{.spec.clusterIP}')
```

2. Send a Test Request

Open a temporary interactive shell inside the cluster to send a request:

```
kubectl run curl-debug --rm -it \
  --image=cfmanteiga/alpine-bash-curl-jq \
  --namespace="${NAMESPACE}" \
  --env="IP=$IP" \
  -- /bin/bash
```

Inside the shell, send a completion request:

```
curl -X POST http://${IP}/v1/completions \  
-H 'Content-Type: application/json' \  
-d '{  
  "model": "Qwen/Qwen3-32B",  
  "prompt": "How are you today?"  
}' | jq
```

Cleanup

To remove all resources created in this guide:

```
helm uninstall ${GUIDE_NAME} -n ${NAMESPACE}  
kubectl delete namespace ${NAMESPACE}
```

llm-d Accelerators

llm-d supports multiple accelerator vendors and we are expanding our coverage.

Support

Maintainers for each accelerator type are listed below. See our well-lit path guides for details of deploying on each hardware type.

Vendor	Models	Maintainers
AMD	ROCm	Kenny Roche (Kenny.Roche@amd.com), Vincent Cave (Vincent.Cave@amd.com)
CPU	x86_64	Hongming Zheng (@ZhengHongming888, hongming.zheng@intel.com)
Google	TPU	Edwin Hernandez (@Edwinhr716), Cong Liu (@liu-cong, congliu.thu@gmail.com)
Iluvatar	BI-V150	ShiChun Yu, shichun.yu@iluvatar.com, Mengxuan Li(@archlitchi,mengxuan.li@dynamia.ai)
Intel	XPU	Yuan Wu (@yuanwu2017, yuan.wu@intel.com)
MetaX	C500X GPU	Lianjie Zhang (@lianjiezh, lianjie.zhang@metax-tech.com), Mengxuan Li (@archlitchi, mengxuan.li@dynamia.ai)
NVIDIA	GPU	Will Eaton (weaton@redhat.com), Greg (grpereir@redhat.com)
Rebellions	NPU	Jinmoo Seok (@rebel-jinmoo, jinmoo_seok@rebellions.ai), Minwook Ahn (@rebel-minwook, minwook.ahn@rebellions.ai), Minh Park (@rebel-minhopark, minho.park@rebellions.ai)

Requirements

We welcome contributions from accelerator vendors. To be referenced as a supported hardware vendor we require at minimum a publicly available container image that launches vLLM.

For integration into the well-lit paths our standard for contribution is higher, **requiring**:

- A named maintainer responsible for keeping guide contents up to date
- Manual or automated verification of the guide deployment for each release

[!NOTE] We aim to increase our requirements to have active CI coverage for all hardware guide variants in a future release.

[!NOTE] The community can assist but is not responsible for keeping hardware guide variants updated. We reserve the right to remove stale examples and documentation with regard to hardware support.

NVIDIA GPUs

NVIDIA GPUs are the default accelerator for all llm-d guides. Any NVIDIA GPU is supported, with the specific capabilities determined by the inference container image used. No special cluster configuration is required beyond the NVIDIA device plugin or DRA driver.

CUDA Runtime and Driver Requirements

llm-d currently ships container images based on the **CUDA 12.9.1** runtime. A future release will move to **CUDA 13.0.2**.

CUDA 12.x and CUDA 13.x have non-overlapping driver compatibility ranges — a given driver version supports one major CUDA family, not both:

CUDA Version	Minimum Driver	Maximum Driver
CUDA 12.9.1 (current)	525.60.13	< 580
CUDA 13.0.2 (planned)	580.65.06	N/A

Recommended driver version: 575.x for current llm-d releases using CUDA 12.9.1. This provides the latest features and fixes within the CUDA 12.x compatible driver range.

When llm-d moves to CUDA 13.0.2, the minimum driver version will become **580.65.06**. Users should plan to upgrade their node drivers to 580+ ahead of this transition.

For the full CUDA/driver compatibility matrix, see the [CUDA Toolkit Release Notes](#).

Google TPU

Google Cloud TPUs (v6e, v7) are supported when running on GKE. See the [GKE infrastructure provider docs](#) for cluster setup.

On TPU7x, model servers can be scheduled onto dynamically formed sub-slices (2x2x1 through 2x4x4) instead of statically provisioned node pool topologies. See [TPU Dynamic Slicing on GKE](#).

AMD ROCm

AMD GPUs are supported via ROCm. The specific GPU models supported are determined by the inference container image. See the [AMD device plugin](#) or [DRA driver](#) below for cluster setup.

Intel XPU

Intel Data Center GPU Max 1550 and Intel BMG GPUs (Battlemage G21) are supported. Intel XPU deployments use DRA with a unified accelerator type that automatically handles driver selection for both i915 and xe drivers.

For cluster prerequisites, ensure you have the Intel Resource Drivers for Kubernetes installed.

XPU with RDMA

For P/D disaggregation with RDMA-accelerated KV-cache transfer on Intel XPU, the following additional prerequisites apply:

- An RDMA DRA driver exposing the `dranet-rdma` device class (e.g., Intel Network Operator for Kubernetes).
- GPU-NIC PCIe alignment for optimal transfer performance.
- UCX transport configured with `ib,rc,ze_copy`.

The RDMA overlay (`modelserver/xpu/vllm-rdma/`) reuses the standard XPU vLLM base and adds one RDMA DRA claim per pod plus RDMA-specific UCX transport settings. See the P/D Disaggregation guide for deployment instructions.

Iluvatar

Iluvatar BI-V150 GPUs are supported via the CoreX runtime (Iluvatar's CUDA-compatible stack) with a vLLM fork and a vendor-specific NIXL connector (`IluNixlConnector`). Each board is dual-die (32 GiB per die, 64 GiB per board); with ix-device-plugin `splitboard: false` , `iluvatar.com/gpu` counts boards and vLLM parallel sizes count CUDA devices (2 per board). The CoreX image defaults to eager unless `VLLM_ENFORCE_CUDA_GRAPH=1` is set. The P/D overlay serves `Qwen/Qwen3-32B` on 4 boards / 8 CUDA devices (1 × TP=4 prefill + 1 × TP=4 decode). The optimized-baseline overlay serves `deepseek-ai/DeepSeek-V4-Flash` on 4 boards / 8 CUDA devices (DP=8 + EP).

UCX transport (required for P/D):

`IluNixlConnector` transfers KV cache over NIXL/UCX with `kv_buffer_device=cuda` (KV stays in VRAM). UCX must be configured with CUDA-aware transports:

```
- name: UCX_TLS
  value: "cuda_copy,cuda_ipc,tcp,self,posix,sysv"
- name: UCX_CUDA_IPC_ENABLE_SAME_PROCESS
  value: "y"
```

Without `cuda_copy` / `cuda_ipc` , UCX misdetects VRAM as host memory and the prefill engine crashes (SIGSEGV) during the KV read.

MetaX C500X

MetaX C500X GPUs are supported for community-contributed well-lit paths. The device plugin must expose `metax-tech.com/gpu` . P/D disaggregation uses vLLM `NixlConnector` over TCP; see the P/D Disaggregation guide MetaX overlay (`modelserver/metax/vllm/`).

Rebellions NPU

Rebellions NPUs are supported through `vllm-rbln`, an out-of-tree vLLM platform plugin that registers on the `vllm.platform_plugins` entry point, shipped in a publicly pullable container image. The optimized-baseline overlay (`modelserver/npu/vllm/`) serves `openai/gpt-oss-120b` on two replicas of one NPU each, so the router balances across two model servers.

Cluster prerequisites:

- RBLN NPU Operator, which installs the driver and the `npu.rebellions.ai` DRA DeviceClass
- Kubernetes 1.34 with the `resource.k8s.io/v1` DRA APIs

Device allocation. NPUs are requested through DRA, not an extended resource. One DeviceClass serves every Rebellions product, so a claim reaches a specific NPU only by filtering on `productName` in a CEL selector — see `resource-claim-template.yaml` in the overlay. Omitting the selector lets the claim bind to a different Rebellions product than the guide was measured on.

NUMA alignment. This overlay claims one NPU and no NIC, so it sets no `resource.kubernetes.io/numaNode` match constraint — such a constraint needs two or more requests in one claim to bind together.

Prefix caching is off, so layer the NPU router values. The runtime disables prefix caching for sliding-window models, and `openai/gpt-oss-120b` is one: enabling it reports 0 hits over every query. The guide's default scheduling profile is prefix-cache aware, so apply `router/npu.rbln.values.yaml` after the guide's own values to fall back to the load scorer alone. Both the flag and that file go away once the runtime supports prefix caching here.

Set MODEL when running the guide's steps. The overlay pins `openai/gpt-oss-120b`, while the guide defaults `MODEL` to `Qwen/Qwen3-32B`. Export `MODEL=openai/gpt-oss-120b` so the validation and benchmark steps address the served model.

Out of scope for this release: fast model actuation, because the runtime does not support sleep and wake; LoRA adapters; and multimodal models, which on this runtime need a code path that cannot run alongside a KV connector.

Verification. The maintainers listed above stand up the optimized-baseline overlay once per llm-d release tag against the release image and record the result.

CPU Inferencing

CPU-only inference is supported for deployments without GPU accelerators. This expects 4th Gen Intel Xeon processors (Sapphire Rapids) or later, or equivalent AMD processors. Each replica requires a minimum of 64 CPU cores and 64GB RAM.

Well-Lit Paths

Well-lit paths are curated, end-to-end guides for llm-d's native inference patterns, intelligent routing algorithms, and runtime traffic regulation engines.

These guides are organized into two native architectural groupings:

Foundations

Individual functional features, sophisticated routing algorithms, and physical inference execution paths.

Workloads

Cohesive, production-grade deployment guides that compose our capability building blocks into one horizontal stack tuned for a use case.

Core Capability Building Blocks

Core Capability Building Blocks represent the individual functional optimization, intelligent routing, and physical inference execution features of llm-d.

These guides teach single architectural capabilities that you can configure independently or compose together into comprehensive production workloads.

Intelligent Routing

- **Optimized Baseline:** Strategies for handling the unique challenges of LLM request scheduling, moving beyond traditional round-robin approaches.
- **Predicted Latency-Based Routing:** Using online-trained machine learning models to predict latency and optimize scheduling.
- **Multi-Model Routing:** Serving multiple LLMs and LoRA adapters behind a single Gateway endpoint using the Inference Payload Processor (IPP).

Advanced KV-Cache Management

- **Precise Prefix Cache Routing:** Near-real-time routing based on exact cache state published by model servers.
- **Tiered Prefix Cache:** Efficiently managing KV caches by offloading to CPU RAM, NVMe, or network storage to improve prefix-cache re-use.
- **Enable P2P Prefix Cache Sharing:** Pulling cached prefix KV blocks directly from a peer's CPU offload tier instead of recomputing them, turning per-pod prefix caches into a fleet-wide resource.

Serving Large Models

- **Prefill/Decode Disaggregation:** Separating prefill (compute-bound) and decode (memory-bandwidth-bound) phases for optimized performance.
- **Wide Expert-Parallelism:** Scaling KV cache space for massive MoE models like DeepSeek-R1 using DP/EP deployment patterns.

Traffic Control & Autoscaling

- **Flow Control:** Intelligent request queuing for multi-tenant deployments and managing traffic spikes.
- **Workload Autoscaling:** From simple Kubernetes autoscaling supplemented by EPP load metrics to advanced, SLO-aware capacity optimization for heterogeneous pools via the Workload Variant Autoscaler.
- **Fast Model Actuation:** Cut vLLM startup time with resident sleep/wake instances and a pre-warmed launcher that spawns new instances without re-importing modules, so replica scale-up and model swaps avoid the cold-start penalty on a shared GPU pool.

Optimized Baseline

Traditional HTTP requests are fast, uniform, and cheap. Standard round-robin request scheduling strategies balance this load well.

LLM requests break all three assumptions. They are:

- **Multi-turn** - conversations and agentic tool loops send the same growing prefix repeatedly
- **Slow** - a single request can take over a minute generating tokens
- **Non-uniform** - range from 1000s of reasoning tokens to a 100k+ context tokens

The llm-d Router injects awareness of the LLM-workload into the load-balancing layer considering **prefix-cache affinity** and **server load metrics**.

[!NOTE] This guide demonstrates one approach to prefix- and load-aware routing. The llm-d Router supports other options as well, including session affinity and active request based routing, which make no assumptions about the router's ability to parse the request or probe the servers. See **configuration** for more details on the available scorers, or **precise prefix cache routing** for KV-event-driven scoring.

Deploy

See the optimized baseline guide for manifests and step-by-step deployment.

Architecture

Prefix-Aware Scheduling

EPP maintains a view of each endpoints' prefix-cache state in memory. When a request arrives, it identifies which pod already holds the matching prefix in KV-cache and routes the request there. For multi-turn workloads, this optimization is critical to avoid excessive recomputation in a scale-out setting.

![Prefix-Aware Routing](../assets/prefix-aware-routing.svg)

Load-Aware Scheduling

EPP continuously probes each endpoints' metrics by scraping `/metrics` at a regular interval (50ms default). It scores endpoints on queue depth, running requests, and KV-cache utilization to schedule requests to the endpoint with the lowest load, avoiding hotspots caused by heterogeneous request patterns.

![Load-Aware Routing](../assets/load-aware-routing.svg)

Observability

The optimized baseline balances two routing objectives, prefix-cache affinity and load-aware spread, so the signals that matter are cache hit rate and per-pod load balance watched together. The guide's Observability & Troubleshooting section covers the key metrics for this path and the common failure modes (including the `peakPrefillThroughput` calibration trap on non-default hardware), backed by the shared PromQL and metric references.

Further Reading

See EPP Architecture for more details.

Workloads

A workload guide provides the recommended, cohesive deployment for serving a production workload on llm-d. Each defines the workload, then composes the relevant capability building blocks into one stack tuned to serve it.

Where a well-lit path teaches a single feature, a workload guide starts from a use case and delivers the horizontal deployment that serves it best.

- **Agentic Serving:** long, multi-turn, tool-using agentic programs (e.g. coding agents) — prefix-aware routing, KV-cache offloading, and P/D disaggregation composed for the agentic workload.
- **Multimodal Serving:** image / audio / video workloads — prefix- and load-aware routing that tracks and matches multimodal payloads across aggregated and disaggregated serving.
- **Batch Serving:** large-scale offline or asynchronous jobs — OpenAI-compatible batch gateway and lightweight async queue processors.

Agentic Serving

Agents are becoming the dominant shape of production LLM traffic. A single user goal expands into a long *program* of model calls interleaved with tool execution — coding agents, deep-research loops, tool-using copilots, multi-agent pipelines. But inference stacks are still tuned for the isolated request: each call is served as if it is standalone, so the stack recomputes context it has already seen, evicts long-lived sessions under memory pressure, and hot-spots whichever replica holds a popular prefix.

Three properties of agentic workloads break those request-centric assumptions:

- **Massive context reuse** — branches of a reasoning tree, turns of a tool loop, and children of a swarm share most of their context (system prompts, tool definitions, conversation so far).
- **Program-level objectives** — users care about whole-program completion time, not the latency of any single call; one stalled branch can hold up the entire program.
- **Typed, predictable lifecycles** — durable system prompts and tools versus ephemeral scratchpad thoughts, with predictable flow the stack could anticipate but instead reacts to.

The next large efficiency gain for these workloads does not come from micro-optimizing isolated requests — it comes from making the stack understand the structure of the agentic program, with **model state — today primarily the KV-cache — as the medium that coordinates it.**

Request-centric (today's default)	Program-aware (the direction)
turn 1 → prefill 160K → decode turn 2 → RE-prefill → decode turn 3 → RE-prefill → decode (KV evicted between turns)	turn 1 → prefill 160K → decode turn 2 → reuse cache → decode turn 3 → reuse cache → decode (KV retained / offloaded, restored on resume)

A single engine already reuses cache across turns when a session happens to land on the same replica with free memory; the failure is fleet-level — no cross-replica coordination, eviction blind to which blocks a live session still needs, and no notion of the program behind the requests.

Canonical Workloads

The llm-d project anchors its work to concrete workload shapes, each stressing the stack differently:

- **Long-horizon loops** (coding agents, computer use): one agent iterating reason → act → observe over many turns. Stresses cross-turn KV persistence, typed retention, and context growth.
- **Parallel fan-out** (best-of-N, tree search, RL rollouts, subagents): one shared context spawns many concurrent children. Stresses prefix reuse, proactive replication, and join (slowest-child) latency.
- **Multi-agent pipelines** (programs spanning distinct agents across pods, sequential or fork/join). Stresses reuse and program identity spanning pools, and program-level accounting.

- **Reasoning-heavy generation** (long internal reasoning before answering): shifts work to decode and grows per-request KV footprint.

Deploy

The agentic-serving guide is the operational counterpart. It composes llm-d's existing well-lit paths into a deployment stack — the optimized baseline for prefix- and load-aware routing, tiered KV-cache offloading to keep idle sessions resident, precise prefix-cache routing for exact KV-state visibility, and P/D disaggregation for interactivity under load — into the recommended deployment, realized per accelerator and benchmarked against a shared, realistic agentic workload. See the guide for how each layer maps to the workload.

Direction

The llm-d project agentic northstar sets the direction toward serving that is *program-aware* rather than request-aware, treating the **session as a graph of typed state blocks** the control plane plans over:

- **Session-graph orchestration** — model a session as a graph of typed blocks (system prompt, tool catalog, conversation turn, reasoning branch), built from external hints (e.g. Anthropic `cache_control`, OpenAI `prompt_cache_key`) where available and inferred otherwise. Precise KV-state indexing is the substrate this rides on.
- **Program-aware scheduling** — schedule based on program-level metrics and the critical path that stalls the whole loop, not per-request fairness. Extends the llm-d Router.
- **State reuse and lifecycle ("zero-recompute")** — if a reusable context exists anywhere (HBM, host memory, storage), no node recomputes it whenever reuse is cheaper than regeneration; durable context is pinned and dead branches dropped via typed retention. Builds on KV-disaggregation, tiered offloading, and KV-centric stores such as Mooncake.
- **Proactive state** — when an agent fans out, context is pre-positioned where the new compute will land rather than pulled on demand, with placement co-planned with scheduling from the session graph and declared fan-out.

The contract is two-layer: applications express intent through the caching and metadata APIs they already use, and llm-d does the cross-session, cluster-wide work underneath — better hints sharpen the orchestration, but the stack functions on inferred structure without them. Agent control flow and cognitive decisions stay in the logic layer (agent frameworks, MCP and Skills servers); llm-d provides the hooks to serve them efficiently.

The guiding boundary: **the engine is the mechanism, llm-d is the meaning** — the semantic layer lives one level above the engine, so the work spans vLLM, SGLang, and beyond.

Predicted Latency-Based Scheduling

llm-d's optimized baseline guide leverages load signals and prefix-cache affinity to schedule requests, combining the signals together with heuristics.

This path is for operators who want to adopt predicted latency-based scheduling - which uses an XGBoost model trained online - to make scheduling decisions. This strategy is useful when:

- Your workload has **high variance in prompt and completion length**, and queue depth alone is a poor proxy for true load.
- Your clients can express **per-request latency SLOs** (interactive vs. batch) and you want the gateway to enforce them.
- Static weight tuning between cache affinity and load has become **fragile** as traffic shifts.

[!NOTE] Predicted latency is not a fit when the pool is **heterogeneous** — mixed GPU types, model variants (e.g. prefill vs decode), or serving configurations in the same pool will produce inaccurate predictions, because the predictor assumes a single pod shape.

Deploy

See the Predicted Latency guide for manifests and step-by-step deployment.

Architecture

![Latency Predictor](../assets/latency-predictor.svg)

The setup deploys an EPP with the predicted latency sidecar containers:

- **Training Server** - trains the XGBoost model to predict TPOT and TTFT based on observed traffic
- **Prediction Servers** - predict TPOT and TTFT of the request based on current server state

During the standard request flow:

- Request arrives at the proxy, which forwards the request to the EPP
- EPP queries the prediction server
- EPP (using `latency-scorer`) selects optimal endpoint based on the prediction
- Proxy forwards request to the vLLM endpoint
- vLLM endpoint processes the request, returns response to proxy
- Proxy sends results to the training server, which uses samples to update the model

Further Reading

- [Latency Predictor Architecture](#) — plugin pipeline, ML model, scaling characteristics, metric reference.
- [llm-d/llm-d-latency-predictor](#) — source for the training and prediction server Python code.
- [Predicted Latency-Based Scheduling for LLMs - Blog](#) — design rationale and benchmark results.

Multimodal Workload Serving

Multimodal inputs are fundamentally changing the shape of production LLM traffic. A single prompt expands beyond text to include dense, non-text modalities—high-resolution images, video frames, or audio clips. Traditional HTTP requests are fast, uniform, and cheap. Standard round-robin request scheduling strategies balance this load well. LLM requests break all three assumptions. Multimodal LLM requests (containing images, video, or audio) break them even further:

- **Massive Context Inflation** — A single high-resolution image, audio clip, or video file drastically inflates the context window (often by thousands of tokens).
- **Heavy Prefill Cost** — Running vision/auditory encoders and prefilling thousands of tokens is highly resource-intensive.

The **llm-d Router** extends text-based prefix scheduling across both aggregated and disaggregated inference architectures by tracking, hashing, and matching complex multimodal payloads across a distributed cluster. The router intelligently directs incoming requests to the specific backend worker that already hold the corresponding pre-computed encoder cache and key-value (KV) blocks in memory. Whether operating in a unified topology or a decoupled precode-prefill-decode landscape, this targeted routing maximizes hardware efficiency and eliminates redundant processing.

Deploy

Multimodal Aggregated Guide

See the multimodal aggregation guide for aggregated guide manifests and step-by-step deployment.

Multimodal Disaggregated Guide

See the multimodal e-disaggregation guide for disaggregated guide manifests and step-by-step deployment.

Architecture & Scheduling

The llm-d-router schedules multimodal requests using prefix cache affinity and server load metrics.

[!NOTE] For the high-level scheduling architecture flow and EPP load-balancing diagrams, see the [Optimized Baseline](#) guide.

Prefix-Aware Scheduling

Approximate Prefix-Cache Aware Routing

EPP maintains a view of each endpoints' prefix-cache state in memory including both text and multi-modality assets, and prioritizes routing an incoming request to an endpoint that has high prefix cache matching. Extending from its approximate prefix cache matching algorithm for text input, EPP mathematically estimates the virtual token footprint of each multimodal asset.

EPP uses two highly customizable **Token Estimation Strategies**:

Providing Asset Metadata

Both strategies need per-asset metadata (dimensions for images; duration, FPS, and resolution for video) to estimate an asset's token footprint. EPP resolves this metadata as follows:

- **Images** — When an image is embedded inline (raw bytes) in the request, EPP reads its width and height directly from the image content. When an image is supplied as a URL, EPP does **not** fetch the remote content and falls back to its configured defaults.
- **Video** — EPP cannot inspect video content directly, so clients should supply video metadata via request headers. Any header that is omitted falls back to the corresponding EPP default:
- `x-llm-d-video-fps` — the video's frames per second.
- `x-llm-d-video-duration-seconds` — the video's length in seconds.
- `x-llm-d-video-resolution` — the video frame resolution as `WIDTHxHEIGHT` (e.g. `1280x720`).

A. Dimension-Based Approximation (e.g., Qwen-VL)

Image

Estimate tokens based on image width and height:
$$\text{Tokens} = \frac{\text{Image Width} \times \text{Image Height}}{\text{Factor}}$$

- The `Factor` parameter is configurable per-EPP:
- For **Qwen 2.5 VL**: `factor = 784` (which is 28×28)
- For **Qwen 3.5 VL**: `factor = 1024` (which is 32×32)

Video

Estimate tokens as the per-frame token count multiplied by the number of sampled frames:
$$\text{Tokens} = \text{tokensPerFrame} \times \text{Number of frames}$$

- **tokensPerFrame** — in `dynamic` mode this reuses the image formula ($\frac{\text{width} \times \text{height}}{\text{factor}}$).
- **Number of frames** — Qwen3-VL uses the **sampled** strategy: the video is sampled at `sampleFPS`, clamped to `[minFrames, maxFrames]`, then every `temporalPatchSize` sampled frames are merged into one token group:
$$\text{Number of frames} = \frac{\text{clamp}(\text{duration} \times \text{sampleFPS}, \text{minFrames}, \text{maxFrames})}{\text{temporalPatchSize}}$$

The total is capped by `maxVideoTokens` . Example per-EPP config for Qwen3-VL:

```
estimate:
video:
  tokensPerFrame:
    mode: dynamic
    dynamic:
      factor: 1024      # 32 × 32
  frames:
    mode: sampled
    minFrames: 4
    maxFrames: 768
    sampled:
      sampleFPS: 2
      temporalPatchSize: 2
    maxVideoTokens: 12288
```

B. Configuration-Based Fixed Allocation (e.g., Gemma 4)

Directly use fixed values from user configuration matching the model's support levels:

Image

- Gemma 4 supported values: 70, 140, 280 (default), 560, or 1120 tokens per image.

Video

Estimate tokens as the fixed per-frame allocation multiplied by the number of frames: $\text{Tokens} = \text{tokensPerFrame} \times \text{Number of frames}$

- **tokensPerFrame** — a constant per-frame value taken directly from user configuration (`static` mode).
- **Number of frames** — Gemma 4 uses the **strided** strategy: frames are taken every `frameStride` source frames, clamped to `[minFrames, maxFrames]` : $\text{Number of frames} = \text{clamp}\left(\frac{\text{duration}}{\text{sourceFPS}} \times \text{frameStride}, \text{minFrames}, \text{maxFrames}\right)$

Example per-EPP config for Gemma 4:

```
estimate:
video:
  tokensPerFrame:
    mode: static
    static:
      numTokensPerFrame: 296
  frames:
    mode: strided
    minFrames: 1
    maxFrames: 8
    strided:
      frameStride: 4
```

Precise Prefix-Cache Aware Routing

This routing strategy bases routing decisions directly on the precise, real-time physical memory block states of individual model server endpoints. It requires the router to tokenize the input, and subscribe to the KV-events channels of model server endpoints. Internally the router maintains an **indexer** which maintains a block key → model server endpoints mapping for every block resident across the fleet. For incoming requests the router breaks the tokenized input into blocks and matches the block keys with the indexer to determine which model server endpoint has the longest prefix match. Multi-modal assets are converted to block keys considering the asset hash and size so that they are correctly accounted.

Load-Aware Routing

EPP continuously probes each endpoints' metrics by scraping `/metrics` at a regular interval (50ms default). It scores endpoints on queue depth, running requests, and KV-cache utilization to schedule requests to the endpoint with the lowest load, avoiding hotspots caused by heterogeneous request patterns.

Further Reading

- See [Optimized Baseline](#) for details on text-based scheduling and general load-balancing.
- See [EPP Architecture](#) for more details.
- See [KV-Cache Indexer](#) for details on precise event-driven indexing.

Precise Prefix Cache Routing

The model server is the most accurate source of truth for what's cached on its own GPUs and memory tiers. vLLM, SGLang and NVIDIA TensorRT-LLM publish every cache change as an event; llm-d subscribes to that stream, builds a near-real-time view of resident blocks across the fleet, and scores requests against it. The prefix-affinity score is combined with the standard load-aware scorers, similarly to the Optimized Baseline path.

KV-events have become the ecosystem-standard substrate for exposing accurate cache state — where reusable inference state lives and how it changes over time. As KV-cache orchestration grows more sophisticated and agentic workloads stretch prefixes longer, cache state becomes something the control plane needs to observe and act on. The same view scales naturally to:

- tier-aware cache tracking across GPU HBM, CPU DRAM, local NVMe, and shared storage;
- policies that account for explicit prompt-cache placement and dynamic KV-offloading;
- cache movement and prefetching workflows for fleet-wide KV reuse;
- advanced KV retention and eviction policies for agentic patterns;
- hybrid-attention models where layer groups (full, sliding-window, linear) evict independently.

Deploy

See the precise prefix cache routing guide for manifests and step-by-step deployment.

Architecture

The split is straightforward: **model servers** produce KV-events on every cache change; the **llm-d Router** consumes them to score pods for better routing decisions. The two sides are decoupled — model server and llm-d Router replicas scale independently.

Inside the llm-d Router:

- An **indexer** consumes the event stream and maintains a `block key → pods` mapping for every block resident across the fleet.
- A **scorer** derives block keys deterministically from the input and queries the index. It returns the longest consecutive prefix each candidate pod has cached, weighted by tier.

Events flow from model-server pods to the llm-d Router over ZMQ via **pod discovery**: each model-server pod binds its own ZMQ socket and every llm-d Router replica subscribes to every pod independently. All Router replicas converge to the same index, enabling active-active HA out of the box. The reference guide ships with two Router replicas behind one Service by default, scalable down to one for small fleets.

Further Reading

See KV-Cache Indexer for the full architecture

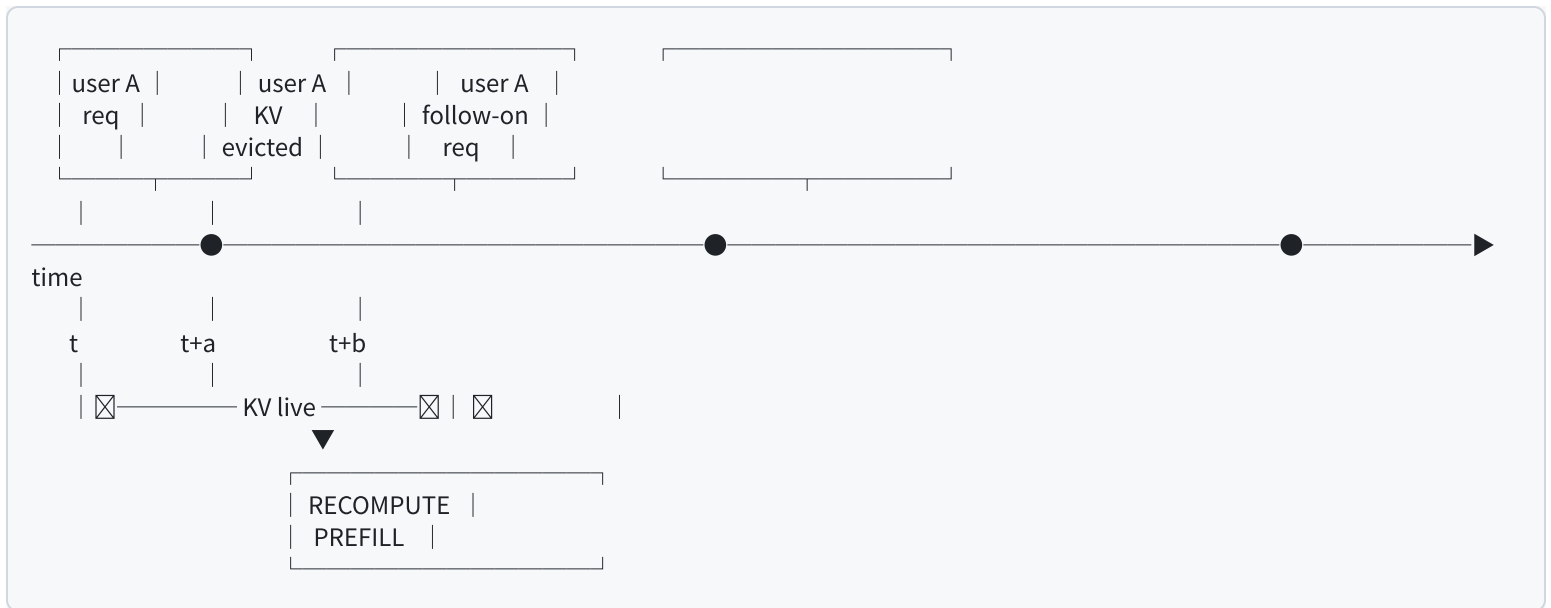
Tiered Prefix Cache

Given the multi-turn nature of agentic workloads, prefix-cache re-use is a critical factor for high performance inference.

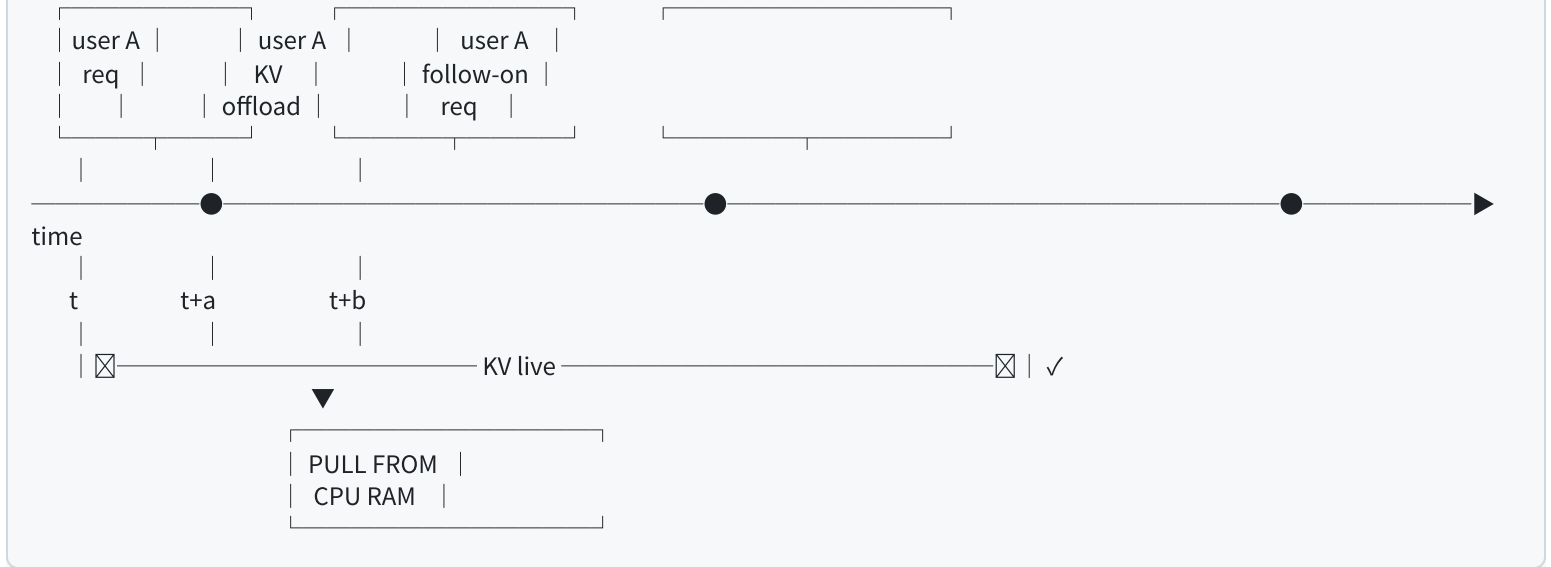
Model servers hold KV-caches in GPU RAM with an LRU eviction scheme. Once space runs out from other requests consuming the resources, the KV caches are evicted. Follow on requests then must recompute the prefill. However, rather than evicting KV caches from GPU memory, we can instead leverage other system resources such as CPU RAM, local NVMe drives, and network storage systems to hold the evicted KVs - pulling them back into GPU RAM on demand.

This increases the **KV-working set size**, growing the **receptive-field** (the amount of time KV caches are retained in the system).

- Without KV offloading:



- With KV offloading:



[!IMPORTANT] CPU KV Cache offloading is low overhead and introduces ~no additional complexity. It can be enabled in almost all deployments. Storage offloading requires additional consideration.

Storage Tiers

Offloaded KV caches can live on several tiers, ordered by read/write latency: frequently accessed caches stay closest to the accelerator, while larger or colder caches move to slower, higher-capacity tiers. We recommend always enabling the HBM and CPU RAM tiers, and adding a filesystem tier when the working set grows beyond HBM + CPU RAM.

- **CPU RAM** — Low operational overhead and typically far larger than accelerator HBM, making it the default offload target. Loading from CPU RAM is faster than recomputing prefill in most cases, and asynchronous offload adds little overhead.
- **Local disk** — Increases capacity further, but is slower than CPU RAM. Suitable when the workload tolerates the added latency and local capacity is sufficient.
- **Shared (remote) storage** — Provides capacity independent of deployment size, KV-cache sharing across replicas, fast scale-up (new replicas reuse existing cache), and persistence across restarts and failures. Latency and throughput depend on the underlying system, so evaluate that the transfer cost does not outweigh the savings. Mature enterprise systems (for example CephFS, GCP Lustre, IBM Storage Scale, AWS EFS) integrate through standard POSIX file access.
- **P2P sharing** — Inference replicas can share caches in CPU memory over a peer-to-peer network, extending sharing without additional storage resources. See [Enable P2P Prefix Cache Sharing](#).

Deploy

See the [Tiered Prefix Cache](#) guide for manifests and step-by-step deployment.

Architecture

llm-d leverages the following architectures for offloading.

CPU KV Cache Offloading

Each model server offloads to host CPU memory through its own native mechanism: vLLM uses the `OffloadingConnector`, and SGLang uses HiCache. Pods are configured with the connector enabled and increased CPU memory requests (e.g., 400 GB). Evicted KV-cache blocks move to host CPU memory instead of being discarded, extending the effective cache size with negligible overhead. The EPP maintains a global index of which blocks exist on which pods and tiers, adding a second `prefix-cache-scorer` plugin for CPU-tier blocks.

![CPU KV Cache Offloading](../../assets/cpu-offloading.svg)

Tiered Offloading to Filesystem

vLLM's `OffloadingConnector` natively supports a multi-tier hierarchy: HBM → CPU RAM → filesystem. Configured with `TieringOffloadingSpec` and a `secondary_tiers` entry of `type: fs`, evicted blocks spill from CPU RAM to a `ReadWriteMany` PVC mounted at `/mnt/files-storage` (backed by Lustre, CephFS, IBM Storage Scale, AWS EFS, or similar). I/O is asynchronous and uses GPU DMA, parallelized across read/write threads. Because the tier is shared, newly scaled pods read existing cache immediately and the cache persists across pod restarts; capacity is bounded only by the storage system.

The connector does not evict data from the shared tier -- capacity is managed by the storage system or by an external controller (a reference PVC evictor is available in the `llm-d-kv-cache` repository).

![Tiered Offloading to Filesystem](../../assets/fs-offloading.svg)

Further Reading

- Tiered Prefix Cache guide — manifests and step-by-step deployment.
- vLLM KV offloading connector — design of the native `OffloadingConnector` and its tiering.
- Multi-tier KV offloading RFC — the upstream tiering design.
- LMCache and SGLang HiCache — alternative offloading implementations supported by this path.

[Experimental] Enable P2P Prefix Cache Sharing

Prefix caches are per-pod, but their content is often fleet-wide: shared system prompts, common documents, session histories. Prefix-aware routing sends each request to the pod that caches its prefix, but routing cannot always follow the cache: a hot prefix's owner saturates, a working set outgrows any single pod, a session is rebalanced. Those requests recompute KV tensors that already exist on a peer.

P2P prefix cache sharing closes that gap: a model server pulls cached prefix KV blocks from a peer's CPU offload tier instead of recomputing them. The transfer is CPU-to-CPU over NIXL. The source pod's GPU is never touched, so serving a pull costs the source no prefill capacity.

The pull fires when a request shares a prefix with an earlier one but is scheduled to a different pod. Two requests share a prefix whenever they begin with the same tokens: the next turn of a conversation, another question against the same document, another session on a shared system prompt. The first request's pod is the **KV cache source**: it computed the prefix and holds a copy in its CPU tier. When the router schedules a prefix-sharing request to a different pod, it names the source on the request, and the scheduled pod - the **consumer** - pulls the prefix instead of recomputing it:

```
sequenceDiagram
    participant R as llm-d router
    participant S as KV cache source pod
    participant C as consumer pod
    R->>S: request 1
    Note over S: computes the prefix KV, caches it, offloads a copy to its CPU tier
    Note over R: request 2 arrives sharing request 1's prefix, but placement picks a different pod
    R->>C: request 2 + header naming the source pod
    alt without P2P prefix cache sharing
        Note over C: recomputes the full shared prefix
    else with P2P prefix cache sharing
        C->>S: request the prefix blocks
        S-->>C: prefix KV blocks, CPU tier to CPU tier over NIXL
        Note over C: computes only the remainder (request 2's unshared tokens)
    end
```

[!IMPORTANT] P2P prefix cache sharing builds on the Tiered Prefix Cache path: peers serve pulls from their CPU offload tier. The shipped configuration uses the same `--block-size`, `PYTHONHASHSEED`, and tensor parallel layout on every peer. Other layouts need a compatible peer session fingerprint; the guide documents the tensor-parallel and model-runner constraints. Size the CPU tier to retain useful blocks; making it larger than the per-pod GPU KV cache is the guide's recommended starting point, not a correctness requirement. The guide's Best Practices covers each requirement, its sizing rule, and its failure mode.

When It Pays

Recompute cost grows with prefix length. On the measured H200/RDMA setup, the CPU-to-CPU pull also grows with prefix length, but much more slowly. For `openai/gpt-oss-120b`, the pull was faster at every measured point from 2,048 to

49,152 tokens. The crossover is model-, hardware-, and transport-specific. The router requests a pull only when the selected source holds at least `minCachedTokenDelta` more prefix tokens than the scheduled pod, so calibrate that value on the target deployment.

Whether the pull helps also depends on the placement policy in front of it:

- **Ownership is stable and uncontended:** prefix-aware routing avoids the transfer. A local hit is cheaper than a pull, so the pull stays quiet.
- **A hot prefix saturates its owner, or the working set outgrows the caches:** load-aware placement plus the pull serves the same content from the whole fleet. On the warm 16-pod document Q&A benchmark this delivered 1.5x better p99 TTFT and 35% higher throughput than prefix-affinity routing.
- **GPU KV capacity itself is the bottleneck:** cache co-location can use capacity more efficiently. Concurrent same-prefix requests on one pod share one copy of the blocks; spreading pays a per-pod copy whether the prefix is pulled or recomputed.

The guide ships prefix affinity plus the pull as the general-purpose default. Reach for load-aware placement plus the pull when many concurrent sessions contend on their owner pods. Both regimes are measured in the benchmark report.

Deploy

See the P2P KV Cache Sharing guide for manifests, verification gates, and step-by-step deployment.

Architecture

1. **Model server pods publish KV-cache events** and run `vLLM's OffloadingConnector` with a CPU tier plus a P2P secondary tier: every pod both offloads computed KV to CPU and serves it to peers.
2. **The router builds the precise prefix index** from the KV events (the Precise Prefix Cache Routing mechanism), so it knows which pods hold which prefix blocks.
3. **The `p2p-source-producer` selects a source** from the CPU-tier holders within one index block of the largest cached prefix, weighted to avoid concentrating pulls on a queued source. After scheduling, it sets the KV cache source header only when that source leads the computing pod by at least `minCachedTokenDelta` tokens.
4. **The routing sidecar injects `kv_transfer_params.remote_kv_source`** from the header, and the engine pulls the prefix blocks from the peer's CPU tier over NIXL. Hits load as normal cache hits; a failed lookup is reported as a miss and the scheduled pod computes the missing prefix locally.

Under P/D disaggregation, the prefill worker is the pull consumer because it computes the prompt KV. A decode worker may be the source for generated session history retained in its CPU tier. After prefill completes, the normal NIXL P/D path transfers the request's KV to the selected decoder.

Further Reading

- P2P KV Cache Sharing guide - manifests, verification gates, benchmarking.
- Benchmark report: gpt-oss-120b on H200 - crossover, shared-prefix pools, document Q&A.
- Benchmark report: GLM-5.2 on H200 - C64 policy comparison, four-arm observation, and pull mechanism evidence.

- Tiered Prefix Cache - the offload tiers P2P serves from.
- Precise Prefix Cache Routing - the index that selects the pull source.

P/D Disaggregation

LLM inference has two computationally distinct phases:

- **Prefill** processes the entire input prompt in a single forward pass - it is compute-bound, bottlenecked by the GPU flops available.
- **Decode** generates output tokens one at a time from the KV-cache - it is memory-bandwidth-bound, bottlenecked by how fast data moves from HBM to on-chip memory.

For long context workloads (10:1 ISL:OSL) and medium-to-large models, separating prefill and decode into separate instances enables:

- Improved throughput via specialization or prefill and decode
- Improved quality of service, as long context prefills will not block decode work

llm-d's EPP natively supports the concept of disaggregation, enabling composition with other scorers (e.g. prefix-aware routing).

[!IMPORTANT] NIXL supports TCP transfer, but high-bandwidth networking (IB, RoCE, EFA) is **highly recommended** for production usage.

Deploy

See the P/D Disaggregation guide for manifests and step-by-step deployment.

Architecture

![P/D Disaggregation](../assets/pd-disaggregation.svg)

The setup creates 2 Deployments of vLLM (all are part of the same InferencePool):

- The **prefill** Deployment is 4 replicas of TP=1 vLLM - labeled with `llm-d.ai/role=prefill`.
- The **decode** Deployment is 1 replica of TP=5 vLLM - labeled with `llm-d.ai/role=decode`. All these pods have a routing proxy sidecar.

During the standard request flow:

- Request arrives at the proxy, which forwards the request to the EPP
- EPP schedules the request with P/D disaggregation, using the labels to detect the decode and prefill variants
- Request is routed to the sidecar, which forwards the request to the prefill instance

- Prefill instance processes the prompt, returning metadata about how to retrieve the KV blocks
- Decode instance pulls the KVs over RDMA (IB, RoCE, EFA) with NIXL
- Decode instances processes the decodes

Observability

P/D disaggregation runs prefill and decode as independently scaled pools joined by a NIXL KV transfer, so the signals that matter are pool **balance** and transfer health rather than a single aggregate latency. The P/D guide's Observability & Troubleshooting section covers the key metrics for this path and the common failure modes, backed by the shared PromQL and metric references.

Further Reading

See PD Architecture for more details.

Multi-Node Wide Expert Parallelism

Very large MoE models like DeepSeek-R1 can consume 500GB+ of RAM just to hold the weights of the model, pressuring KV cache space for long context and high throughput serving. This problem is especially magnified for models with MLA attention, which replicates the KV cache when sharded with tensor parallelism.

To address these issues, model servers support DP/EP deployments, which deploys the attention layers with data parallelism and the MLP layers with expert parallelism. This deployment pattern enables scaling the KV cache space, as the pattern:

- **Scales to multiple nodes** - since the collective operations (dispatch/combine) are sparse - tokens are only sent to the expert rank after routing - they consume much less bandwidth than the all-reduces used in TP setups, making them suitable to run over slower interconnects (IB, RoCE rather than NVLink)
- **Avoids KV replication** - since attention is data-parallel (TP=1 in every DP group), there is only one copy of each tokens' KV

The following visualizes the forward pass in a DP/EP deployment in vLLM:

![DP/EP deployment](../assets/dp-ep-deployment.svg)

The following steps occurs:

- Each rank runs attention independently
- MoE router selects the `topk` experts for each token (this is sparse - in the case of DeepSeek, 8 out of 256 experts are selected)
- Tokens are "dispatched" (using the `topk_id`) to the proper expert rank (e.g, the green token on rank 1 is routed to E1 and E3)
- Each expert runs independently
- Tokens are "combined" backed to the original attention rank

[!IMPORTANT] Dispatch/combine uses the **DeepEP** backend over NVSHMEM with GPU-initiated RDMA (`ibgda` transport), requiring full-mesh InfiniBand/RoCE connectivity.

Deploy

See the Wide Expert Parallelism guide for manifests and step-by-step deployment. The model servers are deployed as a `DisaggregatedSet`, which manages the prefill and decode roles as a single versioned unit.

Architecture

Multi-node "WideEP" deployments are typically combined with disaggregated serving because:

- Disaggregation avoids "bubbles" where Rank N is computing a prefill and Rank M is computing a decode
- Specialized kernels for prefill and decode can be used (e.g. DeepEP HT vs DeepEP LL)

As a result, we leverage the following design for the deployment:

- Disaggregated prefill and decode via llm-d's EPP
- `LeaderWorkerSet` to manage multi-node pod group deployment of vLLM
- DP/EP deployment configuration in vLLM

![[Multi-Node Wide Expert Parallelism](../assets/wide-ep.svg)]

The request flow works as follows:

- Request arrives at the proxy, which forwards the request to the EPP
- EPP schedules the request with P/D disaggregation, using the labels to detect the decode and prefill variants. The EPP schedules to specific pods within the LWS
- Request is routed to the sidecar, which forwards the request to the prefill pods
- Prefill instance processes the prompt, executing the forward pass with DP/EP. DeepEP executes the cross-node dispatch/combine collectives. vLLM returns metadata about how to retrieve the KV blocks
- Decode instance pulls the KVs over RDMA (IB, RoCE, EFA) with NIXL
- Decode instances processes the decodes, executing the forward passes with DP/EP. DeepEP executes the cross-node dispatch/combine collectives

Further Reading

See:

- PD Architecture for more details on disaggregation in llm-d
- vLLM docs on DP deployment
- vLLM docs on EP deployment
- vLLM docs on DeepEP and DeepGEMM

Flow Control

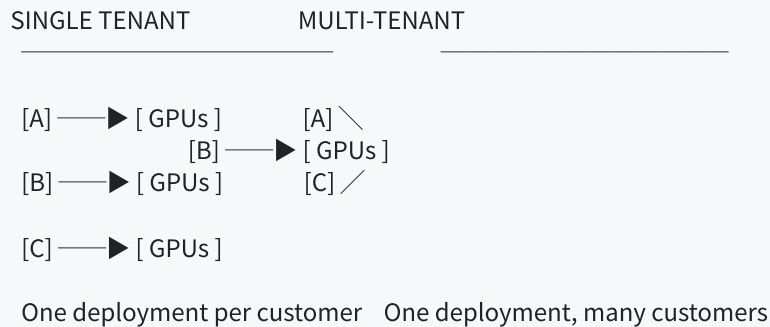
Flow Control feature enables intelligent request queuing. Request queuing is useful for multiple reasons:

Multi-Tenant Deployments

In comparison to a single workload deployment, operators of multi-tenant workloads have additional considerations:

- Certain tenants are **higher-priority** than others (e.g. paid vs unpaid)
- Certain requests have **different-SLOs** than others (e.g. batch vs online)
- Certain tenants are more active than others - we want **fairness** between them

Flow control introduces intelligent queuing to the EPP, allowing operators to factor traffic dynamics into scheduling decisions. This capability addresses noisy-neighbor problems when mixing high- and low-priority traffic; furthermore, it ensures fairness among equal-priority tenants, preventing any single user from starving others of shared pool resources.



Single Workload "No-Regret" Scheduling

In addition to inter-tenant prioritization and fairness, flow control also enables "no-regret" scheduling by holding requests during peak saturation. By delaying the dispatch until load subsides—rather than committing a request to a specific server's queue where it becomes stuck—the EPP ensures requests land on the best available resource.



Deploy

For detailed step-by-step instructions on how to deploy and configure Flow Control, see the Flow Control Guide.

Architecture

![Flow Control](../../assets/flow-control.svg)

Requests arrive to the proxy with headers expressing their tenant ID and traffic priority. EPP leverages these headers to assign a `FlowKey` (tuple of `FairnessID` and `Priority`) to each request and maintains separate in-memory queues for each `FlowKey`. Each `FlowKey` is assigned to a `PriorityBand` (for cases when multiple tenants have the same priority).

Then, in each scheduling cycle, the EPP traverses the queues in 3 tiers:

- **Priority** - the system always services highest `PriorityBand` first
- **Fairness** - within a `PriorityBand`, the **Fairness Policy** determines which flow (i.e. tenant) is dispatched next
- **Ordering** - within a flow (i.e. tenant), the **Ordering Policy** determines which request to serve (e.g. FCFC or SLO-aware)

In the background EPP monitors the model servers for saturation. If it detects saturation, requests are queued until saturation subsides.

[!WARNING] **Trust Boundary:** In a production system, allowing end-users to self-assert their tenant ID or traffic priority (`premium-traffic`) is an abuse vector. In production, these headers should be stripped from external requests and injected by an upstream trusted API gateway, identity provider, or Envoy AuthZ filter based on the API key.

Further Reading

See Flow Control architecture for full details of the design.

Workload Autoscaling

Traditional autoscaling indicators like resource utilization metrics (CPU/GPU) are often lagging indicators — they only reflect saturation after it has already occurred, by which point latency has spiked and requests may be failing. For LLM inference, this problem is compounded by the fact that GPU utilization is often pegged near 100% during active batching regardless of actual load, making it an unreliable signal entirely.

Effective LLM autoscaling requires proactive, SLO-aware signals that reflect the true state of the inference system — queue depth, in-flight request counts, and KV cache pressure — so that capacity can be added before end-user latency is impacted.

The llm-d stack provides two primary paths for workload autoscaling. The EPP path uses an HPA created and managed by KEDA. The WVA path publishes desired-replica signals that can be consumed by an HPA or KEDA, depending on the deployment configuration.

Deploy

See the workload autoscaling guide for manifests and step-by-step deployment instructions for both paths.

Strategies

KEDA + EPP Metrics

This path configures KEDA's Prometheus scaler with signals emitted directly by the Endpoint Picker (EPP). By using metrics such as queue depth and running request counts, KEDA's generated HPA reacts to queueing and active request concurrency, then reduces replicas when demand falls.

- **Best for:** Deployments on homogeneous hardware where each target model-server pool can be isolated by metrics and scaled independently.
- **Signals:** EPP metrics (queue depth, running request count).
- **Components:** KEDA and Prometheus; Prometheus Adapter is not required.

Workload Variant Autoscaler (WVA)

The Workload Variant Autoscaler (WVA) is designed for operators running multiple model variants on shared, potentially heterogeneous GPU hardware. It continuously monitors KV cache utilization, queue depth, and performance budgets to determine optimal replica counts across variants (e.g., A100 vs. L4).

- **Best for:** Multi-variant deployments where cost-aware capacity allocation is required.
- **Signals:** KV cache utilization, queue depth, energy/performance budgets.

- **Components:** WVA controller and KEDA. WVA publishes a `wva_desired_replicas` metric for any KEDA `ScaledObject` (or HPA) annotated with `llm-d.ai/managed: "true"` , and KEDA drives the scale from it. The `VariantAutoscaling` CRD is deprecated as of llm-d 0.8.0.

Choosing a Path

Feature	KEDA + EPP Metrics	Workload Variant Autoscaler (WVA)
Primary Goal	Load-based scaling	Cost-optimized scaling across hardware
Heterogeneous Support	Limited	Native
Complexity	Low (requires KEDA and Prometheus)	Medium (requires WVA controller)
Scale to Zero	Supported	Supported

Observability

Autoscaling is a closed loop, so it is observed as a composed signal across the model servers, the autoscaler, and the resulting pool rather than as any single metric. The guide's Observability & Troubleshooting section walks the loop end to end (demand, decision, convergence, outcome) and the common failure modes for both scaling paths, backed by the shared PromQL and metric references plus the WVA metric definitions.

Multi-Model Routing

Organizations often need to serve multiple large language models behind a single API endpoint. A chatbot application might use a Qwen model for conversational tasks, while a recommendation system uses DeepSeek for complex reasoning. Each base model may also have multiple Low-Rank Adaptation (LoRA) fine-tuned variants serving different use cases.

Traditional path-based routing cannot distinguish between these models when they share the same API path (`/v1/chat/completions`). The **Inference Payload Processor (IPP)** solves this by extracting the model name from the request body and routing to the appropriate InferencePool.

[!NOTE] This capability requires IPP deployment. For simpler deployments serving a single model, see the [Optimized Baseline](#) guide instead.

Deploy

See the [Multi-Model Routing](#) guide for manifests and step-by-step deployment.

Architecture

![Multi-Model Routing Architecture](../../assets/multi-pool-routing.svg)

The setup creates multiple `InferencePools`, each serving a different base model:

- Each **InferencePool** serves one base model and its LoRA adapters.
- **IPP** extracts the model name from the request body and maps it to the base model.
- **HTTPRoutes** match on the `X-Gateway-Base-Model-Name` header to route to the correct pool.
- **EPP** selects the optimal endpoint within each pool.

During the standard request flow:

- Request arrives at the Proxy with `{"model": "food-review-1"}` in the body
- Proxy invokes IPP via ext-proc
- IPP looks up `food-review-1` → base model `Qwen/Qwen3-32B`
- IPP sets header `X-Gateway-Base-Model-Name: Qwen/Qwen3-32B`
- Proxy matches HTTPRoute and routes to `qwen-pool`
- EPP selects endpoint, preserving the original model name
- Model server loads the `food-review-1` LoRA adapter

Further Reading

See [IPP Architecture](#) for more details on the Inference Payload Processor.

Fast Model Actuation

In Kubernetes, once a Pod has been allocated some GPUs it keeps that exclusive allocation for the rest of the Pod's lifetime. Scaling a model server up or down therefore means creating or destroying whole pods, and each new pod pays a full **cold start** — pull the image, initialize the runtime, load gigabytes of weights, and compile CUDA graphs — before it can serve a single request. On a shared GPU pool, swapping between model variants repeats that cold start on every swap.

Fast Model Actuation (FMA) attacks vLLM startup time with two complementary techniques: **vLLM sleep/wake**, which parks a loaded model's tensors in main memory and restores them to the GPU in seconds — skipping model loading and CUDA-graph compilation entirely — and a **launcher** process that loads the vLLM Python modules once and spawns child vLLM instances on demand, skipping that module import. Together they turn model swap-in and replica scale-up from a minutes-scale cold start into a near-instant operation — without changing steady-state serving performance.

[!NOTE] FMA's value is *actuation speed*, not inference throughput. Resident servers add only a small (~2.5%) CPU-memory overhead, and served performance matches a standard deployment. The Workload Autoscaling capability decides *when* to scale; FMA addresses *how fast* the new capacity becomes ready.

Deploy

See the fast model actuation guide for manifests and step-by-step deployment.

For **autoscaled** FMA — see the fast model actuation + KEDA autoscaling guide, where KEDA scales the requester pool on EPP flow-control saturation metrics and each scale-up drives an FMA hot-wake (resident sleeping instance) or warm-create (new instance on an existing launcher).

Architecture

FMA circumvents that exclusive, lifetime-long GPU allocation with a **dual-pods** design coordinated by the FMA controllers:

- **Server-requesting pods** reserve GPU resources through the Kubernetes scheduler and kubelet — keeping cluster capacity accounting accurate — but do not run inference themselves.
- **Launcher pods** (server-providing) run vLLM while requesting **zero** GPUs. They can access all of their node's GPUs through provisions that don't count as scheduler usage, and the FMA controllers point vLLM at the specific GPU(s) reserved by the requesting pod via `CUDA_VISIBLE_DEVICES`.
- **FMA controllers** create and delete launchers, bind requesters to launchers and to the vLLM instances within them, create and delete those vLLM instances, and orchestrate sleep/wake so a fixed GPU pool can host many model variants.

Actuation paths

When capacity is needed, the controller selects a launcher and takes the fastest available path:

- **Hot start** — wake an already-resident, sleeping instance by moving weights from CPU memory back to the GPU over PCIe (sub-second).
- **Warm start** — an existing launcher creates a fresh vLLM instance using preloaded modules (tens of seconds).
- **Cold start (with launcher)** — create a new launcher pod, then initialize the instance; still faster than a no-FMA cold start, which additionally pays pod scheduling and image pull.

To bound memory, the controller keeps at most a configurable number of instances sleeping per accelerator, evicting the least-recently-used instances when that budget is exceeded.

Batch Serving

The **Batch Serving** workload umbrella defines recommended, cohesive deployments for processing large-scale, offline, or latency-insensitive tasks on llm-d infrastructure.

Depending on your integration requirements, scale, and operational environment, llm-d offers two distinct paths for queue-based and batch inference:

- **Batch Gateway:** An enterprise-grade, fully managed **OpenAI-compatible Batch API** (`/v1/batches` , `/v1/files`). Best for multi-tenant environments where clients require formal asynchronous job submission, file storage, status tracking, and strict separation of interactive vs. batch compute.
- **Asynchronous Processing:** A lightweight, low-overhead queue dispatch mechanism (using Redis or GCP Pub/Sub). Best for internal microservice architectures that require low-complexity background task processing or for filling "slack" capacity in your inference pool via dynamic dispatch gating.

Deploy

See the Batch Serving Guide for deployment options, comparative analysis, and operational guides.

Asynchronous Processing

The Asynchronous Processing path enables queue-based inference for latency-insensitive workloads or for filling "slack" capacity in your inference pool. It decouples request submission from execution, allowing clients to submit large volumes of work without maintaining a long-lived HTTP connection.

Deploy

See the asynchronous processing guide for deployment instructions using Helm and supported queue implementations (Redis or GCP Pub/Sub).

Architecture

The **Async Processor** is a lightweight dispatch agent that pulls requests from a message queue and forwards them to the llm-d Router.

Dispatch Gating

To prevent background tasks from impacting real-time traffic, the Async Processor uses **Dispatch Gates**. These gates regulate the flow of requests based on system metrics:

- **Prometheus Gating:** Queries model server saturation (e.g., KV cache pressure, queue depth) and only dispatches when the system has available "slack" capacity.
- **Budget Gating:** Uses a pre-calculated budget to control throughput.
- **Priority & Deadlines:** Requests can be prioritized, and the processor enforces deadlines to ensure stale work is abandoned.

![Async Processor Architecture](../../assets/async-processor.svg)

Resilience

- **Retries:** Transient failures (like rate limits or network issues) are automatically re-queued with exponential backoff.
- **Concurrency Control:** Configurable worker pools allow you to tune the degree of parallelism for background processing.

Use Cases

- **Batch Inference:** Processing large datasets where completion time is measured in minutes or hours rather than milliseconds.

- **Slack Capacity Filling:** Using idle GPU cycles between real-time request spikes to perform background tasks like document summarization or embedding generation.
- **Offline Evaluation:** Running model evaluation pipelines without competing for production resources.

Further Reading

See the Async Processor Architecture for more details on the internal mechanics.

Batch Gateway

Process large-scale batch inference jobs via an OpenAI-compatible API, enabling batch and interactive workloads to coexist efficiently on shared infrastructure.

When to Pick This Path

- You have **offline inference workloads** (such as evaluations, embeddings, dataset processing) that don't need real-time responses.
- You want to **utilize idle accelerator capacity** for batch work while protecting interactive traffic from interference.
- Your clients expect an **OpenAI-compatible Batch API** (`/v1/batches` , `/v1/files`) for job submission, tracking and management.
- You need **multi-tenant isolation** — each tenant's jobs, files, and results are separated.

Prerequisites

- A working llm-d Router, inference pool, and at least one model server. If you don't have this, start with [getting-started/quickstart.md](#).
- PostgreSQL (12+) and Redis (6+) or Valkey (8+) accessible from the cluster.
- S3-compatible storage or a shared PVC with `ReadWriteMany` (RWX) access mode for batch input/output files.
- Helm 3.0+.

Deploy

- [Batch Gateway Deployment Guide](#) — full deployment instructions, configuration options, and troubleshooting.

Related

- [Batch Gateway Architecture](#) — components, data flow, and processing pipeline.
- [Batch Gateway Repository](#) — source code, Helm chart, platform-specific deployment guides, and demo scripts.
- [Asynchronous Processing](#) — complementary queue-based async inference for individual requests.

Architecture

High-level guide to llm-d architecture. Start here, then dive into specific guides.

Core Components

The llm-d architecture is built around three primary concepts: the Router, the InferencePool, and the Model Server.

- **llm-d Router** - The intelligent entry point for inference requests. It provides LLM-aware load balancing, request queuing, and policy enforcement. It is composed of two functional parts:
- **Proxy**: A high-performance L7 proxy conformant with the Gateway API Inference Extension (GAIE) that accepts user requests and consults the EPP via the `ext-proc` protocol to determine the optimal destination.
- **Endpoint Picker (EPP)**: The routing engine that scores and selects model server pods based on real-time metrics, KV-cache affinity, and configured policies.
- **InferencePool** - The API that groups Model Server pods serving the same base model via a label selector. Conceptualized as an "LLM-optimized Service", it serves as the discovery target for the Router. Additionally:
- **Variant** - A logical sub-grouping of Model Server pods within an InferencePool, expressed through pod labels rather than a dedicated resource. Variants distinguish model servers based on shared characteristics such as serving role (e.g., prefill or decode), cost profile, performance profile (throughput, latency), or other operational attributes.
- **Model Server** - The inference engine (such as vLLM or SGLang) that executes the model on hardware accelerators (GPUs, TPUs, HPUs).



Advanced Patterns

llm-d's core design can be extended with optional advanced patterns:

KV Cache Management

llm-d provides a comprehensive ecosystem for managing and reusing the KV cache across the inference pool. This includes:

- **Prefix-Cache Aware Routing**: Heuristic and precise techniques to maximize cache hits.
- **KV-Cache Indexing**: Event-driven tracking of cache state across all model servers.
- **KV Offloading**: Tiered storage hierarchy (CPU, SSD) for extending cache capacity.
- **P2P Prefix Cache Sharing**: Pulling cached prefix KV blocks from a peer's CPU tier instead of recomputing them.

See KV Cache Management for an overview of how these components compose.

Disaggregated Serving

In disaggregated serving, a single inference request is split into multiple phases (e.g., Prefill and Decode) handled by specialized workers. The llm-d Router orchestrates this flow by selecting both a prefill and a decode endpoint and coordinating the KV-cache transfer between them.

See Disaggregation for complete details.

Predicted Latency-Based Routing

The llm-d Router can be extended with "consultant" sidecars that provide advanced signals for routing decisions. The primary implementation is the **Latency Predictor**, which enables routing based on predicted ITL and TTFT.

- Latency Predictor: Trains an XGBoost model online to predict request latency for better endpoint scoring and SLO enforcement.

Batch Inference

Batch and offline inference workloads are handled by two modules that can be deployed independently or together. The Batch Gateway provides an OpenAI-compatible Batch API for job management, while the Async Processor dispatches queued requests with flow-control gating. When composed, the Batch Gateway delegates dispatch to the Async Processor.

See Batch Inference for details on the batch inference design.

Autoscaling

llm-d supports proactive, SLO-aware autoscaling driven by metrics exported by the EPP. KEDA's Prometheus scaler reads those signals — queue depth, pool saturation, token backlog, or estimated latency against an SLO — and creates and owns the HPA that scales model server replicas. No custom controller or Prometheus Adapter is required.

The Workload Variant Autoscaler (WVA), which globally optimized replica placement across variants and inference pools, is deprecated.

See Autoscaling for complete details.



Advanced llm-d Arch

InferencePool

The `InferencePool` is the central resource that bridges the gap between the Gateway, the Endpoint Policy Provider (EPP), and the collection of model server instances. It serves as the source of truth for both endpoint discovery and service mesh/gateway integration.

Functional Overview

The `InferencePool` performs two primary roles in the inference infrastructure:

1. **Endpoint Discovery for the EPP:** It defines how the EPP should find and monitor the model server Pods that are eligible to serve requests.
2. **Service Integration for the Gateway:** It provides the necessary metadata for the Gateway controller to locate the EPP and connect it to the proxy as an external processing (`ext-proc`) service.

Architecture and Relations

The following diagram visualizes how the `InferencePool` resource is involved in the control path of both the EPP and Gateway Controller:

![InferencePool](../../assets/gateway-design.svg)

1. Endpoint Discovery (EPP Perspective)

The EPP uses the `InferencePool` to discover which pods it can pick from.

- **Selector-based Discovery:** The `InferencePool` defines a `selector` (label matching). The EPP watches for Pods that match these labels within the same namespace.
- **Dynamic Membership:** As model server Pods are scaled up or down, or as their readiness state changes, the EPP automatically updates its internal list of healthy candidates.
- **Port Mapping:** The `targetPorts` in the `InferencePool` tell the EPP which ports on the discovered Pods are listening for inference traffic (e.g., port 8000 for vLLM).

2. Gateway Integration (Controller Perspective)

When an `InferencePool` is used as a `backendRef` in an `HTTPRoute`, the Gateway controller uses the resource to configure the underlying proxy.

- **EPP Connectivity:** The `endpointPickerRef` (or `extensionRef`) in the `InferencePool` points to the EPP service. The Gateway controller uses this information to configure the proxy's `ext_proc` filter, ensuring that every request directed to the pool is first processed by the EPP.

- **Routing Logic:** The proxy is configured to "park" the request and wait for the EPP's decision. The EPP then instructs the proxy—via the `ext_proc` protocol—on which specific Pod IP from the discovered pool should receive the request.
- **Failure Handling:** The `failureMode` defined in the `InferencePool` (e.g., `FailOpen` or `FailClose`) tells the Gateway controller how to configure the proxy's behavior if the EPP becomes unresponsive.

Key Relationships

- **One-to-One Mapping:** Typically, one `InferencePool` corresponds to one logical deployment of a model (e.g., Gemma4) and is served by one EPP deployment.
- **Decoupled Scaling:** The model servers can scale independently of the EPP. The `InferencePool` ensures the EPP is always aware of the current set of available endpoints.
- **Namespace Scoped:** All discovery and references (Pods, EPP Service, and the `InferencePool` itself) are strictly contained within the same Kubernetes namespace to maintain security and isolation boundaries.

Latency Predictor

The Latency Predictor is the llm-d component behind predicted latency-based scheduling. Instead of scoring pods by coarse utilization signals alone, the EPP asks an online-trained ML model to predict **Time To First Token (TTFT)** and **Time Per Output Token (TPOT)** for each candidate pod, then routes on those predictions — optionally gated by per-request Service Level Objectives (SLOs).

This page is a reference for the component: its design, EPP plugins, ML model, failure modes, and scaling characteristics. For step-by-step adoption — Helm enablement, SLO header usage, verification, troubleshooting — see the Predicted Latency well-lit path. Design rationale and benchmarks are in the blog Predicted Latency-Based Scheduling for LLMs.

Why Predicted Latency?

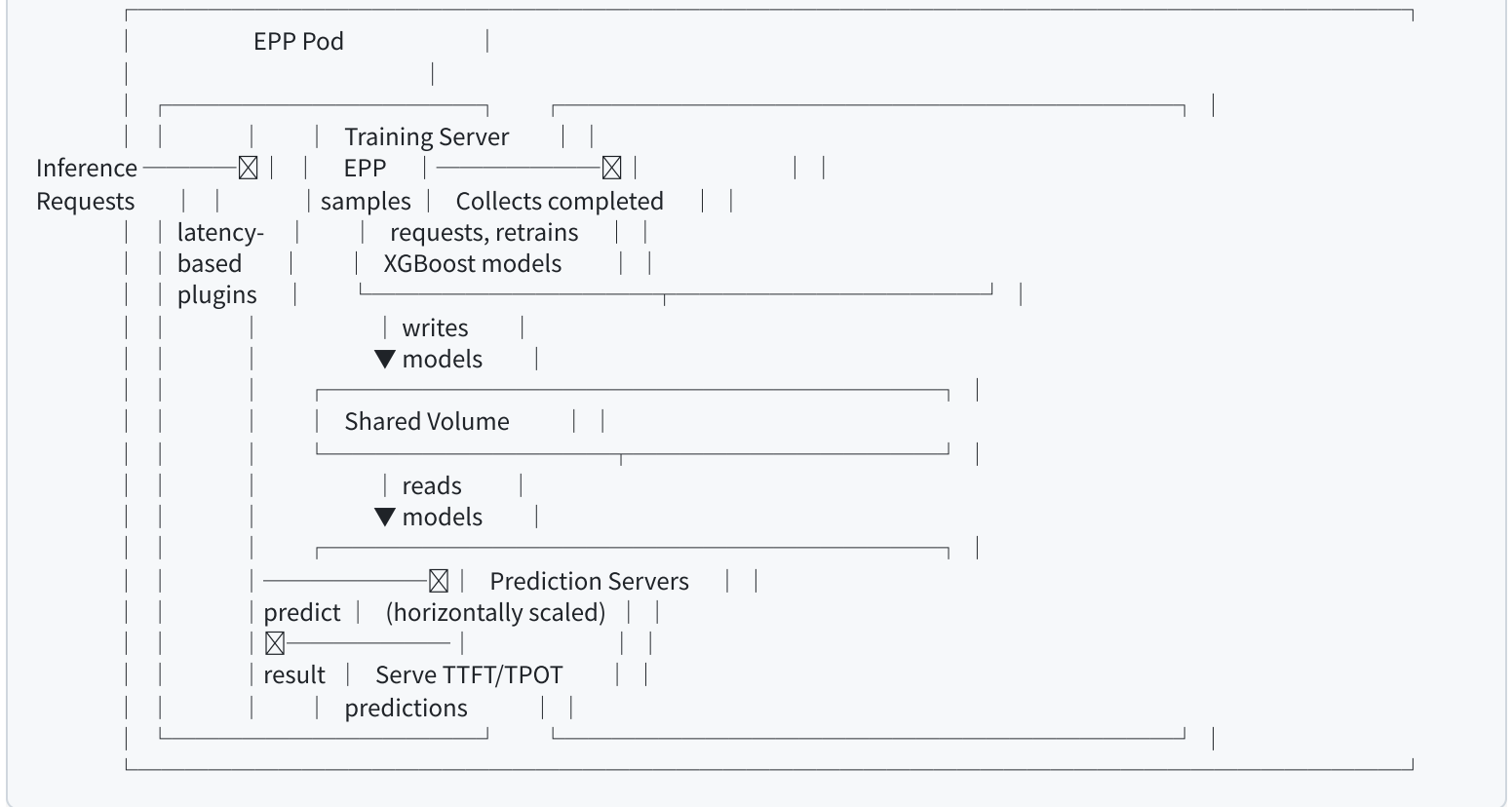
Utilization-based load balancing has structural gaps for LLM workloads:

- **Request costs vary enormously.** A 10-token prompt and a 10,000-token prompt both count as one queued request, but their prefill and decode costs differ by orders of magnitude.
- **Cache reuse is unstable.** Prefix hit rates shift as traffic patterns change, so static weights between cache affinity and utilization drift out of tune.
- **Conflicting objectives.** Optimizing TPOT favors spreading load; optimizing TTFT and cache reuse favors consolidating it. No fixed weight configuration is correct in all regimes.
- **No SLO awareness.** Utilization scorers cannot answer "can this pod serve this request within the caller's latency budget?"

The Latency Predictor closes those gaps by learning the mapping from (pod state, request features) → latency directly from live traffic, and letting the EPP reason about headroom against SLOs instead of hand-tuned weights.

Architecture

The predictor ships as a set of sidecars colocated with the EPP in the same pod. A **training server** continuously retrains on completed requests, and one or more **prediction servers** that read the latest model from a shared volume and answer predictions from the EPP on the hot path.



In the EPP, latency-based scheduling is implemented as a series of composable EPP plugins (more on this later). The **predicted-latency-producer** plugin drives interactions with the predictor. For each request, it calls the predictor to obtain TTFT and TPOT predictions for every candidate endpoint, conditioned on the endpoint's state (KV cache utilization, queue depth, prefix cache match score). After the request is served, the producer sends the observed TTFT and ITL latencies back to the predictor as training samples, so the model is continuously retrained on live traffic.

If the prediction server is unreachable or fails to return a prediction, the latency scorer falls back to a composite score built from KV cache utilization, queue depth, and prefix cache match — so a predictor outage degrades to baseline heuristic routing rather than dropping traffic.

ML Model

The prediction model is an **XGBoost** regression one trained in realtime. Two models are maintained — one for TTFT, one for TPOT — and retrained on a sliding window of completed requests.

Training uses **stratified bucketing** — samples are partitioned by KV cache utilization (10% steps), prefix cache hit rate (0.25 steps), and similar features. Each bucket has its own sample cap, so traffic regimes that are rare in the current window (for example, a cold prefix cache during low load) are not forgotten by the model. Across benchmark runs the models achieve approximately **5% Mean Absolute Percentage Error** on both targets.

The predictor assumes a **homogeneous inference pool** — every pod in the pool must share the same GPU type, model weights, and serving configuration. The features the model is trained on describe pod state (KV cache utilization, queue depth, running requests, prefix hit rate) without encoding pod shape, so predictions across heterogeneous hardware or serving configs would conflate regimes the model treats as identical. Heterogeneous pools are not yet modeled.

TTFT features

Feature	What it captures
KV Cache Usage %	Memory saturation and its effect on prefill scheduling
Input Length	Prefill cost proxy — longer prompts dominate TTFT
Queue Depth	Backlog before scheduling — more waiting requests delay first token
Running Requests	GPU concurrency — contention with in-flight decode
Prefix Cache Match %	KV reuse potential — high match rates collapse prefill work
Input Tokens In Flight	Tokens dispatched but not yet prefilled, plus tokens still resident in KV — captures incoming prefill pressure

TPOT features

Feature	What it captures
KV Cache Usage %	Memory pressure during decode
Input Length	Input token count (affects attention cost)
Queue Depth	Queue contention that leaks into decode batching
Running Requests	Active decode batch size / GPU concurrency
Tokens Generated	Output tokens produced so far

Scalability

Each prediction sidecar sustains roughly 300 QPS of prediction work on a `c4-standard-192` node (~192 vCPUs). Because the EPP makes one prediction per candidate pod, total prediction load scales with `cluster QPS × pod count` . Scale horizontally by adding prediction sidecars and updating the predictor URL list.

Cluster QPS	Avg prediction latency (ms)	p99 prediction latency (ms)	Prediction servers
100	3.5	46	1
1,000	5.0	49	2
5,000	~27	~74	2
7,500	~35	~96	3
10,000	~48	~137	4

Streaming Mode

The `predicted-latency-producer` plugin has two training modes, exposed via a `streamingMode` parameter:

- **`streamingMode: false`** (default) — Trains on end-to-end request latency. TTFT is recorded at response completion (effectively e2e latency); TPOT is not trained. **Use this mode if** your workload mixes streaming and non-streaming responses, or if you only need latency-aware routing without per-request SLO enforcement.
- **`streamingMode: true`** — Trains separate TTFT and TPOT models. TTFT is recorded on the first streamed chunk; TPOT is sampled across subsequent tokens. **Use this mode if** your workload is fully streaming and you need meaningful `x-llm-d-slo-ttft-ms` / `x-llm-d-slo-tpot-ms` enforcement — a mixed workload in this mode will produce incorrect measurements for the non-streamed responses.

Scheduling Strategy

As indicated before, latency-based scheduling is implemented in the EPP as a set of composable plugins. Using the latency predictions obtained by `predicted-latency-producer` plugin, scheduling is done using a sequence of filtering and scoring plugins to determine the optimal placement. When the latency predictor is enabled via the Helm chart, the full sequence is wired up automatically. SLO-specific plugins are no-ops when a request does not include SLO headers, so the same set of plugins handles both SLO and non-SLO annotated traffic.

Filtering Strategy

Two filters narrow the candidate set before scoring.

- **`prefix-cache-affinity-filter`** narrows the candidate set to cache-warm endpoints when any endpoint's prefix cache match score exceeds the affinity threshold (default `0.80`). If no endpoint clears the threshold, the filter is a no-op. The filter implements an epsilon-greedy exploit/explore over cache locality, with a TTFT-based escape hatch:
- **Exploit** (default path). The filter narrows to cache-warm endpoints so downstream scoring concentrates reuse on them.
- **Explore** (small probability). On a configurable fraction of requests the filter bypasses itself entirely, letting traffic land on cache-cold pods so they can seed new entries — this prevents a single pod from permanently owning a hot prefix.
- **TTFT load gate**. Even on the exploit path, if the best cache-warm pod's predicted TTFT is materially worse than the best overall pod's (by more than a configurable penalty), the filter breaks affinity and yields the full candidate set. This stops a hot prefix from piling up behind a saturated pod while cooler pods sit idle.
- **`slo-headroom-tier-filter`** splits endpoints into a **positive** tier (predicted to meet SLO) and a **negative** tier (predicted to violate SLO), with probabilistic exploration of the negative tier so recovering pods still receive traffic. No-op when SLO headers are absent.

Scoring Strategy

Three plugins handle scoring and final selection.

- **`latency-scorer`** scores endpoints. Without SLO headers, lowest predicted latency wins. With SLO headers, the score is derived from headroom (`SLO - predicted`) via the `headroomSelectionStrategy` parameter:

- **least** (default). Bin-pack: prefer the endpoint closest to the SLO boundary — smallest positive headroom if any pod meets the SLO, smallest negative deficit otherwise. Maximizes utilization and keeps less-loaded pods free for bursty arrivals.
- **most** . Spread: prefer the endpoint with the most positive headroom. More conservative, leaves slack for unexpected spikes. For negative headroom, **least** is always used regardless of this setting.
- **latency-slo-admitter** rejects *sheddable* requests (priority < 0) when no endpoint can meet the SLO, rather than wasting capacity on a guaranteed miss. No-op when SLO headers are absent.
- **weighted-random-picker** selects an endpoint via weighted random selection over the scores. This spreads load while still favoring better-scoring endpoints, and avoids the "everyone piles onto the current best pod" failure mode of pure arg-max selection.

Observability

When the latency predictor is enabled, the EPP exposes Prometheus metrics for actual vs. predicted latency, prediction duration, and SLO violation tracking. The primary series are:

Metric	Description
llm_d_epp_request_ttft_seconds	Actual TTFT distribution.
llm_d_epp_request_predicted_ttft_seconds	Predicted TTFT distribution.
llm_d_epp_request_ttft_prediction_duration_seconds	Time spent generating TTFT predictions.
llm_d_epp_request_streaming_tpot_seconds	Actual TPOT distribution (streaming).
llm_d_epp_request_predicted_tpot_seconds	Predicted TPOT distribution.
llm_d_epp_request_tpot_prediction_duration_seconds	Time spent generating TPOT predictions.
llm_d_epp_request_slo_violation_total	Counter of SLO violations (with label <code>type="ttft"</code> or <code>type="tpot"</code>).
llm_d_epp_inference_request_metric	Consolidated gauge for request metrics including TTFT, TPOT, SLOs, and prediction durations.

All latency and prediction-duration series are Prometheus **histograms**, so dashboards should query them via `histogram_quantile` (and the counters via `rate`) rather than reading instantaneous values. Pairing actuals with predictions lets operators validate predictor accuracy in-situ; SLO violation counters are the primary signal for alerting on SLO breaches.

Source

- **EPP plugins** (`predicted-latency-producer` , `prefix-cache-affinity-filter` , `latency-scorer` , `weighted-random-picker` , `slo-headroom-tier-filter` , `latency-slo-admitter`) live in `llm-d Router`. Per-plugin configuration references live alongside each plugin in that repo.

- **Training and prediction server code** (the Python ML sidecars, XGBoost models, stratified sampler) lives in `llm-d/llm-d-latency-predictor`.

Further Reading

- Predicted Latency Well-Lit Path — how to adopt this path: Helm enablement, request headers, verification, troubleshooting.
- Predicted Latency-Based Scheduling for LLMs — design rationale and benchmark results.
- EPP Scheduling — how the plugins fits into EPP request handling.

Model Servers

The model server is the component that runs inference on a model. llm-d supports vLLM, SGLang, and TensorRT-LLM (`trtllm-serve`) as model server backends.

Functionality

A model server loads a model onto one or more accelerators (GPUs, TPUs, etc.) and exposes a supported API, such as OpenAI-compatible API, for inference requests. In the llm-d architecture, model servers are the compute layer -- they execute the actual prefill and decode steps that generate tokens.

Model servers are the lowest layer in the llm-d stack:

![[Architecture](../assets/basic-architecture.svg)]

Model servers are deployed independently from the rest of the llm-d stack. They join an `InferencePool` automatically via Kubernetes label selectors, and the EPP begins routing traffic to them once they are healthy.

Key responsibilities:

- **Serve inference requests** via an OpenAI-compatible API (`/v1/completions` , `/v1/chat/completions`)
- **Expose metrics** (KV-cache utilization, queue depth, active requests) that the EPP uses for intelligent scheduling
- **Manage KV-cache** on GPU memory, including prefix caching for repeated prompt prefixes
- **Support parallelism strategies** such as Tensor Parallelism (TP), Data Parallelism (DP), and Expert Parallelism (EP) for large models

EPP <-> Model Server Protocol

This is the protocol between the EPP and the model servers.

Metrics Reporting

By default, the EPP is configured to scrape metrics from the model servers to make optimal request scheduling decisions. In this mode of operation, the model servers **MUST** provide the following metrics via a Prometheus endpoint. The exact metric names don't necessarily need to be the same as the recommended names here, however the metric types and semantics **MUST** follow this doc.

Metric	Type	Description	vLLM metric	Triton Ten:
TotalQueuedRequests	Gauge	The current total number of requests in the queue.	vllm:num_requests_waiting	nv_trt_llm_request_metrics{request
TotalRunningRequests	Gauge	The current total number of requests actively being served on the model server.	vllm:num_requests_running	nv_trt_llm_request_metrics{request
KVCacheUtilization	Gauge	The current KV cache utilization in percentage.	vllm:kv_cache_usage_perc	nv_trt_llm_kv_cache_block_metrics
[Optional] BlockSize	Labeled/Gauge	The block size in tokens to allocate memory, used by the prefix cache scorer. If this metric is not available, the BlockSize will be derived from the prefix plugin config.	name: vllm:cache_config_info , label name: block_size	nv_trt_llm_kv_cache_block_metrics
[Optional] NumGPUBlocks	Labeled/Gauge	The total number of blocks in the HBM KV cache, used by the prefix cache scorer. If this metric is not available, the NumGPUBlocks will be derived from the prefix plugin config.	name: vllm:cache_config_info , label name: num_gpu_blocks	nv_trt_llm_kv_cache_block_metrics



To correctly map metrics names, model server Pods should be labeled with the model server type they are running as demonstrated below. Pods without the engine-type label will default to vLLM metrics names.

metadata:

labels:

llm-d.ai/engine-type: vllm # other options: sglang, trtllm-serve, triton-tensorrt-llm

[!NOTE] **TensorRT-LLM (trtllm-serve) requirements.** Unlike vLLM/SGLang, trtllm-serve exposes the metrics above at `/prometheus/metrics`. The plain `/metrics` route returns JSON iteration-stats the EPP cannot parse, so point the EPP's metrics data source at path: `/prometheus/metrics`. The gauges are emitted only when the server is started with **both** `return_perf_metrics: true` **and** `enable_iter_perf_stats: true` (both default `false`, passed via `--extra_llm_api_options`). The first mounts the Prometheus endpoint, and the second starts the iteration-stats loop that populates the dynamic gauges (`trtllm_num_requests_waiting`, `trtllm_num_requests_running`, `trtllm_kv_cache_utilization`). They require **TensorRT-LLM v1.3.0rc12 or newer** (added in PR #12545). Earlier releases (including 1.2.1 GA) expose only request-lifecycle histograms. See the **optimized-baseline TensorRT-LLM recipe** for a working configuration.

LoRA Adapter Serving

Model servers that support dynamic LoRA serving can benefit from the LoRA affinity algorithm. Note the current algorithm in the reference EPP is highly biased towards vLLM's current dynamic LoRA implementation.

The model servers **MUST** support serving a LoRA adapter specified in the `model` argument of the request, provided the requested adapter is valid.

The model server **MUST** expose the following LoRA adapter metrics via the same Prometheus endpoint:

- Metric name implemented in vLLM: `vllm:lora_requests_info`
- Metric type: Gauge
- Metric value: The last updated timestamp (so the EPP can find the latest).
- Metric labels:
 - `max_lora`: The maximum number of adapters that can be loaded to GPU memory to serve a batch. Requests will be queued if the model server has reached `MaxActiveAdapter` and cannot load the requested adapter. Example: `"max_lora": "8"`.
 - `running_lora_adapters`: A comma separated list of adapters that are currently loaded in GPU memory and ready to serve requests. Example: `"running_lora_adapters": "adapter1, adapter2"`
 - `waiting_lora_adapters`: A comma separated list of adapters that are waiting to be served. Example: `"waiting_lora_adapters": "adapter1, adapter2"`

Prefix Cache Reuse

The EPP supports prefix cache optimized request scheduling. To benefit from the optimal prefix aware request scheduling, model servers **SHOULD** support prefix cache reuse, such as the vllm automatic prefix caching feature.

Health Checks

Model servers are expected to expose health endpoints that Kubernetes uses for liveness and readiness probes:

- **Liveness:** `GET /health` -- confirms the server process is alive
- **Readiness:** `GET /health` -- confirms the server is ready to accept requests

Further Reading

- [vLLM Documentation](#)
- [SGLang Documentation](#)
- [TensorRT-LLM Documentation](#)
- [InferencePool](#) -- how model servers are discovered and managed
- [EPP](#) -- how the router routes requests to model servers informed by model servers metrics

Autoscaling

With autoscaling, model servers are added or removed automatically to keep serving capacity aligned with inference demand. llm-d scales on signals emitted by the Endpoint Picker (EPP) — queued requests, active concurrency, token backlog, or estimated latency against an SLO — because accelerator utilization is an unreliable proxy for LLM load: GPU utilization can sit near 100% during active batching whether the server is lightly loaded or saturated.

KEDA + EPP Metrics (Recommended)

KEDA's Prometheus scaler reads EPP metrics, evaluates them into a scaler value, and creates and owns the HPA that scales model server replicas. There is no custom controller and no Prometheus Adapter in the loop.

See KEDA with EPP Metrics for the pipeline, the metric isolation rules, ownership semantics, scale-to-zero, and limitations.

Four signals share that pipeline, each with its own deployable guide:

Signal	Scales on	Guide
Queue depth	Absolute per-replica queued-request and running-request targets	keda-epp-queue
Pool saturation	A normalized saturation ratio (0.0–1.0+), more portable across hardware	keda-epp-saturation
Token backlog	Seconds of prefill queue wait plus KV cache occupancy — for widely varying prompt sizes	keda-epp-token-aware
Estimated latency	Predicted (or measured) TTFT/TPOT against latency SLOs	slo-aware

The latency-driven variant is the one design worth reading separately: it collapses its triggers into a single formula whose output is the desired replica count. See SLO-Aware Autoscaling with KEDA — the control law for the derivation, and the SLO-aware autoscaling guide for the deployable setup.

Workload Variant Autoscaler (Deprecated)

[!WARNING] WVA is deprecated. Use KEDA + EPP for new deployments.

WVA was a global optimizer and controller that, given an inventory of available accelerators, decided how to place model servers — potentially serving different base models, on heterogeneous hardware, in disaggregated prefill/decode roles — onto those accelerators, publishing a `wva_desired_replicas` metric for an HPA to consume. It accounted for cost across variants, fair-shared scarce accelerators, and incorporated pending pods into its decisions.

See Workload Variant Autoscaler (WVA) for the retained design reference and Migrating off WVA for the move to KEDA + EPP.

What Changed with the Deprecation

KEDA + EPP does not replace WVA feature for feature. Cost-aware placement across variants has no current equivalent; the rest is covered, though in some cases by a different layer than the autoscaler:

Capability	WVA (deprecated)	KEDA + EPP
Multiple variants of one model	Optimally placed across models and topologies to minimize cost	Each Deployment scales independently; no cost-aware preference between variants
Limited accelerators	Fair-share allocation across pools	Handled below the autoscaler by Kueue — per-model quota floors in a shared cohort, with borrowing and preemption
Pending pods awareness	Incorporated pending (unscheduled) pods into decisions	Available — a trigger can read replica counts from kube-state-metrics; the slo-aware path discounts its ask by <code>readyReplicas/replicas</code>
Strong latency SLOs	Learned supply/demand dynamics (experimental)	Scales directly on estimated latency vs. SLO (slo-aware)
Scale to zero	Supported	Supported (KEDA, without the <code>HPAScaleToZero</code> feature gate)
Operational complexity	Medium — requires the WVA controller (plus a metrics adapter)	Low — KEDA and Prometheus

If a deployment depends on cost-aware placement across variants or on fair sharing of a scarce accelerator pool, stay on WVA for now and track the workload autoscaling guides for a replacement.

KEDA with EPP Metrics

The Endpoint Picker (EPP) and KEDA integration scales model server replicas using demand-side signals from EPP. Rather than relying on coarse resource utilization, KEDA queries EPP metrics from Prometheus, exposes evaluated scaler values through its metrics server, and creates an HPA that consumes those external metrics. This approach is well-suited for homogeneous deployments where each target model-server pool can be isolated by metrics and scaled independently.

This is the recommended autoscaling path in llm-d. The WVA path is deprecated.

Functionality

The KEDA+EPP integration provides:

- **Demand-driven scaling:** Replicas are added or removed based on EPP signals — queue depth, active request count, pool saturation, in-flight tokens, or estimated latency — which reflect inference demand rather than accelerator utilization.
- **Standard Kubernetes actuation:** KEDA owns an `autoscaling/v2` HPA, so HPA status and behavior remain available through standard Kubernetes APIs.
- **Optional scale-to-zero:** KEDA can remove all model server replicas when traffic is idle and reactivate the Deployment from an EPP Flow Control queue.
- **No custom controller and no Prometheus Adapter:** the whole loop is KEDA, Prometheus, and a `ScaledObject`.

Design

The LLM Autoscaling Problem

During active token generation, GPU utilization can remain high whether a model server is lightly loaded or saturated. A utilization-based autoscaler cannot reliably distinguish spare serving capacity from an overloaded batch.

EPP Flow Control moves excess demand to the gateway, where it can be measured before end-user latency degrades. EPP exposes that demand as metrics that map directly onto scaling decisions — reactive saturation signals (queued requests), proactive concurrency signals (running requests, in-flight tokens), and objective-relative signals (pool saturation, estimated latency against an SLO).

Architecture



KEDA + EPP Architecture

Scaling Pipeline

The scaling pipeline is:

1. **Metric emission:** EPP exposes its metrics (for example `llm_d_epp_flow_control_queue_size` and `llm_d_epp_request_running`) on its metrics endpoint.
2. **Metric collection:** Prometheus scrapes EPP through the router's `ServiceMonitor`.
3. **Metric evaluation:** KEDA's Prometheus scaler evaluates one scalar or single-element PromQL result per trigger. Optionally, `spec.advanced.scalingModifiers` combines several triggers into one formula.
4. **Metric publication:** KEDA exposes evaluated scaler values through its metrics server to the Kubernetes External Metrics API.
5. **Scaling decision:** The HPA generated by KEDA computes the desired replica count and reconciles the target Deployment.

The user creates a `ScaledObject`, not a second HPA. Multiple HPAs targeting the same Deployment conflict with one another.

Prometheus Adapter is not part of this pipeline because the KEDA Prometheus scaler queries Prometheus directly. The Prometheus Adapter path is deprecated.

Scaling Signals

The pipeline above is signal-agnostic: what changes between the deployable paths is which EPP metrics the triggers read and how thresholds are derived.

Signal	EPP metrics	Threshold is	Guide
Queue depth	<code>llm_d_epp_flow_control_queue_size</code> , <code>llm_d_epp_request_running</code>	An absolute per-replica target, tuned per deployment	<code>keda-epp-queue</code>
Pool saturation	<code>llm_d_epp_flow_control_pool_saturation</code> , <code>llm_d_epp_request_running</code>	A normalized ratio (0.0–1.0+), more portable across hardware	<code>keda-epp-saturation</code>
Token backlog	<code>llm_d_epp_inflight_tokens</code> , per-pod KV cache occupancy	Seconds of prefill queue wait, derived from a share of the TTFT SLO and a calibrated <code>peakPrefillThroughput</code>	<code>keda-epp-token-aware</code>
Estimated latency	EPP predicted/actual TTFT and TPOT histograms	Latency ÷ SLO, with a hysteresis band	<code>slo-aware</code>

Queue depth and pool saturation are lagging-to-leading counts of unmet demand. Token backlog exists because request counts rate an 8192-token prompt the same as a 512-token one, while the prefill work differs by $16\times$. Estimated latency scales against the objective itself rather than a proxy for it; its control law is derived in SLO-Aware Autoscaling with KEDA.

Dual-Metric Strategy

Most of these paths configure two triggers rather than one, so that unmet demand and active concurrency are both represented. The queue-based path is the canonical example:

Metric	Signal	KEDA metric type	Interpretation
EPP Flow Control queue size	Unfulfilled demand	AverageValue	Target aggregate queued requests per model server replica
Running requests	Active concurrency	AverageValue	Target in-flight requests per model server replica

Each `AverageValue` threshold is a per-replica target. For each metric, the HPA calculates a desired count from the aggregate value and target; roughly, the aggregate must exceed $\text{threshold} \times \text{current replicas}$ to scale beyond the current count. When all configured metrics are available, Kubernetes uses the largest desired replica count.

The SLO-aware path is the exception: it collapses its triggers into a single `scalingModifiers` formula whose output *is* the desired replica count, against a pass-through target of `1`. Use that shape when the triggers must be combined arithmetically rather than raced against each other.

Pending and Warming Pods

A trigger is a PromQL query, so it is not restricted to EPP series: a `ScaledObject` can also read the target's replica counts from kube-state-metrics and reason about capacity that is provisioned but not yet serving — pods that are pending, scheduling, or still loading the model.

The SLO-aware path does this. It reads `kube_deployment_status_replicas` (provisioned, `n`) alongside `kube_deployment_status_replicas_ready` (ready, `r`), and scales its demand by an in-flight credit `r/n` on the scale-up branch. Because the latency signal is measured on ready pods only, those pods over-report the post-warmup load while a scale-up is in flight; the credit makes each ask cover only the deficit beyond the pods already on their way, instead of racing to `maxReplicas` while nothing has become Ready. See SLO-Aware Autoscaling with KEDA for the derivation.

The simpler per-metric `AverageValue` paths do not carry this term. There, the HPA's own stabilization windows and scale-up policies are what keep a slow-to-schedule pool from over-asking.

Sharing a Contended Accelerator Budget

Each `ScaledObject` scales its own Deployment against its own signal, so two pools sharing one GPU budget can both scale up and collectively ask for more accelerators than exist. This is not resolved in the autoscaling loop, and it does not need to be: admission is a scheduling concern, and it is handled one layer down by Kueue.

Every replica pod becomes a Kueue `Workload`, held at a scheduling gate until quota is free for it. Each model gets a guaranteed floor of GPUs in a shared cohort, lends what it is not using to the others, and preempts to take its floor back when its own demand returns. The HPA is untouched and unannotated, so KEDA keeps sole ownership of the HPA it generates and the autoscaling configuration above is unchanged: over-budget replicas simply stay Pending until quota frees.

See the Kueue-based replica rebalancing guide. It supersedes the experimental replica rebalancer, which enforced the same budget from *above* the HPA by patching `maxReplicas` on annotated HPAs — an approach that requires HPA annotations KEDA does not propagate. The two must never run together.

Metric Isolation

Each PromQL query aggregates to one value and must isolate the EPP/InferencePool associated with the target Deployment. Label values such as `namespace`, `model_name`, and `inference_pool` depend on the deployment and scrape configuration. The running-request metric does not expose `inference_pool`. Because multiple pools can serve the same model, selectors may also need EPP scrape labels such as `service`. Verify isolation against live Prometheus series.

Ownership and Scaling Behavior

KEDA owns the generated HPA for the lifetime of the `ScaledObject`:

- KEDA evaluates trigger activation and handles zero-to-one activation.
- The generated HPA handles scaling between one and N replicas.
- `cooldownPeriod` controls scale-down to zero; it does not control ordinary HPA scale-down while the minimum replica count is one or greater.
- `ScaledObject.spec.advanced.horizontalPodAutoscalerConfig.behavior` is copied to the generated HPA and controls stabilization and scaling policies.
- Deleting the `ScaledObject` removes the generated HPA but does not delete the target Deployment.

Scale to Zero

EPP Flow Control enables scale-from-zero by buffering requests while no model server endpoint is ready. A queued request raises the queue-size metric, KEDA activates the target Deployment, and EPP dispatches the request after a model server becomes Ready.

Scale-to-zero should be enabled only after one-to-N scaling is validated. Model cold-start time becomes request latency, and clients must allow enough time for the model to load.

Limitations

- **No cross-variant placement.** Each `ScaledObject` scales one Deployment against its own metrics. Pools serving the same model on different hardware scale independently, with no cost-aware preference between them.
- **No cost-aware choice of hardware.** The autoscaler asks for more of the variant it is pointed at; it will not decide that demand is better served by a cheaper accelerator. Sharing a contended accelerator budget between pools *is* solved, but below the autoscaler — see [Sharing a Contended Accelerator Budget](#).
- **Warmup is paid at every scale-up.** Model load and torch-compile time sit between the decision and the capacity. Cutting pod-ready time is usually the highest-leverage tuning available; see the warmup patch discussed in [SLO-Aware Autoscaling with KEDA](#).

Deployment Guides

Start from the workload autoscaling guides, which cover the reusable router values, `ScaledObject`s, authentication notes, and verification steps for each of the signals in the table above.

Batch Inference

llm-d supports batch and offline inference workloads through two components that can be deployed independently or together:

- **Batch Gateway** — an OpenAI-compatible Batch API (`/v1/batches` , `/v1/files`) for submitting, tracking, and managing batch inference jobs. Includes an API server and a processor that dispatches individual inference requests to the llm-d Router.
- **Async Processor** — a lightweight dispatch agent that pulls individual inference requests from message queues (such as Redis and Google Pub/Sub) and sends them to the llm-d Router, adjusting the flow based on system metrics.

When deployed together, the Batch Gateway hands off individual requests to the Async Processor instead of dispatching directly, gaining metric-based flow control.

Batch Gateway Architecture

Batch Gateway adds OpenAI-compatible batch inference processing to the llm-d stack. It sits between batch API clients and the llm-d Router, managing the lifecycle of batch jobs — from job creation through request dispatching to result collection.

![Batch Gateway Architecture](../../assets/batch-gateway.svg)

API Server

The API server exposes OpenAI-compatible REST endpoints:

- `POST /v1/files` — Upload batch input files (JSONL format).
- `GET /v1/files` — List files.
- `GET /v1/files/{id}` — Retrieve file metadata.
- `GET /v1/files/{id}/content` — Download file content (input or output files).
- `DELETE /v1/files/{id}` — Delete a file.
- `POST /v1/batches` — Create a batch job referencing an uploaded input file.
- `GET /v1/batches/{id}` — Get job status, progress counts, and output file IDs.
- `POST /v1/batches/{id}/cancel` — Cancel a running job.
- `GET /v1/batches` — List batch jobs.

The API server validates input files format constraints, stores metadata in a database (typically PostgreSQL), enqueues jobs into a priority queue (typically in Redis), and stores files in S3 or a filesystem.

Batch Processor

The processor is the core execution engine. It:

1. **Polls** the priority queue for the next job.
2. **Ingests** the input file — parses model IDs and system prompts, groups requests by model, and builds per-model execution plans.
3. **Executes** plans concurrently — launches per-model goroutines, sends individual inference requests to the llm-d Router with concurrency limits, and writes results to an output file.
4. **Updates** the job status during processing, and listens to job events, such as cancellation.
5. **Finalizes** — uploads the output file and updates the job status.

Concurrency Control

The processor uses two-level concurrency control to prevent overloading the llm-d Router:

- **Global concurrency** — caps total in-flight requests across all models.
- **Per-model concurrency** — caps in-flight requests to a single model.

Crash Recovery

On startup, the processor scans for jobs that were `in_progress` when a previous instance crashed. If a partial output file exists, the processor uploads it and marks the job as failed. Otherwise, the job is re-enqueued for a full retry. Recovery concurrency is capped to avoid overwhelming the system during restart storms.

Storage Layer

All Batch Gateway components share a pluggable storage layer:

Function	Options	Notes
Jobs and files metadata	PostgreSQL, Redis, Valkey	PostgreSQL for production; Redis and Valkey for development/test only
Priority queue	Redis, Valkey	Sorted set with SLO-based priority
Event channels	Redis, Valkey	Pub/Sub for job cancellation and future supported events
Status updates	Redis, Valkey	Job processing progress tracking
File storage	S3, Filesystem	S3 for multi-replica deployments; filesystem (PVC) for single-node

Garbage Collector

The GC runs on a configurable interval and removes:

- Expired batch jobs (past their `completion_window`).
- Expired files (past their configured expiration).

It limits concurrent deletions to avoid database pressure.

Authentication and Multi-Tenancy

Batch Gateway delegates authentication and authorization to the upstream gateway infrastructure:

- The API server extracts a tenant identifier from an HTTP header (configurable).
- All data queries are filtered by tenant ID. Cross-tenant access is rejected.
- The processor forwards configurable pass-through headers with each inference request, so the llm-d Router can authenticate and authorize the end user per-request.
- File paths are hashed by tenant ID to prevent enumeration.

The batch route authenticates (verifying identity), while the inference route authorizes (verifying model access). This separation ensures that batch job creation doesn't require per-model permissions — authorization is enforced at inference time.

Observability

- **Prometheus metrics:** request counts, job processing times, queue wait duration, worker utilization, per-model concurrency, and token counts.
- **OpenTelemetry tracing:** distributed traces across API server, processor, database, and llm-d Router calls.
- **Grafana dashboards:** pre-built dashboards for API server and processor metrics, included in the Helm chart.
- **Prometheus alerting rules:** configurable alerts for high queue wait time, job failure rate, expired jobs, and worker saturation.

Related

- [Batch Gateway Deployment Guide](#) — deployment instructions, configuration options, and usage.
- [Batch Gateway Repository](#) — source code, Helm chart, deployment and usage guides.
- [Batch Gateway Design Documents](#) — detailed design documents.
- [Async Processor](#) — composes with Batch Gateway for gated request dispatching.

KV Cache Management

Key-Value (KV) cache management is the foundation of high-performance LLM serving in llm-d. By efficiently tracking, preserving, and reusing the KV cache—the intermediate state generated during LLM inference—llm-d significantly reduces latency and increases the overall throughput of the inference pool.

The KV cache management ecosystem in llm-d consists of three core architectural pillars. By composing these layers, llm-d allows an inference pool to scale its "effective cache capacity" far beyond physical HBM limits, sustaining high hit rates even under heavy, diverse workloads. The three pillars are:

Prefix-Cache Aware Routing

The "intelligence" layer managed by the **llm-d Router** (via its **EPP** component) that uses the index to determine the optimal model server Pod for each incoming request. It aims to maximize "cache hits" by routing requests to replicas that already contain the relevant KV cache for the request's prompt prefix.

See **Prefix-Cache Aware Routing** for a deep dive into the two different routing implementations that the llm-d Router offers: the Approximate (heuristic-based) and the Precise (event-driven) implementations.

KV-Cache Indexing

The "observability" layer that continuously monitors the state of the pool, knowing exactly what is cached and where (including offloaded tiers and across all active model servers). It consumes high-frequency events from engines like vLLM to track the movement and eviction of individual token blocks.

See **KV-Cache Indexer** for a deep dive into how the indexer processes `KVEvents` and provides the source-of-truth for precise routing decisions.

KV Offloading

The "capacity" layer that extends the cache beyond the limited high-bandwidth memory (HBM) of accelerators (GPUs/TPUs). It enables model servers to "spill" or offload cache entries to CPU memory or local SSDs, effectively creating a tiered storage hierarchy for the KV cache.

See **KV Offloader** for a deep dive into the design of the multi-tier storage API and its integration with the inference engine.

P2P Sharing

The "sharing" layer that composes the three pillars into a fleet-wide cache: the index knows which peer holds a request's prefix, the router stamps the request with that source, and the model server pulls the blocks directly from the peer's CPU offload tier over NIXL instead of recomputing them. The source pod's GPU is never touched, so serving a pull costs it no prefill capacity, and an ordinary lookup miss or partial match degrades to a normal recompute rather than failing the request (a promised-but-undelivered block is a documented limitation - see the guide).

See [Enable P2P Prefix Cache Sharing](#) for the mechanism and the when-to-use rule, and the **[P2P KV Cache Sharing guide](#)** for deployment and benchmarks.

Prefix-Cache Aware Routing

Prefix-cache aware routing is a core technique managed by the **llm-d Router** (specifically via its **Endpoint Picker (EPP)** component) to reduce tail latency and increase throughput. By routing requests to model server replicas that already contain the relevant Key-Value (KV) cache for a prompt's prefix, the system avoids redundant "prefill" computation, saving both time and accelerator (GPU/TPU) resources. This technique expects the underlying model servers to support KV-caching across requests, such as vLLM's Automatic Prefix Caching feature.

llm-d provides two distinct implementations of this capability, catering to different operational requirements and precision needs.

1. Approximate Implementation

The approximate implementation is designed to be lightweight and requires no external dependencies beyond the standard EPP deployment.

Components

- **approx-prefix-cache-producer** (DataProducer plugin)
- **prefix-cache-scorer** (Scorer plugin)

How it Works

1. **Approximation:** Since the EPP does not natively contain a tokenizer, it approximates tokens using character-to-token ratios.
2. **Hashing:** The **approx-prefix-cache-producer** splits the incoming prompt into fixed-size blocks (e.g., 16 tokens approximated as characters) and builds a rolling hash chain.
3. **Local Index:** The EPP maintains an in-memory LRU index of which prefix hashes were recently sent to which Pods.
4. **Scoring:** The **prefix-cache-scorer** reads the match information and assigns a score based on the ratio of matched blocks to total prompt blocks.
5. **Learning:** After a routing decision is made, the EPP updates its local index, assuming the selected Pod will now host that prefix.

Pros & Cons

- **Pros:** Extremely lightweight; no need for a tokenizer sidecar; no network connectivity required to model servers (ZMQ); does not require explicit model server integration as it doesn't expect the model servers to communicate KV-cache events.

- **Cons:** Can diverge from actual model server state (e.g., if a Pod evicts a prefix due to memory pressure); less precise than token-based matching.
-

2. Precise Implementation

The precise implementation provides 100% accuracy by leveraging actual token data and real-time state updates from the model servers.

Components

- **token-producer** (DataProducer plugin)
- **precise-prefix-cache-producer** (DataProducer plugin) — owns the KV-block index
- **prefix-cache-scorer** (Scorer plugin) — scores using the producer's match info via `prefixMatchInfoProducerName: precise-prefix-cache-producer`
- **KV-Cache Indexer** (EPP Data Layer component)

How it Works

1. **Exact Tokenization:** The `token-producer` plugin sends the prompt to vLLM's HTTP render endpoint (`/v1/completions/render`) — typically a `vllm launch render` sidecar in the EPP pod (loopback) or a shared render Service — to get exact Token IDs. (The legacy gRPC-over-UDS tokenizer backend is deprecated.)
2. **Real-time Events:** Model servers (like vLLM) are configured to emit `KVEvents` over ZeroMQ (ZMQ) whenever their internal KV cache changes (blocks added or evicted).
3. **Global Index:** The **KV-Cache Indexer** subscribes to these events and maintains a precise, globally consistent view of exactly which token blocks reside on which Pods.
4. **Precise Matching:** The `prefix-cache-scorer`, reading the `precise-prefix-cache-producer`'s match info, scores each candidate pod by how much of the exact Token-ID prefix is resident in this global index.
5. **Speculative Indexing:** To close the "blind spot" between a routing decision and the arrival of the subsequent `KVEvent`, the producer can proactively add "speculative" entries to the index immediately after routing.

Pros & Cons

- **Pros:** 100% precision; handles complex cache eviction policies; natively supports Prefill/Decode disaggregation (by identifying specific blocks for transfer).
 - **Cons:** Requires additional infrastructure (vLLM render endpoint, ZMQ connectivity); slightly higher resource overhead; requires model server support for emitting KV-cache events.
-

Comparison Summary

Feature	Approximate	Precise
Precision	Heuristic (Character-based)	100% (Token-based)
State Source	Local EPP assumptions	Real-time KVEvents from Model Servers
Dependencies	None	vLLM render endpoint, ZMQ
Use Case	Simple, homogeneous workloads	Complex, high-scale production serving
P/D Disagg Support	Basic	Advanced/Native

Composition with KV Cache Management

Both implementations are part of the broader **KV Cache Management** ecosystem in llm-d. While the Approximate implementation is self-contained, the Precise implementation relies on the KV-Cache Indexer and can work in tandem with KV Offloading to manage cache state across accelerator and host memory boundaries.

llm-d Router

The **llm-d Router** is the intelligent entry point for inference requests in the llm-d stack. It provides sophisticated, LLM-aware load balancing, request queuing, and policy enforcement without reimplementing a full-featured network proxy.

Composition

The llm-d Router is composed of two primary functional parts:

Proxy

Any conformant industry-grade L7 proxy. The proxy handles the data plane, including connection management, TLS termination, and request forwarding. Supported implementations range from self-managed application load balancers (e.g., Istio, agentgateway, Envoy AI Gateway) to compliant cloud-managed services, such as Google Cloud's Application Load Balancer.

See the **Proxy deep dive** to learn about deployment modes (Standalone vs. Gateway Mode), request flow, and Gateway API integration.

llm-d Endpoint Picker (EPP)

A specialized service that the proxy consults for every request. The EPP contains the routing "intelligence," using real-time signals from model servers to make optimal placement decisions.

See the **llm-d EPP deep dive** to learn about the the routing engine's architecture, plugin pipeline (Filters, Scorers, Pickers), and flow control mechanisms.

How it Works

When an inference request arrives at the Proxy, the Proxy "parks" the request and initiates a callback to the EPP via the `ext-proc` (External Processing) protocol.

The EPP evaluates the request against the current state of the InferencePool—considering factors like KV-cache locality, current load, and priority—and returns the address of the optimal model server pod back to the Proxy. The Proxy then forwards the original request to that specific destination.

This decoupled architecture allows llm-d to leverage the performance and reliability of production-grade proxies while providing a highly extensible framework for LLM-specific routing logic.

Terminology

To ensure clarity across the project, we use the following standard terminology:

- **llm-d Router:** The complete intelligent entry point, comprising both the **Proxy** and the **Endpoint Picker (EPP)**. This term replaces "Router" in all contexts.
- **llm-d Endpoint Picker (EPP):** The specific component that implements the routing intelligence and scoring logic. Use this term when referring to capabilities or configurations specific to the EPP itself, rather than the request routing system as a whole.
- **Inference Gateway:** A synonym for the **llm-d Router** when operating in **Gateway Mode**.
- **Request Scheduler:** A sub-component within the EPP that acts as the core decision-making engine for intelligent request routing by filtering and scoring candidate model servers (aka endpoints).

Proxy

The proxy is the entry point for inference requests in llm-d Router, receiving client traffic and routing it to the optimal model server via the EPP. Supported implementations range from self-managed application load balancers (e.g., Istio, agentgateway, Envoy AI Gateway) to compliant cloud-managed services, such as Google Cloud's Application Load Balancer. It supports two primary deployment modes: **Standalone Mode**, where a self-managed proxy runs alongside the EPP in the same pod, and **Gateway Mode**, which integrates with L7 load balancers via the Kubernetes Gateway API.

Functionality

llm-d leverages External Processing to extend production-grade proxies with the "LLM inference-aware" request routing implemented in the llm-d EPP. In this way, llm-d re-uses the rich existing ecosystem of high-performance, production-quality proxy technologies in the Kubernetes ecosystem.

The proxy's job is to:

- **Accept incoming inference requests** from clients (OpenAI-compatible API)
- **Consult the EPP** via ext-proc to determine the optimal backend endpoint
- **Route the request** to the selected model server pod
- **Stream responses** back to the client

Request Flow (Both Modes)

Regardless of the deployment mode, the request flow is the same:

1. Client sends an inference request to the proxy
2. The proxy's ext-proc filter calls the EPP
3. The EPP evaluates endpoints using its plugin pipeline (handlers, filters, scorers, picker)
4. The EPP returns the selected endpoint address
5. The proxy forwards the request to that model server pod
6. The model server sends the response back to the proxy (which streams the result through the EPP for post-processing)

Standalone Mode

The standalone mode deploys a proxy as a sidecar to the EPP, offering a lightweight, flexible deployment mode without requiring Gateway API infrastructure.

When to Use

Standalone deployments are intended for workloads where the machinery of Gateway API creates too much operational overhead. This includes:

- Clusters using legacy Ingress controllers.
- Basic testing and local evaluations.
- Batch inference workloads.
- RL post-training pipelines.

Architecture

In standalone mode, the conformant proxy runs alongside the EPP in the same pod.

- **Communication:** `ext-proc` communication happens over `localhost`.
- **Simplification:** No `Gateway`, `HTTPRoute`, or gateway controller is needed.
- **Access:** Traffic is sent directly to the EPP pod's externally exposed port.

![Standalone Design](../assets/standalone-design.svg)

Gateway Mode (Inference Gateway)

Gateway Mode, also known as the **Inference Gateway**, leverages the official Kubernetes Gateway API project focused on L4 and L7 networking, representing the next generation of Kubernetes Load Balancing and Service Mesh APIs.

When to Use

Gateway Mode is targeted at production environments that require:

- **Shared Infrastructure:** A single, shared Gateway can host multiple HTTP/gRPC routes for both inference workloads (represented as `InferencePool`) and traditional applications (standard Kubernetes `Service` objects).
- Integration with cloud-native L7 networking solutions (Istio, GKE Gateway, agentgateway, Envoy AI Gateway).
- Multi-cluster load balancing.
- Advanced traffic management (weighted splitting, mirroring).
- Exposure of endpoints to external workloads with robust control.

Architecture

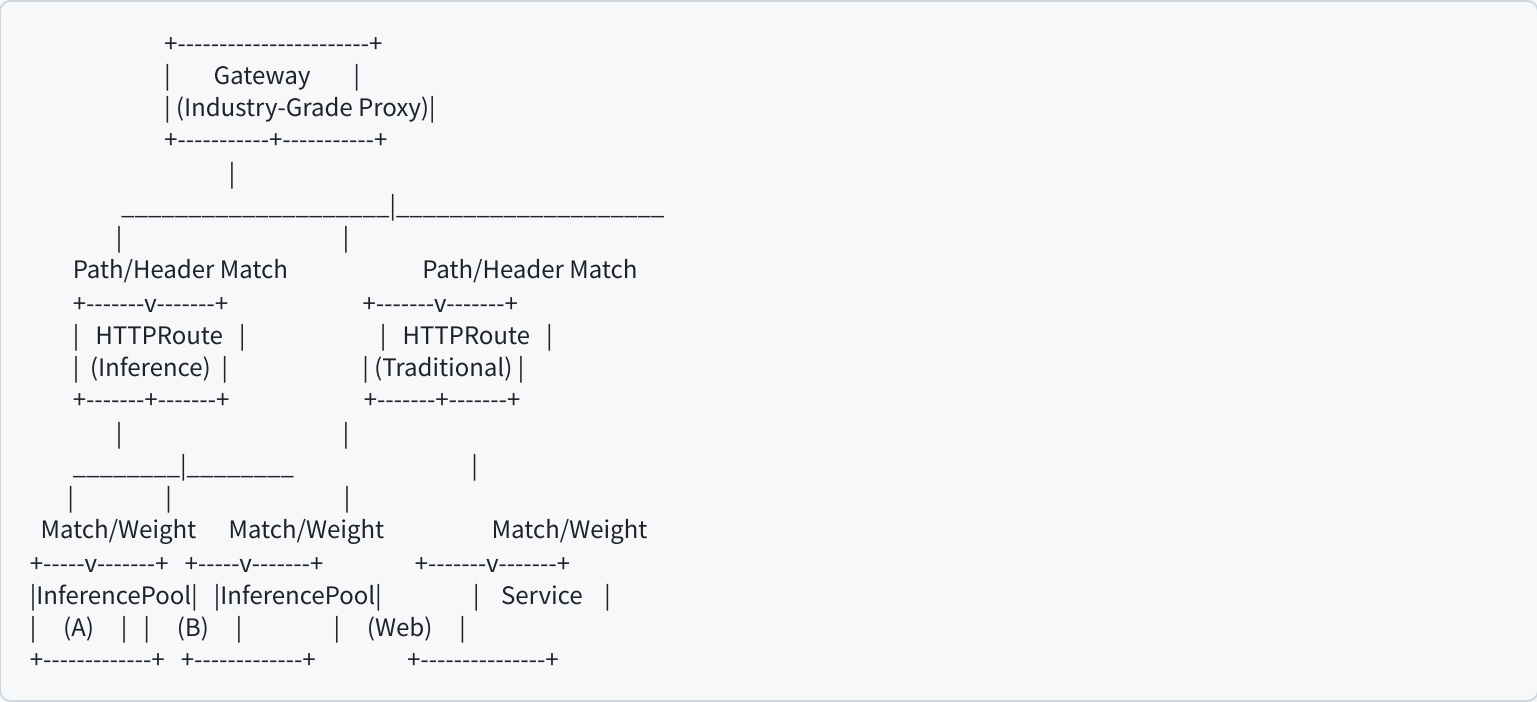
The Gateway API Inference Extension (GAIE) extends Gateway API by leveraging External Processing to inject LLM-aware load balancing into the production-grade networking provided by the Gateway provider.

![Gateway Design](../assets/gateway-design.svg)

To connect an **InferencePool** (and the EPP it selects) to a **Gateway**, you use an **HTTPRoute** that references the pool as its backend. One of the powerful features of Gateway Mode is the ability to host complex routing topologies: a single

Gateway can host multiple HTTPRoute objects, and each route can be configured with multiple InferencePool backends (e.g., for canary rollouts or traffic splitting) or a standard Kubernetes Service.

The following diagram illustrates the hierarchical routing structure in Gateway Mode, showing how requests are matched and distributed across multiple InferencePools and traditional Services:



Traffic is first matched against an HTTPRoute rule (typically using path or header matches). If the selected backend in that rule is an InferencePool, the request is parked while the EPP configured for that InferencePool is consulted for an inference-optimized routing decision.

Example Gateway	Example HTTPRoute
<pre>apiVersion: gateway.networking.k8s.io/v1 kind: Gateway metadata: name: my-gateway spec: gatewayClassName: istio listeners: - name: http protocol: HTTP port: 80</pre>	<pre>apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name: my-http-route spec: parentRefs: - group: gateway.networking.k8s.io kind: Gateway name: my-gateway rules: - backendRefs: - group: inference.networking.k8s.io kind: InferencePool name: my-infpool port: 8000 matches: - path: type: PathPrefix value: /</pre>

llm-d provides Gateway Mode deployment guides for the following Gateways:

- Istio
- GKE Gateway
- agentgateway
- Envoy AI Gateway

SLO-Aware Autoscaling with KEDA — the control law

This is the full derivation of the control law used by the SLO-aware autoscaling guide: how the EPP's latency histograms become a single saturation signal, how the KEDA formula maps that signal to a desired replica count, and why each asymmetry in the design exists. For the deployable manifests, tunables, and benchmark results, use the guide; read this when you want to understand or modify the loop itself.

This is a specialization of the KEDA + EPP design: the same pipeline and the same KEDA-owned HPA, with latency rather than queue depth as the signal.

Five steps, evaluated continuously. Steps 1–2 are the parts you deploy (recording rules + formula); 3–4 are standard HPA/Kubernetes mechanics that shape the response; 5 is why it converges.

1. Signal chain (recording rules, every 15 s)

The saturation signal is the pool's P90 latency measured against your SLOs — the worst of the two targets:

$$s_{\text{raw}} = \max\left(\frac{\text{P90 TTFT}}{S_{\text{ttft}}}, \frac{\text{P90 TPOT}}{S_{\text{tpot}}}\right), \quad \text{if } \text{P90 TTFT} \neq 0 \text{ or } \text{P90 TPOT} \neq 0, \quad \text{otherwise } \text{NaN/no traffic} \mapsto 0$$

Each P90 is `histogram_quantile(0.9, ...)` over 1-minute bucket rates, preferring the predicted histogram and falling back to the actual one (the `>= 0` filter drops a present-but-dead predicted series, whose quantile is NaN). `s_raw < 1` means the tail is within SLO; `> 1` means the SLO is breached; an idle pool reads 0.

Then clamp and smooth:

$$s = \text{avg_over_time}(\min(s_{\text{raw}}, C)) [45s], \quad \text{where } C = 2.0$$

Don't skip the clamp: at the queueing knee, predicted latency can spike to $10\text{--}90 \times \text{SLO}$ in one sample. Anything above the cap already means "add capacity at the maximum rate" — letting the raw magnitude through only poisons the moving average so it stays inflated long after the pool recovers.

2. Desired replicas (the KEDA formula)

With `s` = smoothed saturation, `n` = provisioned replicas, `r` = ready replicas, and thresholds `θup = 0.55`, `θdn = 0.40` (in the ScaledObject these are the `saturation`, `replicas`, and `readyReplicas` triggers):

$$d = \begin{cases} n + \left\lceil n \left(\frac{s \cdot c}{\theta_{\text{up}}} - 1 \right) \right\rceil & \text{if } s > \theta_{\text{up}} \quad \text{(scale up, credited)} \\ n - \left\lfloor n \left(1 - \frac{s}{\theta_{\text{dn}}} \right) \right\rfloor & \text{if } s < \theta_{\text{dn}} \quad \text{(scale down, uncredited)} \\ n & \text{otherwise} \quad \text{(hysteresis band)} \end{cases}$$

where the **in-flight credit** $c = r/n$ (when $0 < r < n$, else 1) is applied on the scale-up branch only. Three deliberate asymmetries, each worth its weight:

- **Hysteresis band (0.40–0.55):** the pool holds steady between the boundaries instead of flapping around a single set-point.
- **Credit:** the signal is measured on ready pods only. While a scale-up is in flight the ready pods over-report the post-warmup load; scaling the demand by r/n makes the ask cover only the deficit beyond pods already warming, instead of racing to `maxReplicas` every cycle.
- **Scale-down uses the uncredited signal:** the credit discounts demand, so during warmup it could push an over-threshold signal below the scale-down boundary and shed replicas mid-scale-up. A pool may only shed when the measured signal itself has headroom.

Why the thresholds sit low: under healthy load the P90-of-predicted-latency signal sits around 0.2–0.45 of the SLO and then goes *vertical* at the queueing knee. A threshold near 1.0 fires only after the SLO budget is nearly burned; 0.55 fires on the pre-knee rise. If your SLO is close to your base latency, raise the band.

In the `ScaledObject`, `d` is the `scalingModifiers` formula output, and the composite target is `1` (`AverageValue`) — a pass-through: the formula output *is* the desired replica count.

3. HPA layer (rate limiting and stabilization)

The KEDA-generated HPA clips `d` to `[minReplicaCount, maxReplicaCount]` and applies the `behavior` block: scale-up bursts at `max(100 %, 4 pods)` per 60 s with a 30 s stabilization window (min over window); scale-down walks 1 pod per 120 s with a 180 s window (max over window). Fast up, deliberate down.

4. Ready replicas (pod warmup)

Additions take a warmup time T_w to become ready (model load + torch-compile); removals are immediate:

$$R(t) = \min_{\tau \in [t - T_w, t]} N_{\text{spec}}(\tau)$$

T_w is why the credit in step 2 exists, and it is the knob most worth engineering: pod warmup is roughly *half* the scale-up transient, so cutting it directly shrinks the violations paid at every scale-up. A torch-compile cache volume + a fast `startupProbe` took our pod-ready time from ~2 m 15 s to ~100 s; that patch ships with the guide as `slo-aware/decode-warmup-patch.yaml` (recommended — apply it to the decode Deployment, see the guide's Deploy section). This is the patch used to produce the guide's benchmark results.

5. Closing the loop

More ready replicas → lower per-pod load → the predicted P90 falls → `s` re-enters the band and the ask stops. Under-provisioning raises `s` and the loop adds capacity; over-provisioning drops `s` below 0.40 and the slow drain begins. The system settles wherever `s` lands inside the band — empirically ~1.4–1.5 rps per H100 TP=2 replica for a 4000-token-prefill workload at these SLOs.

Async Processor Architecture

The Async Processor is a lightweight dispatch agent that pulls inference requests from message queues and forwards them to the llm-d Router. It uses dispatch gates, isolated worker pools, request body transforms, and merge policies to regulate dispatch rates and process background workloads efficiently without overflowing inference servers.

![[Async Processor High-Level Architecture]](../../assets/async-processor.svg)

Internal Pipeline Architecture

![[llm-d-async Internal Pipeline Architecture (4 Queues and 2 Worker Pools)]](../../assets/async-processor-internal.svg)

How It Works

1. **Queue Gate & Poll** — Before pulling messages from queues, requests pass through queue-level **Dispatch Gates** to regulate message flow and ingestion based on system metrics and capacity.
2. **Merge** — Polled messages from multiple input queues or topic subscriptions are grouped by target worker pool and combined into per-pool merged channels according to the globally configured **Request Merge Policy**.
3. **Worker Pool Admission Gate** — Merged requests pass through a **Worker Pool Admission Gate** (e.g., local max concurrency, pool-level saturation or budget gates) that verifies whether the worker pool has capacity and budget to accept new work.
4. **Transform** — Admitted requests pass through **Request Body Transforms** to normalize payload schemas or inject routing headers and parameters.
5. **Dispatch** — Concurrent workers in the designated **Worker Pool** dispatch HTTP requests to the llm-d Router with deadline propagation.
6. **Result** — On success, results are written back to a result queue. On retryable failure (rate limiting, transient errors), the request is re-queued with exponential backoff.

Dispatch Gates

Dispatch gates regulate the flow of requests by controlling message ingestion rates when polling requests from message queues, as well as worker pool admission and downstream dispatch rates. By evaluating metrics before messages are pulled and admitted into worker pipelines, gates prevent overload on downstream inference servers and maintain system stability.

Gates can be configured at two distinct levels:

- **Queue / Subscription Level:** Controls flow and ingestion before messages are polled and merged.

- **Worker Pool Admission Level:** Controls entry into the worker pool execution pipeline. Multiple gates can be combined for a pool using a **composite gate**.

Gate type	Behavior
constant	Always open — no throttling.
redis	Reads budget values from Redis keys, allowing external orchestrators to dynamically adjust rate limits per queue or pool.
prometheus-saturation	Queries Prometheus for model server saturation metrics (e.g., KV cache pressure, queue depth). Allows ingestion and admission when saturation is below a configurable threshold.
prometheus-budget	Computes available downstream capacity directly from metrics.
local-max-concurrency	Restricts local concurrency within a worker pool to cap in-flight requests.

Worker Pools & Request Merge Policies

Worker Pools

The Async Processor uses a multi-tenant pipeline model based on **Worker Pools**.

[!NOTE] An `llm-d-async` worker pool is **not** an inference pool (such as a downstream model server pool). It is an internal dispatch pipeline construct within `llm-d-async` that configures the **pool name**, **number of workers**, and **admission gate**. The worker pool configuration is fully user-configurable.

- **Configurable Pipeline:** Defines the pool name, number of concurrent workers, and admission gate settings.
- **Composite Gates:** Multiple gates (e.g., local max concurrency combined with Prometheus saturation) can be configured for a single pool using a **composite gate**.
- **Backend Isolation:** It is recommended to use different worker pools for different `llm-d-router` endpoints so that saturation or backpressure on one backend model server does not block processing for other endpoints.
- **Isolation:** Saturated or blocked worker pools do not affect the processing or throughput of other worker pools.
- **Queue-to-Pool Mapping:** Multiple queue configurations can route requests to the same worker pool, but a single queue configuration can only route to a single worker pool.

Request Merge Policies

A single **Request Merge Policy** is configured globally at the pipeline level for the entire `llm-d-async` instance (not per pool). Rather than configuring different policy types on individual pools, `llm-d-async` groups input queue channels by their assigned `worker_pool_id` and executes the global merge policy independently across each pool's merging lanes.

The two supported global merge policies are:

Policy	Description
random-robin	Default policy. Randomly picks messages from all queues configured for a each pool.
tier-priority	Buckets requests into 6 strict priority lanes using routing tags ((classification, tier)). Within each bucket, it round-robins across different client channels and stamps lane objectives (x-llm-d-inference-objective).

Because merging lanes are grouped and evaluated independently per worker pool, backpressure from one pool's merged channel only impacts its associated worker pool, maintaining multi-tenant isolation.

Request Body Transforms

Request body-transform plugins handle rewriting the outgoing body and Content-Type based on per-message metadata. The default JSON path is preserved byte-for-byte when no plugin applies.

Request and Result Messages

The Async Processor defines standardized JSON schemas for requests published to input queues and results returned to result queues/topics. All worker pools push completed inference results to a single, unified **Result Message Queue** (or topic) configured for the llm-d-async processor instance.

Request Message Schema

Field	Type	Description
id	string	Unique identifier for result mapping (required).
created	int64	Created timestamp in Unix seconds.
deadline	int64	Deadline in Unix seconds (required, must be positive).
payload	object	Inference request payload.
metadata	map[string]string	Optional caller-supplied pass-through data (e.g., tracing IDs, user labels).
endpoint	string	Optional per-request dispatch path; overrides the queue-level default when set.

Example Request Message:


```
{
  "id": "19933123533434",
  "created": 1764044000,
  "deadline": 1764045130,
  "payload": {
    "model": "food-review",
    "prompt": "hi",
    "max_tokens": 10,
    "temperature": 0
  },
  "metadata": {
    "user": "batch-job-42"
  }
}
```

Result Message Schema

Field	Type	Description
id	string	Unique identifier mapped to the corresponding request.
payload	bytes / object	Inference result payload returned from downstream model server.
status_code	int	HTTP response status code (non-zero value indicates an HTTP response was received).
error_code	string	Error classification code (e.g., DEADLINE_EXCEEDED , GATE_DROPPED , GATE_ERROR , INFERENCE_ERROR , INVALID_REQUEST).
error_message	string	Human-readable explanation of the error.

Example Result Messages:

Success Result:

```
{
  "id": "19933123533434",
  "payload": {
    "choices": [
      {
        "text": "Great food!"
      }
    ]
  },
  "status_code": 200
}
```

Error Result:

```
{
  "id": "19933123533434",
  "error_code": "GATE_DROPPED",
  "error_message": "Pool gating dropped request"
}
```

Message Queue Integrations

Queue configurations define the input message queue source parameters (such as Redis keys or Pub/Sub subscription IDs) and specify the target `llm-d-router` **endpoint information** (such as URL path and target model endpoint) for dispatched requests.

Implementation	Characteristics
Redis Sorted Set	Persisted, priority-ordered by deadline. Supports per-queue gate configurations.
Redis Pub/Sub (deprecated — prefer Redis Sorted Set)	Ephemeral, fan-out delivery. Still functional; scheduled for removal in a future release.
GCP Pub/Sub	Cloud-native, scalable. Supports per-subscription gating.

Configuration

The Async Processor is configured via high-level schema definitions for message queue sources and worker pools.

Message Queue Configurations

Queue configurations specify input message queue parameters, target `llm-d Router` endpoints, routing tags, and optional queue-level dispatch gates.

Field	Type	Required	Description	Example Value
queue_name / subscriber_id	String	Yes	Name of the input queue or Pub/Sub subscriber ID.	"batch_queue"
worker_pool_id	String	Yes	Target worker pool ID to route messages polled from this queue.	"worker_pool_1"
igw_base_url	String	Yes	Base URL of the target ILM-d Router endpoint.	"http://localhost:80/"
request_path_url	String	Yes	Request endpoint path on the target router.	"/v1/completions"
inference_objective	String	Optional	Routing tag for inference classification.	"batch-task"
labels	Object	Optional	Key-value pairs for message metadata and routing tags.	{"tier": "standard"}
gate_type	String	Optional	Type of queue-level dispatch gate (e.g., redis , constant).	"redis"
gate_params	Object	Optional	Key-value parameters configuring the queue dispatch gate.	{"address": "localhost:6379", "budget_key": "my-budget-key"}

Example Queue Configuration:

```
{
  "queue_name": "batch_queue",
  "inference_objective": "batch-task",
  "request_path_url": "/v1/completions",
  "igw_base_url": "http://localhost:80/",
  "worker_pool_id": "worker_pool_1",
  "gate_type": "redis",
  "gate_params": {
    "address": "localhost:6379",
    "budget_key": "my-budget-key"
  }
}
```

Worker Pools Configuration

Worker pool configurations define dedicated worker concurrency limits and pool admission gates for isolated execution pipelines.

Field	Type	Required	Description	Example Value
id	String	Yes	Unique pool identifier referenced by queue/topic configurations (worker_pool_id).	"worker-pool_1"
workers	Integer	Yes	Number of concurrent workers dedicated to this pool. Must be positive.	32
gate_type	String	Optional	The type of dispatch gate to apply to the pool (e.g., local-max-concurrency , prometheus-saturation).	"prometheus-saturation"
gate_params	Object	Optional	Key-value parameters configuring the pool admission gate.	{"pool": "inference_pool_1", "threshold": "0.8"}

Example Worker Pool Configuration:

```
[
{
  "id": "worker-pool_1",
  "workers": 32,
  "gate_type": "prometheus-saturation",
  "gate_params": {
    "pool": "inference_pool_1",
    "threshold": "0.8"
  }
}
```

For more detailed configuration options, see the llm-d-async README.

Concurrency and Retries

- **Configurable Parallelism:** Worker pools define independent concurrency limits (default 64 per pool) for parallel processing.
- **Deadline Enforcement:** Each request carries a deadline from the queue message. Workers abandon requests that cannot complete before their deadline.
- **Exponential Backoff:** Retryable failures are re-queued with backoff (base 2s, max 60s, with jitter). Fatal errors (bad payload, unrecoverable failures) are not retried.

Observability

Prometheus metrics include request totals, success/failure counts, retry counts, deadline-exceeded counts, shedded request counts, gate decisions, dispatch budgets, and request latency histograms.

Related

- Async Processor Well-Lit Path — a guide for deploying the Async Processor.
- Async Processor Repository — source code and Helm chart.

- Batch Gateway — composes with the Async Processor for batch job management.

KV-Cache Indexer

The **KV-Cache Indexer** is a component of the **llm-d Router** (residing within the **EPP**) that enables precise prefix-cache-aware routing functionality.

[!NOTE] This page assumes familiarity with the EPP's design. See [EPP architecture](#) for more details.

Functionality

The kv-cache indexer subscribes to `KVEvents` emitted from model servers to maintain a near-realtime view of the KV cache state. The `precise-prefix-cache-producer` (consumed by the `prefix-cache-scorer`) uses this information during the EPP filter → score → pick flow.

The precise view offers improved precision for harder-to-approximate scenarios:

- **Multi-Modal Models** — Multi-modal content hashes (images, audio) are folded into block keys, so two prompts with the same text but different images produce different keys and are routed to the pod with matching multimodal KV-cache.
- **Hybrid-Attention Models** — In hybrid models, KV cache usage per layer groups (full, sliding-window, linear) scales non-linearly making byte-based trees imprecise.
- **Advanced KV-Cache Orchestration** — As model-server KV-cache management policies grow beyond simple LRU, approximate views become increasingly unreliable and complex to create; the event-driven view tracks the actual state.

[!NOTE] Hybrid-attention-aware scoring is a work in progress.

Architecture

At the top level there are two components: the **Model Servers** and the **EPP**. Model servers publish KV-Events whenever their cache state changes; the EPP watches those events, stores them in an index, and consults the index during scheduling.

Zooming into the EPP, four cooperating components do the work:

- **Index** — stores the event-driven view of which blocks are resident on which pods.
- **Token Producer** — tokenizes prompts and extracts multimodal features into `TokenizedPrompt`.
- **Precise Data Producer** — consumes `TokenizedPrompt`, derives block keys, consults the Index, and publishes per-endpoint `PrefixCacheMatchInfo`.

- **Scorer** — consumes `PrefixCacheMatchInfo` to score each candidate pod by how much of the request's prefix it already holds.

```
flowchart LR
    Req([Inference Request]) --> TP
    subgraph EPP["EPP"]
        direction TB
        TP["Token Producer<br>(token-producer)"]
        DP["Data Producer<br>(precise-prefix-cache-producer)"]
        Scorer["Scorer<br>(prefix-cache-scorer)"]
        Index["Index<br>(block key → pods)"]
        TP --> DP
        DP --> Scorer
        DP <-->|lookup| Index
    end
    subgraph MS["Model Servers"]
        direction TB
        vllm["vLLM / SGLang"]
    end
    MS -.KV-Events over ZMQ.-> Index
    style EPP fill:#eef,stroke:#447,color:#000
    style MS fill:#fff3e0,stroke:#ff9800,color:#000
```

Each component has its own configuration surface; the rest of this page describes them in turn.

Index

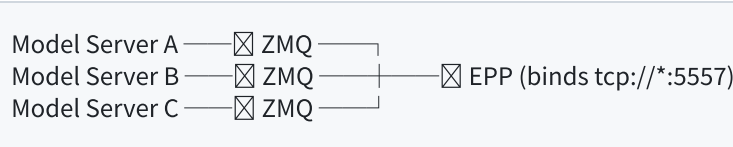
The Index holds a `block key → pods` mapping built up from KV-Events published by the model servers. Model servers (vLLM and SGLang today) publish three event types over ZMQ whenever their KV-cache state changes:

- **BlockStored** — blocks with the given content hashes have been created on a specific device tier. Payload includes the chained parent hash, the token chunk, any LoRA ID/name, and any multimodal extra keys.
- **BlockRemoved** — blocks with the given hashes have been evicted from a specific device tier and/or attention group.
- **AllBlocksCleared** — the pod dropped its entire cache (a reset). This can occur in RL weights rollouts and other scenarios. The indexer drops all entries associated with the pod.

Event Delivery Modes

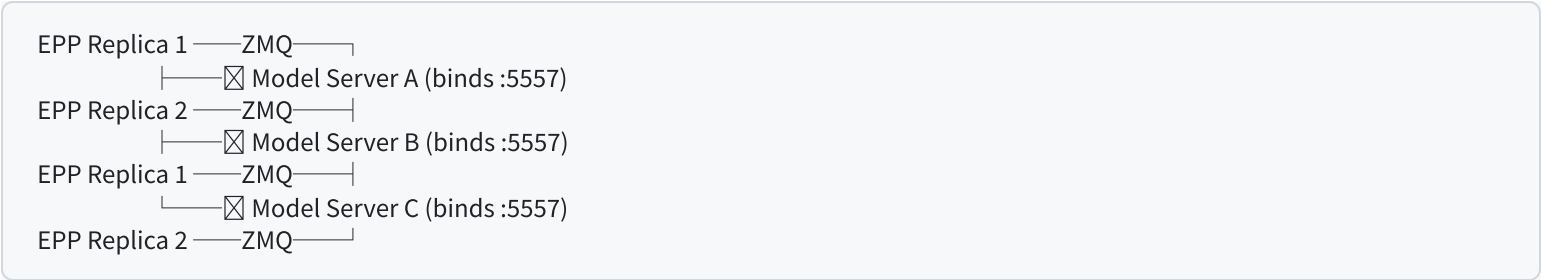
Two shapes are supported for getting events from the model servers to the EPP:

- **Centralized** — every model-server pod connects (`zmq.PUB`) to a single endpoint hosted by the EPP (`zmq.SUB`). Works naturally with a single EPP replica.



- **Pod discovery** — each model-server pod binds its own ZMQ socket. The EPP discovers pods via Kubernetes label selectors and creates per-pod subscribers. This is the mode to use for active-active multi-EPP: every EPP replica

independently subscribes to every pod and sees the full event stream.



Backends

The Index is the hot data structure of the system: every scoring call queries it, every KV-event updates it. Multiple backends are available depending on desired memory and replication model:

Backend	Storage	When to use	Tradeoff
In-Memory (default)	Two-level LRU: outer cache keyed by block hash, inner cache of pods per block	Default choice for most deployments	Lowest latency; fixed entry count (default 100M keys × 10 pod entries) makes sizing predictable
Cost-Aware Memory	Ristretto cache with admission control and cost-based eviction	Workloads where per-entry size varies a lot (multimodal, variable-length LoRA metadata)	Budget specified in bytes (e.g. 2GiB) rather than entry count; probabilistic admission can reject entries under pressure
Redis / Valkey	External server (TCP; Valkey is Redis-wire-compatible, BSD-licensed)	Need for persistent or very-long lived index (uncommon)	Adds a network hop per lookup and ties EPP availability to the external store; shared state gives strong consistency across replicas but is rarely necessary

[!IMPORTANT] In-memory is typically the best option, offering low-latency, simple operations, and high availability via multi-replica deployment (each EPP replica in pod-discovery mode subscribes to every model-server's events independently and converges to the same index).

Sizing. In-memory backends size independently per replica; plan for roughly `keys × pod_entries` with overhead for the two-level LRU. The cost-aware backend is easier to bound because you specify a byte ceiling; it is the safer choice when per-entry size is hard to predict. For Redis / Valkey, the key space is proportional to unique blocks across the fleet, not to request volume.

Token Producer

The Token Producer runs early in the scheduling cycle: it renders chat templates and tokenizes the prompt once per request (and extracts any multimodal features), writing `TokenizedPrompt` onto the request so downstream components do not re-tokenize.

This role is implemented by the `token-producer` plugin. It does not access the KV-cache Index or score endpoints.

The plugin uses the tokenizer-free `estimate` backend by default. For precise routing, select the exact-tokenization `vllm` backend with `modelName` and `vllm.url`. It calls vLLM's render endpoints — `/v1/completions/render` and `/v1/chat/completions/render` over HTTP. Those endpoints are served by `vllm serve <model>` or by the GPU-less `vllm launch render <model>`, deployed either as a sidecar in the EPP pod (loopback) or as a dedicated render Service shared across EPP replicas.

Precise Prefix Cache Producer

The `precise-prefix-cache-producer` consumes `TokenizedPrompt`, derives KV-block keys, looks them up in the event-driven Index, and publishes per-endpoint `PrefixCacheMatchInfo` for the `prefix-cache-scorer`.

It also receives endpoint lifecycle notifications so it can manage per-pod KV-event subscriptions. Wire it to `endpoint-notification-source` in `dataLayer`:

```
plugins:
- type: token-producer
  parameters:
    modelName: Qwen/Qwen3-32B
    vllm:
      url: http://render:8000
- type: endpoint-notification-source
- type: precise-prefix-cache-producer
  parameters:
    speculativeIndexing: true
- type: prefix-cache-scorer
  parameters:
    prefixMatchInfoProducerName: precise-prefix-cache-producer

dataLayer:
  sources:
  - pluginRef: endpoint-notification-source
  extractors:
  - pluginRef: precise-prefix-cache-producer
```

The `prefix-cache-scorer` must reference this producer by name. Without that reference, it uses an automatically created `approx-prefix-cache-producer` instead.

Speculative Indexing

Confirmed KV-events arrive after a request has been routed. Back-to-back requests with the same prefix can be scheduled before `KVEvents` have propagated, breaking affinity.

With `speculativeIndexing: true`, the precise producer inserts short-lived predicted entries in the Index for the selected pod (and, under P/D disaggregation, the selected prefill pod) right after the routing decision. Subsequent requests match against those entries until a confirming `BlockStored` arrives or a TTL (default `2s`) expires.

The default 2-second TTL is tuned to comfortably exceed the typical routing-to-event latency without outliving a genuinely failed speculation.

Scorer

The `prefix-cache-scorer` is stateless. It consumes `PrefixCacheMatchInfo` from the configured data producer and finds, for each candidate pod, the length of the **longest consecutive prefix** of the request's block sequence that the pod has

cached.

KV-cache blocks form a chain where block i depends on all blocks $0..i-1$. Due to the causal nature of attention, a server can reuse a cached block only if it holds the unbroken prefix leading up to it.

For example, consider a prompt with block keys $[B_0, B_1, B_2, B_3, B_4]$ and three candidate pods:

Block keys: $B_0 \ B_1 \ B_2 \ B_3 \ B_4$

Pod A: yes yes yes yes no $\rightarrow$ score = 4 blocks

Pod B: yes yes no - - $\rightarrow$ score = 2 blocks (chain breaks at B_2)

Pod C: no - - - - $\rightarrow$ score = 0 blocks (no prefix)

Even if Pod C happened to hold B_3 and B_4 , those entries are unusable without the preceding chain, and the score is zero.

When blocks are stored across memory tiers, each matching block's contribution is weighted by tier. For a block cached on multiple tiers at once, the scorer takes the maximum weight. Defaults are $\text{gpu} = 1.0$, $\text{cpu} = 0.8$.

Raw scores are then normalized to $[0.0, 1.0]$ before being returned to the EPP, where they are combined with other scorers (queue depth, KV-cache utilization, etc.) through the standard Filter-Score-Pick pipeline.

Multimodal, LoRA, and Hybrid Attention

Many deployment patterns cache KV blocks based on more than text. The indexer supports these by folding additional features into block keys:

- **Multimodal** — Multimodal content hashes (images, audio) are folded into the block-key chain. vLLM emits an `extra_keys` field on `BlockStored` events, which the indexer parses and re-computes on the read side by walking the multimodal placeholders in the tokenized prompt. Two prompts identical in text but differing in image content hash differently and route independently.
- **LoRA** — On `BlockStored`, if a `LoraName` is present, it is used in place of the base model name when deriving block keys. Different adapters produce different key chains for the same token sequence, and cache hits are correctly scoped to the adapter.
- **Hybrid attention** (*target design — work in progress*) — Hybrid models partition the KV-cache into layer groups (full, sliding-window, linear/state-space) that evict independently. For a single token range, full-attention blocks can still be resident while sliding-window blocks have rolled out of the attention window — the same "prefix" is cached in one group but not in another. Scoring for hybrid models therefore needs to classify each prefix match as **full**, **partial**, or **miss**, using the model's window sizes to decide whether a partial hit is still routable.

Further Reading

- **llm-d-kv-cache** — the indexer library. See `architecture.md` for the in-depth technical architecture (block-key hashing, dual-key design, event adapters, module breakdown) and `configuration.md` for the full configuration reference.
- **llm-d Router** — source for the `precise-prefix-cache-producer`, `prefix-cache-scorer`, and `token-producer` plugins. Plugin lifecycle, EPP extension-point wiring, and request scheduling profile examples.

Workload Variant Autoscaler (WVA) — Deprecated

[!WARNING] **Deprecated.** The Workload Variant Autoscaler is deprecated and is no longer the recommended way to autoscale llm-d workloads. It is retained here as a design reference for existing deployments; expect no new features, and plan a migration.

The recommended path is KEDA with EPP Metrics, deployed from the workload autoscaling guides. See [Migrating off WVA](#) below.

Within the deprecated path, the `VariantAutoscaling` (VA) CRD was itself already superseded by HPA objects consuming WVA's `wva_desired_replicas` metric — see [Migration from VA to HPA + WVA](#).

Functionality

The Workload Variant Autoscaler (WVA) is an optimizer and a Kubernetes controller that automatically scales LLM inference workloads based on real-time resource utilization and performance metrics. As an optimizer, it analyzes supply and demand signals across all `InferencePool` variants to produce globally cost-efficient scaling decisions. As a controller, it reconciles those decisions by managing the replica count of model-serving deployments (`Deployments`, `StatefulSets`, or `LeaderWorkerSets`), observing vLLM metrics scraped via Prometheus.

WVA is **variant-aware**. A **variant** is one of multiple model servers in an `InferencePool` that all serve the same base model but differ in hardware configuration (e.g., GPU type), serving configuration (e.g., tensor parallelism, max batch size, quantization), or both, each with an associated cost. WVA optimizes across these variants to minimize total cost while meeting capacity or latency requirements.

[!NOTE] WVA assumes a 1:1:1 relationship between `InferencePool`, `Endpoint Picker` (EPP), and base model. All variants within an `InferencePool` share the same EPP and therefore the same EPP metrics (e.g., request queue size).

WVA provides two main scaling analyzers:

- **Saturation Analyzer** -- Scales based on resource saturation signals (KV cache utilization, request queue depth, and token-level capacity). When the system detects that model servers are saturated (running out of KV cache space or building up queues), it triggers scale-up on the cheapest available variant. When spare capacity is detected, it scales down the most expensive variant. This is the default analyzer. It has two sub-variants: `saturation-percentage-based` (default) and `saturation-token-based` (experimental).

[!NOTE] These two sub-variants are also referred to as the **V1** (saturation-percentage-based) and **V2** (saturation-token-based) saturation engines. The HPA + WVA guide, the WVA configuration keys, and the controller logs (e.g. v2 saturation analysis completed) use the V1/V2 names; this document uses the descriptive saturation-percentage-based / saturation-token-based names for the same engines.

- **SLO Analyzer (Queueing Model)** (*Experimental*) -- Scales based on latency SLO targets using queueing theory. It uses a Kalman filter to learn hardware-specific performance parameters online, then applies a state-dependent Markovian queueing model to determine the maximum sustainable request rate per replica that meets target TTFT (Time To First Token) and ITL (Inter-Token Latency). The desired replica count is computed as the ratio of observed arrival rate to this capacity.

Both analyzers integrate with a pipeline that includes cost-aware optimization, scale-to-zero enforcement, and optional GPU resource limiting.

Design

Relation to llm-d Components

The following diagram shows how WVA fits into the overall llm-d architecture:

 WVA High-Level Architecture

Scaling Engine Architecture

The WVA scaling engine runs as a background goroutine alongside the Kubernetes controller. Each cycle it groups the active WVA-managed HPAs by `modelID`, scrapes per-replica metrics, and runs them through the pipeline below. Decisions land in an in-memory decision cache, from which an actuator emits the `wva_desired_replicas` external metric that the HPAs consume.

[!NOTE] The engine supports both a saturation-percentage-based and saturation-token-based analysis path. saturation-percentage-based is the default. saturation-token-based is an experimental token-based approach that is not yet production-ready. The architecture described here covers both pipelines.

The main loop runs every 30 seconds and follows a four-stage pipeline pattern:

 Scaling Engine Pipeline

Pipeline stages:

1. **Collect** -- Groups the active WVA-managed HPAs by `modelID` and scrapes per-replica metrics (from Prometheus and, for the EPP flow-control queue, directly from EPP) for each resulting InferencePool.
2. **Analyzer** -- Each analyzer produces capacity signals (required capacity, spare capacity) and a priority score per InferencePool. The analyzer does not make scaling decisions directly; it quantifies how much capacity is needed or can be freed.

3. **Optimizer** -- The optimizer receives scaling requests (one per InferencePool) and produces variant decisions specifying the target replica count per variant. Two modes exist: - **Cost-aware** (default, unlimited mode): Processes each InferencePool independently. Scales up the most cost-efficient variant; scales down the most expensive variant. - **Greedy-by-score** (limited mode, `enableLimiter: true`): Fair-shares available GPU resources across all InferencePools based on priority scores.
4. **Enforcer** -- Applies post-optimization policies: scale-to-zero when an InferencePool is idle (no requests in the retention period), or minimum replica enforcement (at least 1 replica on the cheapest variant) when scale-to-zero is disabled.

The resulting variant decisions are written to the in-memory **decision cache**, and an **actuator** publishes them as the `wva_desired_replicas` external metric. A separate **scale-from-zero engine** polls the EPP flow-control queue every 100 ms and writes directly to the same decision cache, so idle InferencePools can scale up without waiting for the 30-second main loop.

Saturation Analyzer

The saturation analyzer determines scaling needs based on how saturated the InferencePools are.

`saturation-percentage-based` -- Default

The `saturation-percentage-based` analyzer uses percentage thresholds on two dimensions:

- **KV cache utilization** -- A replica is saturated if the KV cache usage meets or exceeds the threshold (default 0.80).
- **Queue length** -- A replica is saturated if the queue length meets or exceeds the threshold (default 5).

Scale-up triggers when the average spare KV capacity drops below the spare trigger (default 0.10) or the average spare queue capacity drops below the queue spare trigger (default 3). It adds 1 replica to the cheapest variant, skipping variants with pending replicas.

Scale-down is safe when at least 2 non-saturated replicas exist and a simulated $N/(N-1)$ load redistribution still leaves adequate headroom. It removes 1 replica from the most expensive variant.

`saturation-percentage-based` blocks all scaling when any variant is transitioning (desired != current replicas).

`saturation-token-based` -- Experimental

The `saturation-token-based` analyzer uses absolute token counts instead of percentages, enabling cross-variant capacity comparison and more precise scaling.

Each replica's capacity is modeled with two bounds:

- **k1 (memory-bound)**: Total KV capacity in tokens multiplied by the KV cache threshold -- how many tokens the KV cache can hold.
- **k2 (compute-bound)**: determined via a priority chain: 1. **Observed**: When the queue is saturated (queue length meets or exceeds the threshold), the current tokens in use is taken directly. 2. **Historical**: Rolling average from previous observations (window size 10). 3. **Derived from deployment args**: Computed from the effective max batched tokens, max num seqs, and average input/output token lengths using a steady-state batching model. 4. **Fallback**: k1 (memory-only).

The **effective capacity** per replica is the minimum of k1 and k2. Per-variant capacity is aggregated using the median across ready replicas.

Demand per replica is the sum of tokens currently in use and the queued requests multiplied by the average input token length. The EPP queue demand (from `llm_d_epp_flow_control_queue_size` and `llm_d_epp_flow_control_queue_bytes` metrics) is added to the InferencePool-level totals.

Scaling signals:

- **Required capacity:** total demand divided by the scale-up threshold, minus anticipated supply. A positive value means scale-up is needed.
- **Spare capacity:** total supply minus total demand divided by the scale-down boundary. A positive value means scale-down is possible.

Default thresholds: scale-up threshold 0.85, scale-down boundary 0.70.

Key improvements over `saturation-percentage-based` :

- Token-level granularity enables cross-variant and cross-InferencePool capacity comparison
- Dual-bound capacity model (memory + compute) captures both resource bottlenecks
- Capacity knowledge is cached for zero-replica variants, enabling accurate cost estimation before scaling
- Supports P/D (Prefill/Decode) disaggregation with per-role capacity attribution
- Considers EPP queue demand (requests queued before reaching any vLLM pod)
- Output-length workload bucketing for compute-bound history (short < 100, medium < 500, long >= 500 tokens)

SLO Analyzer (Queueing Model) -- Experimental

The SLO analyzer scales based on latency targets rather than resource saturation. It is activated by the presence of a `wva-queueing-model-config` ConfigMap.

The analyzer operates in three phases:

1. **Online Parameter Learning (Tuner)** -- Uses a Kalman filter to learn three hardware-specific parameters from observed metrics: - **alpha**: baseline iteration overhead (ms) - **beta**: per-token compute time (ms) - **gamma**: per-token KV cache memory access time (ms)

The tuner builds environment snapshots from per-replica metrics (arrival rate, average input/output tokens, average TTFT, average ITL) and runs the Kalman filter to estimate these parameters.

2. **SLO Target Determination** -- Two approaches: - **Explicit**: From the ConfigMap (`targetTTFT` , `targetITL` in milliseconds). - **Inferred** (default): Uses the idle-latency multiplier approach. The target TTFT and ITL are derived by multiplying the predicted idle latency (based on alpha, beta, gamma, and average token lengths) by the SLO multiplier k (default 3.0, giving a target utilization of approximately 0.67). - **Fallback**: Observed TTFT/ITL multiplied by 1.5 headroom, capped at reasonable maximums.
3. **Capacity Computation** -- Uses an M/M/1/k queueing theory model to compute the maximum request rate per replica that meets the SLO targets. The desired replica count is the total arrival rate divided by this maximum rate, rounded up.

Metrics used by the SLO analyzer:

- `llm_d_epp_scheduler_attempts_total{status="success"}` -- per-pod arrival rate (requests/sec)
- `vllm:time_to_first_token_seconds` -- TTFT histogram (sum/count for average)
- `vllm:time_per_output_token_seconds` -- ITL histogram (sum/count for average)
- `vllm:request_generation_tokens` -- average output tokens (sum/count)
- `vllm:request_prompt_tokens` -- average input tokens (sum/count)

Scale To and From Zero

Scale-to-Zero

Scale-to-zero is handled by the **Enforcer** pipeline stage, which runs after the optimizer produces scaling decisions.

For each InferencePool:

1. Check if scale-to-zero is enabled (per-inferencepool config > global ConfigMap default > `WVA_SCALE_TO_ZERO` env var > false).
2. If enabled, query `sum(increase(vllm:request_success_total{...}[retentionPeriod]))` for the InferencePool.
3. If the request count is 0 over the retention period (default 10 minutes), set all variant target replicas to 0.
4. If scale-to-zero is disabled, ensure at least 1 replica on the cheapest variant.

Scale-to-zero is skipped entirely when any variant has `MinReplicas > 0` in the VA spec.

Scale-from-Zero

Scale-from-zero runs as a separate engine with a fast 100ms polling interval, independent of the main 30-second saturation engine loop. This ensures rapid response to incoming requests for idle InferencePools.

For each VA with 0 replicas:

1. Find the matching InferencePool from the datastore.
2. Query the EPP (Endpoint Picker) metrics source for `llm_d_epp_flow_control_queue_size{target_model_name=<modelID>}`.
3. If queue size > 0 (pending requests exist), scale the deployment directly to 1 replica.
4. Record the scaling decision and trigger controller reconciliation.

The scale-from-zero engine processes inactive VAs concurrently with configurable max concurrency.

Metric Collection

WVA collects metrics through a source registry that holds named metrics source implementations. Two types of sources are used:

- **Prometheus source** -- Registered at startup. Executes PromQL queries for model server metrics (KV cache utilization, queue depth, latency, etc.).
- **Pod scraping sources** -- Added dynamically as InferencePool resources are discovered. When a new pool is set in the datastore, WVA creates a dedicated pod scraping source configured with the pool's EPP service name,

namespace, and metrics port. These sources are used to collect EPP-level metrics (e.g., flow control queue size) directly from pods rather than through Prometheus. Sources are automatically removed when their associated pool is deleted.

Each metrics source provides three main capabilities:

- Returning the query registry for registering query templates
- Executing queries and updating the cache
- Retrieving a cached value for a query with given parameters

Registered Queries

Saturation queries (per-pod, 1-minute windows):

Query Name	PromQL	Purpose
kv_cache_usage	max by (pod) (max_over_time(vllm:kv_cache_usage_perc{...}[1m]))	KV cache utilization (0-1)
queue_length	max by (pod) (max_over_time(vllm:num_requests_waiting{...}[1m]))	Pending request queue depth
cache_config_info	max by (pod, num_gpu_blocks, block_size) (vllm:cache_config_info{...})	KV cache configuration (total capacity)
avg_output_tokens	rate(vllm:request_generation_tokens_sum[5m]) / rate(..._count[5m])	Average output token count
avg_input_tokens	rate(vllm:request_prompt_tokens_sum[5m]) / rate(..._count[5m])	Average input token count
prefix_cache_hit_rate	rate(vllm:prefix_cache_hits[5m]) / rate(vllm:prefix_cache_queries[5m])	Prefix cache hit ratio
scheduler_queue_size	sum(llm_d_epp_flow_control_queue_size{...})	Upstream EPP queue depth
scheduler_queue_bytes	sum(llm_d_epp_flow_control_queue_bytes{...})	Upstream EPP queue bytes

Queueing model queries (per-pod, 1-minute windows):

Query Name	PromQL	Purpose
scheduler_dispatch_rate	sum by (endpoint_name) (rate(llm_d_epp_scheduler_attempts_total{status="success",...}[1m]))	Per-pod arrival rate
avg_ttft	rate(vllm:time_to_first_token_seconds_sum[1m]) / rate(..._count[1m])	Average time to first token
avg_itl	rate(vllm:time_per_output_token_seconds_sum[1m]) / rate(..._count[1m])	Average inter-token latency

Scale-to-zero query:

Query Name	PromQL	Purpose
model_request_count	sum(increase(vllm:request_success_total{...}[retentionPeriod]))	Total requests in retention window

Scale-from-zero uses EPP (Endpoint Picker) metrics directly (not via Prometheus):

Metric	Purpose
llm_d_epp_flow_control_queue_size	Pending requests in the EPP flow control queue

The replica metrics collector orchestrates query execution and populates per-pod metric data for consumption by the analyzers.

The VariantAutoscaling (VA) Object

The VariantAutoscaling (short name: `va`) is the core CRD that represents an autoscaling configuration for a single InferencePool variant. API group: `llmd.ai/v1alpha1`.

Spec

Field	Type	Default	Description
scaleTargetRef	object reference	(required)	Reference to the scalable resource (Deployment, StatefulSet, or LeaderWorkerSet)
modelID	string	(required)	Unique identifier linking this VA to an InferencePool; all VAs with the same value form a variant group
minReplicas	integer	1	Lower bound on replica count. Set to 0 to enable scale-to-zero
maxReplicas	integer	2	Upper bound on replica count
variantCost	string	"10.0"	Cost per replica for this variant (used in cost-aware optimization)

Validation: `minReplicas <= maxReplicas`.

Status

Field	Type	Description
desiredOptimizedAlloc.numReplicas	integer	Target replica count from the optimization engine
desiredOptimizedAlloc.lastRunTime	timestamp	Timestamp of the last optimization run
desiredOptimizedAlloc.accelerator	string	<i>(Deprecated)</i> Accelerator type
actuation.applied	boolean	Whether the scaling decision was successfully applied
conditions	list of conditions	Observable state conditions

Conditions

Type	Description
TargetResolved	Whether the scale target (Deployment/LWS) was found
MetricsAvailable	Whether vLLM metrics are available from Prometheus
OptimizationReady	Whether the optimization engine ran successfully

Multi-Variant InferencePool Example

Multiple VA objects with the same `modelID` represent variants of the same InferencePool, each served on different hardware. The optimizer considers all variants together:

```

# Variant 1: Cost-effective GPU
apiVersion: llmd.ai/v1alpha1
kind: VariantAutoscaling
metadata:
  name: llama-8b-a10g
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: llama-8b-a10g
  modelID: "meta/llama-3.1-8b"
  minReplicas: 1
  maxReplicas: 10
  variantCost: "5.0"
---
# Variant 2: High-performance GPU
apiVersion: llmd.ai/v1alpha1
kind: VariantAutoscaling
metadata:
  name: llama-8b-a100
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: llama-8b-a100
  modelID: "meta/llama-3.1-8b"
  minReplicas: 0
  maxReplicas: 5
  variantCost: "15.0"

```

Configuration

WVA uses a layered configuration system. Configuration sources are resolved in the following precedence: **CLI flags > environment variables > config file > ConfigMap > defaults**.

ConfigMaps provide dynamic runtime configuration and must have the label `app.kubernetes.io/name: workload-variant-autoscaler`. They are watched for changes and reloaded dynamically. Namespace-local ConfigMaps override global ConfigMaps, enabling per-namespace tuning.

Scaling Engine Configuration

General Architecture

The engine is configured through the `wva-saturation-scaling-config` ConfigMap. This ConfigMap controls which analyzer is used, the thresholds for scaling decisions, and the optimizer mode.

Saturation Analyzer Configuration

The `wva-saturation-scaling-config` ConfigMap controls saturation analyzer behavior. Per-InferencePool overrides are supported using `model_id` and `namespace` fields.

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: wva-saturation-scaling-config
  labels:
    app.kubernetes.io/name: workload-variant-autoscaler
data:
  default: |
    # Analyzer selection: "" or omit for saturation-percentage-based / V1 (default),
    # "saturation" for saturation-token-based / V2 (experimental).
    analyzerName: "saturation"

    # saturation-percentage-based thresholds (used when analyzerName is "" or omitted)
    kvCacheThreshold: 0.80    # KV cache saturation threshold (0-1)
    queueLengthThreshold: 5   # Queue depth saturation threshold
    kvSpareTrigger: 0.10     # Scale-up trigger: spare KV capacity below this
    queueSpareTrigger: 3     # Scale-up trigger: spare queue capacity below this

    # saturation-token-based thresholds (used when analyzerName is "saturation")
    scaleUpThreshold: 0.85    # Utilization above this triggers scale-up
    scaleDownBoundary: 0.70   # Utilization below this allows scale-down

    # Optimizer mode
    enableLimiter: false     # false: CostAwareOptimizer, true: GreedyByScoreOptimizer
    priority: 1.0            # Model priority for GreedyByScore (default 1.0)

# Per-model override example
  llama-override: |
    model_id: "meta/llama-3.1-8b"
    namespace: "llm-inference"
    scaleUpThreshold: 0.90
    scaleDownBoundary: 0.75

```

[!NOTE] There are two equivalent ways to select the token-based (V2) saturation engine. The scalar `analyzerName: "saturation"` shown above is the backward-compatible form. The newer list form, used in the HPA + WVA guide, is equivalent:

```

yaml analyzers: - name: saturation

```

Populating the `analyzers` list also enables per-analyzer score and threshold overrides. Omit both to fall back to the percentage-based (V1) engine.

SLO Analyzer (Queueing Model) Configuration -- Experimental

The SLO analyzer is activated by the presence of a `wva-queueing-model-config` ConfigMap with a `"default"` entry. When present, it takes priority over the saturation analyzer.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: wva-queueing-model-config
  labels:
    app.kubernetes.io/name: workload-variant-autoscaler
data:
  default: |
    sloMultiplier: 3.0      # k factor (target utilization rho = 1 - 1/k)
    tuningEnabled: true     # Enable Kalman filter for online parameter learning

# Per-model override with explicit SLO targets
llama-slo: |
  model_id: "meta/llama-3.1-8b"
  namespace: "llm-inference"
  targetTTFT: 500          # Target TTFT in milliseconds
  targetITL: 100           # Target ITL in milliseconds
```

Scale-to-Zero Configuration

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: wva-model-scale-to-zero-config
  labels:
    app.kubernetes.io/name: workload-variant-autoscaler
data:
  default: |
    enable_scale_to_zero: true
    retention_period: "10m"  # Period to check for model inactivity

# Disable scale-to-zero for a specific model
llama-override: |
  model_id: "meta/llama-3.1-8b"
  enable_scale_to_zero: false
```

Metric Collection Configuration

Prometheus connectivity is configured via the unified config:

Setting	Source	Default	Description
Prometheus URL	Config file / CLI	--	Base URL for Prometheus API
TLS config	Config file	--	TLS certificates for Prometheus connection

VA Object Configuration

The VA object itself is the primary user-facing configuration surface. See the VA Object section for the full spec reference.

Key configuration decisions per VA:

- `minReplicas: 0` enables scale-to-zero for this variant.

- **variantCost** determines scaling preference order: cheaper variants scale up first, expensive variants scale down first.
- Multiple VAs with the same **modelID** form a variant group for an InferencePool, optimized together.

CLI Flags

Key controller flags:

Flag	Default	Description
--config-file	--	Path to configuration file
--metrics-bind-address	:8080	Metrics endpoint bind address
--health-probe-bind-address	:8081	Health probe bind address
--leader-elect	false	Enable leader election for HA
--watch-namespace	--	Restrict to a single namespace
--leader-election-lease-duration	60s	Leader election lease duration
--leader-election-renew-deadline	50s	Leader election renew deadline
--rest-client-timeout	60s	Kubernetes API client timeout

Migrating off WVA

WVA is deprecated; the target is the KEDA + EPP path. There is no automated conversion, because the two designs express scaling intent differently: WVA computes one desired replica count per variant globally, while KEDA + EPP scales each Deployment against its own EPP signal.

To migrate:

1. Pick the signal that matches your workload from the **signal table** — queue depth, pool saturation, token backlog, or estimated latency — and follow the corresponding guide to create a **ScaledObject** per model-server Deployment.
2. Delete the WVA-managed HPAs (those carrying `llm-d.ai/managed: "true"`), or the **VariantAutoscaling** objects if you are still on the VA flavor, *before* the **ScaledObject** is created. KEDA generates its own HPA, and two HPAs targeting one Deployment conflict.
3. Uninstall the WVA controller, and Prometheus Adapter if it was installed for WVA and nothing else uses it.

What you lose in the move is cost-aware placement across variants — the choice to satisfy demand on cheaper hardware; see **Limitations**. If your deployment depends on that, stay on WVA for now and track the autoscaling guides for a replacement.

Sharing a contended accelerator budget between pools does *not* need WVA: use Kueue-based replica rebalancing, which gates replica pods against per-model quota floors below the HPA and so composes with any of the KEDA + EPP signals.

Migration from VA to HPA + WVA

[!NOTE] This section is historical. It documents migration *within* the now-deprecated WVA path, from the VariantAutoscaling CRD to WVA-published metrics. New deployments should use KEDA + EPP instead.

The HPA + WVA approach replaces the VariantAutoscaling (VA) CRD with standard Kubernetes HPA objects that consume the wva_desired_replicas external metric published by WVA. This removes the need to manage VA resources and makes scaling intent visible through the standard kubectl get hpa surface.

Key Differences

	VA (Superseded)	HPA + WVA (Deprecated)
Scaling object	VariantAutoscaling CRD	Standard HorizontalPodAutoscaler
Scale target	Managed by WVA via scaleTargetRef in VA spec	Managed by HPA via standard scaleTargetRef
WVA discovery	WVA watches VariantAutoscaling resources	WVA discovers HPA via llm-d.ai/managed: "true" annotation
Scaling metric	WVA directly sets replica counts	WVA publishes wva_desired_replicas ; HPA acts on it
CRD requirement	Requires llmd.ai/v1alpha1 VariantAutoscaling CRD	No custom CRDs required
Observability	kubectl get va	kubectl get hpa

Migration Steps

1. **Delete existing VA objects** for each deployment you are migrating:

```
bash kubectl delete va <va-name> -n <namespace>
```

[!NOTE] Deleting a VA object stops WVA from managing that deployment's replica count. Scale the deployment to its desired replica count manually before deleting the VA if you need to avoid disruption.

2. **Create HPA objects** with the llm-d.ai/managed: "true" annotation. WVA discovers these and publishes wva_desired_replicas labeled with variant_name and exported_namespace :

```
yaml apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: <deployment-name> namespace: <namespace>
annotations: llm-d.ai/managed: "true" spec: scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: <deployment-name>
minReplicas: 1 maxReplicas: <max> metrics: - type: External external: metric: name: wva_desired_replicas selector: matchLabels:
variant_name: <deployment-name> exported_namespace: <namespace> target: type: AverageValue averageValue: "1"
```

The variantCost previously set on the VA can be carried over as an annotation: llm-d.ai/variant-cost: "<cost>" . WVA reads this annotation to preserve cost-aware optimization across variants.

3. **Verify** that WVA is discovering the HPA and publishing the metric:

```
bash kubectl get hpa <deployment-name> -n <namespace> kubectl get --raw  
"/apis/external.metrics.k8s.io/v1beta1/namespaces/<namespace>/wva_desired_replicas" | jq .
```

4. **Remove the VA CRD** once all VAs have been migrated and the HPAs are healthy:

```
bash kubectl delete crd variantautoscalings.llmd.ai
```

See the [HPA + WVA guide](#) for a complete end-to-end setup walkthrough including Prometheus Adapter and WVA controller installation.

KV-Cache Offloading

KV-Cache offloading extends the effective cache capacity beyond GPU HBM by moving KV blocks to lower-cost tiers like CPU DRAM and shared storage.

llm-d works with any KV-cache connector compatible with vLLM or SGLang. Two integration patterns are supported:

- **Native (vLLM OffloadingConnector)** — vLLM's built-in offloading path. Targets CPU RAM directly, and a shared filesystem via the llm-d FS backend.
- **Out-of-tree connectors** — third-party cache engines (e.g., LMCache, Mooncake, NVIDIA KVBM) that plug into the model server through its KV-cache connector API and own their own indexing, memory management, and storage.

[!NOTE] Pairs with the **llm-d Router's** cache-aware routing — the Router (specifically the **EPP**) picks replicas that can reuse cached blocks; offloading grows the cache each replica can hold.

Functionality

Transformer inference computes Key and Value tensors during prefill, then reuses them during decode. For long contexts or repeated prefixes (system prompts, agentic loops, multi-turn conversations), recomputing these tensors wastes significant GPU cycles. KV-Cache offloading addresses two scaling limitations:

1. **Capacity** — GPU HBM is limited (tens of GB per GPU). CPU RAM adds another order of magnitude, but storage can scale nearly infinitely.
2. **Sharing** — Local caches are isolated per model-server instance. Shared storage enables cross-node cache reuse, faster scale-up for new replicas, and persistence across pod restarts.

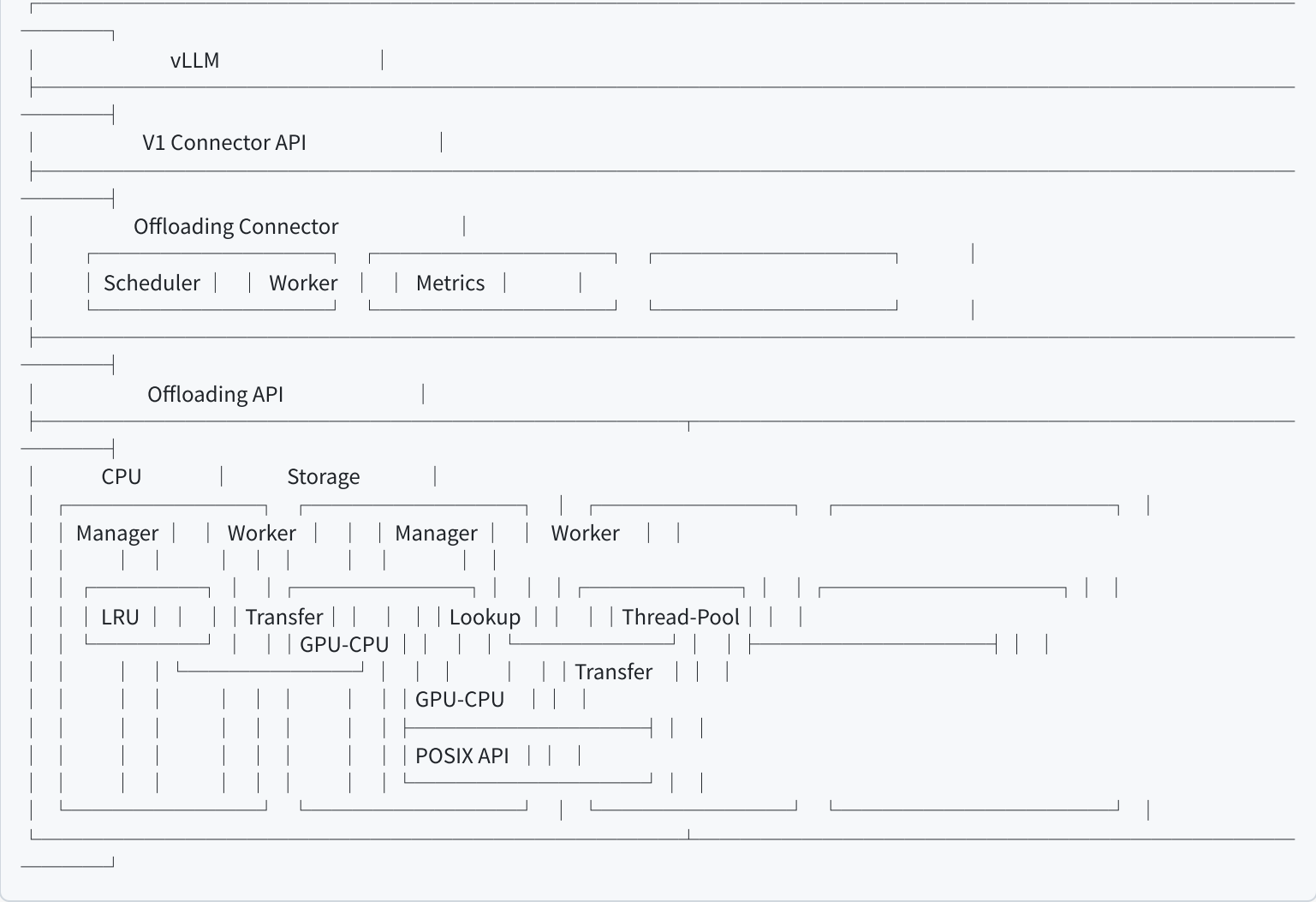
The offloading system generally operates asynchronously. Writes to lower tiers happen in the background without blocking inference. Reads from storage still require waiting, but loading cached blocks is typically faster than recomputing them—up to 16x faster for long prompts.

Architecture

The two integration patterns map to distinct architectures.

Native (vLLM OffloadingConnector)

The native path lives entirely inside the vLLM stack. The `OffloadingConnector` dispatches blocks to either the CPU tier or the shared-storage tier:



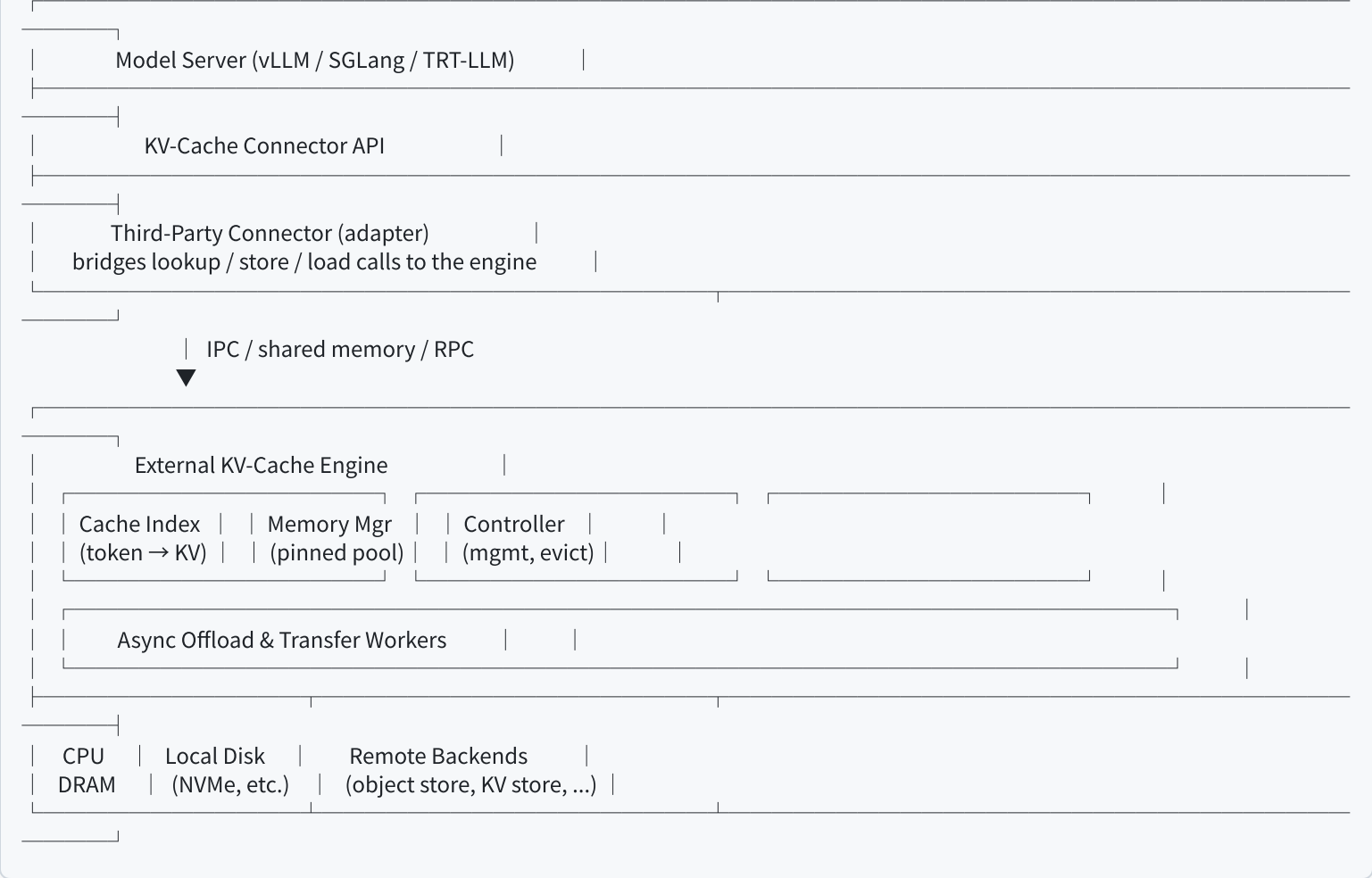
Target	Latency	Capacity	Scope	Best For
CPU RAM	Low	~250GB/GPU	Per-node	High-frequency reuse, preemption recovery
Shared Storage	Higher	TB+	Cross-cluster	Cross-node sharing, persistence, massive scale

Today, the two targets operate as independent options — choose one offloading target based on your workload requirements.

[!NOTE] **Hierarchical KV-cache offloading** — where blocks flow GPU → CPU → Storage as a unified tiered hierarchy — is under active development in the native path.

Out-of-tree Connectors

Third-party connectors (see Other Connectors) adapt an external KV-cache engine to the model server through its KV-cache connector API. The same pattern is present across major serving stacks — vLLM's V1 Connector API, SGLang's HiCache, and TensorRT-LLM's KV Cache Connector API. Unlike the native path, the cache logic — indexing, memory management, tiering, eviction, and remote storage — lives in a separate engine, often a distinct process or service:



This pattern trades deployment complexity for flexibility: the external engine is independently versioned, can coordinate across multiple inference replicas, and typically supports a wider range of storage backends.

Components

vLLM Native CPU Offloading

vLLM's `OffloadingConnector` manages the GPU-to-CPU tier. It uses a hardware DMA engine for high-throughput transfers with minimal GPU core interference. The connector:

- Allocates pinned CPU memory for staging buffers
- Transfers KV blocks asynchronously using GPU DMA, avoiding interference with GPU compute cores.
- Uses a contiguous memory layout (introduced in vLLM 0.12.0) that groups all layers into single physical blocks, improving transfer throughput by 4-5x

CPU offloading requires no external infrastructure. The simplest way to enable it is via vLLM's dedicated top-level flags:

```
--kv-offloading-backend native --kv-offloading-size <size_in_GB>
```

Equivalent to passing `--kv-transfer-config '{"kv_connector":"OffloadingConnector","kv_role":"kv_both",...}'` directly — the top-level flags are a convenience wrapper around the connector JSON.

llm-d Filesystem Connector

The `llmd_fs_backend` is a storage backend that plugs into vLLM's `OffloadingConnector`. It stores KV blocks as files on a shared filesystem and loads them back on demand, using the filesystem directory as the index of cached blocks.

Key properties:

- **Filesystem agnostic** — Relies on standard POSIX file operations, works with any filesystem (CephFS, Lustre, IBM Storage Scale, local NVMe)
- **KV sharing across instances and nodes** — Multiple vLLM servers reuse cached prefixes by accessing the same shared path
- **Persistence across restarts** — KV data survives pod restarts, rescheduling, and node failures
- **Fully asynchronous I/O** — Reads and writes run without blocking the inference path
- **High throughput via parallelism** — I/O operations parallelized across worker threads with NUMA-aware scheduling
- **Minimal GPU interference** — Uses GPU DMA by default, reducing interference with compute kernels

[!NOTE] The storage connector does not handle cleanup or eviction. Storage capacity management must be handled by the underlying storage system or an external controller. A reference implementation, the PVC Evictor, can automatically clean up old KV-cache files when storage thresholds are exceeded.

For implementation details and advanced configuration, see the [llm-d FS backend documentation](#).

MooncakeStoreConnector

Mooncake Store is a distributed KV cache system built on the Mooncake Transfer Engine. A centralized Mooncake Master manages object metadata, replica placement, leases, and eviction, while the Transfer Engine moves the underlying bytes between registered memory regions using RDMA or TCP. Together, they pool CPU DRAM across nodes—and optionally SSD/NVMe—into a shared cache tier without requiring a shared filesystem or PVC for the cache data path.

The `MooncakeStoreConnector` integrates this store with vLLM's V1 Connector API. It converts vLLM's content-addressed KV cache chunks into Mooncake object keys and maps their tensor data to the byte ranges stored and transferred by Mooncake. This allows multiple vLLM instances to retrieve and reuse cached prefixes, reducing redundant prefill computation.

- At the vLLM boundary, cache data is identified as content-addressed KV-cache blocks or chunks derived from vLLM block hashes.
- At the Mooncake boundary, each content-addressed key identifies a variable-length byte object. The connector maps that object to one or more registered memory ranges containing the corresponding K/V tensor data.

[!NOTE] `MooncakeStoreConnector` (distributed cache offloading) is distinct from `MooncakeConnector` (point-to-point KV transfer for P/D disaggregation). They share the same Transfer Engine for RDMA data movement but serve different purposes and are configured independently. They can be composed via vLLM's `MultiConnector` when both P/D disaggregation and shared cache offloading are needed.

Architecture

The system consists of four components:

- **Mooncake Master** — Centralized metadata service managing keyed objects, their replicas and placement, leases, and eviction. It is unaware of vLLM's KV-cache block semantics. Deployment manifests are provided in `helpers/mooncake-master-store/` . Required by both deployment modes.
- **Mooncake Client** — A standalone process that allocates CPU DRAM and optionally SSD storage, registers those resources with the Master, and serves RDMA read/write requests from vLLM ranks. Only used in standalone-store mode. Deployment manifests are provided in `helpers/mooncake-client/` .
- **Mooncake Transfer Engine** — Byte-oriented data mover that transfers data between registered GPU or CPU memory regions and the distributed DRAM/SSD pool using RDMA or TCP.
- **MooncakeStoreConnector** (in each vLLM process) — Converts vLLM content-addressed cache chunks into Mooncake object keys and maps each object to one or more physical memory address-and-size ranges.

Two deployment modes are available:

Mode	Description	global_segment_size	Components
Embedded	Each vLLM rank contributes CPU DRAM to the distributed pool in-process	> 0 (e.g., "80GB")	Master + vLLM
Standalone-store	External Mooncake Client process owns the CPU + SSD pool; vLLM ranks are pure requesters	0	Master + Client + vLLM

Embedded mode is simpler to deploy — the DRAM pool scales automatically with the number of vLLM instances. Standalone-store mode decouples storage from compute and enables the SSD/NVMe tier by delegating resource ownership to the Mooncake Client. An example of when you might need the standalone mode as compared to the embedded mode, would be if the CPU being contributed to the distributed pool does not belong to the vLLM servers, and so you need some other process to manage the physical memory and disk.

Mooncake Master

The Mooncake Master handles block metadata, eviction, and snapshots. Key configuration parameters (set in `configmap.yaml`):

Parameter	Default	Description
eviction_high_watermark_ratio	0.95	Trigger eviction when pool is 95% full
eviction_ratio	0.05	Evict 5% of pool capacity per cycle
default_kv_lease_ttl	5000	Lease TTL in ms
default_kv_soft_pin_ttl	1800000	Soft pin TTL in ms (30 min)
enable_snapshot	true	Periodic snapshots to PVC for recovery
snapshot_interval_seconds	60	Snapshot frequency

The Master exposes gRPC (port 50051), HTTP metadata (port 8080), and Prometheus metrics (port 9003). See `helpers/mooncake-master-store/` for deployment manifests.

Mooncake Client

The Mooncake Client (`mooncake_client`) is the process that owns physical memory and disk in the distributed pool. It is only used in standalone-store mode.

On startup, the Client allocates a CPU DRAM segment (`--global_segment_size`) and optionally opens a local SSD path for persistence (`--enable_offload=true`). It registers these resources with the Mooncake Master, making them available to the cluster. These allocations are fixed for the lifetime of the process — the Client exposes no runtime API to resize segments, change SSD paths, or toggle offloading. Any change to resource sizing requires restarting the Client. vLLM ranks never communicate with the Client directly — when a rank needs to store or load a block, it queries the Master for the block's location, and the Transfer Engine then moves the bytes peer-to-peer over RDMA between the rank and the Client's registered memory.

The two-tier storage within the Client works as follows:

- **Writes** land in CPU DRAM synchronously and are asynchronously persisted to SSD.
- **Reads** check DRAM first and fall through to SSD on a DRAM miss.
- **Eviction** is coordinated by the Master — when the DRAM pool hits the high watermark, the Master instructs the Client to spill blocks to SSD.

Decoupling storage from compute means the pool survives vLLM pod restarts, storage can be placed on nodes without GPUs (e.g., CPU-only nodes with large DRAM and NVMe), and the SSD tier is managed in one place per node rather than per GPU. See `helpers/mooncake-client/` for deployment manifests.

Content-Addressable Storage and PYTHONHASHSEED

KV cache blocks are stored with content-addressable keys derived from vLLM's block hash mechanism. Python randomizes its `hash()` seed per process by default — if two vLLM instances compute different hashes for the same input tokens, they can never share cached blocks.

All vLLM instances sharing a Mooncake Store **must** set `PYTHONHASHSEED` to the same fixed value:

```
PYTHONHASHSEED=0 vllm serve ...
```

mooncake_config.json Reference

Each vLLM instance requires a Mooncake configuration file, pointed to by the `MOONCAKE_CONFIG_PATH` environment variable.

Embedded mode:

```
{
  "mode": "embedded",
  "metadata_server": "P2PHANDSHAKE",
  "master_server_address": "mooncake-master-store.mooncake.svc.cluster.local:50051",
  "global_segment_size": "80GB",
  "local_buffer_size": "4GB",
  "protocol": "rdma",
  "device_name": "",
  "enable_offload": false
}
```

Standalone-store mode:

```
{
  "mode": "standalone-store",
  "metadata_server": "P2PHANDSHAKE",
  "master_server_address": "mooncake-master-store.mooncake.svc.cluster.local:50051",
  "global_segment_size": 0,
  "local_buffer_size": "4GB",
  "protocol": "rdma",
  "device_name": "",
  "enable_offload": true
}
```

Field	Description
mode	"embedded" (in-process DRAM pool) or "standalone-store" (external client)
metadata_server	"P2PHANDSHAKE" for direct peer-to-peer metadata exchange
master_server_address	Mooncake Master gRPC endpoint (host:port)
global_segment_size	CPU memory per GPU contributed to pool. Must be > 0 in embedded, 0 in standalone-store
local_buffer_size	Private buffer per GPU for its own operations
protocol	"rdma" (production) or "tcp" (fallback)
device_name	RDMA device name (e.g., "mlx5_0"). Empty string for auto-discovery
enable_offload	Enable SSD/NVMe staging. Must match mooncake_master and mooncake_client flags

Environment Variables

Variable	Default	Description
MOONCAKE_CONFIG_PATH	(required)	Path to mooncake_config.json
PYTHONHASHSEED	(random)	Must be set to same fixed value across all instances sharing the store

For deployment recipes, see the Tiered Prefix Cache Guide — Mooncake Store.

Other Connectors

Out-of-tree engines coexist with the native path through a common integration contract on the llm-d side:

- **Serving-stack side** — each engine is already connector-compatible with one or more of vLLM, SGLang (via HiCache), and TensorRT-LLM, so the model server drives lookups, stores, and loads through its standard KV-cache connector API.
- **Scheduling side** — connectors integrate with llm-d through **KV-Events**: cache mutation notifications that the KV-Cache Indexer consumes to maintain a global view of cache distribution, enabling prefix-aware routing regardless of which backend is in use.

[!NOTE] llm-d's deployment guides cover LMCache and Mooncake Store today. The integration pattern is the same for KVBM and other connector-compatible engines — they work out-of-the-box on the serving-stack side.

For deployment recipes, see the Tiered Prefix Cache Guide.

Configuration

CPU Offloading (vLLM Native)

Flag	Type	Default	Description
--kv-offloading-backend	string	-	Set to <code>native</code> for vLLM's built-in CPU offloading
--kv-offloading-size	integer	-	CPU offloading buffer size in GB (per vLLM instance, across all workers)

For advanced use and older vLLM releases, the equivalent `--kv-transfer-config` JSON form is supported. See the vLLM offloading connector blog for details.

Storage Offloading (llm-d FS Backend)

Field	Type	Default	Description
shared_storage_path	string	<code>/tmp/shared-kv</code>	Base path for KV-cache files
block_size	integer	256	Tokens per file (must be multiple of GPU block size)
threads_per_gpu	integer	64	I/O worker threads per GPU

For the full configuration reference including GDS modes and environment variables, see the llm-d FS backend README.

Examples

CPU Offloading with vLLM


```
args:
- "--model=Qwen/Qwen3-32B"
- "--tensor-parallel-size=2"
- "--kv-offloading-backend=native"
- "--kv-offloading-size=100"
```

Storage Offloading with llm-d FS Backend

```
args:
- "--model=Qwen/Qwen3-32B"
- "--tensor-parallel-size=2"
- "--block-size=16"
- "--distributed-executor-backend=mp"
- "--kv-transfer-config"
- |
{
  "kv_connector": "OffloadingConnector",
  "kv_role": "kv_both",
  "kv_connector_extra_config": {
    "spec_name": "SharedStorageOffloadingSpec",
    "spec_module_path": "llmd_fs_backend.spec",
    "shared_storage_path": "/mnt/kv-cache/",
    "block_size": 256,
    "threads_per_gpu": 64
  }
}
volumeMounts:
- name: kv-cache
  mountPath: /mnt/kv-cache
```

Distributed Offloading with MooncakeStoreConnector

At the vLLM server layer you might serve with the following configurations:

```
args:
- "--model=Qwen/Qwen3-32B"
- "--tensor-parallel-size=2"
- "--kv-transfer-config"
- '{"kv_connector": "MooncakeStoreConnector", "kv_role": "kv_both"}'
env:
- name: MOONCAKE_CONFIG_PATH
  value: /etc/mooncake/mooncake_config.json
- name: PYTHONHASHSEED
  value: "0"
```

For the full `mooncake_config.json` reference, see [MooncakeStoreConnector — mooncake_config.json Reference](#) above.

Metrics

The FS backend populates vLLM's built-in offloading metrics (`vllm:kv_offload_*`) for transfer bytes, time, and size distribution. See the [llm-d FS backend documentation](#) for the full metrics reference.

Performance Considerations

CPU offloading: Should always be enabled if CPU DRAM is larger than GPU HBM space. It has minimal overhead when the cache fits in HBM, and provides significant benefits when it doesn't—recovering preempted requests without recomputation and extending effective cache capacity with low latency.

Storage offloading: Best when cache working set exceeds single-node capacity, when cross-node sharing is valuable (repeated system prompts across replicas, agentic workflows), or when persistence across restarts matters. Storage offloading is most effective when the storage network is fast enough to allow low-latency loads and stores.

Storage selection: The FS backend works with any POSIX-compliant filesystem and is not tied to a specific vendor. Your choice trades off:

- **Sharing scope** — local media (e.g., NVMe SSDs) are per-node only; networked or distributed filesystems (e.g., NFS, CephFS, Lustre, IBM Storage Scale, Weka, cloud file services) enable cross-node reuse.
- **Throughput and latency** — depend on the underlying hardware, network, and filesystem configuration rather than the KV-cache layer; parallel and distributed filesystems generally scale best for large deployments.

Any POSIX filesystem is a candidate; the best choice for a given deployment depends on existing storage infrastructure, required scale, and latency targets.

Block size tuning: Larger `block_size` values (256-512 tokens) improve I/O efficiency but require longer matching prefixes for a cache hit. Match to your typical prefix lengths.

Distributed offloading (MooncakeStoreConnector): Best when cross-instance cache sharing is critical and RDMA networking is available. Eliminates the shared filesystem dependency and provides built-in eviction via the Mooncake Master. Embedded mode is recommended for most deployments; standalone-store mode adds complexity but enables the SSD tier and decouples storage from compute. Requires RDMA for production performance — TCP fallback is available but significantly slower.

Further Reading

- Tiered Prefix Cache Guide — Step-by-step deployment guides
- llm-d KV-Disaggregation Roadmaps — Planned features and improvements across offloading and KV-cache management
- llm-d FS Backend — Implementation details, configuration, and metrics
- vLLM KV Offloading Connector — Deep dive into vLLM's native offloading
- Mooncake Store — Upstream Mooncake project and documentation
- vLLM MooncakeStoreConnector Usage Guide — vLLM-side configuration reference

Endpoint Picker (EPP)

Functionality

The EPP is the "brains" of an llm-d deployment. It is focused on two key objectives:

- **Intelligent routing** - selecting which **model server pod** within an InferencePool should process each inference request, using the internal state of model server pods (KV-cache utilization, prefix cache locality, request queue depth, and active request counts)
- **Fairness and prioritization** - selecting which **inference requests** should run at any given time, enabling consolidation of multiple workloads with varying priorities onto a single set of Model Servers

The EPP is an extensible component that integrates with the proxy layer via Envoy's External Processing (ext-proc) protocol.

[!NOTE] The project provides tools for automatic Envoy installation. However, if you install or configure it yourself, please note that the only supported `request_body_mode` and `response_body_mode` is `FULL_DUPLEX_STREAMED`

When a request arrives at the proxy, the proxy calls the EPP to select a backend endpoint, and the EPP returns the optimal pod address according to the Endpoint Picking Protocol.

Design

Request Flow

The following diagram shows the end-to-end lifecycle of a request as it flows through the EPP plugin pipeline:

![EPP Design](../../assets/epp-design.svg)

The steps are:

1. **Request arrival** -- An inference request arrives at the proxy (Gateway).
2. **External processing** -- The proxy invokes the EPP via the ext-proc protocol, passing the request headers and body to the EPP.
3. **Request handling** -- Parses the request (from e.g. OpenAI format, vllm gRPC) into the internal request data structure.
4. **Flow control** -- If enabled, queues requests and prioritizes and ensures fairness across different tenants within a priority, and holding requests when the pool is "saturated".

5. **Request Scheduling** -- Selecting the optimal endpoint from the available InferencePool, which involves evaluating each request against a configured set of scheduling plugins, such as filters and scorers.
6. **Request Proxying** -- The EPP returns the address of the selected endpoint to the proxy, which then forwards the request to the corresponding model server endpoint.

Asynchronously, the **Data layer** watches the Kubernetes API server for updates to relevant objects like InferencePools and Pods for endpoint discovery. It is also responsible for model servers metrics probing, and maintaining an internal state—such as a prefix cache tree—to inform the request processing components, flow control and request scheduling.

Components

The EPP is modular and pluggable, consisting of the following components:

Ext-Proc Server

The Ext-Proc Server protocol is very well defined & specific, deviations could cause the EPP to become unusable or unstable. Extension is ill-advised. The Ext-Proc is simply the standard interface by which the Proxy talks to the EPP.

Request Handler (Extensible)

The **Request Handler** is the first step of the request flow in the EPP. The key responsibility is to convert the user's request into the internal EPP data structure via the Parser plugin. The EPP provides out-of-the-box parsers for common formats like the OpenAI HTTP and vLLM gRPC.

In addition, users can write a custom parser for their own protocol. The rest of the functionality in EPP is agnostic to the original request protocol, enabling easy adaptation of the EPP to new request formats.

See Request Handling for more details on the design.

Flow Control (Extensible)

Flow control's primary purpose is to manage the admission, queuing, and dispatching of requests to prevent overloading backend model servers while ensuring fairness and priority. Specifically:

- **Admission Control & Throttling:** Rather than allowing the inference pool to be overwhelmed, it intercepts incoming requests and holds them in an in-memory queue if saturation is reached. This is configurable via `SaturationDetector` plugins (such as the `Concurrency` plugin, which is based on active in-flight requests accounting per endpoint).
- **Priority-Based Queuing:** It categorizes traffic into "Priority Bands." Real-time, latency-sensitive tasks (like chat) are prioritized over batch background tasks (like summarization).
- **Resource Fairness:** It prevents "noisy neighbor" scenarios by isolating traffic streams, ensuring a single flow (e.g., a user or application) cannot monopolize all available inference slots. Fairness is configurable via two pluggable policies: `FairnessPolicy` governs the distribution of dispatch opportunities among competing Flows within the same priority band (e.g., Round Robin), and `OrderingPolicy` plugins customize ordering of requests within a flow (e.g., FIFO, SLO-based).
- **Dynamic Dispatching:** It monitors saturation to send requests to model servers only when they are ready to process them.

See Flow Control for more details on the design.

Request Scheduler (Extensible)

The scheduler acts as the core decision-making engine for intelligent request routing. It operates through a modular Filter → Score → Pick plugins pipeline orchestrated by a ProfileHandler plugin, allowing it to evaluate and select the most suitable model server endpoints for each incoming request.

By leveraging custom plugins at each stage—filtering out unavailable endpoints, scoring them based on metrics like "least-loaded" or "affinity," and picking final candidates—the EPP ensures high performance and efficient resource distribution across inference pools.

See Request Scheduling for more details on the design.

Data Layer (Extensible)

The **Data Layer** operates asynchronously, consuming and storing data from a variety of sources:

- Kube API Server about which pods are active in the InferencePool
- Model Servers about the current internal state (running requests, kv cache utilization)
- In-memory data structures, such as prefix cache trees for prefix-aware routing
- "Consultant" sidecars like the latency predictor, the kv-indexer or tokenizer for advanced routing

Other modules in the EPP consult the **Data Layer** during request processing.

See Data Layer for more details on the design.

How to Configure

The EPP is configured by passing an EndpointPickerConfig YAML to the binary via the `--config-file` or `--config-text` command-line arguments. This configuration defines the plugins, feature gates, and operational parameters that govern request handling, flow control, and scheduling.

[!IMPORTANT] The configuration is only read on startup. Any updates to the configuration require a restart of the EPP process to take effect.

For a detailed guide on the configuration schema, mental model, and examples, see the Configuration Guide.

Request Handler

Functionality

The Request Handler manages the lifecycle of an inference request before and after the request scheduling phase within the EPP. It handles parsing the request payload, preparing and managing state for the Request Scheduler, interacting with Flow Control and processing the response from the model server.

Design

Architecture Overview

```

flowchart TD
    %% Style Definitions
    classDef highlight fill:#e1f5fe,stroke:#01579b,stroke-width:2px;
    classDef accent fill:#fff8e1,stroke:#ffb300,stroke-width:2px;
    classDef danger fill:#ffebee,stroke:#c62828,stroke-width:2px;
    classDef flowcontrol fill:#fff3e0,stroke:#ff9800,color:#000

    %% Nodes
    Req([Incoming Request])
    Client([Return to Client])
    Reject[Reject Request]:::danger

    subgraph EPP ["EPP Request Handling"]
        direction TB
        Parse[Parser]:::highlight
        Prep[DataProducer]:::highlight
        FC[FlowControl]:::flowcontrol
        Admit[Admitter]:::highlight
    end

    Scheduler[Scheduler]:::accent

    %% Connections
    Req --> Parse
    Parse -->|InferenceRequest| FC
    FC --> Prep
    FC -->|"Rejected"| Reject
    Prep --> Admit
    Admit -->|"Admit"| Scheduler
    Admit -->|"Deny"| Reject
    Scheduler --> Endpoint[Selected Endpoint]
    Endpoint --> Client
  
```

Core Components

- **Parser**: Responsible for parsing the incoming request to a structured internal representation consumable by the request scheduler, and parsing the response to extract usage data if reported by the model server.
- **DataProducer**: A pluggable extension that allows customizing request pre-processing and producing per-request state needed for scheduling, such as tokenization, prefix-cache matches, predicted processing latency etc.
- **Admitter**: Decides whether to admit a request based on criteria like latency SLOs. Runs after dataProducer but before request scheduling. Requests failing admission are rejected, while admitted requests proceed to the request scheduling phase.

Advanced Hooks

The framework also supports advanced, auto-resolved hooks in the request control layer. If a plugin implements these interfaces, it is automatically wired into the execution flow:

- **PreRequest** : Executes before the request is processed (e.g., for incrementing in-flight counts).
- **ResponseHeaderProcessor** : Executes when response headers are received from the backend.
- **ResponseBodyProcessor** : Executes during response streaming (e.g., for usage tracking on completion).

[!NOTE] In practice, these interfaces are often implemented by Data Producers to maintain state or track metrics across the request lifecycle. For example, the `predicted-latency-producer` implements these hooks to track request latency.

Concrete Plugins

Parsers

Parser plugins understand the payloads of requests and responses. This is key for features like prefix-cache aware scheduling and response usage tracking.

- **openai-parser** : The default parser supporting the OpenAI API. Supported endpoints: `/conversations` , `/responses` , `/chat/completions` , `/completions` , `/embeddings` .
- **vllmgrpc-parser** : Handles requests for the vLLM gRPC API. Supported methods: `Generate` , `Embed` .
- **passthrough-parser** : Model-agnostic parser that passes content through without interpretation. Note that payload-related scheduling (e.g., `prefix-cache-scorer`) is not supported with this parser.

Admitter Plugins

- **latency-slo-admitter** : Rejects sheddable requests (`priority < 0`) when no endpoint can meet latency SLO constraints.

Data Producers

- **predicted-latency-producer** : Trains XGBoost models via a sidecar and generates per-endpoint TTFT/TPOT predictions. It calculates SLO headroom, collects training data, and tracks per-endpoint running request queues.
- **inflight-load-producer** : Tracks the number of in-flight requests and estimated tokens for each endpoint. It increments counts in `PreRequest` and decrements them in `ResponseBodyProcessor` on end-of-stream.

- **approx-prefix-cache-producer** : Prepares data for approximate prefix cache aware scheduling by hashing prompts in blocks and matching them against an indexer of cached prefixes on servers.

Metrics & Observability

The Request Handling subsystem exposes metrics tracking request volume, success, latency, and token usage. Unless otherwise noted, request and latency metrics carry the label set `{model_name, target_model_name, fairness_id, priority}`.

Request Volume & Success

Metric	Type	Description	Labels
llm_d_epp_request_total	Counter	Total request count	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_error_total	Counter	Total error count	model_name , target_model_name , fairness_id , priority , error_code
llm_d_epp_request_running	Gauge	Current active running requests	model_name , target_model_name , fairness_id , priority

Latency & SLOs

Metric	Type	Description	Labels
llm_d_epp_request_duration_seconds	Histogram	End-to-end response latency	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_ntpot_seconds	Histogram	Normalized Time Per Output Token (NTPOT)	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_ttft_seconds	Histogram	Time to first token (TTFT)	model_name , target_model_name , fairness_id , priority , streaming
llm_d_epp_request_streaming_tpot_seconds	Histogram	Average time per output token (streaming)	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_streaming_itl_seconds	Histogram	Inter-token latency (streaming)	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_predicted_ttft_seconds	Histogram	Predicted TTFT	plugin_name , plugin_type , model_name , target_model_name
llm_d_epp_request_ttft_prediction_duration_seconds	Histogram	Time spent predicting TTFT	plugin_name , plugin_type , model_name , target_model_name
llm_d_epp_request_predicted_tpot_seconds	Histogram	Predicted TPOT	plugin_name , plugin_type , model_name , target_model_name
llm_d_epp_request_tpot_prediction_duration_seconds	Histogram	Time spent predicting TPOT	plugin_name , plugin_type , model_name , target_model_name
llm_d_epp_request_slo_violation_total	Counter	Total count of requests violating SLO	plugin_name , plugin_type , model_name , target_model_name , type

Payload & Token Usage

Metric	Type	Description	Labels
llm_d_epp_request_size_bytes	Histogram	Incoming request body size in bytes	model_name , target_model_name , fairness_id , priority
llm_d_epp_response_size_bytes	Histogram	Outgoing response body size in bytes	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_input_tokens	Histogram	Input token count per request	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_output_tokens	Histogram	Output token count per request	model_name , target_model_name , fairness_id , priority
llm_d_epp_request_cached_tokens	Histogram	Distribution of prompt tokens read from cache	model_name , target_model_name , fairness_id , priority

Note: Response-level metrics (response sizes, output tokens, NTPOT) require Envoy body mode to be set to `Buffered` or `Streamed` . For vLLM streaming responses with usage data, include `stream_options: {"include_usage": true}` in the request.

Processing Overhead, Model Rewrite & Request Gauges

Metric	Type	Description	Labels
llm_d_epp_request_processing_duration_seconds	Histogram	Time from request receipt until request body has been handled	<i>None</i>
llm_d_epp_response_processing_duration_seconds	Histogram	EPP response processing latency	<i>None</i>
llm_d_epp_inference_request_metric	Gauge	Consolidated gauge for request metrics	plugin_name , plugin_type , model_name , target_model_name , type
llm_d_epp_model_rewrite_decisions_total	Counter	Total number of model rewrite decisions	model_rewrite_name , model_name , target_model

In-Flight Load

Emitted when an `InFlightLoadProducer` is configured:

Metric	Type	Description	Labels
llm_d_epp_inflight_requests	Gauge	Requests currently in flight on each endpoint	endpoint_name , namespace , producer_name , fairness_id , priority
llm_d_epp_inflight_tokens	Gauge	Tokens currently in flight on each endpoint (uncached prompt tokens, optionally plus estimated output)	endpoint_name , namespace , producer_name , fairness_id , priority

ext_proc Streams

Opt-in metrics covering the ext_proc gRPC stream lifecycle (enabled via --enable-grpc-stream-metrics):

Metric	Type	Description	Labels
llm_d_epp_extproc_streams_inflight	Gauge	Number of ext_proc gRPC streams currently open	<i>None</i>
llm_d_epp_extproc_stream_duration_seconds	Histogram	Duration an ext_proc gRPC stream stays open in seconds	<i>None</i>
llm_d_epp_extproc_streams_total	Counter	Total ext_proc gRPC streams completed by gRPC status code	code

Flow Control

The Flow Control layer within the EPP is a critical mechanism for pool defense and multi-tenancy. It protects the pool of model servers from overload by shifting intelligent queuing to the gateway, enforcing strict priority and tenant-aware fairness.

[!IMPORTANT] EPP Flow Control is currently behind the `flowControl` feature gate. You must explicitly enable it in your `EndpointPickerConfig` to use these capabilities.

The LLM Queuing Problem

Traditional API gateways rate-limit by Requests Per Second (RPS), assuming every request consumes roughly the same resources. In LLM serving, this approach is ill-suited. Resource consumption is driven by long input contexts and the unpredictable **autoregressive decode loop** (output tokens). A single request can consume orders of magnitude more compute and memory than another, making serving capacity highly variable per request. Static RPS limits either starve the pool by being too conservative or cause overload by being too aggressive.

Without Flow Control, the following issues arise:

- **The Noisy Neighbor:** A single tenant sending massive prompts can monopolize an endpoint's KV cache and queue, starving others.
- **Scheduling Regret:** Once a request is dispatched to a model server's local queue, the EPP cannot move it. Premature routing locks requests into suboptimal candidates, preventing them from utilizing a better endpoint that may free up soon.
- **Priority Inversion:** Model servers batch based on arrival or sequence length, not business priority. Critical real-time requests can get stuck behind large offline batches.
- **Resource Asymmetry:** A single request with a massive context can consume more resources than hundreds of short requests.

By shifting queuing to the EPP, we can govern physical capacity (KV cache, queue saturation) rather than just raw request rates or connection counts.

Architecture Overview

The Flow Control layer intercepts requests at the EPP and holds them in centralized, policy-aware queues when the backend pool is saturated. This prevents requests from piling up in isolated endpoint queues where they cannot be dynamically re-routed or prioritized.

Queuing Topology & The 3-Tier Dispatch

To understand how policy plugins act, it helps to visualize the queuing topology as a grid defined by two dimensions: **Priority and Flow (Fairness ID)**.

1. **Flow Identification:** Every request is assigned a `FlowKey` — a tuple of `(FairnessID, Priority)`.
 - `FairnessID` is extracted from the `x-llm-d-inference-fairness-id` header (e.g., tenant ID).
 - `Priority` is derived from the `InferenceObjective`.
2. **Separate Queues:** Each unique `FlowKey` maps to its own in-memory queue.

A **Priority Band** is a logical grouping of all queues (flows) that share the same external priority value. The system services bands in strict order, but cycles through the queues *within* a band according to the Fairness Policy.

The system selects the next request to dispatch by traversing these queues in three strict tiers:

- **Tier 1: Priority (Band Selection):** The system always services the highest-priority band first. This is hardcoded and not pluggable. It yields all queues belonging to that priority level.
- **Tier 2: Fairness (Flow Selection):** Within that priority band, the **Fairness Policy** plugin determines which specific flow (tenant) gets the next turn.
- **Tier 3: Ordering (Item Selection):** Finally, within that selected flow's queue, the **Ordering Policy** plugin determines which individual request to serve.

Client Integration: Tagging Traffic

To place requests into the correct queues, clients must attach specific headers to the incoming HTTP request.

Here is an example of how to target a specific `InferenceObjective` and ensure fair resource sharing among tenants:

```
curl -X POST http://${IP}:${PORT}/v1/completions \  
-H 'Content-Type: application/json' \  
-H 'x-llm-d-inference-fairness-id: tenant-a' \  
-H 'x-llm-d-inference-objective: premium-traffic' \  
-d '{  
  "model": "default-model",  
  "prompt": "Say hello"  
}'
```

[!WARNING] **Trust Boundary:** In a production system, allowing end-users to self-assert their tenant ID or traffic priority (`premium-traffic`) is an abuse vector. In production, these headers should be stripped from external requests and injected by an upstream trusted API gateway, identity provider, or Envoy AuthZ filter based on the API key.

[!TIP] **Fallbacks:**

- If the **Fairness ID** header is missing, the request falls back to a single global bucket for that priority level.
- If the **Inference Objective** header is missing, or references an objective without a priority set, the request defaults to **Priority 0**.

Two Levels of Priority

The system distinguishes between two levels of priority:

1. **External Priority (Traffic Classes):** Coarse-grained prioritization across tenants and workloads (e.g., Premium vs. Best-Effort). This is governed by `InferenceObjective` and acts at **Tier 1**. Negative values are allowed and designate requests as "shedddable". Unlike legacy admission mode (which immediately drops negative priority requests upon saturation), Flow Control admits them into queues but subjects them to operator-configured band capacity limits, enabling controlled load shedding by strictly bounding lower-priority queues.
2. **Internal Priority (Request Ordering):** Fine-grained prioritization of requests *within a single flow* (e.g., Earliest Deadline First). This is governed by the `OrderingPolicy` and acts at **Tier 3**.

To visualize how these priority tiers and flows interact, the following diagram represents the centralized queuing topology in the EPP:

flowchart TD

%% Custom Styles

classDef client fill:#e3f2fd,stroke:#b0bec5,stroke-width:2px,color:#263238,font-weight:bold

classDef premium fill:#e3f2fd,stroke:#1e88e5,stroke-width:2px,color:#0b3c5d

classDef standard fill:#f3e5f5,stroke:#8e24aa,stroke-width:2px,color:#4a148c

classDef batch fill:#ffebee,stroke:#e53935,stroke-width:2px,color:#b71c1c

classDef logic fill:#fff8e1,stroke:#ffb300,stroke-width:2px,color:#ff6f00,font-weight:bold

classDef gate fill:#fff3e0,stroke:#e65100,stroke-width:3px,color:#e65100,font-weight:bold,stroke-dasharray: 5 5

classDef endpoint fill:#e8f5e9,stroke:#43a047,stroke-width:2px,color:#1b5e20

classDef scheduler fill:#e0f7fa,stroke:#00acc1,stroke-width:2px,color:#006064,font-weight:bold

Client(["Incoming HTTP Connections"]):::client --> AdmissionController

subgraph EPP["EPP Flow Control Layer"]

AdmissionController["1. Admission Controller
Assign FlowKey"]:::logic

subgraph B100["Band 100: Highest QoS"]

direction TB

Q1(["Premium-A
Ordering: FCFS"]):::premium

F1(["Fairness:
Round-Robin"]):::logic

Q1 --> F1

end

subgraph B0["Band 0: Standard QoS"]

direction TB

Q2(["Standard-A
Ordering: FCFS"]):::standard

Q3(["Standard-B
Ordering: FCFS"]):::standard

F0(["Fairness:
Round-Robin"]):::logic

Q2 --> F0

Q3 --> F0

end

subgraph BM10["Band -10: Best Effort / BG"]

direction TB

Q4(["Batch-A
Ordering: FCFS"]):::batch

FM1(["Fairness:
Round-Robin"]):::logic

Q4 --> FM1

end

%% Routing

AdmissionController -->|"Pri: 100"| Q1

AdmissionController -->|"Pri: 0"| Q2

AdmissionController -->|"Pri: 0"| Q3

AdmissionController -->|"Pri: -10"| Q4

%% Dispatcher

Disp["2. Strict Priority Dispatcher"]:::logic

F1 == "1. Always First" ==> Disp

F0 -- "2. If Band 100 Empty" --> Disp

FM1 -. "3. If Band 0 Empty" .-> Disp

%% Gatekeeper

Gate["3. Saturation Detector
(Example: concurrency-detector)
Pool Saturation: 44/45 (98%)"]:::gate

Disp -->|"Selects Next Req"| Gate

%% Explicit label syntax forces a clean loopback curve

Gate -.->|"Pool Saturated
(Head-of-Line Block)"| Disp

Sched["4. Late-Binding Scheduler
(High Affinity Routing)"]:::scheduler

Gate == "Capacity Available (Saturation < 100%)
 (Dispatch)" ==> Sched

end

```

subgraph Backend ["Backend Pool: Model Servers (vLLM)"]
  direction LR
  P1["Endpoint 1<br/>Active Batch: 10/10<br/>Local Queue: 4/5<br/>Saturation: 93%"]:::endpoint
  P2["Endpoint 2<br/>Active Batch: 10/10<br/>Local Queue: 5/5<br/>Saturation: 100%"]:::endpoint
  P3["Endpoint 3<br/>Active Batch: 10/10<br/>Local Queue: 5/5<br/>Saturation: 100%"]:::endpoint
end

Sched == "Route to Best Candidate" ==> P1
Sched -.-> P2
Sched -.-> P3

```

Example Configuration

To ground the diagram above, here are the example manifests that would produce this behavior.

1. InferenceObjectives (Traffic Classes)

These resources define the priority of the traffic. Note the use of a negative priority for best-effort workloads.

```

apiVersion: llm-d.ai/v1alpha2
kind: InferenceObjective
metadata:
  name: premium-traffic
spec:
  priority: 100
  poolRef:
    name: default-pool
---
apiVersion: llm-d.ai/v1alpha2
kind: InferenceObjective
metadata:
  name: standard-traffic
spec:
  priority: 0
  poolRef:
    name: default-pool
---
apiVersion: llm-d.ai/v1alpha2
kind: InferenceObjective
metadata:
  name: best-effort-traffic
spec:
  priority: -10
  poolRef:
    name: default-pool

```

2. EPP Configuration

This snippet from the `EndpointPickerConfig` shows how the Flow Control layer is configured to handle these bands.

apiVersion: llm-d.ai/v1alpha1

kind: EndpointPickerConfig

metadata:

name: default-config

featureGates:

- flowControl

plugins:

- **type:** round-robin-fairness-policy

- **type:** fcfs-ordering-policy

- **type:** concurrency-detector

parameters:

maxConcurrency: 15

concurrencyMode: requests

headroom: 0.0

... other plugins ...

flowControl:

saturationDetector:

pluginRef: concurrency-detector

maxBytes: "10Gi"

maxRequests: "1k"

defaultRequestTTL: "60s"

priorityBands:

- **priority:** 100

maxRequests: "500"

fairnessPolicyRef: round-robin-fairness-policy

orderingPolicyRef: fcfs-ordering-policy

- **priority:** 0

maxRequests: "200"

fairnessPolicyRef: round-robin-fairness-policy

orderingPolicyRef: fcfs-ordering-policy

- **priority:** -10

maxRequests: "50"

fairnessPolicyRef: round-robin-fairness-policy

orderingPolicyRef: fcfs-ordering-policy

... other sections (schedulingProfiles, dataLayer, etc.) ...

Understanding the execution of this configuration requires examining the core components that implement the logic and enforce resource guardrails.

Core Components

- **Flow Controller:** The main orchestrator managing the dispatch loop and queue evaluation.
- **Saturation Detector:** Queries the aggregate health of the pool (e.g., KV cache utilization, queue depth) to determine if the gate should be open or closed.
- **Queues:** The in-memory storage structures organized by `FlowKey` (implemented as lists or heaps depending on the Ordering Policy).

Resource Guardrails & Memory Isolation

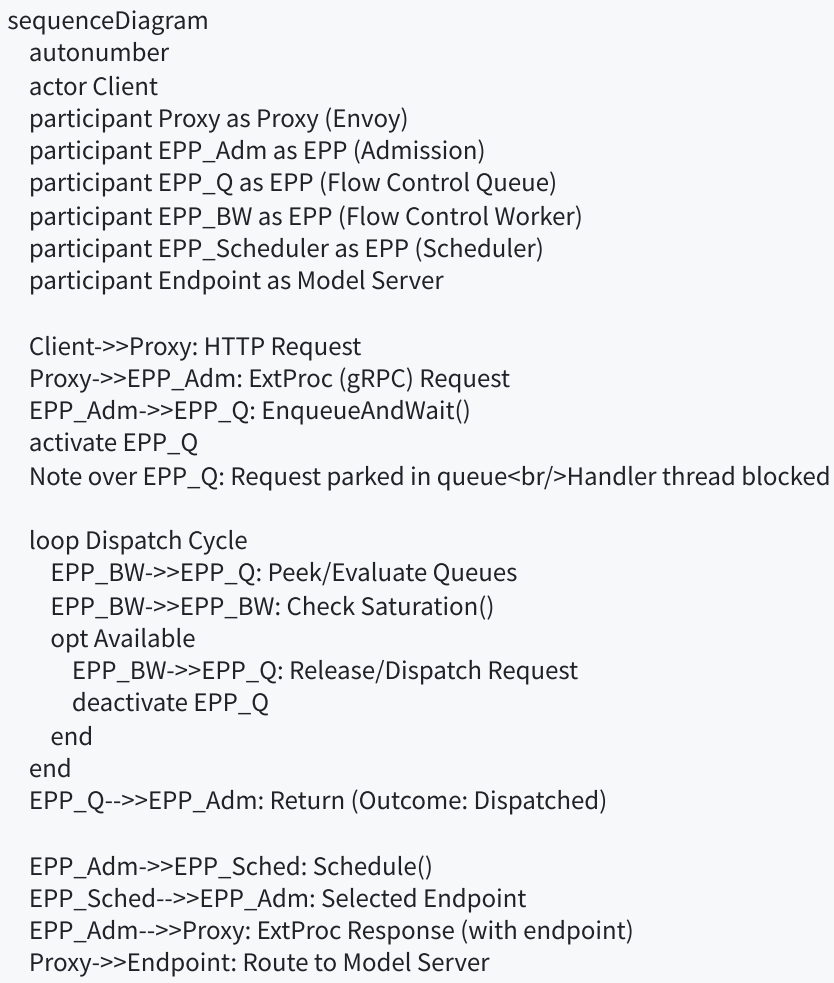
To prevent the EPP itself from resource exhaustion when queues grow, Flow Control enforces configurable capacity limits. See the Global Fields section in the Configuration Guide for details on setting `maxBytes` and `maxRequests`:

- **Global Limits:** Configured via `maxBytes` (payload size) and `maxRequests` (count) across all priority bands. These limits support both plain integers and Kubernetes Quantity format (e.g., `10Gi`, `1k`). If a new request would exceed these limits, it is immediately rejected with an HTTP 429 (Too Many Requests).
- **Per-Band Limits:** Each priority band can have its own `maxBytes` and `maxRequests` limits. This provides **memory isolation** between bands; heavy traffic in a lower-priority band cannot fill the queue space reserved for higher-priority bands.

[!NOTE] **Proxy vs. GPU Protection:** It is important to distinguish between these flow control limits and the Saturation Detector. The `maxBytes` and `maxRequests` limits protect the **Gateway's** memory footprint and prevent proxy-level Resource Exhaustion. The Saturation Detector (described below) protects the **GPU's** physical compute and KV-cache capacity.

The Dispatch Lifecycle

While the Request Scheduler operates on a per-request synchronous lifecycle, the Flow Control layer maintains an asynchronous, continuous **Ingress & Buffer → Policy Evaluation → Gated Dispatch** loop:



1. **Ingress & Buffer:** Incoming requests are classified by `FlowKey` (Priority + Fairness ID) and placed into the appropriate queue. Requests are subject to a configurable TTL (Time-To-Live); if a request remains in the queue past its TTL, it is expired and rejected, preventing the system from processing stale work.
2. **Policy Evaluation:** Flow Control workers continuously evaluate the queues. They select the highest-priority band with work, apply the **Fairness Policy** to pick a flow, and use the **Ordering Policy** to select the candidate request.

3. **Gated Dispatch:** Before releasing the request, the **Saturation Detector** is queried. If the pool has capacity, the request is dispatched to the Router. If saturated, the dispatch cycle halts (Head-of-Line blocking), holding the request safely until capacity frees up.

In-Flight Eviction

In-flight eviction terminates an already-dispatched, negative-priority (`priority < 0`) request to reclaim capacity for a higher-priority request blocked at its dispatch ceiling. Gated dispatch decides only whether to release new work, so it cannot recover capacity that lower-priority requests are already holding.

When `enableEviction: true` (see Flow Control configuration), Flow Control may end an already-dispatched, **negative-priority** (`priority < 0`) request to make room for blocked higher-priority demand. Eviction only occurs while a higher-priority band is blocked at its dispatch ceiling with requests queued behind it, and only targets in-flight requests in negative-priority bands. Under the default Usage Limit Policy that ceiling is full pool saturation; under `priority-holdback-policy` each band holds back at a lower ceiling, so eviction can begin before the pool is full. The default victim policy selects the lowest priority first, then the most recently dispatched, so requests that have already accrued the most work are evicted last.

```
sequenceDiagram
    autonumber
    participant EPP_BW as EPP (Flow Control Worker)
    participant EPP_Proc as EPP (ext_proc Handler)
    participant Proxy as Proxy (Envoy)
    participant Endpoint as Model Server
    actor Client as Evicted Client

    Note over EPP_BW: A higher-priority band is blocked at its<br/>dispatch ceiling with negative-priority requests in flight
    EPP_BW->>EPP_Proc: Revoke selected negative-priority request
    EPP_Proc->>Proxy: ImmediateResponse (HTTP 429)
    Proxy-->>Endpoint: Reset upstream stream
    Proxy->>Client: 429 + x-llm-d-request-dropped-reason: evicted
    Note over Endpoint: Aborts generation and frees KV blocks. Capacity returns to the saturation signal.
```

Eviction ends the evicted request; the capacity returns only once the model server aborts generation on the upstream reset and frees its KV blocks. Flow Control assumes a bounded engine reclaim time and does not observe it directly, so verify on your engine that aborted requests free capacity promptly before enabling eviction in production. The evicted caller (the client or an upstream gateway) owns retry, and its retry path needs idempotency or request-level dedup so a retried request cannot produce two final results. Reserve eviction for lower-priority work whose owner can safely retry (for example, batch jobs on a negative-priority band).

Evictions are observable through the `revocations_issued_total` and `revocations_total` metrics (see Metrics & Observability) and the `x-llm-d-request-dropped-reason: evicted` response header.

System Capabilities & Limits

Understanding the guaranteed capabilities and inherent boundaries of the Flow Control layer is essential for effective capacity planning.

What It Solves (Capabilities & Guarantees)

- **Multi-Tenant Fairness:** Provides a centralized point to enforce business policies (priority, fairness, and starvation prevention).
- **Dynamic Late Binding:** Delays routing decisions until the last possible moment to match requests to the most optimal candidate (e.g., one with high prefix cache affinity).
- **Work-Conserving Nature:** Acts as a **work-conserving** system (meaning it never artificially throttles traffic if GPUs have spare capacity); if backend capacity exists, it dispatches.
- **TPOT Protection:** By preventing context-thrashing on the GPU (managing **arithmetic intensity**), it ensures predictable generation times (**Time-Per-Output-Token**) once dispatched. (Note: While this can also be managed at the model server level via settings like `max_num_seq` in vLLM, doing so lacks global awareness. This protection is primarily beneficial for **chat streaming** use cases where maintaining human reading pace is critical.)

What It Does Not Solve (Limits & Trade-offs)

- **Absolute Capacity Shortages:** Flow Control only handles *when* and *in what order* requests are dispatched. It cannot make the pool faster or create capacity that doesn't exist.
- **TTFT Shifts (Not Elimination):** Flow Control cannot remove wait time when the system is over capacity. By enabling it, you make the **explicit choice to protect TPOT at the expense of queue time**. Its core function is simply controlling **where** and **for whom** TTFT (Time-To-First-Token) is accrued (accruing it safely in the EPP rather than letting it context-thrash the GPU).
- **Non-Persistence:** Queues are stored purely in-memory. If the EPP process restarts or fails, queued requests are lost. During a graceful shutdown, the EPP evicts queued requests with a retryable HTTP 503 (Service Unavailable) — signaling transient unavailability so clients and upstream gateways retry rather than treating the drain as a hard server fault — whereas abrupt crashes will result in hard connection drops for the client.

Failure Modes & Error Mapping

Operators should also be familiar with the system's failure modes and how internal states map to client-facing errors. The following table maps the internal `QueueOutcome` to the public error contract returned to Envoy and the client.

```

stateDiagram-v2
    [*] --> Arriving
    Arriving --> Dropped: Band Limit Exceeded<br/>(Load Shedding - HTTP 429)
    Arriving --> Queued: Admitted

    state Queued {
        [*] --> Waiting
        Waiting --> Waiting: Band Ceiling Reached<br/>(Closed Gate)
    }

    Queued --> Expired: TTL Expired<br/>(HTTP 503)
    Queued --> Cancelled: Client Disconnect<br/>(HTTP 503)
    Queued --> Drained: Controller Shutdown<br/>(HTTP 503)
    Queued --> Failed: Internal Error<br/>(HTTP 500)
    Queued --> Dispatched: Saturation < Band Ceiling<br/>(Open Gate)

    Dispatched --> Evicted: Reclaimed for higher priority<br/>(In-Flight Eviction - HTTP 429)
    Dispatched --> [*]: Sent to Scheduler
    Dropped --> [*]
    Expired --> [*]
    Cancelled --> [*]
    Drained --> [*]
    Evicted --> [*]
    Failed --> [*]

```

The `Drop Reason` column lists the value emitted in the `x-llm-d-request-dropped-reason` response header, which lets operators distinguish the precise cause of a drop on the wire (e.g., for alerting or client-side retry logic).

Queue Outcome	Internal Error Code	HTTP Status	Drop Reason (x-llm-d-request-dropped-reason)	Description
QueueOutcomeRejectedCapacity	ResourceExhausted	429 (Too Many Requests)	rejected-saturated	Rejection because queue capacity limits were met (Global or Per-Band).
In-flight eviction (enableEviction: true)	<i>(ImmediateResponse, no internal code)</i>	429 (Too Many Requests)	evicted	Post-dispatch reclamation, not a QueueOutcome : an already-dispatched, negative-priority request is ended to reclaim capacity for blocked higher-priority demand. Returned as an Envoy ImmediateResponse , so it carries no internal error code. The evicted caller owns retry. See In-Flight Eviction.
QueueOutcomeEvictedTTL	ServiceUnavailable	503 (Service Unavailable)	rejected-ttl-expired	Eviction from queue because the request's TTL expired.
QueueOutcomeEvictedContextCancelled	ServiceUnavailable	503 (Service Unavailable)	rejected-context-cancelled	Eviction from queue because the client disconnected.
QueueOutcomeRejectedOther / EvictedOther (graceful drain)	ServiceUnavailable	503 (Service Unavailable)	rejected-shutting-down	The flow controller is draining queued requests during a graceful shutdown (e.g., rolling update), while the EPP is still alive and its ext_proc stream is open. Transient and retryable, not an internal fault.
QueueOutcomeRejectedOther / EvictedOther (other)	Internal	500 (Internal Server Error)	<i>(none)</i>	Genuine internal flow control error. No drop-reason header is set, since this is an unexpected failure rather than a specific removal policy.

[!IMPORTANT] **These mappings only apply while the EPP is reachable.** The table above describes outcomes the EPP returns over a live ext_proc stream — including the 503 graceful-drain case, where the EPP is still running long enough to finalize its queued requests. Once the EPP process is actually gone (abrupt crash, or the tail of a shutdown after the ext_proc connection closes), none of these codes apply, because Envoy never receives a response from the extension. What happens then is governed by the InferencePool's failureMode :

- **FailOpen** (set by the llm-d router recipe): Envoy **bypasses the extension and routes the request directly to a model-server endpoint**. The request is *not* rejected — but it also receives **no flow control**: no queuing, fairness, or saturation gating. This trades pool-defense guarantees for availability, so a request can land on an already-saturated backend during the window the EPP is down.
- **FailClose** (the API default): Envoy fails the request itself rather than routing around the dead EPP.

In other words, the EPP's error contract is best-effort and contingent on the EPP being reachable; under the default llm-d (FailOpen) configuration, an EPP outage degrades to unprotected pass-through rather than to any status code in this table.

Extension Points

The Flow Control layer behavior is customizable via several extension points implemented as plugins. For details on how to register and reference these plugins in your config, see the Flow Control section in the Configuration Guide:

1. **Fairness Policy**: Determines how to share dispatch opportunities between different flows within the exact same Priority level.
2. **Ordering Policy**: Determines the order in which requests are served within a specific flow.
3. **Saturation Detector**: Evaluates whether the pool has capacity for more dispatches.

Concrete Plugins

Fairness Policies

- **global-strict-fairness-policy** : Ignores flow isolation and serves all requests in a single global order based on the Ordering Policy. Ideal when strict global ordering must be enforced across all requests within the band and fairness is not a concern.
- **round-robin-fairness-policy** : Guarantees fair sharing by cycling through active flows one by one. Prevents a single high-volume flow from starving others (solving the "Noisy Neighbor" problem).

Ordering Policies

- **fcfs-ordering-policy** : First-Come, First-Served based on arrival time. (Default)
- **edf-ordering-policy** : Earliest Deadline First, prioritizing requests with the closest expiration time.
- **slo-deadline-ordering-policy** : Orders requests by an SLO-based deadline computed from arrival time. Uses the x-llm-d-slo-ttft-ms header. Requests without this header are placed behind all SLO requests, risking starvation.

Saturation Detectors

The behavior of the saturation detector depends on whether flow control is enabled:

- **Flow Control enabled:** When the pool is saturated, request dispatch is paused and incoming requests are buffered in the flow control memory queues (respecting priority and fairness policies) until backend capacity frees up. The Saturation Detector acts as the gatekeeper for these centralized queues; see the Dispatch Lifecycle section for details.
- **Flow Control disabled** (default): When the pool is saturated, "sheddable" requests (those with negative priority) are immediately rejected with HTTP 429 (Too Many Requests). All other requests pass directly to the model servers.

Available plugins:

- **utilization-detector** : Closed-loop detector reacting to real-time telemetry (queue depth, KV cache). Highly accurate but subject to telemetry lag ("thundering herd"). In heterogeneous pools, it treats all endpoints equally (unweighted average), meaning a small saturated endpoint can trigger global backpressure. (Default)
- **concurrency-detector** : Open-loop detector based on active in-flight request accounting. Instantaneous reaction but blind to actual hardware memory pressure (KV cache filling). In heterogeneous pools, it biases toward the state of larger endpoints (aggregate capacity model).

[!NOTE]

The "Healthy Buffer" Principle

Regardless of which detector you use, the core goal of tuning saturation detection is to maintain a small, **"healthy buffer"** of requests queued locally on the model servers themselves.

This buffer should be just large enough to ensure continuous batching engines never starve for work, but small enough that the vast majority of queuing happens centrally in the EPP where priority and fairness can be enforced.

Tuning the Buffer:

Tuning this buffer involves adjusting the specific parameters of the configured Saturation Detector. For instance:

- In the `utilization-detector` , this is typically controlled via the `queueDepthThreshold` parameter.
- In the `concurrency-detector` , it is controlled by setting `maxConcurrency` just above your model server's effective batch size.

See the linked READMEs for each detector above for full details on their available knobs.

Advanced Use Cases: Autoscaling

True Demand Autoscaling

Traditional metrics like GPU utilization fail to quantify unfulfilled demand because LLM compute is highly non-linear. Shifting the queue to the EPP provides a definitive "True Demand" metric (Queue Depth). External scalers like KEDA can use this metric to scale out replicas based on the exact volume of traffic waiting to be served. See Autoscaling for more details.

Metrics & Observability

The Flow Control layer exposes detailed metrics to track queuing dynamics and system health.

Metric Name	Metric Type	Description	Labels
llm_d_epp_flow_control_request_queue_duration_seconds	Histogram	Time requests spend in the Flow Control layer from enqueue to final outcome.	fairness_id , priority , outcome , inference_pool , model_name , target_model_name
llm_d_epp_flow_control_queue_size	Gauge	Current number of requests actively held in the Flow Control queue.	fairness_id , priority , inference_pool , model_name , target_model_name
llm_d_epp_flow_control_queue_bytes	Gauge	Current total size in bytes of requests actively held in the Flow Control queue.	fairness_id , priority , inference_pool , model_name , target_model_name
llm_d_epp_flow_control_dispatch_cycle_duration_seconds	Histogram	Time taken for each internal dispatch cycle.	None
llm_d_epp_flow_control_request_enqueue_duration_seconds	Histogram	Time taken to enqueue requests into the Flow Control layer.	fairness_id , priority , outcome
llm_d_epp_flow_control_pool_saturation	Gauge	Pool saturation signal gating dispatch (1.0 is gating set point).	inference_pool
llm_d_epp_flow_control_stale_endpoints	Gauge	Number of candidate endpoints whose metrics are missing or older than staleness threshold.	detector
llm_d_epp_flow_control_requests_total	Counter	Total requests processed by the Flow Control layer by outcome.	outcome , priority , inference_pool
llm_d_epp_flow_control_revocations_issued_total	Counter	Total in-flight eviction revocations issued, labeled by the demand band's priority. Emitted when enableEviction is set.	priority , inference_pool
llm_d_epp_flow_control_revocations_total	Counter	Total in-flight eviction revocations by terminal outcome	outcome , inference_pool

Metric Name	Metric Type	Description	Labels
		(confirmed , timed_out). Every issued revocation eventually increments exactly one outcome.	
llm_d_epp_flow_control_reclaim_target	Gauge	Last computed reclamation deficit, in saturation-gauge units.	inference_pool
llm_d_epp_flow_control_pending_reclaim	Gauge	Sum of outstanding and cooling pending-reclaim debits, in saturation-gauge units.	inference_pool
llm_d_epp_flow_control_revocation_confirmation_seconds	Histogram	Time from revocation issue to confirmed stream termination.	inference_pool
llm_d_epp_program_aware_jains_fairness_index	Gauge	Jain's fairness index over average wait time across active programs.	None
llm_d_epp_program_aware_avg_wait_time_milliseconds	Gauge	Cumulative mean of flow-control queue wait time per program in milliseconds.	program_id
llm_d_epp_program_aware_attained_service_tokens	Gauge	Time-decayed attained service (weighted tokens consumed) per program.	program_id

The five revocation and reclaim metrics above apply only when `enableEviction: true` . Watch `revocations_total{outcome="timed_out"}` during rollout: it counts revocations whose stream termination was never confirmed, which the controller treats as confirmed so a hung stream cannot hold the pacing gate closed. A sustained nonzero rate means reclamation is being sized against capacity that may not have come back.

Grafana Dashboard

A pre-configured Grafana dashboard is available to visualize these metrics, making it easy to monitor queue depths, dispatch latency, and saturation state transitions.

To load this dashboard, follow the Observability Setup guide, which installs Prometheus and Grafana and loads the llm-d dashboards.

 Flow Control Dashboard

Request Scheduler

The Request Scheduler is a highly modular and extensible component within the EPP designed to select the optimal model server (endpoint) for an inference request. It leverages a plugin-based architecture, allowing for sophisticated endpoint picking strategies based on real-time metrics, prefix cache tracking, and model-specific requirements like LoRA adapters.

Architecture Overview

At its core, the scheduler follows a **Filter -> Score -> Pick** lifecycle for every request. It orchestrates multiple **SchedulerProfiles**, each defining a specific set of plugins for filtering and scoring candidate endpoints.

flowchart TD

```
Req[Inference Request] --> S[Scheduler.Schedule]
```

```
subgraph Cycle [Scheduling Cycle]
```

```
direction TD
```

```
S --> Pick[ProfileHandler.Pick]
```

```
Pick -->|Profiles| Loop{For each Profile}
```

```
subgraph Exec [Profile Execution]
```

```
direction TD
```

```
Loop --> Filters[Filters]
```

```
Filters --> Scorers[Scorers]
```

```
Scorers --> Picker[Picker]
```

```
Picker --> Result[ProfileResult]
```

```
end
```

```
Result -->|Collect| Pick
```

```
Pick -->|Done| PRs[ProfileHandler.ProcessResults]
```

```
end
```

```
PRs --> Target["Selected Endpoint(s)"]
```

```
%% Styling
```

```
style Req fill:#e1f5fe,stroke:#01579b,color:#000
```

```
style S fill:#e8f5e9,stroke:#2e7d32,color:#000
```

```
style Pick fill:#fffde7,stroke:#fbc02d,color:#000
```

```
style Loop fill:#fff,stroke:#333,color:#000
```

```
style Filters fill:#fff3e0,stroke:#ff9800,color:#000
```

```
style Scorers fill:#fff3e0,stroke:#ff9800,color:#000
```

```
style Picker fill:#fff3e0,stroke:#ff9800,color:#000
```

```
style Result fill:#f1f8e9,stroke:#558b2f,color:#000
```

```
style PRs fill:#fce4ec,stroke:#c2185b,color:#000
```

```
style Target fill:#f3e5f5,stroke:#7b1fa2,color:#000
```

Core Components

- **Scheduler:** The main orchestrator that manages the scheduling cycle. It invokes the configured `ProfileHandler` to pick profiles and then runs the selected profiles to obtain target endpoints.

- **InferenceRequest:** A structured internal representation of the incoming request produced by the `Parser`, including the target model, parsed body (`Completions`, `ChatCompletions`, etc.), headers, and objectives.
- **Endpoint:** Represents a candidate serving engine, with its metadata (e.g., Pod name, namespace and port) and state (e.g., active models, queue depth and KV-cache). Note that a Pod may run one or more endpoints each on a different port, this is the case in the data parallel deployment mode.

Extension Points

The scheduler's logic is distributed across several extension points, implemented via plugin interfaces:

1. **ProfilePicker:** (Implemented by `ProfileHandler`) Selects which `SchedulerProfile`s to run based on the request and previous cycle results.
2. **Filter:** Narrows down the list of candidate endpoints (e.g., based on health, SLO headroom, or cache affinity).
3. **Scorer:** Assigns a score between `0.0` and `1.0` to each filtered endpoint. Multiple scorers can be weighted and combined.
4. **Picker:** Selects the final endpoint(s) from the scored list (e.g., highest score, weighted random).
5. **ProcessResults:** (Implemented by `ProfileHandler`) Aggregates the results from all executed profiles to produce the final `SchedulingResult`.

Scheduling Profile

A `SchedulingProfile` is a configured pipeline consisting of:

- **Filters:** A list of `Filter` plugins run sequentially.
- **Scorers:** A list of `WeightedScorer` objects, where each contains a `Scorer` plugin and its relative weight.
- **Picker:** A single `Picker` plugin that makes the final selection.

When a profile runs, it first filters the candidate endpoints. If any remain, it calculates a weighted aggregate score for each and then passes the scored list to the picker. The final score for an endpoint is calculated by multiplying the score returned by each scorer (which is bounded between `0.0` and `1.0`) by its configured weight, and summing these weighted scores together. For example, if Scorer A (weight `2.0`) returns `0.8` and Scorer B (weight `1.0`) returns `0.5`, the endpoint's final score is $(0.8 * 2.0) + (0.5 * 1.0) = 2.1$.

Concrete Plugins

[!IMPORTANT] Not all of the plugins listed below are configured by default. Only a curated subset is enabled in the default configuration.

Filters

- **prefix-cache-affinity-filter**: A probabilistic filter that narrows candidates to "sticky" endpoints (those with high prefix cache scores). It includes a "TTFT load gate" to break stickiness if sticky endpoints are significantly slower than non-sticky ones.

- **slo-headroom-tier-filter** : Filters endpoints based on SLO headroom tiers to ensure quality of service.
- **label-selector-filter** : Keeps endpoints that matches a configured label selector.
- **prefill-filter** : Retains endpoints with `prefill`, `encode-prefill`, `prefill-decode`, or `encode-prefill-decode` roles.
- **decode-filter** : Retains endpoints with `decode`, `prefill-decode`, or `encode-prefill-decode` roles, and passes through endpoints without an `llm-d.ai/role` label. The former `both` role is removed; use `prefill-decode` instead.
- **encode-filter** : Retains endpoints with `encode`, `encode-prefill`, or `encode-prefill-decode` roles.

Scorers

For details on exactly how each scorer calculates its score (0.0 to 1.0), please refer to the specific plugin's documentation.

- **kv-cache-utilization-scorer** : Prefers endpoints with lower KV cache utilization to avoid fragmentation.
- **latency-scorer** : Scores endpoints based on predicted latency headroom, defined as the gap between the predicted request latency and the user's SLO if set.
- **lora-affinity-scorer** : Prefers endpoints that already have the requested LoRA adapter active or have capacity to load it.
- **prefix-cache-scorer** : Scores based on the length of the prefix cache match.
- **queue-depth-scorer** : Prefers endpoints with shorter request queues.
- **running-requests-size-scorer** : Scores based on the number of currently active requests.
- **token-load-scorer** : Scores based on the total token load (input + output) handled by the endpoint.
- **precise-prefix-cache-producer** (DataProducer, paired with `prefix-cache-scorer`): Publishes `PrefixCacheMatchInfo` from its event-driven KV-cache index. Set `prefixMatchInfoProducerName: precise-prefix-cache-producer` on the scorer to use this precise cache state.

[!NOTE] Without an explicit `prefixMatchInfoProducerName`, `prefix-cache-scorer` falls back to the auto-instantiated `approx-prefix-cache-producer`.

- **session-affinity-scorer** : Assigns a maximum score to the specific endpoint that handled previous requests for the same session, while all other endpoints receive the minimum score.
- **no-hit-lru-scorer** : For cold requests (zero cache hits), the scorer prioritizes endpoints that have never handled one, followed by those used least recently. This ensures an even distribution of the intensive "prefill" workload across the cluster. If a request has existing cache hits, the scorer assigns equal scores to all endpoints (scorer has no impact).

Pickers

- **max-score-picker** : Selects the endpoint with the absolute highest score. This is the **default picker** — auto-injected into any profile that references no picker with its default `maxNumOfEndpoints: 1` (return the single best endpoint).
- **random-picker** : Selects an endpoint randomly from the candidates.
- **weighted-random-picker** : Selects an endpoint randomly, using the scores as relative probabilities (lottery scheduling).

Profile Handlers

- **single-profile-handler** : Runs a single configured primary profile.
- **disagg-profile-handler** : Runs two scheduling profiles, one for prefill and one for decode. The **decode endpoint** is set as the primary destination for the proxy to forward the original request, while the **prefill endpoint** is injected into the request as a specialized header.

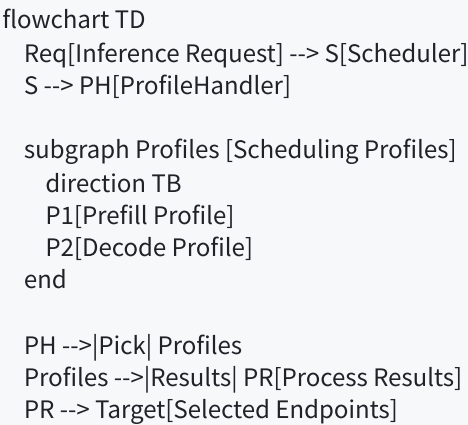
[!NOTE] An older handler, `data-parallel-profile-handler` , is **deprecated** and kept only for backward compatibility, use `single-profile-handler` instead.

Advanced Use Cases: Prefill/Decode Disaggregation

The scheduler natively supports advanced scheduling paradigms, such as **Prefill/Decode Disaggregation (P/D Disagg)**. This is a serving technique where the initial prompt processing (prefill) and the subsequent token generation (decode) are handled by separate, specialized model servers.

In a P/D Disagg setup, the `ProfileHandler` orchestrates two separate `SchedulerProfiles` :

1. **Prefill Profile**: Evaluates and scores endpoints specialized for compute-heavy prompt processing. It may use filters and scorers focused on prefix cache affinity, queue depth, or token load.
2. **Decode Profile**: Evaluates and scores endpoints specialized for memory-bandwidth-bound token generation.



The `ProfileHandler` uses the `Pick` extension point to determine which profiles need to run for a given request (e.g., if a request needs both prefill and decode, or just decode if the KV cache is already transferred). If both are needed, the prefill and decode endpoints are picked at the same time. The `ProfileHandler` then uses the `ProcessResults` extension point to merge the results from both profiles. This merging ensures that the **decode endpoint** is returned as the primary destination for the proxy to forward the original request. Simultaneously, the **prefill endpoint** is injected into the request as a specialized header. When the request reaches the decode worker, the **sidecar** running alongside the decoder intercepts it, extracts the prefill endpoint from the header, and coordinates a remote prefill from the selected prefill worker before the decoding process begins.

See Disaggregated Serving for more details on the design and request flow.

Metrics & Observability

The EPP Scheduler exposes detailed metrics to track pool health and scheduling decisions.

Pool & Scheduling Metrics

The following metrics provide visibility into the InferencePool health and scheduling decisions.

Pool Health Metrics

Metric	Type	Labels	Description
llm_d_epp_ready_endpoints	Gauge	name	Number of ready endpoints in the pool
llm_d_epp_average_kv_cache_utilization	Gauge	name	Average KV cache utilization across the pool
llm_d_epp_average_queue_size	Gauge	name	Average number of pending requests across the pool
llm_d_epp_average_running_requests	Gauge	name	Average number of running requests across the pool
llm_d_epp_std_dev_kv_cache_utilization	Gauge	name	Standard deviation of KV cache utilization across the pool
llm_d_epp_std_dev_queue_size	Gauge	name	Standard deviation of pending requests across the pool
llm_d_epp_std_dev_running_requests	Gauge	name	Standard deviation of running requests across the pool
llm_d_epp_per_endpoint_queue_size	Gauge	name , model_server_endpoint	Queue size for each individual endpoint

Scheduler Performance Metrics

Metric	Type	Labels	Description
llm_d_epp_scheduler_attempts_total	Counter	status , target_model_name , endpoint_name , namespace , port	Number of scheduling attempts and their outcomes
llm_d_epp_scheduler_e2e_duration_seconds	Histogram	None	End-to-end scheduling latency
llm_d_epp_plugin_duration_seconds	Histogram	extension_point , plugin_type , plugin_name	Processing latency for each plugin
llm_d_epp_plugin_data_scope_violations_total	Counter	extension_point , plugin_type , plugin_name , access	Endpoint attribute accesses rejected due to undeclared DataKey (read or write)

In-Flight Load Metrics

Metric	Type	Labels	Description
llm_d_epp_inflight_requests	Gauge	endpoint_name , namespace , producer_name , fairness_id , priority	Requests currently in flight on each endpoint
llm_d_epp_inflight_tokens	Gauge	endpoint_name , namespace , producer_name , fairness_id , priority	Tokens currently in flight on each endpoint

Disaggregation Metrics

Metric	Type	Labels	Description
llm_d_epp_disagg_decision_total	Counter	plugin_name , plugin_type , model_name , decision_type	Disaggregation routing decisions (decode-only , prefill-decode , encode-decode , encode-prefill-decode)
llm_d_epp_pd_decision_total	Counter	plugin_name , plugin_type , model_name , decision_type	Prefill/Decode decisions (deprecated handler)
llm_d_epp_disaggregatedset_strict_header_no_match_total	Counter	plugin_type , plugin_name , selector	Strict header selections that matched no endpoint
llm_d_epp_disaggregatedset_revision_gating_share	Gauge	plugin_type , plugin_name , mode , revision	Current weighted revision gating share

Prefix Cache Metrics (Approximate)

Metric	Type	Labels	Description
llm_d_epp_prefix_indexer_size	Gauge	plugin_name , plugin_type	Size of the approximate prefix indexer
llm_d_epp_prefix_indexer_hit_ratio	Histogram	plugin_name , plugin_type	Hit ratio for prefix matches
llm_d_epp_prefix_indexer_hit_bytes	Histogram	plugin_name , plugin_type	Bytes matched in prefix cache lookup

Data Layer Errors

Metric	Type	Labels	Description
llm_d_epp_datlayer_poll_errors_total	Counter	source_type	Data-source poll failures
llm_d_epp_datlayer_extract_errors_total	Counter	source_type , extractor_type	Extractor failures

Multimodal Encoder Cache

Metric	Type	Labels	Description
llm_d_epp_encoder_cache_queries_total	Counter	plugin_type , plugin_name , modality	Encoder-cache lookups
llm_d_epp_encoder_cache_hits_total	Counter	plugin_type , plugin_name , pod , modality	Encoder-cache hits
llm_d_epp_encoder_cache_hit_ratio	Histogram	plugin_type , plugin_name	Hit ratio for encoder cache lookups

System Info

Metric	Type	Labels	Description
llm_d_epp_info	Gauge	commit , build_ref	EPP build information

Data Layer

The Data Layer is a pluggable and extensible component within the EPP responsible for gathering, processing, and storing real-time state for model server endpoints. It provides the foundational data—such as queue depths, KV cache utilization, and LoRA adapter states—that request scheduling and flow control plugins use to make informed decisions.

Architecture Overview

The Data Layer operates as a reactive and polling-based system that enriches `Endpoint` objects with metadata and dynamic attributes. It follows a **Source -> Extract -> Attribute** lifecycle.

```

flowchart LR
    subgraph Sources [Data Sources]
        direction TB
        Poller[Polling Source] ~~~ Notifier[K8s Notification Source] ~~~ EpEvent[Endpoint Event Source]
    end

    subgraph Runtime [Datalayer Runtime]
        direction TB
        Collector[Collector] --- Mapping[Source-to-Extractor Mapping]
    end

    subgraph Extractors [Extractors]
        direction TB
        Ext1[Metrics Extractor] ~~~ Ext2[GVK Extractor] ~~~ Ext3[Lifecycle Extractor]
    end

    Sources --> Collector
    Collector --> Mapping
    Mapping --> Extractors
    Extractors -->|Update| Attr[(Endpoint Attributes)]

    %% Styling
    style Sources fill:#fff3e0,stroke:#ff9800,color:#000
    style Runtime fill:#e8f5e9,stroke:#2e7d32,color:#000
    style Extractors fill:#e1f5fe,stroke:#01579b,color:#000
    style Attr fill:#f3e5f5,stroke:#7b1fa2,color:#000
  
```

Core Concepts

- **Endpoint Attributes:** A thread-safe, type-safe map stored on each `Endpoint`. Attributes are the primary way information is shared between the Data Layer and other EPP components.
- **Data Source:** A plugin that retrieves raw data from external systems (e.g., Prometheus metrics, Kubernetes API, or local events).
- **Extractor:** A plugin that transforms raw data from a source into structured attributes and stores them on an endpoint.

- **Runtime:** The logic that manages the lifecycle of sources and extractors, orchestrates polling intervals, and handles Kubernetes event bindings.
-

Extension Points

The Data Layer is built around three primary source types, each paired with a corresponding extractor interface:

1. Polling Data Sources

Used for periodic retrieval of data from model servers, primarily metrics.

- **Source Interface:** `PollingDataSource` (e.g., `metrics-data-source`)
- **Extractor Interface:** `Extractor` (e.g., `core-metrics-extractor`)
- **Workflow:** The runtime starts a `Collector` for every new endpoint. The collector polls the source at a configured interval and passes the result to associated extractors. Metrics pulling is the primary motivation for this type of data source.

2. Notification Sources

Used for reacting to changes in Kubernetes resources.

- **Source Interface:** `NotificationSource` (e.g., `k8s-notification-source`)
- **Extractor Interface:** `NotificationExtractor`
- **Workflow:** The runtime binds these sources to the Kubernetes controller-runtime manager. When a resource (e.g., a Pod or Service) changes, the source dispatches an event to its extractors.

3. Endpoint Event Sources

Used for reacting to endpoint lifecycle events (Add, Update, Delete) within the EPP itself.

- **Source Interface:** `EndpointSource` (e.g., `endpoint-notification-source`)
 - **Extractor Interface:** `EndpointExtractor`
 - **Workflow:** When the EPP's endpoint pool changes, these sources notify extractors to perform setup or cleanup (e.g., initializing caches or purging stale metrics).
-

Concrete Plugins

Sources

- `metrics-data-source` - polls a Prometheus-compatible metrics endpoint of a model server and parses the response into a structured format for extraction.

- **k8s-notification-source** — can be configured to watch a single Kubernetes GVK (either a CRD or a k8s core API) and dispatches `NotificationEvent` s to registered `NotificationExtractor` s when objects are created, updated, or deleted. Multiple extractors can be notified of GVK changes by a single source. There are currently no upstream extractors that require this source.
- **endpoint-notification-source** — delivers endpoint (aka model server engine) lifecycle events (add, update, delete) to registered `EndpointExtractor` s whenever an endpoint is added to or removed from the datastore.

Extractors

- **core-metrics-extractor** - responsible for extracting a set of model server metrics sourced from `metrics-data-source` and storing them as endpoint attributes (such as `KVCacheUsagePercent` and `WaitingQueueSize`), which scorers like `kv-cache-utilization-scorer` consume. It supports multiple inference engines and can be configured to map engine-specific metric names to a standard set of internal keys.

Configuration

The `EndpointPickerConfig` is the central configuration for the Endpoint Picker (EPP), defining the graph of plugins and parameters that drive request handling, flow control, and scheduling decisions.

The configuration text has the following form:

```
apiVersion: llm-d.ai/v1alpha1
kind: EndpointPickerConfig
plugins:
- ....
- ....
featureGates:
...
requestHandler:
...
flowControl:
...
schedulingProfiles:
- ....
- ....
dataLayer:
...
```

[!IMPORTANT] While the configuration syntax looks like a Kubernetes Custom Resource, it is **not** a Kubernetes CRD. The configuration is not reconciled by a controller and is only read on startup. Updating the configuration requires a restart of the EPP.

- **Metadata:** The first two lines of the configuration are constant (`apiVersion` and `kind`) and must appear as is.
- **Plugins:** Defines the set of plugins that will be instantiated and their parameters.
- **Feature Gates:** Enables or disables specific experimental or optional features (such as Flow Control).
- **Request Handling:** Manages the full lifecycle of requests around the scheduling phase, spanning protocol parsing, state preparation via data producers, and final admission decisions.
- **Flow Control:** Manages pool defense and multi-tenancy by queuing requests at the gateway to enforce priority and fairness, while evaluating pool saturation to prevent overload (configured via the `flowControl` field, including its nested `saturationDetector`).
- **Scheduling:** Defines the profiles and plugins used to select the optimal model server candidate for each request (via Filter -> Score -> Pick lifecycle).
- **Data Layer:** Configures the backend sources and metrics collection used for smart scheduling decisions and observability.

Configuration Mental Model: Plugins and Wiring

The `EndpointPickerConfig` forms a configuration **graph** that defines how the EPP operates across three layers:

- **Plugins (The Nodes):** In the `plugins` section, you instantiate specific implementations (e.g., a custom scorer or a fairness policy) and provide their parameters.
- **Wiring (The Edges):** In structural sections like `schedulingProfiles` or `flowControl`, you link these plugins by name to specific architectural roles (e.g., telling a profile to use a specific scorer).
- **Static Runtime Configuration:** Alongside the graph, flat configuration parameters (like `maxBytes` or `defaultRequestTTL`) set static operational limits and defaults for the runtime.

This design allows you to define a plugin once and reuse it across multiple profiles or priority bands without duplicating its parameters.

[!NOTE] **Auto-Wiring:** Some subsystems support automatic binding. If a plugin is declared in the top-level `plugins` list and implements a specific Go interface (like `Admitter`, `DataProducer`, or advanced hooks like `PreRequest`, `ResponseHeaderProcessor`, and `ResponseBodyProcessor`), the system will automatically discover and bind it to its role without requiring an explicit edge in the structural configuration.

To ensure the integrity of this graph, the following **validation rules** apply across all layers:

- **Valid References:** Any field that references a plugin (e.g., `pluginRef` in `schedulingProfiles` or `flowControl.saturationDetector`) must reference a valid name defined in the top-level `plugins` section.
- **Unique Names:** All instances within lists that require naming (like `schedulingProfiles`) must have unique, non-empty names.
- **Data Dependencies:** The system validates that metrics extractors form a Directed Acyclic Graph (DAG) without circular dependencies, ensuring correct execution order.

Using the `EndpointPickerConfig`

Use the `--config-file` command-line argument to specify the path to the configuration file. For example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ${EPP_NAME}
...
spec:
...
template:
...
spec:
...
containers:
- name: epp
  args:
  - --pool-name
  - "${POOL_NAME}"
...
- --config-file
- "/etc/epp/epp-config.yaml" # Typically mounted from a ConfigMap
```


If the configuration is passed as inline text, use the `--config-text` command-line argument. For example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ${EPP_NAME}
...
spec:
...
template:
...
spec:
...
containers:
- name: epp
  args:
  - --pool-name
  - "${POOL_NAME}"
...
  - --config-text
  - |
    apiVersion: llm-d.ai/v1alpha1
    kind: EndpointPickerConfig
    plugins:
    - type: prefix-cache-scorer
    - type: approx-prefix-cache-producer
    parameters:
      blockSizeTokens: 5
      maxPrefixTokensToMatch: 1280
      lruCapacityPerServer: 31250
    schedulingProfiles:
    - name: default
      plugins:
      - pluginRef: prefix-cache-scorer
        weight: 1 # Default
```

Configuration Guide

plugins

This section declares the set of plugins to be instantiated along with their parameters.

Each plugin can also be given a name, enabling the same plugin type to be instantiated multiple times, if needed (such as when configuring multiple scheduling profiles). Each entry in this section has the following form:

```
- name: aName
type: a-type
parameters:
  param1: val1
  param2: val2
```

The fields in a plugin entry are:

- `name` which is optional, provides a name by which the plugin instance can be referenced. If this field is omitted, the plugin's type will be used as its name.
- `type` specifies the type of the plugin to be instantiated.

- `parameters` which is optional, defines the set of parameters used to configure the plugin in question. The actual set of parameters varies from plugin to plugin.

featureGates

The `featureGates` section enables optional or experimental features in the EPP. Features listed here are activated; if omitted, they remain disabled.

```
featureGates:  
- flowControl
```

Supported Feature Gates:

- `flowControl` : Enables the Admission and Flow Control layer. This must be enabled to use the `flowControl` configuration section.

Removing a Feature Gate

To ensure backward compatibility, a feature gate should usually be removed over two releases:

1. **First Release:** Mark the feature as stable and enable it by default, but keep the feature gate in the configuration as a deprecated, still-functional gate so existing configurations remain valid and operators retain a temporary rollback mechanism by disabling the feature if needed. During this phase, inform users (e.g., via release notes) that the feature gate is deprecated and will be removed in the next release.
2. **Second Release:** Completely remove the feature gate from the configuration and code.

Request Handling

This section covers components that process requests and responses before they reach the scheduling phase, or after a backend has been selected.

For full architectural details and a list of available parsers, admitters, and data producers, see the [Request Handling](#) reference.

Parsers

The `requestHandler.parsers` section configures how the EPP understands protocol messages (e.g., OpenAI or vLLM payloads). By default, when `requestHandler.parsers` is unspecified, the EPP configures three built-in parsers, `openai-parser`, `anthropic-parser`, and `vllmhttp-parser`, instantiated automatically without `plugins` entries. A request is handled by the first configured parser that claims its path (for example `/v1/chat/completions` by `openai-parser`, `/v1/messages` by `anthropic-parser`, `/inference/v1/generate` by `vllmhttp-parser`).

Setting `parsers` replaces this default list. To use a non-default parser, you must first instantiate it in the `plugins` section and then reference its name in an entry of the `parsers` list:

```
plugins:
- name: myParser
  type: vllmgrpc-parser
# ...
requestHandler:
  parsers:
- pluginRef: myParser
```

Admitters & Data Producers

Admitters and Data Producers are specialized plugins that execute during the initial request processing phase:

- **Admitters** perform early checks to accept or reject requests before they enter the queue.
- **Data Producers** gather per request contextual information (like predicted latency or prefix cache status) required by downstream components.

As introduced in the Mental Model, these plugins support automatic interface-based binding. This reduces boilerplate configuration that would otherwise be needed to wire them explicitly.

If an admitter or data producer plugin is declared in the top-level `plugins` list, the system automatically recognizes it by its capabilities at startup and binds it to the appropriate lifecycle hook:

- **Admitters:** Automatically bound if they implement the Go interface for admitting or rejecting requests early.
- **Data Producers:** Automatically bound if they implement the Go interface for gathering per-request data (like latency predictions) needed by other components.

To enable these plugins, simply list them in the `plugins` section:

```
plugins:
- name: latency-admitter
  type: latency-slo-admitter
  parameters: ...
# Add the predicted latency data producer which does the computation of predicted latency. The predicted latency is consumed by
the latency-slo-admitter.
- name: latency-producer
  type: predicted-latency-producer
  parameters: ...
```

They are automatically active and do not need to be referenced elsewhere in the configuration.

Flow Control

See Flow Control for more architectural details on how the EPP's flow control layer works internally.

The `flowControl` section configures the EPP's Flow Control layer, which acts as a pool defense mechanism by buffering requests before they reach backend model servers. Flow Control implements a 3-tier dispatch hierarchy: **Priority → Fairness → Ordering**. For a visual breakdown of how this looks in practice, see the Queuing Topology diagram in the Flow Control reference.

When flow control is enabled (via the `FlowControl` feature gate), incoming requests are queued in memory and dispatched according to configured priority bands, fairness policies, and ordering policies. When the pool is saturated (as determined by the saturation detector), requests are held in the queue until capacity frees up.

The following example demonstrates a complete `EndpointPickerConfig` with flow control enabled, showing how to configure the `featureGates`, `plugins`, and `flowControl` sections (including the nested `saturationDetector`) to work together.

```
apiVersion: llm-d.ai/v1alpha1
kind: EndpointPickerConfig

featureGates:
- flowControl

plugins:
- type: round-robin-fairness-policy
- type: fcfs-ordering-policy
- type: global-strict-fairness-policy
- type: utilization-detector
# ... other plugins ...

flowControl:
  maxBytes: 0 # Default: unlimited
  maxRequests: 0 # Default: unlimited
  defaultRequestTTL: "0s" # Default: uses client context deadline
  enableEviction: false # Default: in-flight eviction disabled

saturationDetector:
  pluginRef: utilization-detector # Default

defaultPriorityBand:
  maxBytes: "1Gi" # Default
  maxRequests: 0 # Default: unlimited
  orderingPolicyRef: fcfs-ordering-policy # Default
  fairnessPolicyRef: global-strict-fairness-policy # Default

priorityBands: # Only showing overrides; fields not specified inherit from defaults
- priority: 100
  maxBytes: "5Gi"
  maxRequests: 500
  fairnessPolicyRef: round-robin-fairness-policy

- priority: 50
  maxBytes: "2Gi"
  maxRequests: 200

# ... other sections (schedulingProfiles, dataLayer, etc.) ...
```

Global Fields

- `maxBytes` : Global capacity limit across all priority levels. Supports Kubernetes resource quantity format (e.g., `10Gi`, `512Mi`) or plain integers (bytes). If `0` or omitted, no global limit is enforced (unlimited).
- `maxRequests` : Optional global maximum request count limit. If `0` or omitted, no global limit is enforced (unlimited).
- `defaultRequestTTL` : Fallback timeout for requests that do not carry a deadline. If `0` or omitted, it defaults to the client context deadline (which may wait indefinitely).
- `enableEviction` : When `true`, already-dispatched **negative-priority** (`priority < 0`) requests can be evicted to reclaim capacity when higher-priority requests are blocked by pool saturation. Pacing and sizing self-configure from the selected saturation detector. Disabled by default. See In-Flight Eviction for the lifecycle and response mapping.

- `defaultPriorityBand` : A template used to dynamically provision priority bands that are not explicitly configured in `priorityBands` .
- `priorityBands` : A list of explicit configurations for specific priority levels.

Priority Band Fields

These fields apply to both `defaultPriorityBand` and entries in `priorityBands` :

- `priority` : (Required for `priorityBands` entries) Integer priority level; higher values mean higher priority.
- `maxBytes` : Aggregate byte limit for the band. Default: `1Gi` .
- `maxRequests` : Concurrent request limit for the band. Default: no per-band limit.
- `orderingPolicyRef` : References a plugin name for request ordering within the band. Default: `fcfs-ordering-policy` .
- `fairnessPolicyRef` : References a plugin name for fairness policy within the band. Default: `global-strict-fairness-policy` .

For a full list of available Fairness and Ordering policies, see the Flow Control reference.

Saturation Detector

The `saturationDetector` section configures the mechanism that evaluates whether the backend `InferencePool` is overloaded. It is nested under `flowControl` and has the following form:

```
flowControl:
  saturationDetector:
    pluginRef: utilization-detector # Default
```

Fields

- `pluginRef` : References a plugin instance defined in the global `plugins` section. Defaults to `utilization-detector` if omitted or empty. *Note: If a `utilization-detector` is not explicitly defined in your `plugins` array, the gateway will automatically instantiate one under the hood using standard default parameters.*

For a full list of available Saturation Detector plugins, see the Flow Control reference.

Scheduling Profiles

The `schedulingProfiles` section configures the EPP's Scheduling component. For full architectural details and a list of available filters, scorers, and pickers, see the Scheduling reference.

Incoming requests are routed to candidate model servers by executing a pipeline of filters, scorers, and a final picker defined in these profiles.

The following example demonstrates how to configure a scheduling profile with concrete values that are recommended for a typical production setup:

schedulingProfiles:

- **name**: default

plugins:

- **pluginRef**: label-selector-filter # *Optional: not in default profile*

- **pluginRef**: prefix-cache-scorer # *Recommended: not in default profile*

weight: 3.0

- **pluginRef**: kv-cache-utilization-scorer # *Recommended: not in default profile*

weight: 2.0

- **pluginRef**: queue-scorer # *Recommended: not in default profile*

weight: 2.0

- **pluginRef**: max-score-picker # *Default picker (auto-injected if omitted)*

[!NOTE] To use **precise** prefix-cache routing (exact, KV-event-driven) instead of the approximate default, declare a `precise-prefix-cache-producer` in the top-level `plugins` section and set `prefixMatchInfoProducerName: precise-prefix-cache-producer` on the `prefix-cache-scorer`; otherwise the scorer falls back to the approximate producer. See the [Precise Prefix Cache Routing guide](#).

Scheduling Profile Fields

- `name`: The unique name of the scheduling profile.
- `plugins`: A list of plugins that make up the scheduling pipeline for this profile.

Profile Plugin Fields

- `pluginRef`: References a plugin by its name (or type if name was omitted) defined in the top-level `plugins` section.
- `weight`: Optional float weight applied if the referenced plugin is a Scorer. If omitted for a scorer, it defaults to `1.0`.

[!CAUTION] If you define multiple pickers in the top-level `plugins` section and omit `schedulingProfiles`, the auto-generated `default` profile will include references to **all** of them, which will cause an error during initialization (see [Multiple Pickers](#) below).

► Defaulting Behaviors

Profile Execution Rules

While the YAML configuration presents a flat list of plugins within a profile, the framework processes them with specific rules:

- **Interface Roles**: Internally, the framework categorizes referenced plugins by their role (Filter, Scorer, or Picker) based on the interfaces they implement.
- **Execution Order**: Plugins are executed in this order: Filters first, then Scorers, and finally the Picker.
- **Multiple Pickers**: A scheduling profile **cannot** have more than one picker. Referencing more than one picker in a profile's `plugins` list will cause a runtime error during profile initialization.
- **Scorer Weights**: If the `weight` field is omitted for a scorer, it defaults to `1.0`. Scores from multiple scorers are accumulated after multiplying by their respective weights.

Profile Handlers and Use Cases

- **Multiple Profiles:** While a single profile is sufficient for simple serving, advanced use cases like **disaggregated prefill** require two or more profiles to handle different types of requests differently.
- **Profile Handler:** When multiple profiles are defined, you must instantiate and configure a **Profile Handler** plugin in the top-level `plugins` section. The Profile Handler determines which `SchedulingProfile` to use for each incoming request.
- **Single Profile Default:** If only one profile is defined, the system implicitly uses a `SingleProfileHandler` to route all requests to that profile, so no explicit handler configuration is required.

For a popular plugin like `prefix-cache-scorer`, you configure it in the top-level `plugins` section and reference it in a profile:

```
plugins:
- type: prefix-cache-scorer
  # Also add the approx-prefix-cache-producer (data producer) when passing parameters to the prefix cache scorer.
- type: approx-prefix-cache-producer
  parameters:
    blockSizeTokens: 64
    maxPrefixTokensToMatch: 16384 # 256 blocks * 64 tokens/block
    lruCapacityPerServer: 31250 # Default

# ...

schedulingProfiles:
- name: default
  plugins:
  - pluginRef: prefix-cache-scorer
    weight: 3.0
```

► Advanced Example: Multiple Profiles and Profile Handler

`dataLayer`

The `dataLayer` section configures the backend sources and metrics collection used for smart scheduling decisions and observability. It defines a list of data sources and the extractors that pull data from them.

For full details and a list of available data sources and extractors, see the Data Layer reference (TODO: add link to `datalayer.md` once written).

```
dataLayer:
sources:
- pluginRef: metrics-data-source # References a plugin in the 'plugins' section
extractors:
- pluginRef: core-metrics-extractor # References a plugin in the 'plugins' section
```

Fields

- `sources` : A list of data sources to be polled or monitored.
- `pluginRef` : References a plugin instance defined in the global `plugins` section that implements the `DataSource` interface.
- `extractors` : A list of extractors associated with this data source.

- `pluginRef` : References a plugin instance defined in the global `plugins` section that implements the `Extractor` interface.

[!NOTE] The `metrics-data-source` and `core-metrics-extractor` are injected **by default**, so standard metrics collection works without configuring `dataLayer` at all. Injection is additive — even when you supply your own `dataLayer`, the default metrics source is appended with your sources. Unless you set `injectDefaults: false` or `dataLayer.sources` already contains a `metrics-data-source` source

High Availability

To deploy the `EndpointPicker` in a high-availability (HA) active-passive configuration, set `replicas` to be greater than one. In such a setup, only one "leader" replica will be active and ready to process traffic at any given time. If the leader pod fails, another pod will be elected as the new leader, ensuring service continuity.

To enable HA, ensure that the number of replicas in the EPP Deployment is greater than 1.

Monitoring

The EPP exposes a Prometheus-compatible metrics endpoint on **port 9090** at `/metrics`. These metrics provide visibility into request processing, scheduling decisions, flow control behavior, and backend pool health.

For full upstream documentation, see the [Gateway API Inference Extension Metrics & Observability Guide](#).

EPP Metrics by Subsystem

Metrics are organized by the subsystem that owns the logic. For detailed tables of metrics available in each subsystem, see:

- **Request Handling Metrics:** Request volume, latency, token usage, and success rates.
- **Flow Control Metrics:** Queue sizes, dispatch cycles, and pool saturation.
- **Routing Metrics:** Router performance and pool health state.

Monitoring Stack

The recommended monitoring stack is **Prometheus + Grafana**. A pre-built Grafana dashboard is available at `tools/dashboards/inference_gateway.json` in the upstream repository.

Pre-configured alert rules are also available upstream, covering:

- **High P99 latency** — triggers when P99 request latency exceeds 10 seconds
- **High error rate** — triggers when the error rate exceeds 5%
- **High queue size** — triggers when model server queue depth exceeds 50 requests
- **High KV cache utilization** — triggers when KV cache utilization exceeds 90%

Operational Excellence

Operational Excellence guidelines focus on overarching Day-2 site reliability engineering, cluster-wide telemetry frameworks, and safe lifecycle rollout strategies for generative AI inference deployments.

While well-lit path guides teach how to configure llm-d's native intelligent routing algorithms and inference optimizations, this top-level section governs enterprise cluster observability, alerting, and zero-downtime model updates.

Cluster Observability

End-to-end telemetry setup, OpenTelemetry tracing, standard Prometheus metrics, PromQL dashboards, and monitoring architectures.

Disaggregated Serving Operations

Operational considerations and engine-specific guides (vLLM and SGLang) for dynamic connections, request cancellation, fault tolerance, and safe rollouts.

Zero-Downtime Rollouts

Production rollout strategies including Blue-Green updates and live LoRA adapter hot-swapping without dropping active client traffic.

Model-Aware Readiness Probes

Kubernetes HTTP probe configurations using vLLM API endpoints to ensure pods are only marked Ready when models are fully loaded.

Graceful Shutdown & Request Draining

Draining in-flight requests during scale-down, rolling updates, and node drains for general serving: the Kubernetes termination sequence, vLLM `--shutdown-timeout`, request cancellation on client disconnect, and EPP flow-control drain semantics.

Serve External APIs

Deploy LiteLLM Proxy or Kong AI Gateway to route traffic seamlessly between self-hosted llm-d inference stacks and external cloud provider LLM APIs.

Router Operations

Operational best practices, high availability scaling modes, standalone proxy architectures, and container resource sizing for Llm-d Router deployments.

Async Processor Operations

Throughput modeling, concurrency sizing (backed by a measured sweep), container resource sizing, and horizontal scaling for the Async Processor batch-dispatch agent.

vLLM Model-Aware Readiness Probes

Overview

Proper health checking for vLLM inference containers requires understanding three distinct lifecycle stages:

1. **Container Running** - Kubernetes container lifecycle
2. **API Server Ready** - vLLM OpenAI-compatible API server is accepting connections
3. **Model Loaded** - Model is loaded and ready to serve inference requests

This guide explains how to configure Kubernetes probes to ensure pods are only marked Ready when models are fully loaded and operational.

Problem Statement

When deploying vLLM inference servers, there's a significant time gap between when the container starts and when the model is fully loaded. Using only basic health checks can lead to:

- Premature traffic routing to pods that aren't ready to serve requests
- Failed requests during model loading phase
- Need for arbitrary sleep times in deployment pipelines
- Unreliable E2E testing and CI/CD workflows

The vLLM `/health` endpoint only indicates that the server process is running, **not** that models are loaded and ready to serve.

Solution: Model-Aware HTTP Probes

Use Kubernetes HTTP probes with vLLM's OpenAI-compatible API endpoints to implement model-aware readiness checking.

Recommended Probe Configuration

```
containers:
- name: vllm
  ports:
  - containerPort: 8000 # or 8200 for decode pods
    protocol: TCP

# Startup Probe: Wait for model to load during initialization
# Protects liveness/readiness probes from firing too early
startupProbe:
  httpGet:
    path: /v1/models
    port: 8000
  initialDelaySeconds: 15 # Time before first probe
  periodSeconds: 30 # How often to probe during startup
  timeoutSeconds: 5 # HTTP request timeout
  failureThreshold: 60 # Max attempts (30s * 60 = 30min max startup time)

# Liveness Probe: Is the server process alive?
# Simple health check, restarts container if failing
livenessProbe:
  httpGet:
    path: /health
    port: 8000
  periodSeconds: 10 # Check every 10s
  timeoutSeconds: 5
  failureThreshold: 3 # Restart after 3 failures

# Readiness Probe: Is the model loaded and ready?
# Controls traffic routing, removes from service if failing
readinessProbe:
  httpGet:
    path: /v1/models
    port: 8000
  periodSeconds: 5 # Check frequently for fast recovery
  timeoutSeconds: 2
  failureThreshold: 3
```

Port Configuration by Role

Different pod roles use different ports:

Pod Role	Port	Description
Prefill	8000	Direct vLLM API access
Decode	8200	Proxied through sidecar (8200 → 8000)
Standalone	8000	Single-node deployments

Always configure probes to match the pod's serving port.

How It Works

/health Endpoint

The `/health` endpoint provides a basic liveness check:

```
$ curl http://localhost:8000/health
{}

```

Behavior:

- Returns `200 OK` immediately when vLLM server starts
- Does **not** wait for model loading
- Use for `livenessProbe` only

`/v1/models` Endpoint (OpenAI-Compatible)

The `/v1/models` endpoint is model-aware and indicates true readiness:

```
$ curl http://localhost:8000/v1/models
{
  "object": "list",
  "data": [
    {
      "id": "meta-llama/Llama-3.1-8B-Instruct",
      "object": "model",
      "created": 1704321600,
      "owned_by": "vllm"
    }
  ]
}

```

Behavior:

- Returns `503` or connection refused during model loading
- Returns `200 OK` with model metadata once ready
- Ideal for `startupProbe` and `readinessProbe`

Probe Lifecycle

```

Container Start
↓
[startupProbe on /v1/models]
↓ (30s intervals, up to 30min)
✓ Model loaded → Startup complete
↓
[livenessProbe on /health] ← Restarts if server crashes
[readinessProbe on /v1/models] ← Routes traffic when ready

```

Benefits

HTTP Probes vs Exec Probes

HTTP Probes (Recommended):

- ☒ Lightweight, no exec overhead
- ☒ Compatible with cloud load balancers
- ☒ Native Kubernetes integration
- ☒ Better observability and metrics
- ☒ Uses existing vLLM endpoints

Exec Probes (Legacy):

- ☒ Higher overhead (fork/exec per probe)
- ☒ Incompatible with many cloud load balancers
- ☒ Requires custom scripts in container
- ⚠ More complex to debug and maintain

For Production Deployments

- ☒ Prevent premature traffic routing
- ☒ Avoid failed requests during startup
- ☒ Enable safe rolling updates
- ☒ Faster detection of unhealthy pods
- ☒ Better integration with service meshes

For E2E Testing

- ☒ Eliminate arbitrary sleep times
- ☒ Faster test execution
- ☒ More reliable test results
- ☒ Better error detection

Examples

Wide Endpoint Deployment

Example from `guides/wide-ep-lws/manifests/modelserver/base/decode.yaml` :

```
apiVersion: v1
kind: Pod
metadata:
  name: vllm-decode
spec:
  containers:
  - name: vllm
    image: ghcr.io/llm-d/llm-d:latest
    ports:
    - containerPort: 8200
      protocol: TCP

  startupProbe:
    httpGet:
      path: /v1/models
      port: 8200
    initialDelaySeconds: 30
    periodSeconds: 10
    timeoutSeconds: 5
    failureThreshold: 60

  livenessProbe:
    httpGet:
      path: /health
      port: 8200
    periodSeconds: 30
    timeoutSeconds: 5
    failureThreshold: 3

  readinessProbe:
    httpGet:
      path: /v1/models
      port: 8200
    periodSeconds: 10
    timeoutSeconds: 5
    failureThreshold: 3
```

Testing Probes

Manual Testing

Test the endpoints directly in a running pod:

```
# Get pod name
POD=$(kubectl get pods -n llm-d -l app=vllm -o name | head -1)

# Test liveness endpoint
kubectl exec -n llm-d $POD -- curl -sf http://localhost:8000/health

# Test readiness endpoint
kubectl exec -n llm-d $POD -- curl -sf http://localhost:8000/v1/models | jq '!
```

Verification

Check probe status in Kubernetes:

```
# Watch pod readiness
kubectl get pods -n llm-d -w
```

```
# Check probe configuration
kubectl describe pod -n llm-d $POD | grep -A 10 "Liveness:"
```

```
# View probe-related events
kubectl get events -n llm-d --field-selector involvedObject.name=${POD##*/}
```

Expected behavior:

1. Pod starts, enters `Running` state
2. Startup probe checks `/v1/models` repeatedly (30s intervals)
3. Once model loads, startup probe succeeds
4. Readiness probe takes over, pod becomes `Ready`
5. Traffic is routed to pod
6. Liveness probe monitors server health continuously

Troubleshooting

Pod Stuck in Not Ready

```
# Check startup probe status
kubectl describe pod -n llm-d $POD | grep -A 5 "Startup:"

# Check if model is loading slowly
kubectl logs -n llm-d $POD | grep -i "loading model"

# Test endpoint manually
kubectl exec -n llm-d $POD -- curl -v http://localhost:8000/v1/models
```

Common causes:

- Model download taking longer than `failureThreshold` allows
- Insufficient resources (CPU/memory/GPU)
- Wrong port in probe configuration
- Network issues preventing model download

Solutions:

- Increase `failureThreshold` or `periodSeconds` in `startupProbe`
- Pre-download models using init containers or persistent volumes
- Verify pod has sufficient resources allocated
- Check probe configuration matches actual serving port

Probe Failures After Startup

Check recent probe failures

```
kubectrl get events -n llm-d | grep -i probe
```

Check pod logs for errors

```
kubectrl logs -n llm-d $POD --tail=100
```

Common causes:

- vLLM server crashed (liveness probe fails)
- Model unloaded or corrupted (readiness probe fails)
- Resource exhaustion (OOM, GPU errors)
- Network connectivity issues

Additional Resources

- [Kubernetes Probe Configuration](#)
- [vLLM OpenAI-Compatible Server](#)
- [llm-d Getting Started](#)
- [llm-d Monitoring Guide](#)

Related Issues

- [vLLM #6073](#) - Request for dedicated `/ready` endpoint
- [vLLM](#) currently relies on `/v1/models` for model-aware readiness checking

Router Operations Guide

This guide covers operational best practices, high availability deployment architectures, and container sizing recommendations for the llm-d Router components.

1. Endpoint Picker Operations

When deploying the Endpoint Picker (EPP) in either **Standalone** or **Gateway** mode, resource allocations and multi-replica scaling behaviors depend on expected query throughput, prefix cache matching complexity, and high availability (HA) requirements.

High Availability & Scaling Modes

When running multiple replicas of the Endpoint Picker (`router.epp.replicas > 1`), its behavior depends on the configured HA mode.

Active-Passive Mode (Default)

By default, multi-replica EPP deployments automatically enable the `--ha-enable-leader-election` flag. One leader replica actively serves routing decisions and coordinates lease status, while remaining replicas act as warm standbys.

- **Sizing & Capacity Impact:** Scaling replica count does not increase total request throughput capacity, as only the single active leader replica handles external processing requests.
- **Fail Open:** EPP is configured to fail open (`router.proxy.failOpen=true`) to the model server in the event of a leader election or pod restart. This ensures that client requests are not dropped and continue to be processed by the model server if EPP is unresponsive or unavailable.

```
router:
  proxy:
    failOpen: true
```

Active-Active Mode

To scale routing throughput concurrently across all EPP replicas, disable leader election by passing `ha-enable-leader-election: false` under `router.epp.flags` :

```
router:
  epp:
    replicas: 3
    flags:
      ha-enable-leader-election: false
```

- **Near-Linear Throughput Scaling:** Multiple EPP replicas share incoming request load concurrently:

Replicas	Scaling Factor
1	1.0x
2	2.0x
3	2.7x
4	3.5x

- **Warning (Plugin & Prefix Compatibility):** In active-active mode, you must only use active-active compatible plugins—specifically plugins that query backend model servers dynamically for real-time metrics and state (such as queue depth or KV-cache utilization scorers). Avoid approximate prefix caching plugins in active-active mode; because replicas do not share local memory state, prefix routing partitions across replicas and degrades cache hit rates significantly.

Container Resource Sizing

CPU Allocation

- **Rule of Thumb:** Allocate **0.5 to 1.0 CPU cores per request/second** of expected throughput for large agentic workloads (~100k input / 1k output tokens).
- **Prefix Matching Overhead:** Increasing `maxPrefixTokensToMatch` increases CPU consumption. At lower throughputs, a large prefix limit (such as 100,000 tokens / 6,250 blocks with `blockSizeTokens: 16`) can increase CPU consumption by over 100% compared to a small limit (4,096 tokens / 256 blocks) due to block search overhead.
- **Idle Scraping Overhead:** Idle CPU consumption scales with total model-serving pods due to background Prometheus scraping. In a cluster with 100 pods, EPP idle consumption reaches approximately **7.5 cores**.

Memory Allocation

- **Inflight Concurrency:** Memory footprint scales directly with concurrent inflight requests and output decode length.
- **Sizing Guidelines:**
 - At 50 to 100 requests/second with 1k output tokens, EPP requires **4 to 6 GiB** of memory.
 - For long-output generation (e.g., 5k+ output tokens), memory footprint can exceed **20 GiB** due to concurrent request state accumulation.

Performance Reference Data

Empirical benchmark reference data for Qwen/Qwen3-8B simulation across 100 serving pods:

Throughput and Prefix Block Sizing (100k Input / 1k Output Tokens)

Configuration	Request Rate (Req/s)	maxPrefixTokensToMatch	Peak CPU (Cores)	Peak Memory (GiB)	Scheduler P50 Latency (s)
Small Prefix Match	5.0	4096	1.19	0.26	0.00010
Large Prefix Match	5.0	100000	3.82	0.65	0.00010
Small Prefix Match	98.7	4096	35.17	2.46	0.00014
Large Prefix Match	98.8	100000	46.50	3.41	0.00020

Output Length Variation (50 Req/s Constant Throughput)

Input Tokens	Output Tokens	maxPrefixTokensToMatch	Peak CPU (Cores)	Peak Memory (GiB)
100k	500	4096	15.13	2.27
100k	500	32768	17.14	3.76
100k	1000	4096	17.51	3.66
100k	1000	32768	20.28	5.23
100k	5000	16384	30.95	12.54
100k	10000	8192	32.53	12.54

2. Proxy Operations in Standalone Mode

The following operational guidelines and proxy scaling architectures apply **exclusively to Standalone Mode** (`llm-d-router-standalone`), where a proxy (Envoy or Agentgateway) intercepts client requests and external-processes them via EPP.

Horizontally Scalable Proxy Service (Service Mode)

By default, the standalone chart deploys the proxy as a sidecar container inside the EPP pod. To scale data plane throughput independently from control plane intelligence, deploy the proxy as a separate horizontally scalable Deployment and Service by setting `router.proxy.mode=service` .

In this decoupled architecture, the proxy communicates with EPP over the in-cluster EPP Service. If EPP undergoes active-passive leader failover or momentary pod restarts, the proxy fails open by default (`router.proxy.failOpen=true`), preserving uninterrupted client request processing.

```
helm install my-standalone-router ./config/charts/llm-d-router-standalone \
--set router.modelServers.matchLabels.app=my-vllm-service \
--set router.inferencePool.create=false \
--set router.proxy.mode=service \
--set router.proxy.replicas=3
```

High Availability with Fail-open

By default, in service mode, fail open is enabled. To disable it, set `router.proxy.failOpen=false` .

Empirical benchmark reference data for Qwen/Qwen2.5-1.5B-Instruct simulation across 2 model server replicas with forceful and graceful Leader EPP pod termination. Across various Envoy proxy replica counts, `router.proxy.failOpen=true` maintains high request availability during leader pod teardown. Residual errors occur during socket teardown at the exact moment of pod termination.

Scenario	Envoy Replicas	EPP Replicas	Total Requests	Successful Requests (Throughput)	Errors
Scenario 1 Leader election with standby pod, pod terminates immediately without graceful termination	1	2	598	598 (100%)	0
	2	2	591	584 (98.8%)	7
Scenario 2 Leader election with no standby pod, pod terminates immediately without graceful termination	1	1	598	588 (98.3%)	1
	2	1	592	589 (99.3%)	4
Scenario 3 Leader election with standby pod, pod terminates with graceful termination	1	2	593	591 (99.7%)	2
	2	2	594	583 (98.1%)	11
Scenario 4 Leader election with no standby pod, pod terminates with graceful termination	1	1	591	591 (100%)	0
	2	1	593	589 (99.3%)	4

Proxy Container Resource Sizing

When running Envoy as the standalone proxy, CPU consumption scales linearly with client request rate, while memory consumption remains stable across workloads.

CPU & Memory Guidelines

- **CPU Allocation:** For < 10 requests/second, **1.2 to 2.0 cores** is sufficient. For 100 requests/second at 100k context lengths, allocate at least **8 cores** (peak observed at 7.27 cores). For high concurrency at smaller context lengths (892 requests/second at 10k context), allocate at least **10 cores**.
- **Memory Footprint:** Envoy memory footprint remains stable between **1.3 and 1.4 GiB** across all tested throughputs and context lengths. Allocate **2 GiB** baseline.

Envoy Performance Reference Data

Input Tokens	Output Tokens	Throughput (Req/s)	Peak CPU (Cores)	Peak Memory (GiB)
100k	1k	10.0	1.20	1.30
100k	1k	100.0	7.27	< 1.40
10k	1k	892.0	8.78	1.40

Helm Resource Override Example

Example `resource_overrides.yaml` configuring container resources for both EPP and standalone Envoy proxy containers supporting 50 requests/second for 100k/1k token workloads:

```
router:
  epp:
    resources:
      requests:
        cpu: "32"
        memory: "64Gi"
    limits:
      memory: "128Gi"

  proxy:
    resources:
      requests:
        cpu: "8"
        memory: "2Gi"
    limits:
      memory: "4Gi"
```

```
helm install optimize-baseline ./config/charts/llm-d-router-standalone -f resource_overrides.yaml
```

llm-d Async Processor Operations Guide

This guide covers operational best practices, scaling behavior, and container sizing recommendations for the Async Processor.

The Async Processor is a **lightweight dispatch agent**, not an inference engine. It pulls requests from a message queue (Redis or GCP Pub/Sub), passes them through dispatch gates, and forwards them to the llm-d Router over HTTP. Because the heavy lifting (token generation) happens on the model servers, the processor is almost entirely **I/O-bound**: each worker spends the vast majority of its lifetime blocked on a long-running HTTP call to the Router. This shapes every sizing decision below — the container is small, and the constraint is concurrency and payload buffering, not compute.

[!NOTE] The **concurrency** guidance in § 2 is backed by the measured sweep in that section. The **CPU/memory** figures in § 3 are architecture-derived starting points — validate them against your own workload and prefer your own measurements where you have them.

1. Throughput Model

Understanding how the processor scales is a prerequisite to sizing it.

- **Workers are the unit of in-flight concurrency.** Each worker pulls one request, dispatches it, and blocks until the Router returns a result (or the deadline expires). The total number of requests the processor can have in flight at once is the sum of its worker counts.
- **The bottleneck is downstream, not the processor.** Dispatch rate is ultimately limited by inference-server capacity and the dispatch gates, not by the processor's CPU. Adding workers beyond what the inference pool can absorb only grows queue backlog and memory, not throughput.
- **Scaling is horizontal and stateless.** The processor is a pull-based consumer. Running N replicas against the same queue multiplies effective concurrency: $\text{total in-flight} = \text{replicas} \times \text{workers-per-replica}$. Queue backends distribute messages across all consumers, so no leader election or coordination is required.

Concurrency Configuration

In the helm chart, concurrency is controlled by `ap.concurrency` (default `64`), or per-pool via `ap.workerPools` when you need independent parallelism for different queues/topics:

```
ap:
  concurrency: 64    # global worker count (default)
  # OR, for per-queue control:
  workerPools:
    - id: "high-priority"
      workers: 16
    - id: "bulk"
      workers: 8
```

When `workerPools` is set, the per-pool `workers` value overrides the global `concurrency` for that pool. Total in-flight concurrency for the replica is the sum of all pool worker counts.

2. Choosing Concurrency (the most important sizing decision)

Concurrency is the single highest-impact knob. The default of `64` is a sane starting point — enough to keep a modest pool busy — but it is **not** automatically right for yours: a single large GPU still has headroom above it, and high-throughput or long-output workloads need considerably more. The sizing rule is Little's Law:

$$\text{required_workers} \approx \text{target_throughput (req/s)} \times \text{avg_request_latency (s)}$$

Because LLM requests are slow (seconds to minutes end-to-end), the latency multiplier is large: with a 2 s average request, `concurrency: 64` caps you at $64 / 2 = 32$ req/s, no matter how much GPU sits behind the Router (the old default of `8` capped you at just 4 req/s).

Measured sweep

We ran a closed-loop sweep (each worker holds one in-flight request for its full duration — exactly an Async Processor worker) against a live llm-d stack: **Llama-3.1-8B on vLLM (`--max-num-seq 1024`), a single H100-80GB**, ~256-token outputs, dispatched through the inference gateway + EPP. Throughput and vLLM KV-cache utilization vs. concurrency:

Concurrency	Throughput (req/s)	Mean latency (s)	GPU KV used	Notes
8	4.2	1.9	<1%	old default — GPU essentially idle
32	14.7	2.2	3%	
64 (default)	24.1	2.6	5%	current default
128	35.8	3.6	12%	approaching saturation
192	40.4	4.7	16%	knee — best throughput/latency
256	44.3	5.8	20%	past knee, latency climbing
512	61.4	9.7	51%	unstable: errors in repeated runs

Takeaways:

- **The 64 default reaches ~60% of peak throughput.** At concurrency: 64 this H100 sustains ~24 req/s (KV 5%) — a solid start, but still short of the ~40 req/s at the knee. The old default of 8 managed only ~4 req/s (KV <1%), roughly **10× below** the knee.
- **The knee for this backend is ~128–192 — about 2–3× the 64 default.** Beyond it, latency rises faster than throughput (256→512 doubled latency for ~40% more throughput).
- **Little's Law held exactly.** In-flight requests (throughput × latency) matched the worker count at every stable point — workers are never idle in the closed loop, so concurrency directly sets the load on the backend.
- **A single replica destabilized past ~256.** At very high concurrency a lone processor overwhelmed the gateway/EPP connection path (stalls and errors recurred). This is a reason to **scale out** rather than push one replica arbitrarily high (see § 4).

Applying it

1. Estimate (or measure) your workload's average end-to-end request latency W at the Router.
2. Set concurrency $\approx \text{ceil}(\text{target_throughput} \times W \times 1.3)$ (the 1.3 is headroom).
3. Cap per-replica concurrency at a level your gateway/EPP handles cleanly ($\approx 128\text{--}256$ in the test above) and add replicas beyond that.
4. **With a saturation/budget gate, you can set concurrency above this number and let the gate throttle dispatch** — but remember a gated worker still holds its pulled message in memory, so don't set it absurdly high.

Quick anchor (derived from the sweep, ~256-token outputs):

Scenario	Target tput	~Latency	Suggested concurrency	Topology
Single small GPU (dev)	~30 req/s	~3–5 s	128	1 replica
Saturate 1× H100 (8B)	~40 req/s	~5 s	~192	1 replica
Larger fleet	scale linearly	per workload	$\text{tput} \times W \times 1.3$	$N \text{ replicas} \times \leq 256$

[!NOTE] For long-output workloads, w grows proportionally (a 5k-token completion is ~10–20× a 256-token one), so the required concurrency grows with it — long-output pipelines often need concurrency in the high hundreds to low thousands, split across replicas.

3. Container Resource Sizing

The chart's container defaults are deliberately conservative — suitable for the default concurrency: 64 with small payloads:

resources:
requests:
 cpu: 100m
 memory: 256Mi
limits:
 cpu: 500m
 memory: 512Mi

Scale these from the two variables that actually drive consumption: **worker concurrency** and **per-request payload size**.

Memory Allocation

Memory is the dimension most likely to require tuning, and it scales predictably:

- **Per-request buffering dominates.** Each in-flight worker buffers its request body and the result payload in memory. A rough model:

$\text{memory} \approx \text{baseline} (\sim 128 \text{ MiB}) + (\text{concurrency} \times \text{peak payload size} \times \text{buffering factor} \sim 2)$

The buffering factor accounts for serialized request + response held simultaneously, plus marshaling overhead.

- **Payload size, not worker count, is what forces memory up.** With small payloads (a few KiB), even high concurrency is cheap: at `concurrency: 192` and `~16 KiB` payloads, payload memory is only $192 \times 16 \text{ KiB} \times 2 \approx 6 \text{ MiB}$ — the default `256Mi` is plenty. Large payloads flip this: a 100k-token prompt with a 5k-token completion is `~1 MiB` per request, so `concurrency: 256` needs $128 \text{ MiB} + 256 \times 1 \text{ MiB} \times 2 \approx 640 \text{ MiB}$. **Raise memory when payloads are large, not merely when concurrency is high.**
- **Backlog does not accumulate in the processor.** Unlike the Router, undispached requests stay in the message queue (Redis/Pub/Sub), not in processor memory. Only in-flight requests count toward the footprint, which is why memory tracks concurrency rather than queue depth.

CPU Allocation

CPU consumption is low because workers are blocked on I/O most of the time. It scales with **dispatch rate**, not concurrency:

- **Per-dispatch cost** comes from JSON (de)serialization of request and result payloads, gate evaluation, and queue read/write. For typical agentic payloads, budget on the order of **0.5–1.0 vCPU per 100 dispatches/second**. Larger payloads raise the serialization cost proportionally.
- **Gate overhead.** The `prometheus-saturation` and `prometheus-budget` gates poll Prometheus on an interval; the cost is small and independent of dispatch rate. Setting `ap.prometheusCacheTTL` reduces query frequency when many workers share a gate. The `constant` and `redis` gates add negligible CPU.
- **Redis Sorted Set polling.** With the Redis backend, `pollIntervalMs` (default `1000`) and `batchSize` (default `10`) govern queue-read frequency. Aggressive (low-interval, small-batch) polling raises idle CPU and Redis load; tune `batchSize` up before lowering the interval.

Sizing Reference (starting points)

Derived guidance for a single replica, by concurrency and payload profile. Treat as initial requests/limits to validate under load, not measured peaks.

Workload profile	Concurrency	Payload (in/out)	CPU request	Memory request	Memory limit
Default / light	64	small ($\leq 4k/\leq 512$)	100m	256Mi	512Mi
Saturate 1 small GPU	128–192	small ($\leq 4k/\leq 512$)	500m	256Mi	512Mi
Large agentic	256	large (100k/1k)	1	1Gi	2Gi
Long-output	256	large (100k/5k+)	1	2Gi	4Gi

Note how the second row keeps the default memory despite $2\text{--}3\times$ the concurrency — small payloads keep memory flat; only the large-payload rows need more.

Set the **memory limit close to the request** (the footprint is bounded by `concurrency × payload` , so it does not spike with backlog), and keep CPU limits generous relative to requests to absorb dispatch bursts without throttling.

4. Horizontal Scaling

Prefer horizontal scaling over a single large worker pool once a replica is comfortably sized — it improves availability and spreads queue-read and gate-polling load.

- **Increase replicas to raise total in-flight capacity** up to what the inference pool can absorb: `total in-flight = replicas × concurrency` . Beyond the inference pool's capacity, gates will hold requests closed and extra workers sit idle.
- **No coordination required.** Replicas independently pull from the shared queue. Redis Sorted Set and GCP Pub/Sub both distribute messages across consumers; Redis Pub/Sub fans out (each replica sees all messages), so use Sorted Set or Pub/Sub-per-consumer semantics when load should be *shared* rather than duplicated.
- **Gates remain effective across replicas.** Prometheus-based gates observe global inference-server saturation, so independent replicas converge on the same dispatch decisions without sharing state.
- **Graceful shutdown.** The processor drains in-flight work on termination (`drainTimeout` , default `2m`). Ensure your `pod terminationGracePeriodSeconds` is at least `drainTimeout` so rolling updates and scale-down do not abandon in-flight requests.

Helm Resource Override Example

Example values overriding concurrency and container resources for a large-agentic profile (100k/1k payloads) at higher concurrency:

```
ap:
concurrency: 256
resources:
requests:
cpu: "1"
memory: "1Gi"
limits:
cpu: "2"
memory: "2Gi"
```

```
helm install llm-d-async \
oci://ghcr.io/llm-d/charts/llm-d-async \
-f guides/batch-serving/asynchronous-processing/${MQ_PROVIDER}/values.yaml \
-f resource_overrides.yaml \
--set ap.igwBaseUrl=http://${IP}:80 \
-n ${NAMESPACE} --create-namespace --version ${ASYNC_VERSION}
```

Related

- [Asynchronous Processing Well-Lit Path](#) — overview and use cases.
- [Asynchronous Processing Guide](#) — deployment instructions for Redis and GCP Pub/Sub.
- [Async Processor Architecture](#) — internal mechanics, gates, and queue integrations.
- [llm-d Router Operations Guide](#) — sizing for the Router/EPP and standalone proxy.

Graceful Shutdown & Request Draining

Overview

When a pod in an llm-d deployment is terminated — during a scale-down, a rolling update, or a node drain — in-flight inference requests are at risk. Because inference requests are long-running and compute-intensive, abruptly killing a pod can fail requests that were seconds away from completing.

Handling termination gracefully spans **two layers**:

1. **Routing layer** (Router / Endpoint Picker) — stop sending *new* requests to the terminating pod, and drain any requests still queued in the scheduler.
2. **Model server layer** (vLLM) — let *in-flight* inference requests finish (or drain for a bounded window) before the process exits.

This guide covers graceful shutdown for **general (non-disaggregated) serving**. For prefill/decode disaggregation, where in-flight requests span multiple servers and KV caches are held across instances, see Disaggregated Serving Operations (vLLM).

The Kubernetes Pod Termination Sequence

llm-d relies on the standard Kubernetes pod termination process:

1. **Termination triggered** — the pod's state is set to `Terminating`.
2. **InferencePool update** — the pod is removed from the endpoints of its associated `InferencePool`, so the Router stops routing **new** traffic to it. (For standard Kubernetes objects this is equivalent to removal from a `Service`.)
3. **preStop hook** — if defined, the container's `preStop` hook executes.
4. **SIGTERM** — Kubernetes sends `SIGTERM` to the main process in each container.
5. **Termination grace period** — the pod is given `terminationGracePeriodSeconds` (default **30s**) to exit. If it has not exited by the end of this window, Kubernetes sends `SIGKILL` to force termination.

New traffic is handled automatically by step 2. The rest of this guide is about what happens to requests that are **already in flight** when `SIGTERM` arrives.

Draining the Model Server (vLLM)

How vLLM responds to `SIGTERM` determines whether in-flight requests are drained or dropped:

- **Default** — vLLM immediately `aborts` in-flight requests and exits. Those requests fail with an error status code.

- **With** `--shutdown-timeout N` — vLLM catches `SIGTERM` and continues serving the currently running requests for up to `N` seconds. After the timeout, any requests still in flight are `aborted` and returned an error.

[!IMPORTANT] `terminationGracePeriodSeconds` must be **greater than** `--shutdown-timeout` (plus a small buffer for the `preStop` hook and process cleanup). Otherwise Kubernetes sends `SIGKILL` before the drain completes, and the graceful window is never fully used.

Request Cancellation on Client Disconnect

Independent of shutdown, model servers like vLLM support **request cancellation**: when a client disconnects, the in-progress request is freed rather than run to completion. When a request is disconnected, vLLM triggers its `abort` codepath, releasing the KV cache and compute resources held by that request.

This matters during shutdown and rolling updates: clients (or the gateway) that retry against a draining pod should disconnect the original request so its resources are reclaimed promptly instead of being held until the request finishes on its own.

For the disaggregated case — where a cancelled request may have KV blocks held on a separate prefill instance — see Request Cancellation.

Draining the Routing Layer (EPP)

The Endpoint Picker (EPP) queues requests before dispatching them to model servers. During a graceful shutdown, the EPP's flow controller **evicts queued requests with a retryable 503 Service Unavailable** (outcome `rejected-shutting-down`), signaling transient unavailability so that clients and upstream gateways **retry** rather than treating the drain as a hard fault.

These retryable semantics only apply while the EPP is still reachable over its `ext_proc` stream. Once the EPP process is actually gone, the outcome is governed by the `InferencePool`'s `failureMode`. For high-availability Router configurations and `failOpen` behavior during leader teardown, see Router Operations.

For the full set of flow-control outcome codes, see Flow Control.

Recommended Configuration

A model server pod configured for graceful draining:

```

apiVersion: v1
kind: Pod
metadata:
  name: vllm
spec:
  # Must exceed --shutdown-timeout plus a buffer for preStop + cleanup.
  terminationGracePeriodSeconds: 120
  containers:
  - name: vllm
    image: ghcr.io/llm-d/llm-d:latest
    args:
    - "--shutdown-timeout"
    - "90"      # Drain in-flight requests for up to 90s after SIGTERM.
    ports:
    - containerPort: 8000
      protocol: TCP
    # Readiness gates traffic; see readiness-probes.md.
    readinessProbe:
      httpGet:
        path: /v1/models
        port: 8000
      periodSeconds: 5
      timeoutSeconds: 2
      failureThreshold: 3

```

Guidance:

- Set `--shutdown-timeout` to roughly the p99 request duration you want to allow to complete on drain.
- Set `terminationGracePeriodSeconds` above that with headroom (here 120s vs 90s).
- Pair with model-aware readiness probes so pods are only marked `Ready` once the model is loaded — see Readiness Probes.

Verification

Send a long-running request, then delete the pod and confirm the request completes instead of failing:

```

# Start a long generation in the background against a target pod.
# Then trigger termination:
kubectl delete pod -n llm-d <pod-name>

# Watch the drain in the model server logs.
kubectl logs -n llm-d <pod-name> --tail=50 | grep -iE "shutdown|drain|abort|SIGTERM"

# Observe the pod staying in Terminating until the drain window elapses.
kubectl get pod -n llm-d <pod-name> -w

```

Expected behavior with `--shutdown-timeout` set:

1. Pod enters `Terminating`; it is removed from the `InferencePool` (no new traffic).
2. vLLM catches `SIGTERM` and keeps serving in-flight requests.
3. The in-flight request completes successfully.
4. The pod exits before `terminationGracePeriodSeconds` elapses (no `SIGKILL`).

Troubleshooting

In-flight requests still fail on scale-down / rollout

- Confirm `--shutdown-timeout` is set; without it vLLM aborts immediately.
- Confirm `terminationGracePeriodSeconds > --shutdown-timeout`; otherwise `SIGKILL` fires mid-drain.

Clients see hard errors instead of retries during shutdown

- Ensure clients/gateway treat `503` as retryable — the EPP drains queued requests with a retryable `503`, not a fatal error.
- Check the `InferencePool failureMode` for behavior once the EPP is gone.

Additional Resources

- Disaggregated Serving Operations (vLLM)
- Readiness Probes
- Router Operations
- Flow Control
- Kubernetes Pod Termination

Disaggregated Serving Operations

Disaggregated serving separates the **prefill** and **decode** stages of LLM inference onto different model server instances. While disaggregated serving offers superior performance and resource efficiency, it introduces operational complexity around dynamic connections, request cancellation, fault tolerance, and safe rollouts.

For an overview of the architecture, request flow orchestration, and KV cache transfer fundamentals, see the [Disaggregated Serving Concepts](#) page.

Engine Operations Guides

Operational behavior differs by model server engine:

- **vLLM Operations** — Dynamic connections via NIXL peer-to-peer handshake, lease-based KV block management, fault tolerance policies (`fail` vs `recompute`), and rollout compatibility.
- **SGLang Operations** — Lazy connection establishment via prefill bootstrap server, heartbeat-based fault isolation, waiting timeouts, and rollout compatibility.

Disaggregated Serving: Operations (vLLM)

While disaggregated serving can offer superior performance, it introduces additional operational complexity, including:

- Dynamic Connections - how to add or remove P and D instances on the fly when instances require point-to-point RDMA connections
- Request Cancellation - how to free KV caches from the instances when requests stop in a distributed setting
- Fault Tolerance - how to ensure crashes do not create cascading failures and that resources are cleaned up
- Rollouts - how to roll out changes to the service, such as the version of the vLLM image
- Known NIXL Connector Issues and Limitations - current gaps and bugs in the NIXL connector to plan around

This page documents architectural considerations that impact these common operations flows.

Dynamic Connections

In production environments, it is common for model server replicas to be created and destroyed during the running of the service. In a disaggregated configuration, the ability to dynamically add/remove replicas from the deployment is complicated by the need to establish/destroy connections between P and D workers on the fly.

vLLM supports this functionality via NIXL's APIs, which enable dynamically adding and removing connections.

Scale-Up

To create new connections, vLLM executes a "NIXL Handshake" between the D and P instances to setup the RDMA connection. This is a relatively expensive operation (~5s) that is done once per pair, with all subsequent requests leveraging the existing connection. llm-d uses a "dynamic lazy" roll-out strategy, avoiding the need for a centralized bootstrap server maintaining global state.

It works like this:

sequenceDiagram

participant R as Routing Proxy

participant P as Prefill Engine

participant D as Decode Engine

R->>P: Request with do_remote_decode=True

P-->>P: Run prefill

P->>R: Response with KVTransferParams including remote_host, remote_port, remote_request_id, and remote_kv_blocks

R->>D: Request with KVTransferParams

alt No connection

D-->>D: Spawn background thread

D-->>P: Request NIXLMetadata (via ZMQ)

P-->>D: Return NIXLMetadata (via ZMQ)

D-->>D: Bootstrap RDMA connection

end

D-->>P: NIXL READ: pull KV cache blocks via RDMA

D-->>D: Run decodes

D->>R: Response

- Prefill instances run a background server thread to handle requests for `NIXLMetadata`. When the P instances finishes processing, it constructs the `KVTransferParams`, which includes (among other things) `remote_host=VLLM_SIDE_CHANNEL_HOST` (the pod IP) and `remote_port=VLLM_SIDE_CHANNEL_PORT` in the response body.
- Decode instances receive the request with the `KVTransferParams`; if there is not yet a connection to the remote P worker, it runs a background thread to fetch the `NIXLMetadata` and create the RDMA connection. This action does not block core engine execution, enabling other requests to proceed as usually.

Discovery

Since model server instances are added to an `InferencePool` via standard Kubernetes selectors and labels, new prefill and decode instances discovered automatically when their pods status becomes `status: Running`.

As a result, new replicas can be added to a running disaggregated deployment without restarts and without need to coordinate within any specialized service discovery plane.

Scale-Down

In Kubernetes, there is a well-defined pod termination process:

- **Termination Triggered:** The pod's state is changed to **Terminating**.
- **InferencePool Update:** The pod is removed from the list of endpoints for associated the `InferencePool`, preventing new traffic from being routed to it. (note: for standard Kubernetes objects, this is equivalent to removal from a Service)
- **PreStop Hook:** If defined, the preStop hook executes.
- **SIGTERM Signal:** Kubernetes sends a SIGTERM signal to the main process in each container.
- **Termination Grace Period:** The pod is given a set amount of time (default is 30 seconds) to shut down gracefully. If it does not terminate by the end of this period, a SIGKILL is sent to force termination.

For **new requests**, instances are automatically removed from the `InferencePool` so no new traffic is routed to terminating pods.

For **running requests**, we can configure how vLLM handles the `SIGTERM`:

- By default, vLLM immediately aborts existing requests and terminates. This fails the running requests with an error status code.
- vLLM can be configured with a `--shutdown-timeout N`. When this is set, vLLM catches the `SIGTERM` and drains the currently running requests for `N` seconds. After this timeout, it aborts any running requests still in flight, returning an error code.

Scaling Down Decode Replicas

Since prefill instances hold the KV's until the decode instances pull them, it is important to ensure that KV's are released on the prefill instance when decode instances are scaled down.

In vLLM, regardless of whether `--shutdown-timeout` is set, requests are aborted during the shutdown process. As part of the abort process, decode instances with not-yet-started KV transfers send a NIXL notification to the remote prefill instances to free the blocks. Thus, scaling down decode replicas always free requests on the P instance.

Scaling Down Prefill Replicas

When scaling down prefill replicas, decode instances may attempt to pull KV blocks from terminated remote prefill instances.

[!WARNING] At current, regardless of `--shutdown-timeout`, there is no way to delay shutdown of a prefill instance until after all blocks have been retrieved. This functionality is work in progress in vLLM.

As a result, prefill scale down will cause KV load failure for in-progress requests on decode instances. To avoid error codes for failed KV transfers, the decode instances can be configured with `kv_load_failure_policy=recompute` to recompute the prefill on the decode instance.

Request Cancellation

Given the compute intensity and duration of inference requests, model servers like vLLM support "Request Cancellation", where currently in-progress requests are freed when the client disconnects.

In a disaggregation setup, this feature is more complicated, because the resources associated with an inference request are spread across multiple servers (as the P instances holds onto the KV caches until they have been retrieved by the D instance). As a result, if the request is canceled while it is still "in-flight" on the D instance but before the KV transfer occurs, we need to ensure that the resources on the P instance are properly cleaned up.

llm-d accomplishes this functionality by building on top of vLLM's existing request cancellation infrastructure. When requests are disconnected in vLLM, it triggers the abort codepath, which cleans up running resources. When request with `do_remote_prefill=True` are aborted, vLLM sends a NIXL notify message, instructing the remote prefill instance to free the KV cache for the cancelled request.

sequenceDiagram
participant R as Router
participant P as Prefill Instance
participant D as Decode Instance

R->>P: Request (do_remote_decode=True)
P->>P: Run prefill
P->>R: Response (w/ KVTransferParams)

R->>D: Request (do_remote_prefill=True)
D->>D: Request disconnected, calls abort
D->>P: NIXL.send_notif
P->>P: Free KVs

[!WARNING] There is a small window in which request cancellation will not trigger KV freeing on the P instance. If the request is disconnected after it is completed on the P worker but before it reaches the D worker's scheduler (for example, if it disconnects while the request is inside Routing Proxy), the D instance never knows about the request and therefore is unable to free the remote blocks on the P worker. In this case, the KV blocks are held on the P instance until the lease expires (`kv_lease_duration` , default 30s), at which point they are freed automatically.

Fault Tolerance

In llm-d's disaggregated serving design, all D instances are connected to all P instances. This creates a critical operational risk - crashes in workers have the potential for cascading failures if the system is not tolerant of failures.

Prefill Instance Failure

Prefill instance crashes are a critical failure mode, since D instances will attempt to pull KVs from no longer running P instances without performing any liveness checks. Since every D worker is connected to every P worker, it is critical to handle such an error on the D worker.

vLLM handles Prefill instance failure by building on top of NIXL's error handling functionality. When a READ is attempted and fails, NIXL returns an error code such as `NIXL_ERR_BACKEND` . vLLM catches this error and handles it according to the `kv_load_failure_policy` :

- **fail (default, recommended):** Immediately fail the request with an error when KV load fails. This prevents performance degradation by avoiding recomputation of prefill work on the decode instance.
- **recompute:** Recompute failed blocks locally on the decode instance. This may cause performance jitter on decode instances as the scheduled prefill will delay and interfere with other decodes. Furthermore, decode instances configured with low-latency optimizations (such as DeepEP LL for Wide EP deployments) may suffer significant slowdowns.

```

sequenceDiagram
    participant R as Routing Proxy
    participant P as Prefill Worker
    participant D as Decode Worker

    R->>P: Request (do_remote_decode=True)
    P->>P: Run prefill
    P->>R: Response

    note over P: P crashes ☒

    R->>D: Request (do_remote_prefill=True)
    D->>P: NIXL_RDMA_READ
    D->>D: NIXL_ERR_BACKEND

    alt kv_load_failure_policy = fail
        D->>R: 500 Response
    else kv_load_failure_policy = recompute
        D->>D: Run full request locally (including prefill)
        D->>R: 200 Response
    end
end

```

Failed Prefill Worker pods are automatically moved to `status: Terminated` state as part of the standard Pod lifecycle. Since llm-d leverages the Kubernetes API Server for service discovery, no additional traffic will be routed to the failed worker until the pod has been restarted and returns to the `status: Running` state.

In this way, llm-d isolates Prefill instance failure.

Decode Instance Failure

While D instance failures are unlikely to result in P instance crashes (since P instance never initiates RDMA operations), there is a challenge around ensuring that KV cache memory on the P instance is not stranded (since the P instance holds onto the KV cache until it has been explicitly pulled from the D instance).

vLLM addresses this with a **lease-based KV block management** system. When a prefill completes, P holds the KV blocks with an initial lease of `kv_lease_duration` (default `30s`). While the request is queued or running on the D instance, D's scheduler periodically sends heartbeat notifications to P, each extending the lease by `lease_extension = kv_lease_duration * 2 // 3` (default `20s`). If D crashes (or becomes unresponsive), heartbeats stop, and P automatically frees the KV blocks when the last lease extension expires — at most `lease_extension` seconds after the last heartbeat.

This approach keeps the worst-case block hold time short without risking premature block eviction when D instances are heavily loaded — as long as D is alive, the heartbeats keep the lease active indefinitely.

sequenceDiagram
participant R as Routing Proxy
participant P as Prefill Instance
participant D as Decode Instance

R->>P: Request (do_remote_decode=True)
P->>P: Run prefill (holds onto KVs with lease)
P->>R: Response

R->>D: Request (do_remote_prefill=True)
D->>P: Heartbeat (extend lease)
D->>P: Heartbeat (extend lease)
note over D: D crashes ☒
note over P: No heartbeat received
P->>P: Lease expires
P->>P: Free KV Blocks

The `kv_lease_duration` is configurable via `kv_connector_extra_config`:

```
--kv-transfer-config '{"kv_connector_extra_config": {"kv_lease_duration": 10}}'
```

[!NOTE] Lease-based KV block TTL requires **vLLM v0.22.0** or later. In earlier versions, vLLM used a fixed-timeout approach via `VLLM_NIXL_ABORT_REQUEST_TIMEOUT` (default 480s): the P instance would free stranded KV blocks only after that timeout elapsed, regardless of whether the D instance was still alive. This made the worst-case hold time very long, and reducing the timeout risked premature eviction when D instances were heavily loaded.

Rollouts

In disaggregated serving, rolling out a new version of the model server (e.g. a new version of vLLM or a new configuration) requires care, since prefill-decode instance pairs communicate with each other to execute the KV transfer operation. As a motivating example, vLLM has multiple attention kernel implementations, each of which can have slightly different KV cache layouts - since NIXL pulls the KVs directly from the GPU KV cache memory of the remote instance, we need to ensure these are matching.

By default, vLLM checks for compatibility between instances during the NIXL handshake, failing the request if scheduled to incompatible pods. There is an escape hatch to disable compatibility checking:

```
--kv-transfer-config '{"kv_connector_extra_config": {"enforce_handshake_compat": false}}'
```

[!IMPORTANT] The llm-d EPP currently assumes all P and D instances within an `InferencePool` are compatible and will therefore schedule requests to any arbitrary pair of P and D instances. As a result, it is currently recommended to create a new `InferencePool` for upgrading model servers. When deploying with a `Gateway`, traffic can be gradually shifted to the new `InferencePool` by modifying the `HTTPRoute`.

Known NIXL Connector Issues and Limitations

Beyond the operational flows above, the NIXL connector has a couple of current gaps worth planning deployments around.

Prefill TP > Decode TP Is Not Supported

[!WARNING] **Prefill TP > Decode TP is not supported** for most model architectures. **NixlConnector** only supports the fan-out direction (decode TP $\geq$ prefill TP, e.g. prefill TP=1 $\rightarrow$ decode TP=4). This is a deliberate guard in vLLM's own source, not a bug — always keep decode TP $\geq$ prefill TP.

The reverse direction (prefill TP > decode TP, e.g. prefill TP=8 $\rightarrow$ decode TP=4) is explicitly unsupported for any model that is neither **MLA** nor **Mamba/state-space** based:

```
# num_kv_heads > tp_size with P_TP > D_TP not supported for non-mamba.  
# Mamba models can have replicated FA KV with tp_ratio < 0.  
# MLA models do not need to handle kv replication.  
if not self.use_mla and not self._has_mamba:  
    assert not (  
        tp_ratio < 0 and self.transfer_topo.is_kv_replicated(remote_engine_id)  
    )
```

Source: `vllm/distributed/kv_transfer/kv_connector/v1/nixl/base_worker.py`, function `_validate_remote_agent_handshake`

- v0.26.0: lines 1690-1696
- v0.25.0: lines 1619-1625 (identical logic, not a version regression)

Hitting this path crashes the decode engine with `AssertionError`. A related bug in vLLM's failure-cleanup handler (`_handle_failed_transfer`) often surfaces it as `IndexError: list index out of range` instead, making it look like a generic connector bug rather than an unsupported-topology guard.

Total GPU count (TP $\times$ replicas) and replica counts on each side are unaffected by this constraint — only the TP ratio direction between whichever specific prefill/decode engine pair handles a given request matters.

Stale NIXL Agent Cache After a Prefill Pod Restart

[!WARNING] **Decode-side stale NIXL agent cache after a prefill pod restart** can segfault decode (vllm-project/vllm#49238, open), or leave it silently serving against a dead engine for up to an hour. Until the fixes below ship in an official image, proactively restart any decode replica that was connected to a restarted prefill pod, rather than relying on TTL expiry.

If a prefill pod restarts while decode holds a cached NIXL agent handle for it, decode can segfault, or (after the NIXL-side fix below) silently keep serving against the dead engine's stale cached agent/rkey entries. With a pinned/stable engine ID, decode does not detect the prefill engine is gone until `engine_ttl` expires (default up to one hour), and every request routed to that decode replica fails until then or until the decode pod is restarted.

When decode does *not* segfault, it instead attempts the NIXL RDMA read against the stale agent, which fails with `NIXL_ERR_BACKEND` — the same error path described in Prefill Instance Failure. vLLM then applies `kv_load_failure_policy` (source):

- **recompute** : decode silently falls back to running the prefill itself, logging only an error line for the failed transfer. The client never sees a failed request, so this shows up purely as a throughput/latency regression on that decode replica, not as an error — harder to notice than an outright failure.
- **fail (default)**: the request fails with a 500 response, which at least surfaces the problem to the caller.

Until `engine_ttl` expires, every request that lands on that decode replica repeats this failed-read-then-recompute cycle, so the performance hit persists rather than clearing up after the first failure.

Fix status:

- NIXL-level segfault fixed upstream: ai-dynamo/nixl#1986 (merged ~2026-07-29), returns a clean disconnect error instead of crashing.
- vLLM-side fix, vllm-project/vllm#50047, invalidates decode's cached engine state as soon as the prefill peer is detected gone, instead of waiting out the TTL.

Neither fix is in any published vLLM image yet, including nightly, since vLLM pins an exact NIXL version and the NIXL fix landed after the latest published NIXL release (v1.3.2, 2026-07-24).

Observability

Monitor and debug llm-d deployments with Prometheus metrics, Grafana dashboards, and OpenTelemetry distributed tracing.

[!NOTE] Every well-lit path guide links here for observability setup. Install the stack once and reuse it across guides.

Documentation

- Setup — Install Prometheus and Grafana, load dashboards, and deploy tracing backends
- Metrics — Enable and interpret model server and EPP metrics
- Distributed Tracing — Configure OpenTelemetry across vLLM, the routing proxy, and the EPP
- PromQL Reference — Ready-to-use queries for dashboards and alerting
- Alerting — Apply the default EPP Prometheus alerting rules

Runnable assets

Scripts, Grafana dashboard JSON, alerting rules, and tracing manifests live in `guides/recipes/observability/` in the llm-d repository (not published as website pages).

Metrics

This page covers how to enable and interpret metrics from an llm-d deployment. For Prometheus and Grafana installation, see [Observability Setup](#) first.

[!NOTE] Commands in this page use `${NAMESPACE}` for the namespace where your llm-d workload runs. Set it before following along:

```
bash export NAMESPACE=<your-llm-d-namespace>
```

Prerequisites

- A running llm-d deployment with an InferencePool and model servers — see the [quickstart](#) if needed
- Prometheus and Grafana installed — see [Observability Setup](#)

Step 1: Enable Model Server Metrics

Model server metrics are enabled by default. Configuration varies by deployment method.

Kustomize Deployments

If you deployed your model server using `kustomize build`, add the monitoring component to your `kustomization.yaml`:

components:

```
- ../../recipes/modelserver/components/monitoring    # decode PodMonitor
# - ../../recipes/modelserver/components/monitoring-pd # add for prefill/decode disaggregation
```

The monitoring component creates PodMonitors that scrape model server metrics. See [guides/recipes/modelserver/components/monitoring/](#) for details.

Verify PodMonitors

Verify the PodMonitors exist:

```
kubectl get podmonitors -n ${NAMESPACE}
```

Expected output:

NAME	AGE
decode-podmonitor	5m
prefill-podmonitor	5m

Key vLLM Metrics

Metric	What it measures	Why it matters
vllm:num_requests_running	Active requests being processed	High values indicate GPU saturation; new requests will queue. Watch for sustained spikes
vllm:num_requests_waiting	Requests queued, waiting to be processed	Non-zero means pods are saturated. Primary signal for autoscaling decisions
vllm:kv_cache_usage_perc	KV cache utilization (0.0 to 1.0)	Above 0.9 means GPU memory is nearly full and requests may get preempted or rejected
vllm:time_to_first_token_seconds (histogram)	Time from request arrival to first generated token (TTFT)	Directly impacts user experience. Use histogram_quantile() to query percentiles
vllm:inter_token_latency_seconds (histogram)	Time between consecutive generated tokens (ITL)	Affects streaming response speed. High ITL causes choppy output. Use histogram_quantile() to query percentiles
vllm:prefix_cache_hits_total	Number of prefix cache hits	Compare with prefix_cache_queries_total to get hit rate. Low hit rate suggests the EPP is not routing effectively
vllm:prefix_cache_queries_total	Total prefix cache lookups	Divide prefix_cache_hits_total by this to get hit rate. A dropping ratio indicates routing or prompt pattern changes
vllm:prompt_tokens_total	Total input tokens processed	Use rate() to get tokens/sec per pod. Compare across pods to spot uneven load distribution
vllm:generation_tokens_total	Total output tokens generated	Use rate() alongside prompt tokens to get total throughput. A drop signals degraded model performance

vLLM NIXL KV Transfer Metrics

When vLLM uses NixlConnector for disaggregated serving, it exports metrics for the KV cache transfers between workers. See the vLLM NixlConnector usage guide for configuration details.

Metric	What it measures	Why it matters
vllm:nixl_xfer_time_seconds (histogram)	Time from posting a transfer until the backend reports completion, including submission and data movement	High tail latency can indicate network congestion, large transfers, or backend delays
vllm:nixl_post_time_seconds (histogram)	Synchronous time spent submitting a transfer to the backend	A high value with otherwise normal transfer time points to descriptor setup or submission overhead
vllm:nixl_bytes_transferred (histogram)	Bytes moved per transfer observation	Shows transfer size and KV cache data volume
vllm:nixl_num_descriptors (histogram)	Memory descriptor count per transfer observation	High counts can indicate fragmented or large KV cache allocations
vllm:nixl_num_failed_transfers (counter)	Failed NIXL KV cache transfers	Transfer failures can prevent the consumer from using remote KV blocks
vllm:nixl_num_failed_notifications (counter)	Failed transfer completion notifications	Notification failures can prevent a peer from learning that a transfer completed
vllm:nixl_num_kv_expired_reqs (counter)	Requests whose KV blocks expired before the decoder read them; recorded on the prefill instance	Sustained increases can indicate that kv_lease_duration is too short for the workload or network

[!NOTE] With tensor parallelism, vLLM pools transfer observations from all TP ranks before exporting them. Histogram counts therefore represent rank-level transfer observations, not inference requests, and byte values do not represent the total size of one request across all ranks. These are aggregate Prometheus metrics and are not correlated with individual request or trace IDs.

Key SGLang Metrics

Metric	What it measures	Why it matters
sglang_num_running_reqs	Active requests being processed	High values indicate GPU saturation; new requests will queue
sglang_num_queue_reqs	Requests queued, waiting to be processed	Non-zero means pods are saturated. Primary signal for autoscaling decisions
sglang_token_usage	KV cache token utilization (0.0 to 1.0)	Above 0.9 means GPU memory is nearly full
sglang_cache_hit_rate	Prefix cache hit rate (0.0 to 1.0)	High hit rate indicates efficient KV cache reuse
sglang_time_to_first_token_seconds (histogram)	Time from request arrival to first generated token (TTFT)	Directly impacts user experience. Use histogram_quantile() to query percentiles
sglang_inter_token_latency_seconds (histogram)	Time between consecutive generated tokens (ITL)	Affects streaming response speed. Use histogram_quantile() to query percentiles
sglang_prompt_tokens_total	Total input tokens processed	Use rate() to get tokens/sec per pod
sglang_generation_tokens_total	Total output tokens generated	Use rate() alongside prompt tokens to get total throughput

Step 3: Enable EPP Metrics

EPP (Endpoint Picker) metrics are enabled by default. To verify or enable manually, see the [Monitoring & Tracing Configuration](#) section in the [llm-d-router Helm chart docs](#).

Verify the ServiceMonitor exists:

```
kubectl get servicemonitors -n ${NAMESPACE}
```

Expected output:

```
NAME          AGE
epp-servicemonitor 5m
```

Key llm-d Router EPP Metrics

The EPP exposes Prometheus metrics under the `llm_d_epp_` prefix.

Request and Latency

Metric	What it measures	Why it matters
llm_d_epp_request_total	Total request count per flow ID and priority	Baseline for calculating error rate and throughput per model
llm_d_epp_request_error_total	Error count per flow ID and priority	Rising errors signal backend failures. Alert when error rate exceeds 5%
llm_d_epp_request_duration_seconds	Response latency distribution per flow ID and priority	The SLO metric. Tracks full round-trip time from request to response
llm_d_epp_request_size_bytes	Incoming request size distribution in bytes per flow ID and priority	Helps identify payload size anomalies and exceptionally large incoming prompts
llm_d_epp_response_size_bytes	Outgoing response size distribution in bytes per flow ID and priority	Tracks outgoing bandwidth usage and response payload size distribution
llm_d_epp_request_input_tokens	Input token count distribution per flow ID and priority	Helps identify expensive requests. Long prompts cost more compute
llm_d_epp_request_output_tokens	Output token count distribution per flow ID and priority	Combined with duration, gives normalized cost and generation volume per token
llm_d_epp_request_cached_tokens	Cached prompt token distribution per flow ID and priority	Measures prefix cache utilization reported by model servers
llm_d_epp_request_running	Active request count per flow ID and priority	Shows real-time load concurrency across models
llm_d_epp_request_ntpot_seconds	Normalized time per output token (NTPOT) distribution per flow ID and priority	Key efficiency metric (lower is better). Compare across pods to find stragglers
llm_d_epp_request_ttft_seconds	Time to first token (TTFT) distribution per flow ID and priority	Directly measures user-perceived responsiveness and time to initial output byte
llm_d_epp_request_streaming_tpot_seconds	Time per output token (TPOT) distribution per flow ID and priority; applicable to streaming requests	Tracks ongoing generation speed excluding initial prompt prefill latency
llm_d_epp_request_streaming_itl_seconds	Inter-token latency (ITL) distribution per flow ID and priority; applicable to streaming requests	Measures pacing between consecutive response body chunks; spikes indicate choppy output

Inference Pool

Metric	What it measures	Why it matters
llm_d_epp_ready_endpoints	Number of ready endpoints in the pool	If this drops below expected count, pods are crashing or not scheduling
llm_d_epp_average_kv_cache_utilization	Mean KV cache utilization across the pool	High average utilization signals the pool is nearing memory capacity
llm_d_epp_average_queue_size	Mean queue depth across model servers	Primary signal for pool saturation and autoscaling decisions
llm_d_epp_average_running_requests	Mean running requests across model servers	Tracks concurrency across endpoints
llm_d_epp_std_dev_kv_cache_utilization	Spread of KV cache utilization across endpoints	High standard deviation indicates uneven cache distribution
llm_d_epp_std_dev_queue_size	Spread of queue depth across endpoints	High variance points to routing imbalances
llm_d_epp_std_dev_running_requests	Spread of in-flight requests across endpoints	Helps diagnose stragglers or hotspotting
llm_d_epp_per_endpoint_queue_size	Per-endpoint queue depth	Identifies overloaded model-server replicas

Scheduler, Plugins & Processing Overhead

Metric	What it measures	Why it matters
llm_d_epp_scheduler_attempts_total	Scheduling attempt counts and outcomes	Track failed scheduling attempts. High failure rate indicates filter/scorer misconfiguration
llm_d_epp_scheduler_e2e_duration_seconds	End-to-end scheduling latency distribution	Tracks latency spent within the EPP scheduling cycle
llm_d_epp_plugin_duration_seconds	Per-plugin processing latency distribution	Identifies slow or bottlenecked scheduling plugins
llm_d_epp_plugin_data_scope_violations_total	Total endpoint attribute accesses rejected due to undeclared DataKeys	Non-zero value indicates plugin implementation drift from declaration
llm_d_epp_request_processing_duration_seconds	Time from request receipt until request body has been handled	Measures EPP request ingest and admission overhead
llm_d_epp_response_processing_duration_seconds	EPP response processing latency	Measures response streaming handler overhead
llm_d_epp_info	Build information (commit , build_ref)	Verifies running binary release and build details
llm_d_epp_model_rewrite_decisions_total	Total model rewrite decisions	Tracks request rewrites across configured rewrite rules

In-Flight Load

Metric	What it measures	Why it matters
llm_d_epp_inflight_requests	Requests currently in flight on each endpoint	Per-replica queue depth for load-aware routing and capacity analysis
llm_d_epp_inflight_tokens	Tokens currently in flight on each endpoint	Per-replica token pressure, a finer load signal than request count

Disaggregation & Rollout

Metric	What it measures	Why it matters
llm_d_epp_disagg_decision_total	Routing decisions across disaggregation stages	Tracks breakdown of decode-only, prefill-decode, encode-decode, and full pipeline decisions
llm_d_epp_pd_decision_total	Prefill/decode disaggregation decisions (deprecated handler)	Backward-compatible counter for P/D routing decisions
llm_d_epp_disaggregatedset_strict_header_no_match_total	Strict header selections that matched no endpoint	Surfaces strict selector misconfigurations that fail closed
llm_d_epp_disaggregatedset_revision_gating_share	Current weighted revision gating share (0 to 1)	Monitors canary and rolling deployment shares across disaggregated revisions

Flow Control Metrics

When flow control is enabled (`flowControl` feature gate), these additional metrics are exposed:

Metric	What it measures	Why it matters
llm_d_epp_flow_control_queue_size	Queued request count per flow ID and priority	Growing queue means the pool cannot keep up. Consider scaling or adjusting priority bands
llm_d_epp_flow_control_queue_bytes	Queued payload size in bytes per flow ID and priority	Large queued payloads can exhaust EPP memory. Monitor alongside maxBytes config
llm_d_epp_flow_control_request_queue_duration_seconds	Queuing duration distribution per flow ID and priority	Directly impacts user-perceived latency. High values mean flow control is holding requests too long
llm_d_epp_flow_control_dispatch_cycle_duration_seconds	Internal dispatch cycle duration distribution	Tracks execution speed of the flow control scheduler loop
llm_d_epp_flow_control_request_enqueue_duration_seconds	Request enqueue duration distribution per flow ID and priority	Measures admission overhead entering the flow control queue
llm_d_epp_flow_control_pool_saturation	Pool saturation level (0.0 to 1.0+)	Above 1.0 means demand exceeds capacity and flow control is actively throttling. Scale up or shed load
llm_d_epp_flow_control_stale_endpoints	Number of endpoints with missing or stale telemetry	Non-zero value indicates metrics collection issues rather than genuine overload
llm_d_epp_flow_control_requests_total	Total requests processed by flow control by outcome	Direct signal for rejection (outcome="RejectedCapacity") and eviction rates
llm_d_epp_flow_control_revocations_issued_total	Total in-flight eviction revocations issued by priority	Tracks preemptive capacity reclamation events under contention
llm_d_epp_flow_control_revocations_total	Total in-flight eviction revocations by outcome (confirmed , timed_out)	Confirms stream cancellation success vs timeout
llm_d_epp_flow_control_reclaim_target	Last computed reclamation deficit	Measures capacity deficit driving revocations
llm_d_epp_flow_control_pending_reclaim	Outstanding and cooling pending-reclaim debits	Tracks in-flight reclamation debits before confirmation

Metric	What it measures	Why it matters
llm_d_epp_flow_control_revocation_confirmation_seconds	Revocation issue to confirmed termination duration	Latency signal for in-flight eviction completion
llm_d_epp_program_aware_jains_fairness_index	Jain's fairness index across active programs	Quantifies fairness balance across competing programs
llm_d_epp_program_aware_avg_wait_time_milliseconds	Mean queue wait time per program in ms	Monitors per-program queuing delays
llm_d_epp_program_aware_attained_service_tokens	Time-decayed attained service per program	Tracks historical consumption used for Least Attained Service (LAS) scheduling

Prefix & Multimodal Cache

Metric	What it measures	Why it matters
llm_d_epp_prefix_indexer_size	Entries in the approximate prefix index	Tracks index memory footprint
llm_d_epp_prefix_indexer_hit_ratio	Prefix-match hit ratio distribution	High hit ratio indicates effective prefix caching
llm_d_epp_prefix_indexer_hit_bytes	Bytes matched per prefix lookup	Quantifies data reused from cache
llm_d_epp_encoder_cache_queries_total	Total multimodal encoder-cache lookups	Volume of multimodal items queried against cache
llm_d_epp_encoder_cache_hits_total	Encoder-cache hits per pod	Tracks multimodal embedding cache reuse
llm_d_epp_encoder_cache_hit_ratio	Hit ratio distribution for encoder-cache lookups	Surfaces uneven multimodal cache locality across pods

Predicted Latency & SLO

Metric	What it measures	Why it matters
llm_d_epp_request_predicted_ttft_seconds	Predicted TTFT distribution	Used by latency-based scheduling to evaluate candidates
llm_d_epp_request_ttft_prediction_duration_seconds	Time spent computing TTFT predictions	Measures overhead of the predictor sidecar
llm_d_epp_request_predicted_tpot_seconds	Predicted TPOT distribution	Evaluates token generation latency across backends
llm_d_epp_request_tpot_prediction_duration_seconds	Time spent computing TPOT predictions	Measures TPOT predictor execution latency
llm_d_epp_request_slo_violation_total	Count of requests violating configured latency SLOs	Primary alert signal for SLO degradation
llm_d_epp_inference_request_metric	Consolidated gauge for observed request metrics and SLO states	Flexible single-series export of TTFT, TPOT, and SLO metrics

ext_proc Streams & Data Layer Errors

Metric	What it measures	Why it matters
llm_d_epp_extproc_streams_inflight	Number of open ext_proc gRPC streams (opt-in)	Monitors Envoy↔EPP gRPC stream connections
llm_d_epp_extproc_stream_duration_seconds	Duration ext_proc gRPC streams stay open	Sudden short durations indicate stream reconnect churn
llm_d_epp_extproc_streams_total	Completed ext_proc gRPC streams by status code	Surfaces abnormal stream disconnects and errors
llm_d_epp_datalayer_poll_errors_total	Data-source poll failures per source type	Signals failing telemetry scrapes from model servers
llm_d_epp_datalayer_extract_errors_total	Extractor failures per source/extractor type	Signals parsing failures on scraped telemetry payloads

Key Batch Gateway Metrics

Only relevant if you deployed the Batch Gateway guide. These are the operationally useful series; the definitional source of truth for names, types, and labels is `docs/guides/metrics.md` in the component repo, which also documents the token, cancellation, and startup-recovery counters not listed here.

Unlike the EPP and vLLM metrics above, these names carry no `llm_d_` prefix, so scope queries by `namespace` to avoid picking up unrelated workloads.

Metric	What it measures	Why it matters
http_requests_total	API server requests by method, path, and status	Baseline for API-side error rate — batch submission failures show up here, not in job metrics
http_request_duration_seconds	API server request latency distribution	Submission and status-poll responsiveness
jobs_processed_total	Jobs processed by result (success , failed , skipped , re_enqueued , expired) and reason	The primary outcome signal. expired means jobs aged out before running, which is a capacity problem rather than a failure
job_queue_wait_duration_seconds	Time in the priority queue before pickup	Leading indicator of backlog; rises before end-to-end latency does
job_processing_duration_seconds	End-to-end processing duration by size_bucket	Separates "large jobs are slow" from "everything is slow"
batch_job_e2e_latency_seconds	Submission to terminal state, by status	What the user actually waits for, including queue time
active_workers / total_workers	Active vs configured worker pool size	Their ratio is the saturation signal — sustained near 1.0 means the pool is the bottleneck
processor_inflight_requests	In-flight inference requests during execution	Load the batch path is placing on the inference backends
batch_reconciler_errors_total	GC reconciler cycle errors	Non-zero means orphaned jobs are not being recovered
batch_reconciler_orphans_recovered_total	Orphans recovered by action	Sustained non-zero points at processor crashes upstream
file_storage_operations_total	File storage operations by operation , component , and status	status="exhausted" means retries gave up — input or output data is inaccessible

[!NOTE] The Batch Gateway dashboards compute histogram quantiles by summing buckets across pods before applying histogram_quantile , which yields a fleet-wide approximate percentile rather than an exact one. This is acceptable because each job is processed by exactly one pod, but it can mask a single consistently-slow pod. Add by (le, pod) for a per-pod breakdown.

For alerts built on these metrics, see Alerting.

Step 4: View Dashboards

llm-d provides pre-built Grafana dashboards for common monitoring scenarios.

Access Grafana

[!NOTE] The commands below use namespace and service names from the bundled install script. If you use an existing Prometheus or Grafana instance, adjust the namespace and service names accordingly.

```
kubectl port-forward -n llm-d-monitoring svc/llmd-grafana 3000:80
# Open http://localhost:3000
# Default login: admin / admin
```

Import Dashboards

Load all llm-d dashboards into Grafana:

```
./guides/recipes/observability/load-llm-d-dashboards.sh
```

Verify dashboards were imported:

```
kubectl get configmaps -n llm-d-monitoring -l grafana_dashboard=1
```

Expected output:

NAME	DATA	AGE
llm-d-vllm-overview	1	30s
llm-d-sglang-overview	1	30s
llm-d-failure-saturation-dashboard	1	30s
llm-d-diagnostic-drilldown-dashboard	1	30s
llm-d-performance-kv-cache	1	30s
llm-d-pd-coordinator-metrics	1	30s
llm-d-batch-gateway-apiserver	1	30s
llm-d-batch-gateway-processor	1	30s
llm-d-batch-gateway-gc	1	30s

Or import individual dashboard JSON files manually from `guides/recipes/observability/grafana/dashboards/` :

Dashboard	What it shows
llm-d-vllm-overview.json	General vLLM metrics overview
llm-d-sglang-overview.json	General SGLang metrics overview
llm-d-failure-saturation-dashboard.json	Failure and saturation indicators
llm-d-diagnostic-drilldown-dashboard.json	Detailed diagnostic metrics for troubleshooting
llm-d-performance-kv-cache.json	Performance metrics including KV cache utilization
llm-d-pd-coordinator-metrics.json	Prefill/decode disaggregation metrics
llm-d-batch-gateway-apiserver.json	Batch Gateway API server request rate, latency, and in-flight requests
llm-d-batch-gateway-processor.json	Batch Gateway job throughput, queue wait, worker saturation, and token usage
llm-d-batch-gateway-gc.json	Batch Gateway GC reconciler cycles, orphan recovery, and errors

The three Batch Gateway dashboards are only useful if you deployed the Batch Gateway guide; each has a `namespace` variable to select the namespace it runs in.

Step 5: Query Metrics

Access the Prometheus UI:

```
kubectrl port-forward -n llm-d-monitoring svc/llmd-kube-prometheus-stack-prometheus 9090:9090
# Open http://localhost:9090 (or https://localhost:9090 if TLS is enabled)
```

Cleanup

```
./guides/recipes/observability/install-prometheus-grafana.sh -u -n llm-d-monitoring
```

Troubleshooting

Autoscaler reports "http: server gave HTTP response to HTTPS client"

The autoscaler is configured for HTTPS but Prometheus is serving HTTP. Enable TLS:

```
./guides/recipes/observability/install-prometheus-grafana.sh -u
./guides/recipes/observability/install-prometheus-grafana.sh --enable-tls
```

Metrics not appearing in Prometheus

1. Check that PodMonitors and ServiceMonitors exist:


```
bash kubectl get podmonitors,servicemonitors -n ${NAMESPACE}
```

2. Verify Prometheus is scraping the targets. Open `http://localhost:9090/targets` (after port-forwarding) and check that vLLM and EPP targets show `UP`
3. Confirm pods expose metrics:

```
bash VLLM_POD=$(kubectl get pods -n ${NAMESPACE} -l app=my-model -o jsonpath='{.items[0].metadata.name}') kubectl port-forward -n  
${NAMESPACE} ${VLLM_POD} 8000:8000 curl http://localhost:8000/metrics | head -20
```

Grafana dashboards show "No data"

1. Verify the Grafana datasource points to the correct Prometheus URL
2. Check that metrics are flowing in Prometheus first (use the Prometheus UI)
3. If using TLS, ensure the Grafana datasource is configured for HTTPS with the correct CA certificate

Rollouts

Rollouts are incremental deployment operations that gradually introduce new versions of inference infrastructure with minimal service disruption. Rather than replacing all running instances at once, a rollout shifts traffic progressively — allowing you to monitor behavior at each stage and roll back immediately if problems arise.

Why Rollouts Matter for LLM Inference

LLM inference workloads have properties that make all-at-once replacements especially risky:

- **Long-running requests:** a request mid-generation cannot be migrated. Abrupt pod replacement causes visible failures.
- **Warm-up cost:** new model server pods require time to load model weights, compile CUDA graphs, and fill KV caches. Traffic arriving before warm-up completes sees high latency.
- **Resource intensity:** GPU memory is non-trivially expensive. Rolling back a bad deployment means waiting for a full cold-start cycle.

Gradual rollouts mitigate all three by keeping the stable version fully operational while the new version is validated under real traffic.

Rollout Strategies

Rolling Update

A Rolling Update is the standard Kubernetes deployment strategy that updates pods gradually within a single InferencePool. This approach works in both standalone and llm-d router gateway modes.

How it works:

- Updates pods incrementally (e.g., 25% at a time)
- Old pods continue serving traffic until new pods are healthy
- Built into Kubernetes Deployments

Use Rolling Updates for:

- General, non-critical updates where strict traffic percentages do not matter
- Scenarios where you want to conserve compute resources
- Development and staging environments

Learn more: [Kubernetes Rolling Update Tutorial](#)

Blue-Green Update (HTTPRoute Traffic Splitting)

A Blue-Green Update creates a second complete InferencePool and uses HTTPRoute to control traffic distribution between the old (blue) and new (green) versions. This strategy requires llm-d router gateway mode.

How it works:

- Deploy a complete new InferencePool alongside the existing one
- Use HTTPRoute to gradually shift traffic (e.g., 1% → 5% → 10% → 50% → 100%)
- Instant rollback by adjusting HTTPRoute weights

Use Blue-Green Updates for:

- Critical, high-risk production deployments that require gradual canary rollouts
- Scenarios requiring fast rollbacks
- Header-based routing (e.g., routing beta users to new version)
- Updates that need precise traffic control

Guide: Blue-Green Update

LoRA Adapter Rollout

LoRA (Low-Rank Adaptation) adapter rollouts allow you to update model customizations without changing the base model or infrastructure. This works in both standalone and llm-d router gateway modes.

How it works:

- Use `InferenceModelRewrite` to map model names to specific adapter versions
- Gradually shift traffic between adapter versions
- No infrastructure changes required

Use LoRA Adapter Rollouts when:

- You need to deploy new versions of LoRA adapters without disrupting service
- You want to test adapter changes with a subset of traffic
- You need to maintain multiple adapter versions simultaneously

Guide: LoRA Adapter Rollout

Strategy Comparison

Feature	Rolling Update	Blue-Green Update
Routing Control	Random/Even across all healthy pods	Precise Percentage (e.g., exactly 1% or 10%)
Blast Radius	High (All users exposed randomly)	Low (Isolated to specified target weight)
Rollback Speed	Slow (Requires creating new pods in reverse)	Instant (Flip HTTPRoute weight back to 0)
Resource Costs	Low (Only temporary surge of pods)	High (Requires running two full environments)
Version Coexistence	Simultaneously active inside one Service	Strictly separated across two distinct Services
Deployment Mode	Standalone and Gateway	Gateway only

LoRA adapter rollouts have no infrastructure cost and work in either mode, but apply only when the change is limited to adapter weights — not base model or serving framework changes.

[!NOTE] Capacity management may also play a role in choosing between these strategies.

Serve External APIs

This section covers how to deploy an API gateway or proxy layer on top of **llm-d** to manage traffic across both self-hosted LLM workloads and third-party external model APIs (such as Google Gemini, OpenAI, or Anthropic).

By deploying a unified proxy front-ending your LLM infrastructure, you can:

- **Centralize API Key & Secret Management:** Store external provider keys securely in Kubernetes Secrets rather than distributing them to client applications.
- **Provide a Unified OpenAI-Compatible Interface:** Allow client applications to switch seamlessly between self-hosted llm-d model endpoints and external SaaS LLM providers.
- **Enforce Security & Rate Limiting:** Apply enterprise governance, authentication, token budgeting, and logging at a single entry point.

Architecture Overview

In both configurations, the proxy acts as the single external or in-cluster entry point. Requests targeting self-hosted models are routed directly into the **llm-d Optimized Baseline** infrastructure, while requests targeting external models are authenticated and forwarded to cloud API providers.

flowchart TD

Client["Client Applications"] --> Proxy["Kong / LiteLLM AI Gateway"]

Proxy -->|"Self-Hosted Model Traffic"| Router["llm-d Router"]

Proxy -->|"External Provider Traffic"| External["External API Providers
(Gemini, OpenAI, ...)"]

Router --> Pool["vLLM / SGLang Model Pool"]

```
classDef client fill:#f3e5f5,stroke:#7b1fa2,stroke-width:1.5px,color:#000;
classDef proxy fill:#e1f5fe,stroke:#0288d1,stroke-width:1.5px,color:#000;
classDef llmd fill:#e8f5e9,stroke:#388e3c,stroke-width:1.5px,color:#000;
classDef ext fill:#fff3e0,stroke:#f57c00,stroke-width:1.5px,color:#000;
```

```
class Client client;
class Proxy proxy;
class Router,Pool llmd;
class External ext;
```

Integration Modes with llm-d

Both guides support connecting the external API proxy to **llm-d** via either:

1. **Gateway Mode (Default & Recommended):** Connects to the Kubernetes Gateway IP or endpoint, leveraging llm-d's full Gateway API Inference Extension (GAIE) capabilities and route management.

2. **Standalone Mode (Optional):** Connects directly to the `optimized-baseline-epp` Service endpoint (`http://optimized-baseline-epp...svc.cluster.local:80/v1`), bypassing the Kubernetes Gateway for direct in-cluster routing.
-

Deployment Guides

Select a guide to proceed with deployment:

- **LiteLLM Proxy Guide:** Deploy LiteLLM with PostgreSQL for virtual API key management, user spend tracking, budget caps, and multi-provider routing.
- **Kong AI Gateway Guide:** Deploy Kong in DB-less mode using Kubernetes Gateway API and custom resources (`KongPlugin` , `HTTPRoute`) for high-performance routing.

LiteLLM for Hosting llm-d Inference Stack and External APIs

This guide demonstrates how to deploy a **LiteLLM proxy** (Helm chart, PostgreSQL-backed) on Kubernetes to provide a single, OpenAI-compatible entry point that fronts both an existing **llm-d inference stack** and external LLM provider APIs.

Overview

By placing LiteLLM in front of your infrastructure, a single proxy entry point serves:

1. **In-cluster model endpoints:** An existing llm-d inference stack serving models in your Kubernetes cluster (e.g. Qwen/Qwen3-32B).
2. **External provider APIs:** Third-party LLM services (e.g. Google Gemini via a Google AI Studio API key).

End users can issue OpenAI-compatible API calls using virtual keys generated by LiteLLM to access self-hosted models or external provider APIs without managing individual provider API credentials directly.

Prerequisites

1. **Kubernetes Cluster:** A running Kubernetes cluster with `kubectl` configured.
2. **llm-d Inference Stack:** An active llm-d deployment set up using the Optimized Baseline hosting a model (e.g., Qwen/Qwen3-32B). - **Gateway Mode (Default):** Reached via the Kubernetes Gateway IP (`http://<gateway-ip>/v1`). - **Standalone Mode (Optional):** Reached directly via the Endpoint Picker (EPP) Service: `http://optimized-baseline-epp.llm-d-optimized-baseline.svc.cluster.local:80/v1` .
3. **External Provider API Key:** An API key from Google AI Studio for `gemini/gemini-3.5-flash` .
4. **Local Tools:** `kubectl` , `helm` , `openssl` , and `curl` .

[!NOTE] **Validated Versions:** This guide was tested and verified with the LiteLLM Helm chart `1.94.1` (`oci://ghcr.io/berriai/litellm-helm`), LiteLLM proxy `v1.90.2` (`ghcr.io/berriai/litellm-database`), PostgreSQL `16` , and Redis `7.4-alpine` .

Set up the LiteLLM target namespace:

```
export NAMESPACE=litellm
```

```
kubectl create namespace "$NAMESPACE" --dry-run=client -o yaml | kubectl apply -f -
```

Step 1: Create Secrets for LiteLLM

Generate random keys for administrative management and database encryption, and create the Kubernetes Secrets required by LiteLLM:

```
export DB_PASSWORD="$(openssl rand -hex 24)"
export GEMINI_API_KEY="<your-gemini-api-key>"

# 1) Core secret: Admin master key, salt key (encrypts stored creds), and DB creds
kubectl -n "$NAMESPACE" create secret generic litellm-secrets \
  --from-literal=master-key="sk-$(openssl rand -hex 24)" \
  --from-literal=salt-key="sk-$(openssl rand -hex 24)" \
  --from-literal=db-username="litellm" \
  --from-literal=db-password="$DB_PASSWORD"

# 2) Environment secret: Provider API keys (+ salt key for the chart's env)
SALT=$(kubectl -n "$NAMESPACE" get secret litellm-secrets -o jsonpath='{.data.salt-key}' | base64 -d)
kubectl -n "$NAMESPACE" create secret generic litellm-env \
  --from-literal=LITELLM_SALT_KEY="$SALT" \
  --from-literal=GEMINI_API_KEY="$GEMINI_API_KEY"
```

[!IMPORTANT] Replace <your-gemini-api-key> with your actual Google AI Studio API key.

Step 2: Set Up PostgreSQL and Redis Backends

LiteLLM requires two storage tiers:

1. **PostgreSQL (Durable State):** Stores virtual keys, user spend metrics, model aliases, and audit logs. Deployed as a `StatefulSet` with a persistent volume.
2. **Redis (Ephemeral Coordination):** Coordinates cross-pod rate limiting (RPM/TPM), budget locking, and spend counters across proxy replicas. Deployed as a lightweight `Deployment` using the official upstream image (`redis:7.4-alpine`).

Deploying both components as standalone Kubernetes resources avoids deprecated, unpinned third-party subcharts and provides full control over versions and resources.

1. Deploy PostgreSQL StatefulSet

Create `postgres.yaml` :


```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          env:
            - name: POSTGRES_DB
              value: litellm
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: litellm-secrets
                  key: db-username
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: litellm-secrets
                  key: db-password
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          ports:
            - containerPort: 5432
          readinessProbe:
            exec:
              command: ["sh", "-c", "pg_isready -U \"${POSTGRES_USER}\" -d \"${POSTGRES_DB}\""]
            initialDelaySeconds: 10
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 10Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

2. Deploy Redis Coordination Store

Create `redis.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7.4-alpine
          ports:
            - containerPort: 6379
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 512Mi
          readinessProbe:
            exec:
              command: ["redis-cli", "ping"]
            initialDelaySeconds: 5
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
```

Apply both manifests and wait for readiness:

```
kubectl apply -n "$NAMESPACE" -f postgres.yaml -f redis.yaml

kubectl -n "$NAMESPACE" rollout status statefulset/postgres
kubectl -n "$NAMESPACE" rollout status deployment/redis
```

The LiteLLM proxy communicates with PostgreSQL at `postgres:5432` and Redis at `redis:6379` within the same namespace.

Step 3: Construct LiteLLM Helm Values and Install

Create a `values.yaml` configuration file for the LiteLLM Helm chart. The `model_list` array configures the routing for both self-hosted llm-d models and external providers.

Configuration (`values.yaml`)

```
replicaCount: 3

image:
  repository: ghcr.io/berriai/litellm-database # Bundled Prisma client for DB migrations
  tag: "v1.90.2"

# Run Redis as a standalone Deployment (Step 2) rather than the chart's
# optional bundled subchart, so the image and version stay under your control.
redis:
  enabled: false

# Admin master key secret reference
masterkeySecretName: litellm-secrets
masterkeySecretKey: master-key

# Database integration via in-cluster StatefulSet
db:
  useExisting: true
  deployStandalone: false
  endpoint: "postgres:5432"
  database: litellm
  secret:
    name: litellm-secrets
    usernameKey: db-username
    passwordKey: db-password

# Pass provider keys as environment variables to LiteLLM
environmentSecrets:
- litellm-env

# Execute DB migrations as a Helm hook Job
migrationJob:
  hooks:
    helm: { enabled: true }
    argocd: { enabled: false }

# Disable runtime schema updates on proxy pods (handled by migration job)
envVars:
  DISABLE_SCHEMA_UPDATE: "True"

proxy_config:
  # Global router coordination: cross-replica rate limiting and model routing state
  router_settings:
    redis_host: redis
    redis_port: 6379

model_list:
#
```

```
# 1) DEFAULT: llm-d Gateway Mode
# Points to your Kubernetes Gateway IP (from `kubectl get gateway`).
#
```

```
- model_name: qwen/qwen3-32B
litellm_params:
  model: hosted_vllm/Qwen/Qwen3-32B
  api_base: http://<gateway-ip>/v1
  api_key: "none" # llm-d requires no API key
# Self-hosted models have no public price. These are chargeback rates —
# amortized GPU cost, not a vendor price. Replace with your own:
# tokens_per_hour = 3600 x sustained_output_tok/s x utilization
# output_cost_per_token = (GPU $/hr x GPUs) / tokens_per_hour
```

*# Without these, LiteLLM records spend=0 for llm-d models and the
max_budget below only ever counts Gemini.*

model_info:

input_cost_per_token: 0.0000008 *# \$0.80 per 1M input tokens*

output_cost_per_token: 0.0000024 *# \$2.40 per 1M output tokens*

#

2) OPTIONAL: llm-d Standalone Mode

Points directly to the EPP Service endpoint (bypassing Gateway API).

#

- **model_name:** qwen/qwen3-32b-standalone

litellm_params:

model: hosted_vllm/Qwen/Qwen3-32B

api_base: http://optimized-baseline-epp.llm-d-optimized-baseline.svc.cluster.local:80/v1

api_key: "none"

Self-hosted models have no public price. These are chargeback rates —

amortized GPU cost, not a vendor price. Replace with your own:

tokens_per_hour = 3600 x sustained_output_tok/s x utilization

output_cost_per_token = (GPU \$/hr x GPUs) / tokens_per_hour

Without these, LiteLLM records spend=0 for llm-d models and the

max_budget below only ever counts Gemini.

model_info:

input_cost_per_token: 0.0000008 *# \$0.80 per 1M input tokens*

output_cost_per_token: 0.0000024 *# \$2.40 per 1M output tokens*

#

3) EXTERNAL API: Google Gemini

Uses the Google AI Studio API key injected via GEMINI_API_KEY.

#

- **model_name:** gemini-flash

litellm_params:

model: gemini/gemini-3.5-flash

api_key: os.environ/GEMINI_API_KEY

[!NOTE]

- **What Redis Does and Doesn't Enforce:** Model-level rate limits are checked in Redis, so they hold across all replicas. Virtual key limits (`rpm_limit` on `/key/generate`) are counted in each pod's memory and synced on a delay — with 3 replicas, a key limited to 60 RPM can briefly reach ~180. Spend and budgets are written to PostgreSQL and lag similarly. For a hard per-key ceiling, run a single replica or divide `rpm_limit` by the replica count.
- **Self-Hosted Model Cost Tracking (`model_info`):** Commercial models (like `gemini-3.5-flash`) have pre-configured pricing in LiteLLM's public pricing dictionary. Self-hosted models (`hosted_vllm/*`) require explicit `model_info` with `input_cost_per_token` and `output_cost_per_token` so LiteLLM can track usage spend and deduct from virtual key budgets accurately.

Install via Helm

Deploy LiteLLM using the official OCI Helm chart:

```
helm install litellm oci://ghcr.io/berriai/litellm-helm \
-n "$NAMESPACE" -f values.yaml

kubectl -n "$NAMESPACE" rollout status deploy/litellm
```

Step 4: Verify Both Models

Retrieve the deployed LiteLLM proxy service:

```
kubectl -n "$NAMESPACE" get svc
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
litellm	ClusterIP	<cluster-ip>	<none>	4000/TCP	12h
postgres	ClusterIP	None	<none>	5432/TCP	12h
redis	ClusterIP	<cluster-ip>	<none>	6379/TCP	12h

1. Establish Port Forwarding

In a separate terminal, forward port 4000:

```
kubectl -n "$NAMESPACE" port-forward deploy/litellm 4000:4000
```

2. Retrieve Configured Models (/v1/models)

Fetch the admin master key and query the /v1/models endpoint:

```
MK=$(kubectl -n "$NAMESPACE" get secret litellm-secrets -o jsonpath='{.data.master-key}' | base64 -d)

curl -s http://127.0.0.1:4000/v1/models -H "Authorization: Bearer $MK" | jq .
```

Expected output showing both self-hosted and external models:

```
{
  "object": "list",
  "data": [
    {
      "id": "qwen/qwen3-32B",
      "object": "model",
      "created": 1677610602,
      "owned_by": "openai"
    },
    {
      "id": "qwen/qwen3-32b-standalone",
      "object": "model",
      "created": 1677610602,
      "owned_by": "openai"
    },
    {
      "id": "gemini-flash",
      "object": "model",
      "created": 1677610602,
      "owned_by": "openai",
      "max_input_tokens": 1048576,
      "max_output_tokens": 65535
    }
  ]
}
```

3. Generate Scoped Virtual Key with Budget and Rate Limits

Use the admin master key to provision a scoped virtual key with model access restrictions, a \$10 budget cap, and a 60 RPM rate limit:

```
KEY_RESP=$(curl -s http://127.0.0.1:4000/key/generate \
-H "Authorization: Bearer $MK" \
-H "Content-Type: application/json" \
-d '{
  "models": ["qwen/qwen3-32B", "gemini-flash"],
  "max_budget": 10.0,
  "rpm_limit": 60,
  "duration": "30d",
  "user_id": "team-analytics"
}')

echo "$KEY_RESP" | jq .

# Export the generated virtual key for client inference:
export VIRTUAL_KEY=$(echo "$KEY_RESP" | jq -r .key)
```

Expected output:

```
{
  "key": "<generated-virtual-key>",
  "max_budget": 10.0,
  "spend": 0.0,
  "models": ["qwen/qwen3-32B", "gemini-flash"],
  "rpm_limit": 60,
  "duration": "30d",
  "user_id": "team-analytics"
}
```

4. Test Inference Requests with Virtual Key

Client applications use their scoped virtual key (`$VIRTUAL_KEY`) rather than the admin master key.

A. Call Self-Hosted Model (`qwen/qwen3-32B` via `llm-d`)

```
curl -s -m 45 http://127.0.0.1:4000/v1/chat/completions \
-H "Authorization: Bearer $VIRTUAL_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen/qwen3-32B",
  "messages": [{"role": "user", "content": "hi"}],
  "max_tokens": 16
}' | jq .
```

Expected output (containing `vllm` fingerprint, confirming execution through `llm-d`):

```
{
  "id": "<response-id>",
  "object": "chat.completion",
  "created": 1783624185,
  "model": "qwen/qwen3-32B",
  "system_fingerprint": "vllm-0.23.0-tp2-f1888500",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "<think>\nOkay, the user sent \"hi\". That's pretty casual. I"
      },
      "finish_reason": "length"
    }
  ],
  "usage": {
    "prompt_tokens": 9,
    "completion_tokens": 16,
    "total_tokens": 25
  }
}
```

B. Call External Model (`gemini-flash` via Google AI Studio API)

```
curl -s -m 45 http://127.0.0.1:4000/v1/chat/completions \
-H "Authorization: Bearer $VIRTUAL_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "gemini-flash",
  "messages": [{"role": "user", "content": "hi"}],
  "max_tokens": 16
}' | jq .
```

Expected output (translated OpenAI format returned from Gemini):


```
{
  "id": "<response-id>",
  "object": "chat.completion",
  "created": 1783624118,
  "model": "gemini-flash",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! How can I help you today?"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 2,
    "completion_tokens": 9,
    "total_tokens": 11
  }
}
```

5. Check Spend Tracking and Remaining Budget

Query the database via LiteLLM's key management endpoint to verify spend auditing:

```
curl -s "http://127.0.0.1:4000/key/info?key=$VIRTUAL_KEY" \
-H "Authorization: Bearer $MK" | jq .
```

Expected output:

```
{
  "key": "<generated-virtual-key>",
  "info": {
    "spend": 0.00042,
    "max_budget": 10.0,
    "models": ["qwen/qwen3-32B", "gemini-flash"],
    "rpm_limit": 60,
    "user_id": "team-analytics"
  }
}
```

Cleanup

To remove the LiteLLM proxy, standalone Redis coordination store, PostgreSQL database, and all associated resources:

```
helm uninstall litellm -n "$NAMESPACE"
kubectl delete -n "$NAMESPACE" -f redis.yaml -f postgres.yaml
kubectl delete namespace "$NAMESPACE"
```

Disaggregated Serving: Operations (SGLang)

While disaggregated serving can offer superior performance, it introduces additional operational complexity, including:

- **Dynamic Connections** - how to add or remove P and D instances on the fly when instances require point-to-point RDMA connections
- **Request Cancellation** - how to free KV caches from the instances when requests stop in a distributed setting
- **Fault Tolerance** - how to ensure crashes do not create cascading failures and that resources are cleaned up
- **Rollouts** - how to roll out changes to the service, such as the version of the SGLang image

This page documents architectural considerations that impact these common operations flows for SGLang model servers. For the vLLM counterpart see `vllm.md`, and for an overview of how the EPP and Routing Proxy Sidecar coordinate P/D requests see the [Disaggregated Serving](#) page.

Both engines establish a NIXL (RDMA) connection for each P/D pair and differ mainly in how a pair finds each other and which side moves the data: vLLM negotiates the connection peer-to-peer with no central coordinator and has the decode pull the KVs, while SGLang routes peer discovery through a bootstrap server that runs alongside each prefill instance and has the prefill push the KVs to the decode.

Dynamic Connections

In production environments, it is common for model server replicas to be created and destroyed during the running of the service. In a disaggregated configuration, the ability to dynamically add/remove replicas from the deployment is complicated by the need to establish/destroy connections between P and D workers on the fly.

SGLang coordinates this through a bootstrap server that is started on each prefill instance (default port `8998`). For each request the Routing Proxy Sidecar pairs a P and a D and gives both the same rendezvous tokens; the two then use that bootstrap server to find each other and run the KV transfer directly between the pods, off the sidecar's request path.

Scale-Up

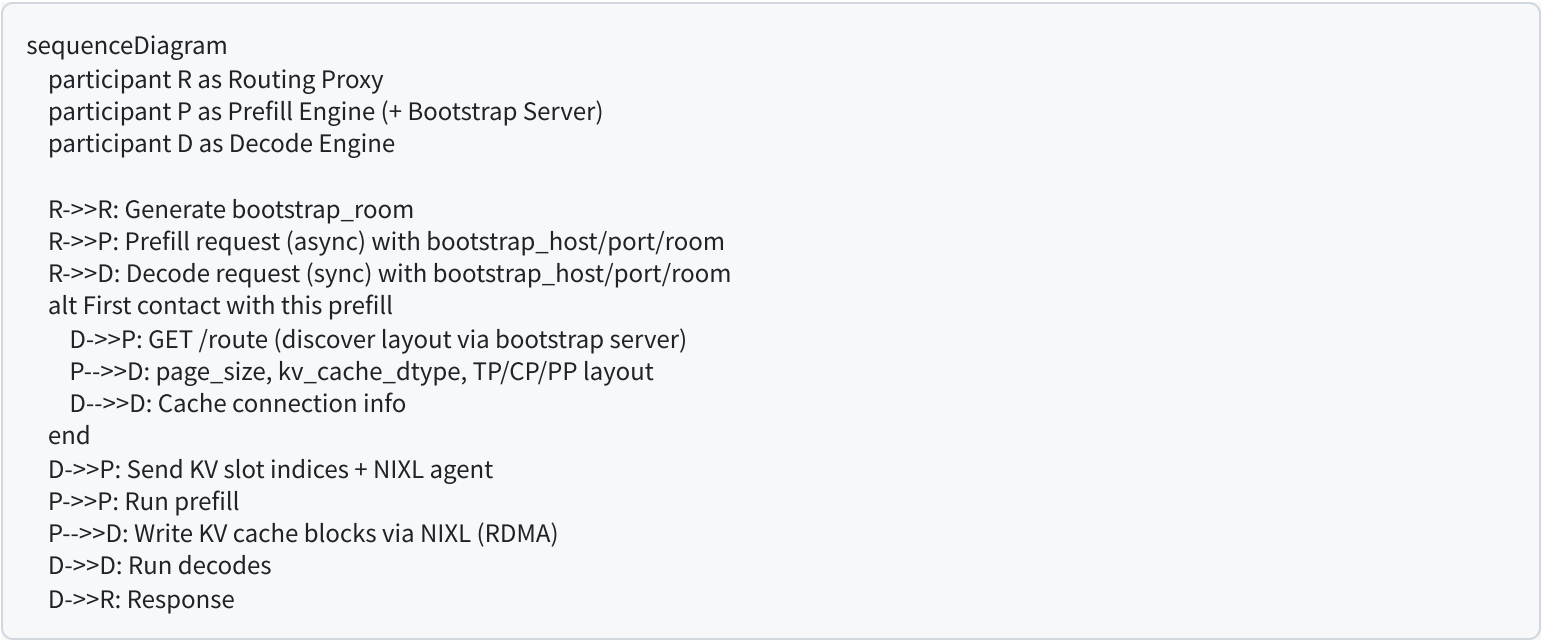
New P and D replicas connect lazily, with no pre-warming step: the first request routed across a given P/D pair triggers a one-time connection setup, and subsequent requests reuse it, so a newly added replica starts serving without restarts.

For each request, the Routing Proxy Sidecar (configured with `--kv-connector=sglang`) generates a unique `bootstrap_room` ID, injects `bootstrap_host`, `bootstrap_port`, and `bootstrap_room` into both the prefill and decode request bodies, and fires the prefill request asynchronously (in a background goroutine) while sending the decode request synchronously. As a result, the decode worker does not wait for prefill to complete before beginning coordination.

On the engine side:

- Prefill instances run a bootstrap server that exposes a `/route` endpoint, and register their parallelism layout (TP/CP/PP ranks, `page_size`, `kv_cache_dtype`) into it on startup.
- Decode instances, on first contact with a given prefill bootstrap address, fetch that prefill's parallel info from the bootstrap `/route` endpoint over HTTP and cache it. Subsequent requests to the same prefill reuse the cached connection, so there is no centralized global-state coordinator beyond the per-prefill bootstrap server.

The request flow looks like this:



Discovery

Since model server instances are added to an `InferencePool` via standard Kubernetes selectors and labels, new prefill and decode instances are discovered automatically when their pod status becomes `status: Running`. The same `llm-d.ai/role` label values (`prefill`, `decode`, `prefill-decode`) are used to filter P and D workers. See `vllm.md#discovery` for details.

As a result, new replicas can be added to a running disaggregated deployment without restarts and without coordinating within any specialized service discovery plane.

Scale-Down

SGLang instances follow the same Kubernetes pod termination process as vLLM (Terminating state, removal from the `InferencePool`, optional `preStop` hook, `SIGTERM`, grace period, then `SIGKILL`). For **new requests**, terminating pods are automatically removed from the `InferencePool` so no new traffic is routed to them.

For **in-flight requests**, note that SGLang does not expose a vLLM-style `--shutdown-timeout` `graceful-drain` flag, so operators should not rely on in-flight requests draining during termination. How any KV cache held for those requests is reclaimed depends on the transfer backend, as described below.

Scaling Down Decode Replicas

When a decode replica is removed, any in-flight rooms it was assigned are abandoned before their KV transfer completes. From the prefill's perspective this is indistinguishable from a decode crash, and how the held tree cache is reclaimed depends on the transfer backend (see `Decode Instance Failure`).

Scaling Down Prefill Replicas

When a prefill replica is removed, decode instances detect it through the heartbeat mechanism (see Prefill Instance Failure). In-flight requests bound to that prefill are marked failed, and new requests are no longer routed to it once it leaves the `InferencePool`.

Request Cancellation

Given the compute intensity and duration of inference requests, model servers support "Request Cancellation", where currently in-progress requests are freed when the client disconnects.

In a disaggregation setup this is more complicated, because the resources associated with a request are spread across the prefill and decode instances. With SGLang, the Routing Proxy Sidecar holds the decode request open for the client while the prefill request runs asynchronously in the background. When the client disconnects:

- The decode instance aborts the in-flight request and releases its tree cache.
- The prefill instance frees the room's tree cache once its KV transfer reaches a terminal state. On the NIXL transfer backend used by this guide, a prefill whose decode disconnected *before* initiating the transfer has no timeout that reclaims it (see the note below). The periodic `SGLANG_DISAGGREGATION_BOOTSTRAP_ENTRY_CLEANUP_INTERVAL` (default 120s) only evicts stale bookkeeping entries from the bootstrap server, not GPU KV cache.

[!NOTE] This differs from vLLM, which sends an explicit `NIXL.send_notif` to free the prefill-side KV cache on abort. With the NIXL transfer backend, SGLang has no equivalent prefill-side abort signal: if a request is cancelled after the decode has been paired but before it initiates the transfer, the prefill can hold that room's KV cache until the prefill pod is restarted. This is analogous to vLLM's `VLLM_NIXL_ABORT_REQUEST_TIMEOUT` stranding window, except that vLLM bounds it with a timeout (default 480s) whereas NIXL-backed SGLang does not. The `SGLANG_DISAGGREGATION_BOOTSTRAP_TIMEOUT` that bounds prefill-side reclaim is implemented for the Mooncake and Mori transfer backends, not NIXL.

Fault Tolerance

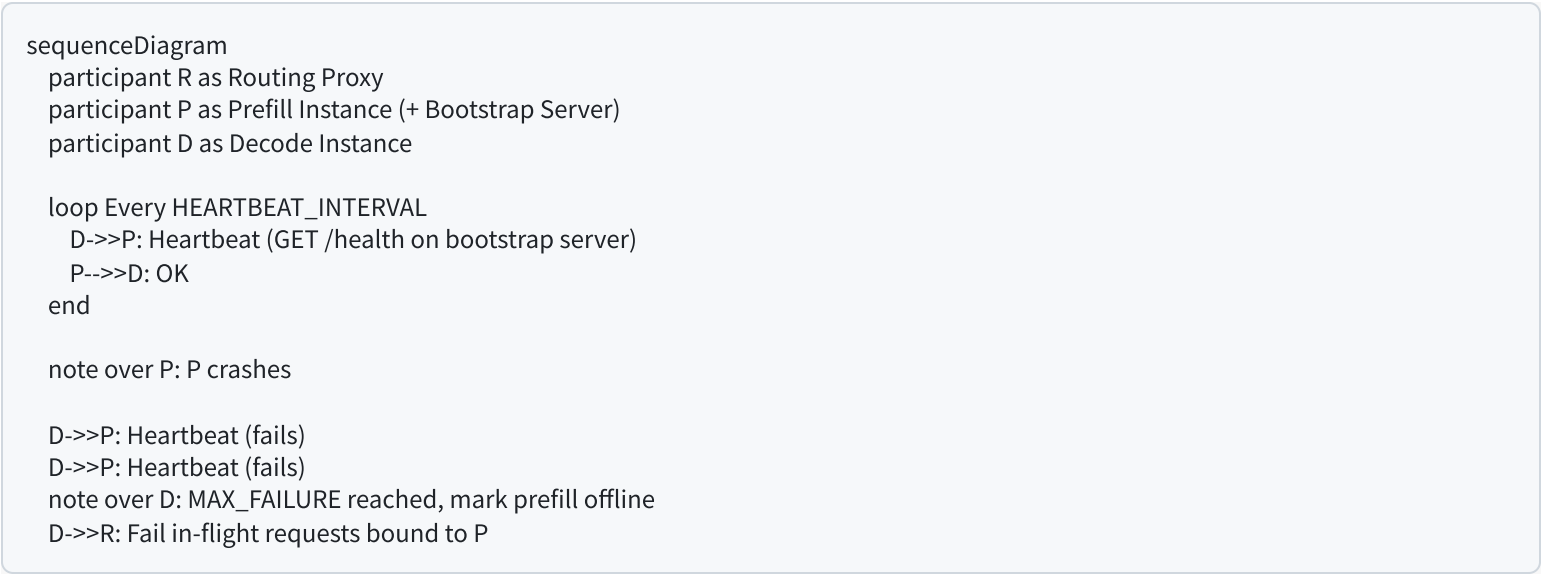
In llm-d's disaggregated serving design, all D instances can connect to all P instances. This creates a critical operational risk - crashes in workers have the potential for cascading failures if the system is not tolerant of failures.

Prefill Instance Failure

Unlike vLLM, which performs no liveness check and only discovers a dead prefill when a per-request NIXL `READ` fails, SGLang decode instances actively health-check the prefill bootstrap servers via heartbeat:

- `SGLANG_DISAGGREGATION_HEARTBEAT_INTERVAL` : interval between health checks to prefill bootstrap servers (default 5s, clamped to a minimum of 2s).
- `SGLANG_DISAGGREGATION_HEARTBEAT_MAX_FAILURE` : consecutive heartbeat failures before a prefill server is marked offline (default 2, minimum 1).

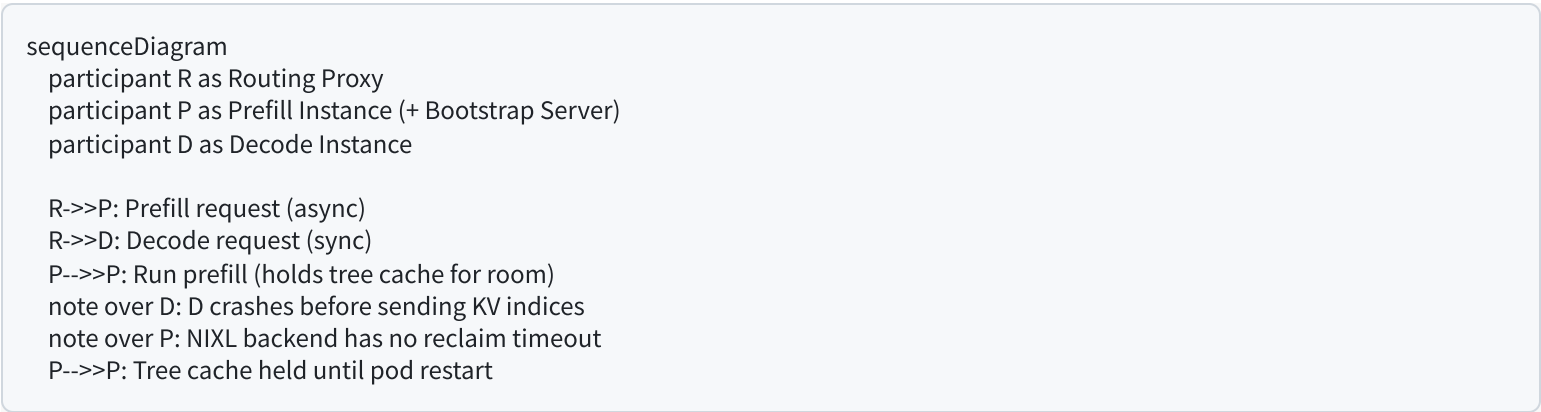
When a prefill server that was previously healthy stops responding, the decode instance marks it offline and fails any in-flight requests bound to it (`KVPoll.Failed`) rather than hanging. Failed Prefill Worker pods are also moved to `status: Terminated` as part of the standard Pod lifecycle, and since llm-d leverages the Kubernetes API Server for service discovery, no additional traffic is routed to the failed worker until it returns to `status: Running` . In this way, llm-d isolates Prefill instance failure.



Decode Instance Failure

While a decode failure is unlikely to crash a prefill instance, there is a risk of KV cache memory being stranded on the prefill, since the prefill holds the tree cache for a room until it has written the KVs to the decode and the transfer completes.

A decode that is still alive but waiting on a failed or slow transfer protects itself with `SGLANG_DISAGGREGATION_WAITING_TIMEOUT` (default 300s): if it never receives the KV-transfer-done signal after bootstrapping, it aborts the request and releases its own tree cache. The prefill side frees a room once its transfer reaches a terminal state - a failed NIXL transfer marks the room `KVPoll.Failed` , and the prefill aborts and releases the tree cache. If the decode dies *before* it initiates the transfer, however, the prefill room never reaches a terminal state on the NIXL backend.



[!WARNING] On the NIXL transfer backend used by this guide, a prefill room whose decode died before initiating the transfer is not reclaimed automatically; it holds its tree cache until the prefill pod is restarted. The `SGLANG_DISAGGREGATION_BOOTSTRAP_TIMEOUT` (default 300s) that bounds this stranding is implemented for the Mooncake and Mori transfer backends, not NIXL. This is comparable to the decode-failure stranding documented for vLLM, which bounds it with `VLLM_NIXL_ABORT_REQUEST_TIMEOUT` (default 480s).

Rollouts

In disaggregated serving, rolling out a new version of the model server (e.g. a new SGLang image or a new configuration) requires care, because NIXL moves KV cache directly between the prefill and decode GPU memory. The KV cache layout must therefore match between any P and D pair that gets scheduled together.

SGLang validates layout compatibility at connection time. When a decode instance fetches a prefill's info from the bootstrap `/route` endpoint, it checks:

- `page_size`: a mismatch raises a `RuntimeError` (Both servers must use the same `--page-size` value).
- `kv_cache_dtype`: a mismatch raises a `RuntimeError` (Both servers must use the same `--kv-cache-dtype` value).

Mismatched tensor-parallel sizes between P and D are supported (the decode resolves a TP/CP/PP rank mapping), but for non-MLA models SGLang warns that performance is not guaranteed.

[!IMPORTANT] As with vLLM, the llm-d EPP currently assumes all P and D instances within an `InferencePool` are compatible and will schedule requests to any arbitrary pair. Because incompatible `page_size` / `kv_cache_dtype` combinations fail at connection time, it is recommended to roll out model-server upgrades by creating a new `InferencePool` rather than mixing versions in one pool. When deploying with a `Gateway`, traffic can be gradually shifted to the new `InferencePool` by modifying the `HTTPRoute`. See [vllm.md#rollouts](#).

Tuning Reference

The values below are set as environment variables on the model server pods (the bootstrap port is read by the routing sidecar).

Variable	Default	Side	Controls
SGLANG_BOOTSTRAP_PORT (sidecar)	8998	sidecar / prefill	Port of the prefill bootstrap server the sidecar points decode workers at. Must match the engine's --disaggregation-bootstrap-port .
SGLANG_DISAGGREGATION_HEARTBEAT_INTERVAL	5s	decode	How often decode workers health-check prefill bootstrap servers (minimum 2s). Lower it to detect dead prefills faster.
SGLANG_DISAGGREGATION_HEARTBEAT_MAX_FAILURE	2	decode	Consecutive heartbeat failures before a prefill is marked offline (minimum 1).
SGLANG_DISAGGREGATION_WAITING_TIMEOUT	300s	decode	How long a decode waits for the KV-transfer-done signal before aborting and releasing its tree cache. Enforced on the NIXL backend.
SGLANG_DISAGGREGATION_BOOTSTRAP_TIMEOUT	300s	prefill	How long a prefill waits for the decode's KV indices before aborting and releasing its tree cache. Implemented for the Mooncake and Mori backends only - not NIXL.
SGLANG_DISAGGREGATION_BOOTSTRAP_ENTRY_CLEANUP_INTERVAL	120s	prefill	How often the bootstrap server evicts stale routing bookkeeping entries. Does not free GPU KV cache.

Distributed Tracing

This guide shows how to enable OpenTelemetry distributed tracing across llm-d components.

[!NOTE] This guide assumes a running llm-d deployment with an InferencePool and model servers. For metrics and dashboards, see Metrics.

Commands in this guide use `${NAMESPACE}` for the namespace where your llm-d workload runs:

```
export NAMESPACE=<your-llm-d-namespace>
```

What Gets Traced

Component	Config Method	Traced Operations
vLLM (prefill + decode)	Kustomize: container args + env vars	Inference engine spans
Routing proxy (P/D sidecar)	Kustomize: container env vars	KV transfer coordination
EPP	Helm: GAIE inferenceExtension.tracing:	Request routing, endpoint scoring, KV-cache indexing

All components export traces via OTLP gRPC to an OpenTelemetry Collector, which filters noise (e.g., `/metrics` scraping spans), batches traces, and forwards them to a backend like Jaeger.

Step 1: Deploy OTel Collector and Jaeger

Deploy the OTel Collector and Jaeger into the same namespace as your llm-d workload:

```
./guides/recipes/observability/install-otel-collector-jaeger.sh -n ${NAMESPACE}
```

[!NOTE] If the OpenTelemetry Operator is installed, the script uses an `OpenTelemetryCollector` CR. Otherwise it deploys a standalone collector Deployment.

Verify the components are running:


```
kubectrl get pods -n ${NAMESPACE} -l app=otel-collector
kubectrl get pods -n ${NAMESPACE} -l app=jaeger
```

Expected output:

```
NAME                READY STATUS  RESTARTS AGE
otel-collector-xxxxxxx-xxxxx 1/1   Running 0      30s
```

```
NAME                READY STATUS  RESTARTS AGE
jaeger-xxxxxxx-xxxxx 1/1   Running 0      30s
```

Manual Deployment

If you prefer to apply manifests directly:

```
# Standalone collector (no operator)
kubectrl apply -n ${NAMESPACE} -f guides/recipes/observability/tracing/jaeger-all-in-one.yaml \
-f guides/recipes/observability/tracing/otel-collector.yaml

# Or with the OTEL Operator installed
kubectrl apply -n ${NAMESPACE} -f guides/recipes/observability/tracing/jaeger-all-in-one.yaml \
-f guides/recipes/observability/tracing/otel-collector-operator.yaml
```

Verify with the same `kubectrl get pods` commands above.

Step 2: Enable Tracing on the Model Server and Routing Proxy

Model servers are deployed with kustomize. The example below uses vLLM; SGLang and other OpenTelemetry-capable model servers use the same `OTEL_*` environment variables — set `OTEL_SERVICE_NAME` to match your engine and role. Add the engine's tracing flags to its `serve` command and the OTEL env vars to the container:

```
# Add to the model server's serve command (vLLM shown):
# --otlp-traces-endpoint http://otel-collector:4317
# --collect-detailed-traces all

# Add to the container env (applies to any OpenTelemetry-capable engine):
env:
- name: OTEL_SERVICE_NAME
  value: "vllm-decode" # name per engine/role, e.g. vllm-decode, vllm-prefill, sglang-decode
- name: OTEL_EXPORTER_OTLP_ENDPOINT
  value: "http://otel-collector:4317"
- name: OTEL_TRACES_SAMPLER
  value: "parentbased_traceidratio"
- name: OTEL_TRACES_SAMPLER_ARG
  value: "1.0"
```

Step 3: Enable Tracing on EPP

Add the tracing configuration to your GAIE values:

```
# In your gaie-*/values.yaml
```

```
inferenceExtension:
```

```
tracing:
```

```
enabled: true
```

```
otelExporterEndpoint: "http://otel-collector:4317"
```

```
sampling:
```

```
sampler: "parentbased_traceidratio"
```

```
samplerArg: "1.0"
```

Step 4: View Traces

Access the Jaeger UI:

```
kubectl port-forward -n ${NAMESPACE} svc/jaeger-collector 16686:16686
```

```
# Open http://localhost:16686
```

Verify traces are flowing:

1. Send an inference request through llm-d
2. Open the Jaeger UI
3. Select a service (e.g., vllm-decode , llm-d-router/epp)
4. Click **Find Traces**

You should see traces with multiple spans covering the request lifecycle. You can also verify via the Jaeger API:

```
curl -s http://localhost:16686/api/services | jq '.data'
```

Expected output:

```
[  
  "vllm-decode",  
  "llm-d-router/epp"  
]
```

If you only see generic GET spans, check that:

- The vLLM container args include --collect-detailed-traces all
- The EPP image includes tracing instrumentation

Production Recommendations

- **Sampling:** Set samplerArg to "0.1" (10%) or lower to reduce overhead
- **Collector:** Use a collector to batch, filter, and route traces to a persistent backend
- **Backend:** Use Jaeger with Elasticsearch/Cassandra storage, or Grafana Tempo for long-term retention
- **Service names:** Set OTEL_SERVICE_NAME per container (e.g., vllm-decode-prod , epp-us-east) to distinguish clusters and environments

Environment Variable Reference

When tracing is enabled, these environment variables are set on vLLM and routing-proxy containers:

Variable	Description
OTEL_SERVICE_NAME	Service identifier (e.g., vllm-decode , routing-proxy)
OTEL_EXPORTER_OTLP_ENDPOINT	Collector endpoint (http://otel-collector:4317)
OTEL_TRACES_SAMPLER	Sampler type (e.g., parentbased_traceidratio)
OTEL_TRACES_SAMPLER_ARG	Sampling ratio (1.0 = 100%, 0.1 = 10%)

Cleanup

```
./guides/recipes/observability/install-otel-collector-jaeger.sh -u -n ${NAMESPACE}
```

Kong AI Gateway for Routing llm-d Inference Stack and External APIs

This guide demonstrates how to deploy **Kong AI Gateway** (Helm chart, DB-less mode, Kong Ingress Controller) on Kubernetes to route traffic across an existing **llm-d inference stack** and external LLM provider APIs.

Overview

A single Kong AI Gateway deployment can front both:

1. **In-cluster model endpoints:** An existing llm-d inference stack serving models in your Kubernetes cluster (via the `ai-proxy` plugin pointing an `openai` provider at the llm-d endpoint).
2. **External provider APIs:** Third-party LLM services (via the `ai-proxy` plugin pointing a `gemini` provider at Google AI Studio).

All Kong components (gateway, routes, plugins, secrets) live in a dedicated `kong` namespace, keeping the llm-d inference stack isolated in its own namespace.

End users can:

- Call external provider APIs using API keys managed centrally in Kong (the provider key is stored in Kubernetes Secrets and never exposed to client applications).
- Call self-hosted models served by the llm-d inference stack via unified OpenAI-compatible routes.

Prerequisites

1. **Kubernetes Cluster:** A running Kubernetes cluster with `kubectrl` configured and Gateway API CRDs installed.
2. **llm-d Inference Stack:** An active llm-d deployment set up using the Optimized Baseline hosting a model (e.g., Qwen/Qwen3-32B). - **Gateway Mode (Default):** Reached via the Kubernetes Gateway IP (`http://<gateway-ip>/v1`). - **Standalone Mode (Optional):** Reached directly via the Endpoint Picker (EPP) Service: `http://optimized-baseline-epp.llm-d-optimized-baseline.svc.cluster.local:80/v1` .
3. **External Provider API Key:** An API key from Google AI Studio for `gemini-3.5-flash` .
4. **Local Tools:** `kubectrl` , `helm` , and `curl` .

[!NOTE] **Validated Versions:** This guide was tested and verified with Kong Gateway `v3.9.3` , Kong Ingress Controller `v3.5.0` (`kong/ingress` Helm chart `v0.24.0`), and Gateway API `v1.2.1` .

Set up the Kong target namespace:

```
export NAMESPACE=kong

kubectl create namespace "$NAMESPACE" --dry-run=client -o yaml | kubectl apply -f -
```

Step 1: Create Secret for External Provider Keys

Kong securely retrieves external provider keys from Kubernetes Secrets via `configPatches` references in the plugin Custom Resources (CRDs), ensuring keys are never stored in plain text within plugin specs.

```
export GEMINI_API_KEY="<your-gemini-api-key>"

kubectl -n "$NAMESPACE" create secret generic ai-provider-keys \
--from-literal=gemini-api-key="\ "$GEMINI_API_KEY\ "" # Note the embedded double quotes
```

[!IMPORTANT] Replace `<your-gemini-api-key>` with your actual Google AI Studio API key. Kong Ingress Controller substitutes `configPatches` values directly as JSON; string values require embedded double quotes (e.g. `"\ "$GEMINI_API_KEY\ ""`), whereas numeric or boolean patch values do not.

Step 2: Install Kong Ingress Controller + Gateway via Helm

Make sure Gateway API is installed on your cluster and verify that the standard CRDs are available:

```
kubectl get crd gateways.gateway.networking.k8s.io httproutes.gateway.networking.k8s.io
```

Add the official Kong Helm repository and install the `kong/ingress` chart (deploys Kong Ingress Controller + DB-less Kong Gateway data plane):

```
helm repo add kong https://charts.konghq.com
helm repo update

helm install kong kong/ingress -n "$NAMESPACE"
kubectl -n "$NAMESPACE" wait --for=condition=Available deploy/kong-controller deploy/kong-gateway --timeout=300s
```

Inspect the deployed services:

```
kubectl get deploy,svc -n "$NAMESPACE"
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE
deployment.apps/kong-controller	1/1	1	1
deployment.apps/kong-gateway	1/1	1	1

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/kong-controller-metrics	ClusterIP	<cluster-ip>	<none>	10255/TCP,10254/TCP
service/kong-controller-validation-webhook	ClusterIP	<cluster-ip>	<none>	443/TCP
service/kong-gateway-admin	ClusterIP	None	<none>	8444/TCP
service/kong-gateway-manager	NodePort	<cluster-ip>	<none>	8002:31071/TCP,8445:31084/TCP
service/kong-gateway-proxy	LoadBalancer	<cluster-ip>	<proxy-ip>	80:32117/TCP,443:30425/TCP

Export the proxy LoadBalancer IP or hostname for verification commands:

```
export PROXY_IP=$(kubectl -n "$NAMESPACE" get svc kong-gateway-proxy \
-o jsonpath='{.status.loadBalancer.ingress[0].ip}{.status.loadBalancer.ingress[0].hostname}')
```

Step 3: Create GatewayClass, Gateway, and Placeholder Service

Create a `GatewayClass` and `Gateway` resource for Kong, along with a placeholder Service (`ai-placeholder`) that the `HTTPRoute` resources in Step 4 will reference to satisfy Gateway API schema requirements.

Create `kong-gateway.yaml` :

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: kong
  annotations:
    konghq.com/gatewayclass-unmanaged: "true"
spec:
  controllerName: konghq.com/kic-gateway-controller
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: kong
spec:
  gatewayClassName: kong
  listeners:
    - name: proxy
      port: 80
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: Same
---
apiVersion: v1
kind: Service
metadata:
  name: ai-placeholder
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
```

Apply the resources:

```
kubectl apply -n "$NAMESPACE" -f kong-gateway.yaml
```

Step 4: Configure Routes and `ai-proxy` Plugins

[!WARNING] **Explicit Port Requirement:** Always include an explicit port in `upstream_url` (e.g., `http://<gateway-ip>:80/...`). Omitting the port causes connection timeouts and returns `503 The upstream server is currently unavailable` on Kong Gateway versions prior to 3.8, where the `openai` driver defaults to port 443 even for `http://` URLs. Specifying `:80` is harmless and ensures compatibility across all Kong Gateway releases.

Retrieve the llm-d Gateway IP to substitute for `<gateway-ip>`:

```
kubectl -n llm-d-optimized-baseline get gateway llm-d-inference-gateway \
-o jsonpath='{.status.addresses[0].value}'
```

Create `models.yaml`:

#

1) *DEFAULT: llm-d Gateway Mode*
Routes to the llm-d Gateway API endpoint (InferenceGateway).
Points to your Kubernetes Gateway IP (from kubectl get gateway).
#

apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
 name: ai-proxy-qwen3-32b
plugin: ai-proxy
config:
 route_type: llm/v1/chat
 model:
 provider: openai
 name: Qwen/Qwen3-32B *# Must match backend model ID*
 options:
 upstream_url: http://<gateway-ip>:80/v1/chat/completions

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
 name: qwen3-32b
 annotations:
 konghq.com/plugins: ai-proxy-qwen3-32b, key-auth, rate-limiting
spec:
 parentRefs:
 - **name:** kong
 rules:
 - **matches:**
 - **path:**
 type: PathPrefix
 value: /qwen3-32b
 backendRefs:
 - **name:** ai-placeholder
 port: 80

#

2) *OPTIONAL: llm-d Standalone Mode*
Routes directly to the EPP Service endpoint (bypassing Gateway API).
#

apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
 name: ai-proxy-qwen3-32b-standalone
plugin: ai-proxy
config:
 route_type: llm/v1/chat
 model:
 provider: openai
 name: Qwen/Qwen3-32B
 options:
 upstream_url: http://optimized-baseline-epp.llm-d-optimized-baseline.svc.cluster.local:80/v1/chat/completions

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:


```
name: qwen3-32b-standalone
annotations:
  konghq.com/plugins: ai-proxy-qwen3-32b-standalone, key-auth, rate-limiting
spec:
parentRefs:
  - name: kong
rules:
  - matches:
    - path:
      type: PathPrefix
      value: /qwen3-32b-standalone
  backendRefs:
    - name: ai-placeholder
      port: 80
```

```
---
#
```

```
# 3) EXTERNAL API: Google Gemini
# Authenticates with Google AI Studio key injected from Secret via configPatches.
#
```

```
apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
  name: ai-proxy-gemini-flash
plugin: ai-proxy
config:
  route_type: llm/v1/chat
auth:
  param_name: key
  param_location: query
model:
  provider: gemini
  name: gemini-3.5-flash
configPatches:
  - path: /auth/param_value
    valueFrom:
      secretKeyRef:
        name: ai-provider-keys
        key: gemini-api-key
```

```
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: gemini-flash
annotations:
  konghq.com/plugins: ai-proxy-gemini-flash, key-auth, rate-limiting
spec:
parentRefs:
  - name: kong
rules:
  - matches:
    - path:
      type: PathPrefix
      value: /gemini-flash
  backendRefs:
    - name: ai-placeholder
      port: 80
```

```
---
#
```

```
# 4) SECURITY & GOVERNANCE: key-auth and rate-limiting
# Enforces client authentication and request rate limits across all routes.
#
```

```
apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
  name: key-auth
plugin: key-auth
config:
  key_names:
    - apikey
  hide_credentials: true
```

```
---
apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
  name: rate-limiting
plugin: rate-limiting
config:
  minute: 60
  policy: local
```

Apply the route definitions and security plugins:

```
kubectl apply -n "$NAMESPACE" -f models.yaml
```

[!NOTE]

- **Fail-Closed Placeholder Backend:** Gateway API `HTTPRoute` requires a valid `backendRef`. Because `ai-proxy` replaces the upstream request path with the target model's `upstream_url`, the `ai-placeholder` Service is not normally reached. Using a selector-less `ClusterIP` Service ensures that if an `ai-proxy` plugin fails to program or is missing, Kong fails closed with an immediate `503 Service Unavailable` rather than proxying to `localhost`.
- **Authentication Header:** Kong's `key-auth` plugin compares the full header value and does not strip a `Bearer` prefix, so clients pass keys via the `apikey` header (`-H "apikey: $CLIENT_KEY"`). OpenAI SDK clients can send this with `default_headers={"apikey": CLIENT_KEY}`. If a client can only send `Authorization: Bearer <key>`, add a Kong pre-function plugin to copy the token into `apikey` and clear the original header before `key-auth` runs.
- **Local Rate Limiting (`policy: local`):** Counters are stored in pod memory with zero external dependencies, meaning the 60 req/min limit applies independently per Kong data-plane replica (e.g., $60 \times N$ req/min total across N replicas). For strict cluster-wide rate limiting across multiple replicas, configure `policy: redis` with a shared Redis instance.
- **Model Pinning:** `ai-proxy` pins the model per route. Clients can omit `"model"` in the request body; if provided, it must match `model.name`, otherwise Kong returns `HTTP 400 Bad Request`.

Next, provision an authorized client consumer and API key credentials:

```
# 1) Generate a client API key and store it in a labeled credential Secret
CLIENT_KEY="key-$(openssl rand -hex 16)"
```

```
kubectl apply -n "$NAMESPACE" -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: client-app-key
  labels:
    konghq.com/credential: key-auth
stringData:
  key: "$CLIENT_KEY"
---
```

```
# 2) Create the KongConsumer referencing the credential Secret
apiVersion: configuration.konghq.com/v1
kind: KongConsumer
metadata:
  name: client-app
annotations:
  kubernetes.io/ingress.class: kong
username: client-app
credentials:
  - client-app-key
EOF
```

Step 5: Verification

1. Verify Gateway API Resource Programming

Check that all Gateway API resources, Kong plugins, and consumers are programmed:

```
kubectl -n "$NAMESPACE" get gateway,httproute,kongplugin,kongconsumer
```

Expected output:

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
gateway.gateway.networking.k8s.io/kong	kong	<proxy-ip>	True	77m

NAME	HOSTNAMES	AGE
httproute.gateway.networking.k8s.io/gemini-flash		77m
httproute.gateway.networking.k8s.io/qwen3-32b		77m
httproute.gateway.networking.k8s.io/qwen3-32b-standalone		77m

NAME	PLUGIN-TYPE	AGE	PROGRAMMED
kongplugin.configuration.konghq.com/ai-proxy-gemini-flash	ai-proxy	69m	True
kongplugin.configuration.konghq.com/ai-proxy-qwen3-32b	ai-proxy	71m	True
kongplugin.configuration.konghq.com/ai-proxy-qwen3-32b-standalone	ai-proxy	71m	True
kongplugin.configuration.konghq.com/key-auth	key-auth	71m	True
kongplugin.configuration.konghq.com/rate-limiting	rate-limiting	71m	True

NAME	USERNAME	AGE	PROGRAMMED
kongconsumer.configuration.konghq.com/client-app	client-app	5m	True

2. Verify Authentication Enforcement (Security Check)

Attempt an unauthenticated request to verify that Kong blocks unauthorized access:

```
curl -i -s http://$PROXY_IP/gemini-flash \
-H "Content-Type: application/json" \
-d '{"messages": [{"role": "user", "content": "hi"}]}'
```

Expected response (HTTP 401 Unauthorized):

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
Connection: keep-alive

{
  "message": "No API key found in request"
}
```

3. Call Configured Models with Client Authentication

A. Call Self-Hosted Model (/qwen3-32b via Gateway Mode)

Pass the client API key via the `apikey` header:

```
curl -s -m 45 http://$PROXY_IP/qwen3-32b \
-H "Content-Type: application/json" \
-H "apikey: $CLIENT_KEY" \
-d '{
  "messages": [{"role": "user", "content": "hi"}],
  "max_tokens": 16
}' | jq .
```

Expected response (showing vLLM system fingerprint from llm-d):

```
{
  "id": "<response-id>",
  "object": "chat.completion",
  "model": "Qwen/Qwen3-32B",
  "system_fingerprint": "vllm-0.23.0-tp2-a536750c",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "<think>\nOkay, the user said \"hi\". That's a greeting. I"
      },
      "finish_reason": "length"
    }
  ],
  "usage": {
    "prompt_tokens": 9,
    "completion_tokens": 16,
    "total_tokens": 25
  }
}
```

B. Call Self-Hosted Model (/qwen3-32b-standalone via Standalone Mode)

```
curl -s -m 45 http://$PROXY_IP/qwen3-32b-standalone \
-H "Content-Type: application/json" \
-H "apikey: $CLIENT_KEY" \
-d '{
  "messages": [{"role": "user", "content": "hi"}],
  "max_tokens": 16
}' | jq .
```

C. Call External Model (/gemini-flash via Google AI Studio API)

Kong validates the client's `apikey`, translates the OpenAI request format into Gemini API format, and injects the upstream Gemini API key:

```
curl -s -m 45 http://$PROXY_IP/gemini-flash \
-H "Content-Type: application/json" \
-H "apikey: $CLIENT_KEY" \
-d '{
  "messages": [{"role": "user", "content": "Reply with just: GEMINI-OK"}],
  "max_tokens": 16
}' | jq .
```

Expected response:

```
{
  "id": "<response-id>",
  "object": "chat.completion",
  "model": "gemini-3.5-flash",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "GEMINI-OK"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 9,
    "completion_tokens": 4,
    "total_tokens": 13
  }
}
```

Cleanup

To remove the Kong AI Gateway deployment and all associated resources:

```
kubectl delete -n "$NAMESPACE" -f models.yaml
kubectl delete -n "$NAMESPACE" -f kong-gateway.yaml
kubectl delete -n "$NAMESPACE" secret client-app-key --ignore-not-found
helm uninstall kong -n "$NAMESPACE"
kubectl delete namespace "$NAMESPACE"
```

PromQL Query Reference

Ready-to-use PromQL queries for monitoring llm-d deployments. Use these in the Prometheus UI or as the basis for Grafana panels. For a default set of ready-to-apply alerts built on these metrics, see Alerting.

To generate traffic and populate error metrics for testing, use the traffic generation script.

Tier 1: Immediate Failure & Saturation Indicators

Start here when something looks wrong.

Metric Need	PromQL Query
Overall Error Rate	<code>sum(rate(llm_d_epp_request_error_total[5m])) / sum(rate(llm_d_epp_request_total[5m]))</code>
Per-Model Error Rate	<code>sum by(model_name) (rate(llm_d_epp_request_error_total[5m])) / sum by(model_name) (rate(llm_d_epp_request_total[5m]))</code>
Request Preemptions	<code>sum by(pod, instance) (rate(vllm:num_preemptions[5m]))</code>
Overall Latency P90	<code>histogram_quantile(0.90, sum by(le) (rate(llm_d_epp_request_duration_seconds_bucket[5m])))</code>
Overall Latency P99	<code>histogram_quantile(0.99, sum by(le) (rate(llm_d_epp_request_duration_seconds_bucket[5m])))</code>
TTFT P99 per model	<code>histogram_quantile(0.99, sum by(le, model_name) (rate(vllm:time_to_first_token_seconds_bucket[5m])))</code>
TTFT P99 per model (SGLang)	<code>histogram_quantile(0.99, sum by(le, model_name) (rate(sglang_time_to_first_token_seconds_bucket[5m])))</code>
Inter-Token Latency P99	<code>histogram_quantile(0.99, sum by(le, model_name) (rate(vllm:inter_token_latency_seconds_bucket[5m])))</code>
Inter-Token Latency P99 (SGLang)	<code>histogram_quantile(0.99, sum by(le, model_name) (rate(sglang_inter_token_latency_seconds_bucket[5m])))</code>
Request Rate	<code>sum by(model_name) (rate(llm_d_epp_request_total[5m]))</code>
GPU Utilization	<code>avg by(gpu, node) (DCGM_FI_DEV_GPU_UTIL or nvidia_gpu_duty_cycle)</code>
EPP E2E Latency P99	<code>histogram_quantile(0.99, sum by(le) (rate(llm_d_epp_scheduler_e2e_duration_seconds_bucket[5m])))</code>
EPP Plugin Latency P99	<code>histogram_quantile(0.99, sum by(le, plugin_type) (rate(llm_d_epp_plugin_duration_seconds_bucket[5m])))</code>

Tier 2: Diagnostic Drill-Down

Basic Model Serving

Metric Need	PromQL Query
KV Cache Utilization	avg by(pod, model_name) (vllm:kv_cache_usage_perc)
KV Cache Utilization (SGLang)	avg by(pod, model_name) (sglang_token_usage)
Request Queue Depth	sum by(pod, model_name) (vllm:num_requests_waiting)
Request Queue Depth (SGLang)	sum by(pod, model_name) (sglang_num_queue_reqs)
Active Requests	avg by(pod) (vllm:num_requests_running)
Active Requests (SGLang)	avg by(pod) (sglang_num_running_reqs)
Total Throughput (tokens/sec)	sum by(model_name, pod) (rate(vllm:prompt_tokens_total[5m]) + rate(vllm:generation_tokens_total[5m]))
Total Throughput (SGLang) (tokens/sec)	sum by(model_name, pod) (rate(sglang_prompt_tokens_total[5m]) + rate(sglang_generation_tokens_total[5m]))
Generation Token Rate	sum by(model_name, pod) (rate(vllm:generation_tokens_total[5m]))
Generation Token Rate (SGLang)	sum by(model_name, pod) (rate(sglang_generation_tokens_total[5m]))

Routing & Load Balancing

Metric Need	PromQL Query
QPS per endpoint	sum by(endpoint_name) (rate(llm_d_epp_scheduler_attempts_total{status="success"}[5m]))
Token distribution per pod	sum by(pod) (rate(vllm:prompt_tokens_total[5m]) + rate(vllm:generation_tokens_total[5m]))
Token distribution per pod (SGLang)	sum by(pod) (rate(sglang_prompt_tokens_total[5m]) + rate(sglang_generation_tokens_total[5m]))
Routing decision latency P99	histogram_quantile(0.99, sum by(le) (rate(llm_d_epp_plugin_duration_seconds_bucket[5m])))

Prefix Caching

Metric Need	PromQL Query
Cache hit rate	sum(rate(vllm:prefix_cache_hits_total[5m])) / sum(rate(vllm:prefix_cache_queries_total[5m]))
Cache hit rate (SGLang)	avg(sglang_cache_hit_rate)
Per-pod hit rate	sum by(pod) (rate(vllm:prefix_cache_hits_total[5m])) / sum by(pod) (rate(vllm:prefix_cache_queries_total[5m]))
Per-pod hit rate (SGLang)	sglang_cache_hit_rate
EPP prefix indexer size	llm_d_epp_prefix_indexer_size
EPP prefix hit ratio P90	histogram_quantile(0.90, sum by(le) (rate(llm_d_epp_prefix_indexer_hit_ratio_bucket[5m])))

Prefill/Decode Disaggregation

Metric Need	PromQL Query
Prefill worker utilization	avg by(pod) (vllm:num_requests_running{pod=~".*prefill.*"})
Prefill worker utilization (SGLang)	avg by(pod) (sglang_num_running_reqs{pod=~".*prefill.*"})
Decode KV cache utilization	avg by(pod) (vllm:kv_cache_usage_perc{pod=~".*decode.*"})
Disaggregation decision ratio	sum(rate(llm_d_epp_disagg_decision_total{decision_type="prefill-decode"}[5m])) / sum(rate(llm_d_epp_disagg_decision_total[5m]))
Average NIXL transfer time (ms)	sum(rate(vllm:nixl_xfer_time_seconds_sum[5m])) / sum(rate(vllm:nixl_xfer_time_seconds_count[5m])) * 1000
NIXL transfer time P95 (ms)	histogram_quantile(0.95, sum by(le) (rate(vllm:nixl_xfer_time_seconds_bucket[5m]))) * 1000
Average NIXL transfer size (MiB)	sum(rate(vllm:nixl_bytes_transferred_sum[5m])) / sum(rate(vllm:nixl_bytes_transferred_count[5m])) / 1048576
NIXL transfer data volume (MiB/sec)	sum(rate(vllm:nixl_bytes_transferred_sum[5m])) / 1048576
Failed NIXL transfers in 5m	sum(increase(vllm:nixl_num_failed_transfers[5m]))
Failed NIXL notifications in 5m	sum(increase(vllm:nixl_num_failed_notifications[5m]))
Expired prefill KV requests in 5m	sum(increase(vllm:nixl_num_kv_expired_reqs[5m]))

NIXL histogram observations are pooled across tensor-parallel ranks. Their counts and average sizes describe rank-level transfer observations, not inference requests or whole-request KV cache sizes.

Flow Control

Requires the `flowControl` feature gate enabled on the EPP.

Metric Need	PromQL Query
Queue size	sum(llm_d_epp_flow_control_queue_size)
Queue size by priority	sum by(priority) (llm_d_epp_flow_control_queue_size)
Queue wait time P99	histogram_quantile(0.99, sum by(le) (rate(llm_d_epp_flow_control_request_queue_duration_seconds_bucket[5m])))
Pool saturation	llm_d_epp_flow_control_pool_saturation

Notes

Metric name prefixes: Current deployments use `llm_d_epp_*`. Older deployments may use `llm_d_inference_scheduler_*`, `inference_objective_*`, `inference_pool_*`, or `inference_extension_*` — update accordingly if panels show "No data".

Histograms: Always include `by(le)` when using `histogram_quantile()` :

```
histogram_quantile(0.99, sum by(le) (rate(metric_name_bucket[5m])))
```

Error metrics only appear after the first error occurs. Use the traffic generation script to populate them for testing.

Observability Setup

This page explains how to set up Prometheus, Grafana, and distributed tracing for an llm-d deployment. All guides reference this page — set this up once and it works across every guide.

[!NOTE] Commands in this page use `${NAMESPACE}` for the namespace where your llm-d workload runs. Set it before following along:

```
bash export NAMESPACE=<your-llm-d-namespace>
```

Step 1: Install Prometheus and Grafana

Skip this step if you already have Prometheus running in your cluster.

```
# Install Prometheus + Grafana into the llm-d-monitoring namespace
./guides/recipes/observability/install-prometheus-grafana.sh
```

For HTTPS/TLS (required by autoscalers like WVA):

```
./guides/recipes/observability/install-prometheus-grafana.sh --enable-tls
```

Verify the installation:

```
kubectl get pods -n llm-d-monitoring
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-llmd-kube-prometheus-stack-alertmanager-0	2/2	Running	0	30s
llmd-grafana-xxxxxxx-xxxxx	3/3	Running	0	30s
prometheus-llmd-kube-prometheus-stack-prometheus-0	2/2	Running	0	30s

Platform-specific notes

OpenShift

OpenShift provides a built-in Prometheus stack via User Workload Monitoring. Enable it instead of installing a separate Prometheus:

- See the OpenShift monitoring documentation to enable User Workload Monitoring

- Prometheus endpoint: `https://thanos-querier.openshift-monitoring.svc.cluster.local:9091`

GKE

GKE clusters include Google Managed Prometheus (GMP) by default. GKE also provides a built-in inference gateway dashboard.

To use GMP as a Grafana data source, follow the GMP Grafana integration guide.

Manual ServiceMonitor Setup (Fallback)

[!NOTE] **The recommended path is the llm-d helm charts**, which create ServiceMonitors automatically when you include `monitoring.values.yaml`. Skip this section if you deployed that way.

Use this manual setup **only as a fallback** for workloads deployed outside the llm-d helm charts — e.g. CRD, KServe, or RHAI — where ServiceMonitors are not created for you. In that case, create them manually as shown below.

Find Your Service Labels

ServiceMonitor selectors **MUST** exactly match your service labels:

```
# Find your EPP/Router service
kubectl get svc -n ${NAMESPACE} --show-labels

# View full labels for a specific service
kubectl get svc -n ${NAMESPACE} <epp-service-name> -o yaml
```

Note the exact `app.kubernetes.io/component` and `app.kubernetes.io/name` values.

Create ServiceMonitors

For EPP:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: <epp-servicemonitor-name>
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app.kubernetes.io/component: <epp-component>
      app.kubernetes.io/name: <epp-name>
  namespaceSelector:
    matchNames:
      - ${NAMESPACE}
  endpoints:
    - port: metrics
      path: /metrics
      interval: 10s
      scheme: http
      bearerTokenFile: /var/run/secrets/kubernetes.io/serviceaccount/token
```

For vLLM:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: <vllm-servicemonitor-name>
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app.kubernetes.io/component: <vllm-component>
      app.kubernetes.io/name: <vllm-name>
  namespaceSelector:
    matchNames:
      - ${NAMESPACE}
  endpoints:
    - port: https
      path: /metrics
      interval: 10s
      scheme: https
      tlsConfig:
        insecureSkipVerify: true
```

Apply:

```
kubectl apply -f servicemonitor-epp.yaml
kubectl apply -f servicemonitor-vllm.yaml
```

Verify

```
kubectl port-forward -n llm-d-monitoring svc/llmd-kube-prometheus-stack-prometheus 9090:9090
curl -sk 'https://localhost:9090/api/v1/targets' | jq '.data.activeTargets[] | select(.labels.namespace=="${NAMESPACE}") | {job:
.labels.job, health: .health}'
```

Targets should show "health": "up" . Then proceed to Step 2 for dashboards.

Step 2: Load Grafana Dashboards

```
./guides/recipes/observability/load-llm-d-dashboards.sh
```

Verify dashboards were imported:

```
kubectl get configmaps -n llm-d-monitoring -l grafana_dashboard=1
```

Then access Grafana:

```
kubectl port-forward -n llm-d-monitoring svc/llmd-grafana 3000:80
# Open http://localhost:3000 (login: admin / admin)
```

Available dashboards:

Dashboard	What it shows
llm-d-vllm-overview	General vLLM metrics overview
llm-d-sglang-overview	General SGLang metrics overview
llm-d-failure-saturation-dashboard	Key failure and saturation indicators
llm-d-diagnostic-drilldown-dashboard	Detailed diagnostic metrics for troubleshooting
llm-d-performance-kv-cache	KV cache utilization and performance
llm-d-pd-coordinator-metrics	Prefill/decode disaggregation metrics
llm-d-inference-gateway	Inference Gateway (EPP) metrics: inference pool, inference objective, and flow control

Step 3: Install Distributed Tracing (Optional)

Deploy the OTel Collector and Jaeger into the same namespace as your llm-d workload:

```
./guides/recipes/observability/install-otel-collector-jaeger.sh -n ${NAMESPACE}
```

Then access the Jaeger UI:

```
kubectl port-forward -n ${NAMESPACE} svc/jaeger-collector 16686:16686
# Open http://localhost:16686
```

For full tracing configuration across vLLM, the routing proxy, and the EPP, see Distributed Tracing.

Cleanup

```
# Remove Prometheus and Grafana
```

```
./guides/recipes/observability/install-prometheus-grafana.sh -u -n llm-d-monitoring
```

```
# Remove OTel Collector and Jaeger
```

```
./guides/recipes/observability/install-otel-collector-jaeger.sh -u -n ${NAMESPACE}
```

Troubleshooting

Autoscaler reports "http: server gave HTTP response to HTTPS client"

The autoscaler is configured for HTTPS but Prometheus is serving HTTP. Enable TLS:

```
./guides/recipes/observability/install-prometheus-grafana.sh -u
```

```
./guides/recipes/observability/install-prometheus-grafana.sh --enable-tls
```

Metrics not appearing in Prometheus

1. Check that PodMonitors and ServiceMonitors exist:

```
bash kubectl get podmonitors,servicemonitors -n ${NAMESPACE}
```

2. Open `http://localhost:9090/targets` (after port-forwarding Prometheus) and check that vLLM and EPP targets show UP

3. Confirm pods expose metrics:

```
bash VLLM_POD=$(kubectl get pods -n ${NAMESPACE} -l app=my-model -o jsonpath='{.items[0].metadata.name}') kubectl port-forward -n  
${NAMESPACE} ${VLLM_POD} 8000:8000 curl http://localhost:8000/metrics | head -20
```

Grafana dashboards show "No data"

1. Verify the Grafana datasource points to the correct Prometheus URL
2. Check that metrics are flowing in Prometheus first
3. If using TLS, ensure the Grafana datasource is configured for HTTPS with the correct CA certificate

Alerting

This page covers default sets of Prometheus alerting rules for the EPP (Endpoint Picker) and for the Batch Gateway. For Prometheus and Grafana installation, see [Observability Setup](#) first, and for the metrics these alerts are built on, see [Metrics](#).

The rules ship as a `PrometheusRule` custom resource, so they require the Prometheus Operator (bundled with the `kube-prometheus-stack` installed by the setup guide).

[!NOTE] Commands on this page use `${NAMESPACE}` for the namespace where your `llm-d` workload runs. Set it before following along:

```
bash export NAMESPACE=<your-llm-d-namespace>
```

Prerequisites

- A running `llm-d` deployment with an `InferencePool` — see the quickstart if needed
- Prometheus and the Prometheus Operator installed — see [Observability Setup](#)
- EPP metrics being scraped — see [Metrics](#) (verify the `epp-servicemonitor` `ServiceMonitor` exists)

Step 1: Apply the Alerting Rules

Apply the bundled `PrometheusRule` :

```
kubectl apply -n ${NAMESPACE} -f guides/recipes/observability/alerts/epp-alerting-rules.yaml
```

[!NOTE] The bundled `install-prometheus-grafana.sh` opens Prometheus' `ruleSelector` so any `PrometheusRule` is discovered (central mode). If you run the installer in individual/scoped mode, Prometheus only selects rules carrying the `monitoring-ns: ${NAMESPACE}` label in a namespace with the same label — add that label to the `PrometheusRule` (and its namespace) to match your `ServiceMonitor` . If you bring your own Prometheus, make sure its `ruleSelector` matches the `app: epp-metrics` label on this resource.

Step 2: Verify

Confirm the rule was created:

```
kubectl get prometheusrules -n ${NAMESPACE}
```


Expected output:

```
NAME      AGE
epp-alerting-rules 10s
```

Then open the Prometheus UI and check that the rules loaded under **Status → Rule Health** (or `http://localhost:9090/rules` after port-forwarding — see Metrics). You should see the `epp.availability` and `epp.selfhealth` groups.

Alert Reference

All metric names use the current `llm_d_epp_*` prefix. Thresholds and `for:` windows are conservative defaults — tune them for your traffic profile (see Customization).

Availability and errors (`epp.availability`)

Alert	Severity	Fires when	Why it matters
EPPHighErrorRatio	warning	Error ratio > 5% for 10m	Backend or routing failures are degrading a meaningful share of requests
EPPCriticalErrorRatio	critical	Error ratio > 20% for 5m	The inference path is likely broken, not just degraded
EPPNoReadyEndpoints	critical	<code>llm_d_epp_ready_endpoints == 0</code> for 2m	The pool has no routable endpoints — every request fails
EPPMetricsAbsent	critical	<code>absent(llm_d_epp_ready_endpoints)</code> for 5m	Coarse cluster-wide signal: no EPP metrics from any pool — every EPP is down or scraping stopped entirely (a silent observability gap)

The error-ratio alerts are 0/0 -safe: with no traffic the expression yields no value and stays silent. On very low-volume workloads you may see startup flapping — add a request-rate floor with `and sum(rate(llm_d_epp_request_total[5m])) > N`.

[!NOTE] `EPPMetricsAbsent` uses `absent()`, which only fires when *no* `llm_d_epp_ready_endpoints` series exist at all. In multi-pool deployments, one pool's EPP can die while the others keep reporting — its series vanish rather than report `0`, so neither this alert nor `EPPNoReadyEndpoints` fires for it. For per-pool coverage, alert on series that existed recently but disappeared:

```
promql llm_d_epp_ready_endpoints offset 15m unless llm_d_epp_ready_endpoints
```

This fires once per vanished pool, at the cost of a fixed lookback window (a pool removed on purpose also fires until the offset ages out).

EPP self-health (`epp.selfhealth`)

Alert	Severity	Fires when	Why it matters
EPPDataLayerPollErrors	warning	llm_d_epp_datalayer_poll_errors_total increasing for 10m	A data source failed to poll — scheduling may be running on stale endpoint state
EPPDataLayerExtractErrors	warning	llm_d_epp_datalayer_extract_errors_total increasing for 10m	A data extractor failed — scheduling may be running on stale state
EPPExtProcStreamErrors	warning	llm_d_epp_extproc_streams_total{code!~"OK\\ Canceled"} increasing for 10m	Envoy↔EPP ext_proc streams are terminating abnormally

[!NOTE] EPPExtProcStreamErrors relies on opt-in metrics enabled by the EPP --enable-grpc-stream-metrics flag. When the flag is unset, the series are absent and the alert never fires. The matcher excludes OK and Canceled (a normal client disconnect) — adjust it for your environment.

Batch Gateway (batch-gateway.rules)

These alerts cover the Batch Gateway processor and GC reconciler rather than the request path, so they are shipped as a separate PrometheusRule and are only relevant if you deployed the Batch Gateway guide.

```
kubectl apply -n ${NAMESPACE} -f guides/recipes/observability/alerts/batch-gateway-alerting-rules.yaml
kubectl get prometheusrules -n ${NAMESPACE}
```

You should then see the batch-gateway.rules group under **Status → Rule Health** in the Prometheus UI.

[!IMPORTANT] Every expression in this file is scoped with namespace="batch-gateway" , the namespace the Batch Gateway guide deploys into. The Batch Gateway metric names are unprefixed and generic (jobs_processed_total , active_workers), so an unscoped rule can pick up unrelated workloads. If you deployed into a different namespace, edit the matchers to match — applying the PrometheusRule into your namespace does **not** scope its queries. If you bring your own Prometheus, its ruleSelector must match this resource's label, app: batch-gateway-metrics (not app: epp-metrics , which is specific to the EPP rule).

Alert	Severity	Fires when	Why it matters
BatchGatewayHighQueueWait	warning	p95 job_queue_wait_duration_seconds > 300s for 15m	Jobs are backing up in the priority queue faster than workers drain it
BatchGatewayHighJobFailureRate	warning	Failed share of jobs_processed_total > 10% for 10m	Job execution is failing at a rate users will notice
BatchGatewayExpiredJobsDetected	warning	Expired share of jobs_processed_total > 3% for 10m	Jobs are aging out before execution — capacity or completion-window problem, not a code failure
BatchGatewayWorkersSaturated	warning	active_workers / total_workers > 90% for 15m	The worker pool is the bottleneck; raise NumWorkers or add replicas
BatchReconcilerErrors	warning	batch_reconciler_errors_total increased over 65m	The GC orphan reconciler is failing, so orphaned jobs are not being recovered

The two ratio alerts use `clamp_min` on the denominator, so they stay silent when no jobs are being processed rather than dividing by zero.

`BatchReconcilerErrors` uses `increase()` over a 65m window rather than `rate()` : the reconciler runs on a 60m interval by default and increments the counter at most once per cycle, which is too sparse for a `rate()` threshold. If you shorten `reconciler.interval` in the GC config, shorten the window to match — a window shorter than the interval can miss a once-per-cycle failure, and a much longer one keeps the alert firing after the error has aged out.

Customization

These rules are a starting point, not a tuned policy. Common adjustments:

- **Thresholds and durations** — edit the `expr` comparison (`> 0.05`) and the `for:` window per alert to match your SLOs and noise tolerance.
- **Routing** — the `severity: warning|critical` labels are the hook for Alertmanager routing (e.g. page on `critical` , Slack on `warning`). Configure routes in your Alertmanager config.
- **Scope** — to alert per pool or per model, add a `by (...)` clause to the error-ratio expressions using labels such as `name` or `model_name` .

See the PromQL Reference for more queries you can promote into alerts (for example latency SLOs and flow-control saturation).

Cleanup

```
kubectl delete -n ${NAMESPACE} -f guides/recipes/observability/alerts/epp-alerting-rules.yaml
kubectl delete -n ${NAMESPACE} -f guides/recipes/observability/alerts/batch-gateway-alerting-rules.yaml
```

Troubleshooting

Rules don't appear in the Prometheus UI

1. Confirm the resource exists: `kubectl get prometheusrules -n ${NAMESPACE}` .
2. Confirm Prometheus' `ruleSelector` matches the rule's labels. The bundled installer sets `ruleSelectorNilUsesHelmValues: false` with an open `ruleSelector` in central mode; in scoped mode add the `monitoring-ns` label (see Step 1).
3. Check the Prometheus Operator logs for rule-rejection errors: `kubectl logs -n llm-d-monitoring -l app.kubernetes.io/name=prometheus-operator` .

An alert never fires

- `EPPExtProcStreamErrors` needs the EPP `--enable-grpc-stream-metrics` flag (see the note above).
- Error-ratio alerts only evaluate once there is traffic — see the `0/0` -safe note above.
- Self-health counters only produce series after the first error occurs.

Infrastructure & Environments

This section covers the hardware and software requirements for llm-d, cluster configuration, accelerator specs, and platform adaptations across diverse physical execution environments.

Kubernetes Infrastructure Providers

Provider-specific cluster setup notes (GKE, AKS, OpenShift, Minikube, DigitalOcean).

Multi-Node Serving Orchestration

Deploying multi-host inference workloads with LeaderWorkerSet (LWS) and Topology Aware Scheduling.

Non-Kubernetes & Bare-Metal Deployments

Running the llm-d routing stack on bare metal, HPC Slurm schedulers, or Ray via file-based worker discovery.

Fast Internode Networking & RDMA

Orchestrating multi-host replica topologies and RDMA networking fabrics.

Gateway & Ingress Resources

Configuring ingress controllers, Gateway API, and service meshes.

llm-d infrastructure

llm-d tests on the following configurations, supporting leading-edge AI accelerators:

- Kubernetes: 1.29 or newer
- Your cluster scheduler must support placing multiple pods within the same networking domain for running multi-host inference
- Kubernetes v1.33.0+ is recommended for complete sidecar init container support (restartPolicy: Always). If using Kubernetes v1.28.x or below, pods may get stuck in Init:0/1 state due to incomplete sidecar support.
- Recent generation datacenter-class accelerators
- AMD MI250X or newer
- Google TPU v5e, v6e, and newer
- NVIDIA L4, A100, H100, H200, B200, and newer

- Fast internode networking
- For accelerators
 - AMD Infinity Fabric, InfiniBand NICs
 - Google TPU ICI
 - NVIDIA NVLink, InfiniBand or RoCE NICs
- For hosts and north/south traffic
 - Fast (100Gbps+ aggregate throughput) datacenter NICs
- Hosts
- 80+ x86 or ARM cores per machine
- 500GiB or more of memory
- PCIe 5+

Older configurations may function, especially slightly older accelerators, but testing is best-effort.

(Optional) vLLM container image

llm-d provides container images derived from the vLLM upstream that are tested with the supported hardware and have all necessary optimized libraries installed. To build and deploy with your own image, you should integrate:

- General
- vLLM: 0.10.0 or newer
- NIXL: 0.5.0 or newer
- UCX: 0.19.0 or newer
- NVIDIA-specific
- NVSHMEM: 3.3.9 or newer

llm-d guides expect a series of conventions to be followed in the vLLM image:

- General
- At least one vLLM compatible Python version must be available (3.9 to 3.12)
 - We recommend at least 3.10+
- Required system libraries must be bundled
 - `LD_LIBRARY_PATH` must contain all necessary system libraries for vLLM to function
- `PATH` must contain the vLLM binary and directly invoking `vllm` should start with the correct Python environment (i.e. a virtual env)
- The default image command (or if not specified, entrypoint) should start vLLM in a serving configuration and accept additional arguments
 - A pod with `args` should see all arguments passed to vLLM
 - A pod with `command: ["vllm", "serve"]` should override any image defaults
- Caches
- Default compilation cache directory environment variables under a shared root path under `/tmp/cache/compile/<NAME>`
 - I.e. set `VLLM_CACHE_ROOT=/tmp/cache/compile/vllm` to ensure vLLM compiles to a temporary directory

- Future versions of vLLM will recommend mounting a pod volume to `/tmp/cache` to mitigate restart for some caches.
- Do not hardcode the model cache directory and model cache environment variables
 - Future versions of llm-d will provide conventions for vLLM model loading
- Hardware
- Follow best practices for your hardware ecosystem, including:
 - Expecting to mount hardware-specific drivers and libraries from a standard host location as a value
 - Ahead Of Time (AOT) compilation of kernels
- NVIDIA specific
 - `LD_LIBRARY_PATH` includes the `/usr/local/nvidia/lib64` directory to allow Kubernetes GPU operators to inject the appropriate driver

Installing on a well-lit infrastructure provider

The following documentation describes llm-d tested setup for cluster infrastructure providers as well as specific deployment settings that will impact how model servers is expected to access accelerators.

- Azure Kubernetes Service (AKS)
- DigitalOcean Kubernetes (DOKS)
- Google Kubernetes Engine (GKE)
- OpenShift (OCP), OpenShift on AWS
- minikube for single-host development

These provider configurations are tested regularly.

Please follow the provider-specific documentation to ensure your Kubernetes cluster and hardware is properly configured before continuing.

Other providers

To add a new infrastructure provider to our well-lit paths, we request the following support:

- Documentation on configuring the platform to support one or more well-lit path guides
- The appropriate configuration contributed to the guide to deal with provider specific variation
- An automated test environment that validates the supported guides
- At least one documented platform maintainer who responds to GitHub issues and is available for regular discussion in the llm-d slack channel `#sig-installation` .

Multi-Node Serving Orchestration

(Optional) Install LeaderWorkerSet for multi-host inference

The LeaderWorkerSet (LWS) Kubernetes workload controller specializes in deploying serving workloads where each replica is composed of multiple pods spread across hosts, specifically accelerator nodes. llm-d defaults to LWS for deployment of multi-host inference for rank to pod mappings, topology aware placement to ensure optimal accelerator network performance, and all-or-nothing failure and restart semantics to recover in the event of a bad node or accelerator.

Use the LWS installation guide to install the recommended 0.9.0 release when deploying an llm-d guide using LWS.

Install LWS

To install LWS release 0.9.0 into the `lws-system` namespace:

```
export LWS_VERSION="0.9.0"
export LWS_NAMESPACE="lws-system"

helm install lws oci://registry.k8s.io/lws/charts/lws \
  --version "${LWS_VERSION}" \
  --namespace "${LWS_NAMESPACE}" \
  --create-namespace
```

[!WARNING] If you installed LWS 0.7.0 or earlier with Helm, do not upgrade directly to 0.9.0. Helm may delete the LWS CRD and cascade-delete existing LeaderWorkerSet resources during upgrade; see [kubernetes-sigs/lws#880](https://kubernetes-sigs.github.io/lws/#880).

Uninstall LWS

To uninstall LWS:

```
export LWS_NAMESPACE="lws-system"

helm uninstall lws --namespace "${LWS_NAMESPACE}"
```

(Optional) Install Kueue and Kueue Populator for Topology Aware Scheduling for multi-host inference

Kueue is a Kubernetes controller for job queueing. When combined with Kueue-Populator, it can schedule a multi-host inference workload for optimal accelerator network performance.

Use the TAS + LWS user guide to setup topology aware scheduling when deploying an llm-d guide using LWS.

No-Kubernetes Deployment

llm-d's reference deployment runs on Kubernetes — workers are managed by Kubernetes `Deployments`, the EPP discovers them through an `InferencePool`, and the platform handles networking and lifecycle. Many environments don't have a Kubernetes control plane, though: HPC schedulers like Slurm or LSF launch workers dynamically, Ray-based stacks run workers as actors, bare-metal inference farms operate without K8s, and a single workstation with a couple of GPUs is often enough for development.

The No-Kubernetes path runs the same routing stack — the llm-d EPP, Envoy, and one or more model servers — directly as host processes or containers. The EPP gets its endpoint inventory from a YAML file on disk via the file-discovery plugin instead of watching an `InferencePool` over the Kubernetes API; everything else (EPP plugin set, scoring, Envoy `ext_proc`, vLLM arguments) is unchanged.

[!IMPORTANT] Without Kubernetes, some pieces of the llm-d stack are out of scope: `InferenceObjective`-driven `FlowControl`, the `InferenceModelRewrite` model-name rewriter, and `PodMonitor`-based Prometheus discovery. Scoring, prefix-cache affinity, saturation-based admission, and Prometheus metrics on `--metrics-port` all work; see the parity caveats for the full list.

Deploy

See the no-Kubernetes deployment guide for manifests and step-by-step deployment.

Architecture

![No-Kubernetes Deployment](../assets/no-kubernetes-deployment.svg)

Traffic flows the same way as the optimized-baseline path:

```
client -> Envoy listener :8081
  -> ext_proc gRPC :9002 (EPP picks endpoint, sets header)
  -> ORIGINAL_DST cluster
  -> reads x-gateway-destination-endpoint from EPP response
  -> <address>:<port> of the vLLM worker chosen by the EPP
```

The EPP's datastore is populated entirely from `endpoints.yaml`. With `watchFile: true` the file is hot-reloaded on every atomic rewrite — adding, removing, or relabelling a worker takes effect without restarting the EPP. The plugin set, weights, and scheduling profile match the Optimized Baseline, so routing behaviour is identical to the Kubernetes-based deployment.

Further Reading

- `file-discovery` plugin source
- "No Kubernetes? No Problem" — full background on the design

Gateway Guides

This directory contains guides for deploying a Kubernetes Gateway as a proxy for the **llm-d Router**. For more information on this capability, see [llm-d Router in Gateway Mode](#).

Why do you need a Gateway?

The **llm-d Router** extends compatible Gateway providers with load balancing optimizations for LLM traffic across model server replicas.

The integration with a Gateway allows self-hosted models to be exposed in a wide variety of network topologies including:

- Internet-facing services
- Internal to your cluster
- Through a service mesh

and take advantage of key Gateway features like:

- Traffic splitting for incremental rollout of new models
- TLS encryption of queries and responses

By integrating with a Gateway -- instead of developing an llm-d specific proxy layer -- llm-d can leverage the high performance of mature proxies and take advantage of existing operational tools for managing traffic to services. Compatible Gateway implementations may use proxies like Envoy or other high-performance data planes under the hood.

Supported Gateway Providers

llm-d requires you select a Gateway implementation that supports the Gateway API Inference Extension. Your infrastructure may provide a default compatible implementation, or you may choose to deploy a gateway implementation onto your cluster.

- **GKE Gateway** - GKE's implementation of the Gateway API is through the GKE Gateway controller which provisions Google Cloud Load Balancers for Pods in GKE clusters. The GKE Gateway controller supports weighted traffic splitting, mirroring, advanced routing, multi-cluster load balancing and more. [Official GKE Docs](#).
- **Istio** - Istio is an open source service mesh and gateway implementation. It provides a fully compliant implementation of the Kubernetes Gateway API for cluster ingress traffic control. [Official Istio docs](#)
- **Agentgateway** - Agentgateway is a high-performance, Rust-based AI gateway for LLM, MCP, and A2A workloads that can also serve as a Gateway API and Inference Gateway implementation. [Official Agentgateway docs](#).

- Envoy AI Gateway - Envoy AI Gateway is an open source project for using Envoy Gateway to handle request traffic from application clients to GenAI services that can also serve as a Gateway API and Inference Gateway implementation. [Official Envoy AI Gateway docs](#).

[!NOTE] Setting up a Gateway generally requires cluster administration rights.

Other Providers

For other compatible Gateway implementations not listed above, follow the installation instructions for your selected Gateway provider. Ensure the necessary CRDs for Gateway API and the Gateway API Inference Extension are installed.

Istio

This guide shows how to deploy llm-d with Istio as your inference gateway. By the end, inference requests will flow from an Istio-managed Gateway to your model servers via the llm-d EPP.

[!NOTE] This guide assumes familiarity with Gateway API and llm-d.

Prerequisites

1. The environment variables `${GUIDE_NAME}`, `${MODEL_NAME}` and `${NAMESPACE}` should be set as part of deploying one of the well-lit path guides.
2. A Kubernetes cluster running one of the three most recent Kubernetes releases
3. Helm
4. jq

Step 1: Install Gateway API and Gateway API Inference Extension CRDs

Install the required CRDs by following the CRD installation guide.

Step 2: Install Istio

Install Istio with inference extension support enabled:

```
ISTIO_VERSION=1.29.2
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=${ISTIO_VERSION} sh -
export PATH="$PWD/istio-${ISTIO_VERSION}/bin:$PATH"
istioctl install -y \
--set values.pilot.env.ENABLE_GATEWAY_API_INFERENCE_EXTENSION=true
```

Verify the installation:

```
kubectl get pods -n istio-system
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
istiod-xxxxxxxx-xxxxx	1/1	Running	0	30s

Step 3: Deploy the Gateway

Set the llm-d version to match your deployment:

```
LLM_D_VERSION=main # Use 'main' for latest, or a release tag like 'v0.7.0'
```

Create a `Gateway` resource. Istio watches this resource and creates an Envoy-based proxy that accepts incoming traffic.

```
kubectl apply -k "https://github.com/llm-d/llm-d/guides/recipes/gateway/istio?ref=${LLM_D_VERSION}" -n ${NAMESPACE}
```

Verify the Gateway is programmed:

```
kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE}
```

Expected output:

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
llm-d-inference-gateway	istio	10.xx.xx.xx	True	30s

Wait until `PROGRAMMED` shows `True` before proceeding.

Step 4: Send a Request

[!IMPORTANT] Before sending requests, you must deploy a well-lit path guide. This sets up a model server deployment, an `InferencePool`, and an `HTTPRoute` to connect the Gateway to the pool.

Get the `Gateway` external address:

```
export IP=$(kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}')
```

Send an inference request via the managed `Gateway` :

```
curl -X POST http://${IP}/v1/completions \  
-H 'Content-Type: application/json' \  
-d '{  
  "model": "${MODEL_NAME}",  
  "prompt": "How are you today?"  
}' | jq
```

Cleanup

```
kubectl delete gateway llm-d-inference-gateway -n ${NAMESPACE}  
istioctl uninstall --purge -y  
kubectl delete namespace istio-system  
kubectl delete gatewayclass istio istio-remote
```

To uninstall the Gateway API and Gateway API Inference Extension CRDs, see the CRD installation guide.

Troubleshooting

Gateway not showing `PROGRAMMED=True`

```
kubectl describe gateway llm-d-inference-gateway -n ${NAMESPACE}
kubectl get pods -n istio-system
kubectl logs -n istio-system deployment/istiod --tail=20
```

Verify Istio was installed with the inference extension flag enabled.

HTTPRoute not accepted

```
kubectl describe httproute ${GUIDE_NAME} -n ${NAMESPACE}
```

Verify that `parentRefs` matches the Gateway name and `backendRefs` matches the InferencePool name.

No response from Gateway IP

```
kubectl get gateway llm-d-inference-gateway -o jsonpath='{.status.addresses[0].value}' -n ${NAMESPACE}
```

If the address is empty, your Gateway may still be waiting for a LoadBalancer service. Check that your cluster supports external load balancers.

GKE

This guide shows how to deploy llm-d with GKE Gateway as your inference gateway. By the end, inference requests will be forwarded by a GKE-managed Gateway to your model servers via the llm-d EPP.

[!NOTE] This guide assumes familiarity with Gateway API and llm-d.

Prerequisites

1. The environment variables `${GUIDE_NAME}`, `${MODEL_NAME}` and `${NAMESPACE}` should be set as part of deploying one of the well-lit path guides.
2. You have verified the version in and sourced in the llm-d repo wide `env.sh` file:

```
bash export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel)) source ${REPO_ROOT}/guides/env.sh
```

3. The following steps from the GKE Inference Gateway deployment documentation should be run:

- Verify your prerequisites
- Configure a proxy-only subnet
- Enable Gateway API in your cluster
- Verify your cluster

Step 1: Install Gateway API and Gateway API Inference Extension CRDs

[!NOTE] GKE automatically installs all GA CRDs for Gateway API and Gateway API Inference Extension on GKE versions 1.34.0-gke.1626000 or later. If using this version or newer, skip to Step 2.

For GKE versions earlier than 1.34.0-gke.1626000, install the CRDs manually:

```
# GAIE_URL is automatically calculated from GAIE_VERSION at ${REPO_ROOT}/guides/env.sh
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api-inference-extension/${GAIE_URL}/v1-manifests.yaml
```

Verify the APIs are available:

```
kubectl api-resources --api-group=gateway.networking.k8s.io
kubectl api-resources --api-group=inference.networking.k8s.io
```

Step 2: Deploy the Gateway

GKE supports two types of Application Load Balancers. Choose the one that matches your network requirements:

Regional External Application Load Balancer

For internet-facing workloads accessible from outside your VPC:

```
LLM_D_VERSION=main # Use 'main' for latest, or a release tag like 'v0.7.0'
GATEWAY_CLASS=gke-l7-regional-external-managed
kubectl apply -n ${NAMESPACE} -k "https://github.com/llm-d/llm-d/guides/recipes/gateway/${GATEWAY_CLASS}?ref=${LLM_D_VERSION}"
```

Regional Internal Application Load Balancer

For private workloads accessible only within your VPC:

```
LLM_D_VERSION=main # Use 'main' for latest, or a release tag like 'v0.7.0'
GATEWAY_CLASS=gke-l7-rlb
kubectl apply -n ${NAMESPACE} -k "https://github.com/llm-d/llm-d/guides/recipes/gateway/${GATEWAY_CLASS}?ref=${LLM_D_VERSION}"
```

Step 3: Verify the Gateway

Verify the Gateway is programmed:

```
kubectl get gateway -n ${NAMESPACE} llm-d-inference-gateway
```

Expected output:

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
llm-d-inference-gateway	gke-l7-regional-external-managed	xx.xx.xx.xx	True	30s

Wait until PROGRAMMED shows True before proceeding.

Step 4: Deploy a Model Serving Workload

Before sending inference requests through the Gateway, you must deploy a model server (such as vLLM), an InferencePool, and an HTTPRoute connecting the Gateway to the pool. You can deploy using a standard baseline or enable High Availability.

Option A: Standard Deployment (Well-Lit Path)

Deploy one of the well-lit path guides with the GKE Gateway provider enabled:

1. Deploy the llm-d Router in Gateway mode:

```
export PROVIDER_NAME=gke
helm install ${GUIDE_NAME} \
  ${ROUTER_GATEWAY_CHART} \
  -f ${ROUTER_BASE_VALUES} \
  ${MONITORING_VALUES} \
  -f ${ROUTER_VALUES} \
  --set provider.name=${PROVIDER_NAME} \
  --set httpRoute.create=true \
  --set httpRoute.inferenceGatewayName=llm-d-inference-gateway \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}
```

2. Deploy the model server with the GKE overlay:

```
export ACCELERATOR_TYPE=gpu # options: gpu, amd, xpu, hpu, tpu/v6, tpu/v7, cpu
export MODEL_SERVER=vllm # options: vllm, sglang, trtllm
kubectrl apply -n ${NAMESPACE} -k
${REPO_ROOT}/guides/${GUIDE_NAME}/modelserver/${ACCELERATOR_TYPE}/${MODEL_SERVER}/gke/
```

Option B: High Availability with Preferred Backends

Deploying Endpoint Picker (EPP) with GKE Preferred Backends enables an active-passive routing topology backed by Cloud Load Balancing. Steady-state ext_proc traffic routes exclusively to the primary EPP replica (epp-0) to keep KV-cache tracking and request state centralized. If epp-0 becomes unhealthy, Cloud Load Balancing shifts traffic to the warm standby replica (epp-1), with InferencePool fail-open mode providing an additional safety net against dropped requests.

Key Benefits

- **State Consistency:** Directs 100% of steady-state ext_proc traffic to the primary EPP instance (epp-0), preserving KV-cache state and request scheduling context.
- **Zero-Downtime Failover:** If the primary EPP pod crashes or undergoes maintenance, Cloud Load Balancer detects the failure via active gRPC health checks and immediately routes traffic to the warm standby replica (epp-1).
- **Fail-Open Resilience:** Paired with failureMode: FailOpen on the InferencePool , transient routing blips bypass EPP and forward directly to model servers without dropping user requests.

1. Deploy the llm-d Router with Preferred Backends enabled:

[!NOTE] When Preferred Backends is enabled, the chart deploys a single StatefulSet where pod ordinals map to target priority tiers and generates corresponding GCPBackendPolicy resources for each tier.

```

export PROVIDER_NAME=gke
helm install ${GUIDE_NAME} \
  ${ROUTER_GATEWAY_CHART} \
  -f ${ROUTER_BASE_VALUES} \
  ${MONITORING_VALUES} \
  -f ${ROUTER_VALUES} \
  --set provider.name=${PROVIDER_NAME} \
  --set provider.gke.preferredBackends.enabled=true \
  --set provider.gke.preferredBackends.preferredReplicas=1 \
  --set provider.gke.preferredBackends.defaultReplicas=1 \
  --set httpRoute.create=true \
  --set httpRoute.inferenceGatewayName=llm-d-inference-gateway \
  -n ${NAMESPACE} --version ${ROUTER_CHART_VERSION}

```

Key configuration parameters (see `values.yaml`):

- * `preferredReplicas` (Default: 1): the replica count for active primary pod ordinals pinned to the `PREFERRED` backend preference tier (`epp-0`). Setting a value greater than 1 scales concurrent active routing capacity.
- * `defaultReplicas` (Default: 1): the replica count for standby pod ordinals pinned to the `DEFAULT` backend preference tier (`epp-1`). Setting a value greater than 1 scales warm standby failover capacity.
- * `balancingMode` (Default: `RATE`): the calculation mode used to determine load thresholds (`RATE` , `UTILIZATION` , or `CONNECTION`).
- * `maxRatePerEndpoint` (Default: 100): the maximum requests per second per pod instance before spilling over to standby tiers.
- * `capacityScalerPercent` (Default: 100): the effective capacity ceiling percentage (0 to 100).

2. Deploy the model server with the GKE overlay:

```

export ACCELERATOR_TYPE=gpu # options: gpu, amd, xpu, hpu, tpu/v6, tpu/v7, cpu
export MODEL_SERVER=vllm # options: vllm, sglang, trtllm
kubectl apply -n ${NAMESPACE} -k
${REPO_ROOT}/guides/${GUIDE_NAME}/modelserver/${ACCELERATOR_TYPE}/${MODEL_SERVER}/gke/

```

3. Discover and attach the primary Preferred Network Endpoint Groups (NEGs) across zones to the generated Google Cloud BackendService :

[!NOTE] The `InferencePool` resource natively targets the standby backup Service (`Service/${GUIDE_NAME}-epp-backup`). Anchoring declarative Gateway ownership to the standby Service ensures that GKE Gateway Controller reconciliation loops do not detach manually attached primary NEGs during pod restarts or updates.

```
export PROJECT="<your GCP project>"
export REGION="<your GCP region>"
export NAMESPACE="<your namespace>"
export GUIDE_NAME="<llm-d guide name>"
```

```
export BACKEND_SERVICE=$(gcloud compute backend-services list --project=${PROJECT} --format="value(name)" | grep "${GUIDE_NAME}-epp")
```

```
export PRIMARY_NEG=$(kubectl get svc ${GUIDE_NAME}-epp -n ${NAMESPACE} -o jsonpath='{.metadata.annotations.cloud.google.com/neg-status}' | jq -r '.network_endpoint_groups["9002"]')
```

```
for ZONE in $(kubectl get svc ${GUIDE_NAME}-epp -n ${NAMESPACE} -o jsonpath='{.metadata.annotations.cloud.google.com/neg-status}' | jq -r '.zones[]'); do
  gcloud compute backend-services add-backend ${BACKEND_SERVICE} \
    --network-endpoint-group=${PRIMARY_NEG} \
    --network-endpoint-group-zone=${ZONE} \
    --region=${REGION} \
    --project=${PROJECT} \
    --balancing-mode=RATE \
    --max-rate-per-endpoint=100
  sleep 15
done
```

[!IMPORTANT] Once zonal NEG's are attached to the Google Cloud BackendService , scaling replica counts (preferredReplicas or defaultReplicas) or restarting pods does not require running gcloud commands again. The GKE NEG Controller automatically synchronizes individual pod IP endpoints into the registered zonal NEG's.

If you delete and recreate the parent Gateway or InferencePool resource, the GKE Gateway Controller recreates the underlying Google Cloud BackendService with a new cloud identifier. In that event, execute the attachment workflow above to attach primary zonal NEG's to the newly created BackendService .

Step 5: Send a Request

Get the Gateway external address:

```
export IP=$(kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}')
```

Send an inference request via the managed Gateway :

```
curl -X POST http://${IP}/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "${MODEL_NAME}",
    "prompt": "How are you today?"
  }' | jq
```

Cleanup

```
kubectl delete gateway llm-d-inference-gateway -n ${NAMESPACE}
```

Troubleshooting

Gateway not showing PROGRAMMED=True

```
kubectl describe gateway llm-d-inference-gateway -n ${NAMESPACE}
```

Verify all prerequisites were applied, especially enabling Gateway API in your cluster and configuring a proxy-only subnet. Also make sure the cluster is running a supported GKE version.

HTTPRoute not accepted

```
kubectl describe httproute ${GUIDE_NAME} -n ${NAMESPACE}
```

Verify that `parentRefs` matches the Gateway name and `backendRefs` matches the InferencePool name.

No response from Gateway IP

```
kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}'
```

If the address is empty, your Gateway may still be waiting for a LoadBalancer service. Check that your cluster supports external load balancers.

Getting fault filter abort response

A couple of issues may cause this:

1. The request doesn't match the routing rules setup on HTTPRoute.
2. A misconfiguration in the gateway's backend routing. When configured correctly, the HTTPRoute status should have a condition of type `Reconciled` and reason `ReconciliationSucceeded`.

Agentgateway

This guide shows how to deploy llm-d with agentgateway as your inference gateway. By the end, inference requests will flow from an agentgateway-managed Gateway to your model servers via the llm-d EPP.

[!NOTE] This guide assumes familiarity with Gateway API and llm-d.

Prerequisites

1. The environment variables `${GUIDE_NAME}`, `${MODEL_NAME}` and `${NAMESPACE}` should be set as part of deploying one of the well-lit path guides.
2. A Kubernetes cluster running one of the three most recent Kubernetes releases
3. Helm
4. jq

Step 1: Install Gateway API and Gateway API Inference Extension CRDs

Install the required CRDs by following the CRD installation guide.

Step 2: Install Agentgateway

Install the agentgateway CRDs and control plane with inference extension support enabled:

```
AGENTGATEWAY_VERSION=v1.1.0
```

```
helm upgrade --install agentgateway-crds \
oci://cr.agentgateway.dev/charts/agentgateway-crds \
--namespace agentgateway-system \
--create-namespace \
--version ${AGENTGATEWAY_VERSION}
```

```
helm upgrade --install agentgateway \
oci://cr.agentgateway.dev/charts/agentgateway \
--namespace agentgateway-system \
--create-namespace \
--version ${AGENTGATEWAY_VERSION} \
--set inferenceExtension.enabled=true
```

Verify the installation:

```
kubect! get pods -n agentgateway-system
kubect! get gatewayclass agentgateway
```

Expected output:

NAME	CONTROLLER	ACCEPTED	AGE
agentgateway	agentgateway.dev/agentgateway	True	30s

Step 3: Deploy the Gateway

Set the llm-d version to match your deployment:

```
LLM_D_VERSION=main # Use 'main' for latest, or a release tag like 'v0.7.0'
```

Agentgateway

This deploys a gateway suitable for `agentgateway`, using the `agentgateway` gateway class. This is the preferred self-installed inference gateway recipe in llm-d.

```
kubect! apply -k "https://github.com/llm-d/llm-d/guides/recipes/gateway/agentgateway?ref=${LLM_D_VERSION}" -n ${NAMESPACE}
```

Agentgateway (OpenShift)

This deploys the preferred OpenShift-oriented `agentgateway` recipe. The rendered `Gateway` uses the `agentgateway` `GatewayClass` and an OpenShift-oriented `AgentgatewayParameters` resource.

```
kubect! apply -k "https://github.com/llm-d/llm-d/guides/recipes/gateway/agentgateway-openshift?ref=${LLM_D_VERSION}" -n ${NAMESPACE}
```

Verify the `Gateway` is programmed:

```
kubect! get gateway llm-d-inference-gateway -n ${NAMESPACE}
```

Expected output:

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
llm-d-inference-gateway	agentgateway	10.xx.xx.xx	True	30s

Wait until `PROGRAMMED` shows `True` before proceeding.

Step 4: Send a Request

[!IMPORTANT] Before sending requests, you must deploy a well-lit path guide. This sets up a model server deployment, an `InferencePool`, and an `HTTPRoute` to connect the Gateway to the pool.

Get the `Gateway` external address:

```
export IP=$(kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}')
```

Send an inference request via the managed `Gateway` :

```
curl -X POST http://${IP}/v1/completions \
-H 'Content-Type: application/json' \
-H 'X-Gateway-Base-Model-Name: "${GUIDE_NAME}"' \
-d '{
  "model": "${MODEL_NAME}",
  "prompt": "How are you today?"
}' | jq
```

Cleanup

```
kubectl delete gateway llm-d-inference-gateway -n ${NAMESPACE}
helm uninstall agentgateway -n agentgateway-system
helm uninstall agentgateway-crds -n agentgateway-system
kubectl delete namespace agentgateway-system
kubectl delete gatewayclass agentgateway
```

To uninstall the Gateway API and Gateway API Inference Extension CRDs, see the CRD installation guide.

Troubleshooting

Gateway not showing `PROGRAMMED=True`

```
kubectl describe gateway llm-d-inference-gateway -n ${NAMESPACE}
kubectl get pods -n agentgateway-system
kubectl logs -n agentgateway-system deployment/agentgateway --tail=20
```

Verify the `agentgateway` `GatewayClass` is present and accepted:

```
kubectl get gatewayclass agentgateway
```

HTTPRoute not accepted

```
kubectl describe httproute ${GUIDE_NAME} -n ${NAMESPACE}
```

Verify that `parentRefs` matches the Gateway name and `backendRefs` matches the InferencePool name.

No response from Gateway IP

```
kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}'
```

If the address is empty, your Gateway may still be waiting for a LoadBalancer service. Check that your cluster supports external load balancers.

Envoy AI Gateway

This guide shows how to deploy llm-d with Envoy AI Gateway as your inference gateway. By the end, inference requests will flow from an Envoy-managed Gateway to your model servers via the llm-d EPP.

[!NOTE] This guide assumes familiarity with Gateway API and llm-d.

Prerequisites

1. The environment variables `${GUIDE_NAME}`, `${MODEL_NAME}` and `${NAMESPACE}` should be set as part of deploying one of the well-lit path guides.
2. A Kubernetes cluster running one of the three most recent Kubernetes releases (minimum Kubernetes 1.32)
3. Helm
4. jq

Step 1: Install Gateway API and Gateway API Inference Extension CRDs

Install the required CRDs by following the CRD installation guide.

Step 2: Install Envoy AI Gateway

Install Envoy Gateway with the AI Gateway integration values, token rate limiting add-on, and InferencePool add-on:

ENVOY_GATEWAY_VERSION=v1.8.1

Install the CRDs first, skipping the Gateway API CRDs installed in the previous step.

#

Note 1: We're using helm template piped into kubectl apply instead of helm install due to known Helm limitation related to large CRDs in the templates/ directory: <https://github.com/helm/helm/pull/12277>

Note 2: We filter the output of the Helm template to remove offending lines until this is fixed: <https://github.com/helm/helm/pull/32217>

```
helm template eg-crd oci://docker.io/envoyproxy/gateway-crd-helm \
--version ${ENVOY_GATEWAY_VERSION} \
--set crds.gatewayAPI.enabled=false \
--set crds.envoyGateway.enabled=true \
| grep -v '^Pulled:' | grep -v '^Digest:' | kubectl apply --server-side -f -
```

Install Envoy gateway

```
helm upgrade -i eg oci://docker.io/envoyproxy/gateway-helm \
--version ${ENVOY_GATEWAY_VERSION} \
--namespace envoy-gateway-system \
--create-namespace \
--skip-crds \
-f https://raw.githubusercontent.com/envoyproxy/ai-gateway/main/manifests/envoy-gateway-values.yaml \
-f https://raw.githubusercontent.com/envoyproxy/ai-gateway/main/examples/inference-pool/envoy-gateway-values-addon.yaml
```

Give permissions to Envoy Gateway to watch InferencePool resources

```
kubectl apply -f - <<EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: envoy-gateway-inference-access
rules:
- apiGroups:
  - inference.networking.k8s.io
  resources:
  - inferencepools
  verbs:
  - get
  - list
  - watch
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: envoy-gateway-inference-access
subjects:
- kind: ServiceAccount
  name: envoy-gateway
  namespace: envoy-gateway-system
roleRef:
  kind: ClusterRole
  name: envoy-gateway-inference-access
apiGroup: rbac.authorization.k8s.io
EOF
```

```
kubectl wait --timeout=2m -n envoy-gateway-system deployment/envoy-gateway --for=condition=Available
```

Install the Envoy AI Gateway CRDs and controller:

```
ENVOY_AI_GATEWAY_VERSION=v0.7.0
```

```
helm upgrade -i envoy-ai-gateway-crd oci://docker.io/envoyproxy/ai-gateway-crds-helm \
--version ${ENVOY_AI_GATEWAY_VERSION} \
--namespace envoy-ai-gateway-system \
--create-namespace
```

```
helm upgrade -i envoy-ai-gateway oci://docker.io/envoyproxy/ai-gateway-helm \
--version ${ENVOY_AI_GATEWAY_VERSION} \
--namespace envoy-ai-gateway-system \
--create-namespace
```

```
kubectl wait --timeout=2m -n envoy-ai-gateway-system deployment/ai-gateway-controller --for=condition=Available
```

Create the GatewayClass

```
kubectl apply -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: envoy-ai-gateway
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller
EOF
```

Verify the installation:

```
kubectl get pods -n envoy-gateway-system
kubectl get pods -n envoy-ai-gateway-system
kubectl get gatewayclass envoy-ai-gateway
```

Expected output:

NAME	CONTROLLER	ACCEPTED	AGE
envoy-ai-gateway	gateway.envoyproxy.io/gatewayclass-controller	True	30s

Step 3: Deploy the Gateway

Set the llm-d version to match your deployment:

```
LLM_D_VERSION=main # Use 'main' for latest, or a release tag like 'v0.7.0'
```

This deploys a gateway suitable for `envoy-ai-gateway`, using the `envoy-ai-gateway` gateway class. This is the preferred self-installed inference gateway recipe in llm-d.

```
kubectl apply -k "https://github.com/llm-d/llm-d/guides/recipes/gateway/envoy-ai-gateway?ref=${LLM_D_VERSION}" -n
${NAMESPACE}
```

Verify the Gateway is programmed:

```
kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE}
```

Expected output:

NAME	CLASS	ADDRESS	PROGRAMMED	AGE
llm-d-inference-gateway	envoy-ai-gateway	10.xx.xx.xx	True	30s

Wait until `PROGRAMMED` shows `True` before proceeding.

Step 4: Send a Request

[!IMPORTANT] Before sending requests, you must deploy a well-lit path guide. This sets up a model server deployment, an `InferencePool`, and an `HTTPRoute` to connect the Gateway to the pool.

Get the `Gateway` external address:

```
export IP=$(kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}')
```

Send an inference request via the managed `Gateway` :

```
curl -X POST http://${IP}/v1/completions \
-H 'Content-Type: application/json' \
-H 'X-Gateway-Base-Model-Name: "${GUIDE_NAME}"' \
-d '{
  "model": "${MODEL_NAME}",
  "prompt": "How are you today?"
}' | jq
```

Cleanup

```
kubectl delete clienttrafficpolicy client-buffer-limit -n ${NAMESPACE}
kubectl delete gateway llm-d-inference-gateway -n ${NAMESPACE}
helm uninstall envoy-ai-gateway -n envoy-ai-gateway-system
helm uninstall envoy-ai-gateway-crd -n envoy-ai-gateway-system
kubectl delete namespace envoy-ai-gateway-system
helm uninstall eg -n envoy-gateway-system
kubectl delete namespace envoy-gateway-system
helm template eg-crds oci://docker.io/envoyproxy/gateway-crds-helm \
--version ${ENVOY_GATEWAY_VERSION} \
--set crds.gatewayAPI.enabled=false \
--set crds.envoyGateway.enabled=true \
| grep -v '^Pulled:' | grep -v '^Digest:' | kubectl delete -f -
kubectl delete gatewayclass envoy-ai-gateway
```

To uninstall the Gateway API and Gateway API Inference Extension CRDs, see the CRD installation guide.

Troubleshooting

Gateway not showing `PROGRAMMED=True`

```
kubectl describe gateway llm-d-inference-gateway -n ${NAMESPACE}
kubectl get pods -n envoy-gateway-system
kubectl logs -n envoy-gateway-system deployment/envoy-gateway --tail=20
```

Verify the `envoy-ai-gateway` `GatewayClass` is present and accepted:

```
kubectl get gatewayclass envoy-ai-gateway
```

Also check the Envoy AI Gateway controller is running:

```
kubectl get pods -n envoy-ai-gateway-system
kubectl logs -n envoy-ai-gateway-system deployment/ai-gateway-controller --tail=20
```

HTTPRoute not accepted

```
kubectl describe httproute ${GUIDE_NAME} -n ${NAMESPACE}
```

Verify that `parentRefs` matches the Gateway name and `backendRefs` matches the InferencePool name.

No response from Gateway IP

```
kubectl get gateway llm-d-inference-gateway -n ${NAMESPACE} -o jsonpath='{.status.addresses[0].value}'
```

If the address is empty, your Gateway may still be waiting for a LoadBalancer service. Check that your cluster supports external load balancers.

Installing Gateway API CRDs

Before deploying any Gateway provider, you must install the Gateway API and Gateway API Inference Extension CRDs.

[!NOTE] GKE automatically installs all GA CRDs for Gateway API and Gateway API Inference Extension on GKE versions 1.34.0-gke.1626000 or later. If using GKE with this version or newer, you can skip the installation steps below.

Option 1: Using the Installation Script

The quickest way to install the CRDs is using the provided script:

```
bash guides/recipes/gateway/install-gateway-crds.sh
```

This script installs both Gateway API and Gateway API Inference Extension CRDs.

Option 2: Manual Installation

If you prefer to install manually or need to customize versions:

```
GATEWAY_API_VERSION=v1.5.1
GAIE_VERSION=v1.5.0
GATEWAY_API_INSTALL_URL=https://github.com/kubernetes-sigs/gateway-
api/releases/download/${GATEWAY_API_VERSION}/standard-install.yaml
GAIE_INSTALL_URL=https://github.com/kubernetes-sigs/gateway-api-inference-extension/releases/download/${GAIE_VERSION}/v1-
manifests.yaml

kubectl apply -f "${GATEWAY_API_INSTALL_URL}"
kubectl apply -f "${GAIE_INSTALL_URL}"
```

Verification

Verify the APIs are available:

```
kubectl api-resources --api-group=gateway.networking.k8s.io
kubectl api-resources --api-group=inference.networking.k8s.io
```

You should see resources like `gateways`, `httproutes`, and `inferencepools`.

Uninstalling Gateway API CRDs

Option 1: Using the Installation Script

The quickest way to uninstall the CRDs is using the provided script:

```
bash guides/recipes/gateway/install-gateway-crds.sh delete
```

Option 2: Manual Uninstall

If you prefer to uninstall manually:

```
GATEWAY_API_VERSION=v1.5.1
```

```
GAIE_VERSION=v1.5.0
```

```
kubectl delete -f https://github.com/kubernetes-sigs/gateway-api/releases/download/${GATEWAY_API_VERSION}/standard-install.yaml
```

```
kubectl delete -f https://github.com/kubernetes-sigs/gateway-api-inference-extension/releases/download/${GAIE_VERSION}/v1-manifests.yaml
```

API Reference

Core Kubernetes APIs

The following Kubernetes APIs are defined in the `inference.networking.k8s.io` (v1) and `llm-d.ai` (v1alpha2) groups.

Resource	API Group	Version	Description
InferencePool	<code>inference.networking.k8s.io</code>	<code>v1</code>	Defines a pool of inference endpoints (model servers) to configure the Endpoint Picker (EPP) and Gateways for inference-optimized routing.
InferenceObjective	<code>llm-d.ai</code>	<code>v1alpha2</code>	Defines performance goals (priority, latency) for specific model workloads within a pool.
InferenceModelRewrite	<code>llm-d.ai</code>	<code>v1alpha2</code>	Specifies rules for rewriting model names in request bodies, enabling traffic splitting and canary rollouts.

Component Configuration

These schemas define the internal configuration for project components and are typically provided via ConfigMaps or local files, rather than as standalone Kubernetes objects.

Schema	API Group	Version	Description
EndpointPickerConfig	<code>llm-d.ai</code>	<code>v1alpha1</code>	Defines the internal configuration for the Endpoint Picker (EPP) , including plugins and request scheduling profiles.

Recognized HTTP Headers

- EPP HTTP Headers Reference: The EPP inspects specific HTTP headers to manage flow control and observability for inference requests.

Supported Request APIs

- EPP HTTP APIs Reference: HTTP APIs such as OpenAI's Chat, Anthropic's Message and vLLM's Generate APIs.
- EPP gRPC APIs Reference: gRPC APIs such as vLLM's gRPC Generate API.

See Also

- Glossary: Definitions of key terms and concepts used across this project.

InferencePool

`InferencePool` is the Schema for the `InferencePools` API. It defines a pool of inference endpoints that can be used by a Gateway.

Group: `inference.networking.k8s.io` **Version:** `v1`

InferencePool

Field	Description
<code>apiVersion</code>	<code>inference.networking.k8s.io/v1</code>
<code>kind</code>	<code>InferencePool</code>
<code>metadata</code>	<code>metav1.ObjectMeta</code>
<code>spec</code>	<code>InferencePoolSpec</code> Required Spec defines the desired state of the InferencePool.
<code>status</code>	<code>InferencePoolStatus</code> Status defines the observed state of the InferencePool.

InferencePoolSpec

`InferencePoolSpec` defines the desired state of the InferencePool.

Field	Description
selector	LabelSelector Required Selector determines which Pods are members of this inference pool. It matches Pods by their labels only within the same namespace; cross-namespace selection is not supported. The structure is intentionally simple to be compatible with Kubernetes Service selectors.
targetPorts	[]Port Required TargetPorts defines a list of ports that are exposed by this InferencePool. Every port will be treated as a distinctive endpoint by EPP, addressable as a podIP:portNumber combination. Max items: 8. Port numbers must be unique.
appProtocol	AppProtocol AppProtocol describes the application protocol for all the target ports. If unspecified, the protocol defaults to http (HTTP/1.1).
endpointPickerRef	EndpointPickerRef Required EndpointPickerRef is a reference to the Endpoint Picker extension and its associated configuration.

InferencePoolStatus

InferencePoolStatus defines the observed state of the InferencePool.

Field	Description
parents	[]ParentStatus Parents is a list of parent resources, typically Gateways, that are associated with the InferencePool, and the status of the InferencePool with respect to each parent. Max items: 32.

Port

Port defines the network port that will be exposed by this InferencePool.

Field	Description
number	int32 Required Number defines the port number to access the selected model server Pods. Must be in range 1 to 65535.

AppProtocol

AppProtocol describes the application protocol for a port.

Supported values:

- http : HTTP/1.1. This is the default.
- kubernetes.io/h2c : HTTP/2 over cleartext. Typically used for gRPC workloads where TLS is terminated at the Gateway.

EndpointPickerRef

EndpointPickerRef specifies a reference to an Endpoint Picker extension and its associated configuration.

Field	Description
group	string Group of the referent API object. Defaults to "" (Core API group).
kind	string Kind of the referent. Defaults to Service . Implementations MUST NOT support ExternalName Services.
name	string Required Name of the referent API object.
port	Port Port of the Endpoint Picker extension service. Required when kind is Service .
failureMode	string Configures how the parent handles cases when the Endpoint Picker extension is non-responsive. Defaults to FailClose . Supported values: FailOpen , FailClose .

ParentStatus

ParentStatus defines the observed state of InferencePool from a Parent, i.e. Gateway.

Field	Description
conditions	[]metav1.Condition Conditions provide information about the observed state. Supported types: Accepted , ResolvedRefs .
parentRef	ParentReference Required Identifies the parent resource this status is associated with.
controllerName	string Name of the controller that wrote this status (e.g., example.net/gateway-controller).

ParentReference

ParentReference identifies an API object, such as a Gateway.

Field	Description
group	string Group of the referent. Defaults to gateway.networking.k8s.io .
kind	string Kind of the referent. Defaults to Gateway .
name	string Required Name of the referent.
namespace	string Namespace of the referenced object. Defaults to the local namespace.

LabelSelector

LabelSelector defines a query for resources based on their labels.

Field	Description
matchLabels	map[string]string Required A set of {key,value} pairs. An object must match every label in this map (AND operation). Max properties: 64.

Condition Types and Reasons

Accepted

Indicates whether the InferencePool has been accepted or rejected by a Parent.

- **True Reasons:**
 - Accepted : Supported by parent.
- **False Reasons:**
 - NotSupportedByParent : Parent does not support InferencePool as a backend.
 - HTTPRouteNotAccepted : Referenced by an HTTPRoute that has been rejected.
- **Unknown Reasons:**
 - Pending

ResolvedRefs

Indicates whether the controller was able to resolve all object references.

- **True Reasons:**

- ResolvedRefs

- **False Reasons:**

- InvalidExtensionRef : Extension is invalid (unsupported kind/group or not found).

Exported

Indicates whether the controller was able to export the InferencePool to specified clusters.

- **True Reasons:**

- Exported

- **False Reasons:**

- NotRequested : No export was requested.
- NotSupported : Export requested but not supported by implementation.

InferenceObjective

`InferenceObjective` represents the desired state of a specific model use case. It allows "Inference Workload Owners" to define performance and latency goals for a model within an `InferencePool`.

Group: llm-d.ai **Version:** v1alpha2

InferenceObjective

Field	Description
<code>apiVersion</code>	llm-d.ai/v1alpha2
<code>kind</code>	<code>InferenceObjective</code>
<code>metadata</code>	<code>metav1.ObjectMeta</code>
<code>spec</code>	<code>InferenceObjectiveSpec</code> Spec represents the desired state of the model use case.
<code>status</code>	<code>InferenceObjectiveStatus</code> Status defines the observed state of the <code>InferenceObjective</code> .

InferenceObjectiveSpec

`InferenceObjectiveSpec` defines the priority and the pool reference for the model workload.

Field	Description
<code>priority</code>	<code>int</code> Optional Defines how important it is to serve the request compared to others in the same pool. Higher values have higher priority. Unset value is treated as <code>0</code> . Requests of higher priority are served first when resources are scarce.
<code>poolRef</code>	<code>PoolObjectReference</code> Required Reference to the inference pool. The pool must exist in the same namespace.

InferenceObjectiveStatus

InferenceObjectiveStatus defines the observed state of InferenceObjective.

Field	Description
conditions	[]metav1.Condition Conditions track the state of the InferenceObjective. Known type: Accepted .

PoolObjectReference

PoolObjectReference identifies an API object within the same namespace.

Field	Description
group	string Group of the referent. Defaults to inference.networking.k8s.io .
kind	string Kind of the referent. Defaults to InferencePool .
name	string Required Name of the referent.

Condition Types and Reasons

Accepted

Indicates if the objective configuration is accepted.

- **True Reasons:**
 - Accepted : Model conforms to the state of the pool.
- **Unknown Reasons:**
 - Pending : Initial state, controller has not yet reconciled the resource.

InferenceModelRewrite

`InferenceModelRewrite` defines rules for rewriting inference requests, such as traffic splitting, A/B tests, or canary rollouts across weighted model targets.

Group: `llm-d.ai` **Version:** `v1alpha2`

InferenceModelRewrite

Field	Description
<code>apiVersion</code>	<code>llm-d.ai/v1alpha2</code>
<code>kind</code>	<code>InferenceModelRewrite</code>
<code>metadata</code>	<code>metav1.ObjectMeta</code>
<code>spec</code>	<code>InferenceModelRewriteSpec</code> Spec defines the desired state of the rewrite rules.
<code>status</code>	<code>InferenceModelRewriteStatus</code> Status defines the observed state of the resource.

InferenceModelRewriteSpec

Field	Description
<code>poolRef</code>	<code>PoolObjectReference</code> Required Reference to the target <code>InferencePool</code> .
<code>rules</code>	<code>[]InferenceModelRewriteRule</code> Required Ordered set of rules. The first rule to match a request is used.

InferenceModelRewriteRule

`InferenceModelRewriteRule` defines the match criteria and corresponding actions (targets).

Field	Description
matches	[] Match Optional Criteria for matching a request. Logical OR if multiple criteria are specified. If empty, matches all requests.
targets	[] TargetModel Optional How to distribute traffic across weighted model targets. Min items: 1.

Match

Match defines the criteria for matching LLM requests.

Field	Description
model	ModelMatch Required Criteria for matching the model field in the JSON request body.

ModelMatch

Field	Description
type	string Kind of string matching to use. Supported value: Exact . Defaults to Exact .
value	string Required The model name string to match against.

TargetModel

TargetModel defines a weighted model destination.

Field	Description
weight	int32 Optional Proportion of requests forwarded to the model. Computed as weight/(sum of all weights) . Min: 1, Max: 1000000. If set for one, must be set for all.
modelRewrite	string Required The static model name to rewrite the request to.

InferenceModelRewriteStatus

Field	Description
conditions	[]metav1.Condition Conditions track the state. Known type: Accepted .

PoolObjectReference

PoolObjectReference identifies an API object within the same namespace.

Field	Description
group	string Group of the referent. Defaults to inference.networking.k8s.io .
kind	string Kind of the referent. Defaults to InferencePool .
name	string Required Name of the referent.

Precedence and Conflict Resolution

- Model Match Precision:** Rules with an Exact model match take precedence over generic matches (empty matches).
- Resource Age:** If multiple resources target the same pool with identical matches, the oldest resource (by creation timestamp) takes precedence.
- Rule Order:** Within a single resource, the FIRST matching rule (in list order) is used.

Condition Types and Reasons

Accepted

Indicates if the rewrite is valid, non-conflicting, and applied to the pool.

- True Reasons:**
 - Accepted : Rewrite is valid and successfully applied.
- Unknown Reasons:**
 - Pending : Initial state, controller has not yet reconciled the resource.

EndpointPickerConfig (EPP Configuration)

`EndpointPickerConfig` defines the internal configuration for the **Endpoint Picker (EPP)**. Unlike Kubernetes resources (like `InferencePool`), this is a configuration schema used to initialize the EPP binary, typically provided via a `ConfigMap` or a local file.

Group: `llm-d.ai` **Version:** `v1alpha1`

EndpointPickerConfig

Field	Description
<code>featureGates</code>	<code>[]string</code> A set of flags to enable experimental features (e.g., <code>flowControl</code>).
<code>plugins</code>	<code>[]PluginSpec</code> Required List of plugins to be instantiated (e.g., scorers, adapters, reporters).
<code>schedulingProfiles</code>	<code>[]SchedulingProfile</code> Required Named profiles that group plugins into routing slots.
<code>dataLayer</code>	<code>DataLayerConfig</code> Configures the <code>DataLayer</code> for metadata extraction and processing.
<code>flowControl</code>	<code>FlowControlConfig</code> Configures global and per-priority admission control. Only respected if the <code>flowControl</code> feature gate is enabled.
<code>requestHandler</code>	<code>RequestHandlerConfig</code> Specifies the handling logic used by the EPP to process incoming requests.

PluginSpec

Defines a plugin instance and its parameters.

Field	Description
name	string Unique name for this plugin instance. If omitted, type is used.
type	string Required The plugin type to instantiate (e.g., least-request , openai-parser).
parameters	json.RawMessage Arbitrary parameters passed to the plugin's factory function.

SchedulingProfile

Groups plugins to define specific routing behavior.

Field	Description
name	string Required Name of the profile.
plugins	[]SchedulingPlugin Required List of plugins associated with this profile.

SchedulingPlugin

Field	Description
pluginRef	string Required Reference to a named plugin in the top-level plugins list.
weight	float64 Weight used if the plugin is a Scorer.

FlowControlConfig

Configures admission control and queuing.

Field	Description
maxBytes	resource.Quantity Global maximum aggregate byte size of all active requests.
maxRequests	resource.Quantity Global maximum number of concurrent requests.
defaultRequestTTL	duration Fallback timeout for queued requests.
defaultPriorityBand	PriorityBandConfig Template for priority levels not explicitly configured.
priorityBands	[]PriorityBandConfig Explicit policies for specific priority levels.
usageLimitPolicyPluginRef	string Reference to a UsageLimitPolicy plugin for adaptive capacity management.
saturationDetector	SaturationDetectorConfig Specifies which saturation detector plugin to use. Defaults to utilization-detector .

PriorityBandConfig

Field	Description
priority	int Integer priority level. Higher is more critical.
maxBytes	resource.Quantity Max bytes allowed for this priority band.
maxRequests	resource.Quantity Max concurrent requests allowed for this band.
fairnessPolicyRef	string Policy governing flow selection (default: global-strict-fairness-policy).
orderingPolicyRef	string Policy governing request selection within a flow (default: fcfs-ordering-policy).

RequestHandlerConfig

Configures request handling behavior.

Field	Description
<code>parsers</code>	<code>[]ParserConfig</code> List of parsing plugins used to process protocol messages. If unspecified, <code>openai-parser</code> , <code>anthropic-parser</code> , and <code>vllmhttp-parser</code> are configured by default.

DataLayerConfig

Field	Description
<code>sources</code>	<code>[]DataLayerSource</code> Required List of metadata sources.

DataLayerSource

Field	Description
<code>pluginRef</code>	<code>string</code> Required Reference to a plugin providing the data source.
<code>extractors</code>	<code>[]DataLayerExtractor</code> Required Plugins that extract specific attributes from the source.

SaturationDetectorConfig

Field	Description
<code>pluginRef</code>	<code>string</code> Reference to a plugin instance for saturation detection.

ParserConfig

Field	Description
<code>pluginRef</code>	<code>string</code> Required Reference to a parser plugin instance.

EPP HTTP APIs Reference

This document lists the HTTP APIs the Endpoint Picker (EPP) supports for inference traffic. Depending on the API, the EPP may parse fields from the request body to do prefix-cache aware routing, and plugin decisions.

Supported HTTP APIs

Endpoint	Source	Supported
/v1/completions	OpenAI Completions API	☒
/v1/chat/completions	OpenAI Chat Completions API	☒
/v1/responses	OpenAI Responses API	☒
/v1/embeddings	OpenAI Embeddings API	☒
/v1/messages	Anthropic Messages API	☒
/inference/v1/generate	vLLM Generate API	☒

Request Examples

The examples below parameterize the model as `${MODEL_NAME}` and the proxy endpoint as `${IP}`. Set `${MODEL_NAME}` to `Qwen/Qwen3-VL-32B-Instruct` from the multimodal aggregation guide, and set `${IP}` to the proxy endpoint IP retrieved per that guide's verification steps.

```
export MODEL_NAME=Qwen/Qwen3-VL-32B-Instruct
```

The `/v1/embeddings` section overrides `${MODEL_NAME}` since chat/instruct models do not expose that route.

OpenAI `/v1/completions`

Request:

```
curl -X POST http://${IP}/v1/completions \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "prompt": "Hello",
  "max_tokens": 10
}' | jq
```

Response:

```
{
  "id": "cmpl-abc123",
  "object": "text_completion",
  "created": 1781036021,
  "model": "Qwen/Qwen3-VL-32B-Instruct",
  "choices": [
    {
      "index": 0,
      "text": "! I am trying to write a story, and",
      "logprobs": null,
      "finish_reason": "length",
      "stop_reason": null
    }
  ],
  "system_fingerprint": "vllm-0.21.0-tp2-5054d0df",
  "usage": {
    "prompt_tokens": 1,
    "total_tokens": 11,
    "completion_tokens": 10
  }
}
```

Streaming request (set `stream: true` ; the response is server-sent events, so drop `jq` and use `curl -N` to flush chunks as they arrive):

```
curl -N -X POST http://${IP}/v1/completions \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "prompt": "Hello",
  "max_tokens": 10,
  "stream": true
}'
```

OpenAI /v1/chat/completions

Request:

```
curl -X POST http://${IP}/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "Describe this image."},
        {"type": "image_url", "image_url": {"url": "https://picsum.photos/640/360"}}
      ]
    }
  ],
  "max_tokens": 10
}' | jq
```

Streaming request:

```
curl -N -X POST http://${IP}/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "Describe this image."},
        {"type": "image_url", "image_url": {"url": "https://picsum.photos/640/360"}}
      ]
    }
  ],
  "max_tokens": 10,
  "stream": true
}'
```

► Streaming response (SSE)

OpenAI `/v1/responses`

Request:

```
curl -X POST http://${IP}/v1/responses \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "input": "Hello",
  "max_output_tokens": 10
}' | jq
```

Response:

```
{
  "id": "resp_abc127",
  "created_at": 1781036107,
  "incomplete_details": {"reason": "max_output_tokens"},
  "model": "Qwen/Qwen3-VL-32B-Instruct",
  "object": "response",
  "output": [
    {
      "id": "msg_abc128",
      "type": "message",
      "role": "assistant",
      "status": "completed",
      "content": [
        {
          "type": "output_text",
          "text": "Hello! How can I help you today?",
          "annotations": []
        }
      ]
    }
  ],
  "status": "incomplete",
  "max_output_tokens": 10,
  "usage": {
    "input_tokens": 9,
    "output_tokens": 10,
    "total_tokens": 19
  }
}
```

OpenAI /v1/embeddings

This endpoint requires an embedding model deployment (for example `Qwen/Qwen3-Embedding-0.6B`). Chat/instruct models do not expose this route.

Request:

```
export MODEL_NAME=Qwen/Qwen3-Embedding-0.6B
curl -X POST http://${IP}/v1/embeddings \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "input": "Hello"
}' | jq
```

Response (embedding vector truncated for readability):

```
{
  "model": "Qwen/Qwen3-Embedding-0.6B",
  "object": "list",
  "data": [
    {
      "index": 0,
      "object": "embedding",
      "embedding": [-0.01350, -0.02152, -0.01368, -0.03032, 0.00941, "..."]
    }
  ],
  "usage": {
    "prompt_tokens": 2,
    "total_tokens": 2,
    "completion_tokens": 0
  }
}
```

Anthropic `/v1/messages`

Request:

```
curl -X POST http://${IP}/v1/messages \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "Describe this image."},
        {"type": "image", "source": {"type": "url", "url": "https://picsum.photos/640/360"}}
      ]
    }
  ],
  "max_tokens": 10
}' | jq
```

Response:

```
{
  "id": "chatcmpl-abc125",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "This image is a close-up, shallow-focus photograph"
    }
  ],
  "model": "Qwen/Qwen3-VL-32B-Instruct",
  "stop_reason": "max_tokens",
  "usage": {
    "input_tokens": 234,
    "output_tokens": 10
  }
}
```

Streaming request:

```
curl -N -X POST http://${IP}/v1/messages \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "Describe this image."},
        {"type": "image", "source": {"type": "url", "url": "https://picsum.photos/640/360"}}
      ]
    }
  ],
  "max_tokens": 10,
  "stream": true
}'
```

► Streaming response (SSE)

vLLM /inference/v1/generate

This endpoint requires the model server to be vLLM. Sampling controls must be nested inside a `sampling_params` object rather than placed at the top level.

Request:

```
curl -X POST http://${IP}/inference/v1/generate \
-H 'Content-Type: application/json' \
-d '{
  "model": "${MODEL_NAME}",
  "token_ids": [9906],
  "sampling_params": {"max_tokens": 10}
}' | jq
```

Response:

```
{
  "request_id": "abc129",
  "choices": [
    {
      "index": 0,
      "logprobs": null,
      "finish_reason": "length",
      "token_ids": [17993, 1894, 7332, 198, 286, 2415, 1140, 259, 4580, 892]
    }
  ]
}
```

EPP gRPC APIs Reference

This document lists the gRPC APIs the Endpoint Picker (EPP) supports for inference traffic. gRPC requests flow through the gateway as HTTP/2 (H2C) traffic, and the EPP decodes the gRPC frames and protobuf payloads to do prefix-cache aware routing, plugin decisions, and response usage tracking.

Unlike the HTTP APIs, gRPC parsing is not enabled by default: the matching parser plugin must be configured in the `EndpointPickerConfig`.

Supported gRPC APIs

gRPC Method	Source	Parser Plugin	Supported
<code>vllm.grpc.engine.VllmEngine/Generate</code>	vLLM gRPC engine API	<code>vllmgrpc-parser</code>	☒
<code>vllm.grpc.engine.VllmEngine/Embed</code>	vLLM gRPC engine API	<code>vllmgrpc-parser</code>	☒

The gRPC API is currently token-out only for `Generate`: responses carry token IDs (`chunk.token_ids` , `complete.output_ids`) rather than decoded text, and clients are responsible for detokenization.

Parser Configuration

Parsers are configured via the `requestHandler.parsers` section of the `EndpointPickerConfig`. Instantiate the parser plugin in `plugins` , then reference it by name:

```
apiVersion: llm-d.ai/v1alpha1
kind: EndpointPickerConfig
plugins:
- name: maxScore
  type: max-score-picker
- name: vllmgrpcParser
  type: vllmgrpc-parser
schedulingProfiles:
# ... omitted for brevity ...
requestHandler:
  parsers:
  - pluginRef: vllmgrpcParser
```

InferencePool Configuration

gRPC requires HTTP/2 end to end. For the gateway to connect to the model server pods with HTTP/2 cleartext (h2c), the `InferencePool` must set `appProtocol: kubernetes.io/h2c` .


```
apiVersion: inference.networking.k8s.io/v1
kind: InferencePool
metadata:
  name: vllm-grpc-qwen3-32b
spec:
  targetPorts:
    - number: 8000
  appProtocol: kubernetes.io/h2c
  selector:
    matchLabels:
      app: vllm-grpc-qwen3-32b
  endpointPickerRef:
    name: vllm-grpc-qwen3-32b-epp
    port:
      number: 9002
```

When deploying with the llm-d-router Helm charts, setting `router.modelServers.protocol=grpc` configures this automatically.

Request Examples

The examples below use `grpcurl` with the proxy endpoint as `${IP}`, set per the relevant guide's verification steps. They require the `vllm_engine.proto` definition, and a model server that exposes the vLLM gRPC engine API.

vLLM `VllmEngine/Generate`

Request (text input; alternatively pass pre-tokenized input via the `tokenized` field):

```
grpcurl -plaintext -proto vllm_engine.proto \
-d '{
  "request_id": "req-1",
  "text": "Hello",
  "sampling_params": {"max_tokens": 10}
}' \
${IP}:80 vllm.grpc.engine.VllmEngine/Generate
```

Response:

```
{
  "complete": {
    "outputIds": [17993, 1894, 7332, 198, 286, 2415, 1140, 259, 4580, 892],
    "finishReason": "length",
    "promptTokens": 1,
    "completionTokens": 10
  }
}
```

Streaming request (set `"stream": true`; the server returns a stream of `GenerateResponse` messages with incremental chunk payloads followed by a final `complete` payload):

```
grpcurl -plaintext -proto vllm_engine.proto \
-d '{
  "request_id": "req-2",
  "text": "Hello",
  "sampling_params": {"max_tokens": 10},
  "stream": true
}' \
${IP}:80 vllm.grpc.engine.VllmEngine/Generate
```

► Streaming response

vLLM `vllmEngine/Embed`

This method requires pre-tokenized input and an embedding model deployment.

Request:

```
grpcurl -plaintext -proto vllm_engine.proto \
-d '{
  "request_id": "req-3",
  "tokenized": {"original_text": "Hello", "input_ids": [9906]}
}' \
${IP}:80 vllm.grpc.engine.VllmEngine/Embed
```

Response (embedding vector truncated for readability):

```
{
  "embedding": [-0.01350, -0.02152, -0.01368, "..."],
  "promptTokens": 1,
  "embeddingDim": 1024
}
```

HTTP Headers

The EPP HTTP headers (request classification, flow control, and SLO headers such as `x-llm-d-inference-objective` and `x-llm-d-inference-fairness-id`) work for gRPC requests exactly as they do for HTTP.

Specify them as gRPC metadata on the call. With `grpcurl`, use `-H`:

```
grpcurl -plaintext -proto vllm_engine.proto \
-H 'x-llm-d-inference-objective: my-objective' \
-H 'x-llm-d-inference-fairness-id: tenant-a' \
-d '{
  "request_id": "req-4",
  "text": "Hello",
  "sampling_params": {"max_tokens": 10}
}' \
${IP}:80 vllm.grpc.engine.VllmEngine/Generate
```

In a Go client, attach the metadata to the outgoing context:

```
ctx = metadata.AppendToOutgoingContext(ctx,  
    "x-llm-d-inference-objective", "my-objective",  
    "x-llm-d-inference-fairness-id", "tenant-a")  
resp, err := client.Generate(ctx, req)
```

In Python, pass `metadata` on the call:

```
stub.Generate(request, metadata=(  
    ("x-llm-d-inference-objective", "my-objective"),  
    ("x-llm-d-inference-fairness-id", "tenant-a"),  
))
```

EPP HTTP Headers Reference

This document describes the HTTP headers that the Endpoint Picker (EPP) inspects to manage and control inference requests, specifically for flow control, performance management, and request classification.

Request Classification and Flow Control

These headers allow the EPP to identify the request's goals, group them for fair resource allocation, and handle model-specific targeting.

Header	Description
x-llm-d-inference-objective	Specifies the name of the <code>InferenceObjective</code> resource associated with the request. The EPP uses this to look up the corresponding objective resource in the same namespace as the <code>InferencePool</code> to apply the defined priority and performance goals.
x-llm-d-inference-fairness-id	Provides a unique identifier for grouping requests for fairness-based flow control. Requests with the same ID share capacity according to the fairness policy. If omitted, the EPP defaults to <code>default-flow</code> .
x-llm-d-model-name-rewrite	Specifies the target model name to be used for the request. This is an alternative approach to model name rewriting; while the <code>InferenceModelRewrite</code> API provides rule-based rewriting on the server side, this header allows for an explicit, per-request override. When present, the EPP uses this value to override the model name in the request body and for recording model-specific metrics.

Service Level Objectives (SLOs)

These headers are used by admission control and load balancing plugins to make decisions based on latency targets.

Header	Description
x-llm-d-slo-ttft-ms	Specifies the target Time To First Token (TTFT) in milliseconds. Used by plugins to determine if a request can be admitted while meeting the latency goal.
x-llm-d-slo-tpot-ms	Specifies the target Time Per Output Token (TPOT) in milliseconds. Used for admission control based on predicted or observed token generation latency.

Deprecated Aliases

llm-d Router v0.9.0 and later accept the previous EPP-managed header names as deprecated read aliases. New integrations and examples should use the canonical `x-llm-d-*` names. Earlier releases expect the previous names, so

mixed-version fleets should coordinate the header migration during rolling upgrades. See the [llm-d Router releases](#) page for available versions.

If a request sends both a canonical header and its deprecated alias, the canonical `x-llm-d-*` value wins deterministically. No alias removal release or date has been announced; treat aliases as temporary compatibility support and prefer the canonical names for all new clients.

Deprecated alias	Canonical header
x-gateway-inference-fairness-id	x-llm-d-inference-fairness-id
x-gateway-inference-objective	x-llm-d-inference-objective
x-gateway-model-name-rewrite	x-llm-d-model-name-rewrite
x-slo-ttft-ms	x-llm-d-slo-ttft-ms
x-slo-tpot-ms	x-llm-d-slo-tpot-ms

This rename applies only to llm-d/EPP-managed user and control headers. Gateway API Inference Extension (GAIE) Endpoint Picker Protocol headers, such as `x-gateway-destination-endpoint*`, are unchanged.

Response Headers

These headers are set by the EPP on responses sent back to the client.

Header	Description
x-llm-d-request-dropped-reason	Indicates why a request was dropped by flow control. Only present on 429 responses generated by the EPP (not forwarded from a model server). The value uses a two-prefix scheme: <code>rejected-*</code> means the request never reached an inference server, <code>evicted-*</code> means it was dispatched and then killed. See the table below for possible values.

Dropped Reason Values

Value	Meaning
rejected-saturated	System at capacity, request rejected before queueing.
rejected-ttl-expired	Request entered the queue but its TTL expired before dispatch.
rejected-context-cancelled	Client disconnected while the request was queued.
evicted	Generic post-dispatch eviction.
evicted-queue-pressure	Evicted after dispatch due to backpressure from new arrivals.
evicted-priority	Evicted after dispatch, preempted by a higher-priority request.

The two-prefix convention helps consumers decide retry strategy: `rejected-*` means no inference work was done (cheap to retry), `evicted-*` means GPU cycles were consumed (factor into backoff).

Implementation Notes

- **Case Sensitivity:** All header lookups are case-insensitive.
- **Source:** These values are typically provided as standard HTTP headers in the incoming request.

Glossary

Quick-reference definitions for terms used throughout the llm-d documentation. For a high-level overview of how these pieces fit together, see the [Architecture Overview](#).

Aggregated Serving — The default serving mode where a single Model Server handles both Prefill and Decode for each request, as opposed to Disaggregated Serving.

Consultant — An optional sidecar component that the EPP queries for additional scoring signals beyond built-in metrics. Examples include the Latency Predictor and the KV-Cache Indexer.

Data Parallelism (DP) — Running independent model replicas on separate GPUs within the same node, each handling different requests for higher aggregate throughput. See [Model Servers](#).

Decode — The second phase of LLM inference that generates output tokens one at a time, each depending on the previous token's KV Cache state. Decode throughput is measured by TPOT. See [Architecture Overview](#).

Disaggregated Serving — A deployment pattern that separates Prefill and Decode into dedicated, independently scalable pools of Model Servers, connected by NIXL for KV-cache transfer. See [Disaggregation](#).

Envoy — A high-performance L7 GAIE-conformant proxy that can be used with llm-d as a data-plane Proxy. It communicates routing decisions with the EPP via ext-proc. See [Proxy](#).

Expert Parallelism (EP) — Distributing the expert layers of MoE models across multiple GPUs, enabling large models like DeepSeek-R1 to be served across nodes. See [Model Servers](#).

ext-proc (External Processing) — An Envoy filter protocol that offloads per-request routing decisions to an external gRPC service — in llm-d, the EPP. This is the communication channel between the Proxy and the routing logic. See [EPP](#).

Flow Control — The EPP subsystem that manages admission, queuing, and dispatch of requests using a Priority, Fairness, and Ordering hierarchy to prevent backend overload. See [EPP](#).

Gateway API — The Kubernetes-native API for configuring L7 traffic routing, succeeding Ingress. llm-d uses Gateway API resources (`HTTPRoute` , `Gateway`) to route external traffic to InferencePools. See [Proxy](#).

Gateway API Inference Extension (GAIE) — An extension to Gateway API that adds inference-aware routing via ext-proc. Defines the InferencePool CRD. See [Proxy](#).

InferencePool — A Kubernetes custom resource, defined by the Gateway API Inference Extension, that represents the set of Model Server pods an EPP considers when routing a request. See [InferencePool](#).

KV Cache — Key-value tensor cache storing intermediate attention states during LLM inference. Reusing cached entries for shared prompt prefixes (Prefix Caching) avoids redundant computation and reduces latency. See [Architecture Overview](#).

KV-Cache Indexer — A Consultant component that maintains a globally consistent view of KV-cache block distribution across Model Servers using KV-Events, enabling precise prefix cache-aware routing. See KV-Cache Indexer.

KV-Events — Events emitted by Model Servers (via ZeroMQ) when KV-cache blocks are created or evicted. Consumed by the KV-Cache Indexer for real-time cache state tracking.

Latency Predictor — A Consultant that uses XGBoost quantile regression models trained on live traffic to predict per-endpoint TTFT and TPOT, enabling SLO-aware routing. See Latency Predictor.

llm-d — A distributed inference serving stack that adds intelligent routing, KV Cache-aware routing, Disaggregated Serving, and autoscaling on top of existing Model Servers. See Introduction.

llm-d Async Processor — A lightweight dispatch agent that pulls individual inference requests from message queues (such as Redis and Google Pub/Sub) and sends them to the llm-d Router. It adjusts the dispatch rate based on system metrics to protect interactive traffic. See Batch Inference.

llm-d Batch Gateway — An OpenAI-compatible API server (`/v1/batches` , `/v1/files`) for submitting, tracking, and managing batch inference jobs. It coordinates with the llm-d Async Processor for throttled request dispatch. See Batch Inference.

llm-d Endpoint Picker (EPP) — The central routing component of llm-d. Receives ext-proc callbacks from the Proxy, evaluates candidate Model Servers through a Plugin Pipeline of filters, scorers, and pickers, and returns the address of the optimal backend. See EPP.

llm-d Router — The intelligent entry point for inference requests. It provides LLM-aware load balancing (e.g., prefix-cache and load-aware routing) and request queuing, and manages disaggregated serving. It is composed of two functional parts: a data-plane **Proxy** (e.g., Envoy) and the **Endpoint Picker (EPP)**. See Architecture Overview.

llm-d Well-Lit Path — A pre-validated, end-to-end deployment recipe (model + hardware + Helm values + benchmarks) that the llm-d community tests and supports as a first-class configuration. See Introduction.

MoE (Mixture of Experts) — A model architecture where only a subset of "expert" sub-networks activate per token, enabling very large models (e.g., DeepSeek-R1) to run efficiently. llm-d supports MoE serving via Wide Expert Parallelism.

Model Server — The inference engine (e.g., vLLM, SGLang) that loads model weights, runs inference on hardware accelerators, and manages a local KV Cache. The EPP routes requests to the optimal server instance. See Model Servers.

NIXL — NVIDIA Inference Xfer Library for high-speed GPU-to-GPU KV-cache transfer over InfiniBand, RoCE, EFA, and TCP. Used between Prefill and Decode workers in Disaggregated Serving.

Plugin Pipeline — The modular Filter, Score, Pick architecture inside the EPP that evaluates and selects Model Server endpoints for each request. Filters narrow candidates, scorers rank them, and pickers make the final selection. See EPP.

Prefix Caching — A technique where the EPP routes requests to Model Servers that already hold matching KV Cache entries for the prompt prefix, eliminating redundant Prefill computation and reducing TTFT. See Architecture Overview.

Prefill — The first phase of LLM inference that processes all input tokens in parallel to populate the KV Cache. Prefill latency is the dominant component of TTFT. See Architecture Overview.

Proxy — The L7 data-plane component that accepts client requests and delegates routing decisions to the EPP via ext-proc. Can be deployed via Gateway API or in Standalone mode with a GAIE-conformant proxy (such as Envoy) running as a sidecar to the EPP. See [Proxy](#).

Saturation Detector — A safety mechanism in the EPP that evaluates whether the backend InferencePool is overloaded based on queue depth and KV-cache utilization, triggering Flow Control or request shedding.

SGLang — An open-source LLM serving engine that can be used as a Model Server backend in llm-d, providing RadixAttention-based Prefix Caching and disaggregation support. See [Model Servers](#).

Tensor Parallelism (TP) — Sharding model layers across multiple GPUs within a node to serve models that exceed single-GPU memory. See [Model Servers](#).

KV Cache Management — A comprehensive ecosystem for managing and reusing the KV cache across the inference pool. It includes **Prefix-Cache Aware Routing**, **KV-Cache Indexing**, and **KV Offloading** to scale effective cache capacity beyond hardware limits. See [KV Cache Management](#).

TPOT (Time Per Output Token) — The average latency to generate each subsequent token during Decode. A key metric for streaming response quality. See [Architecture Overview](#).

TTFT (Time To First Token) — The latency from request arrival to the first generated output token, dominated by Prefill time. Prefix Caching is the primary optimization for reducing TTFT. See [Architecture Overview](#).

vLLM — An open-source high-throughput LLM serving engine and the default Model Server in llm-d. Provides PagedAttention, continuous batching, Prefix Caching, and KV-Events for cache-aware routing. See [Model Servers](#).

Wide Expert Parallelism — A deployment pattern for large MoE models that combines Data Parallelism and Expert Parallelism across multiple nodes, maximizing KV-cache space for long-context serving. See [Introduction](#).

Workload Variant Autoscaler (WVA) — A multi-model, SLO-aware autoscaler that optimizes cost on heterogeneous hardware by measuring instance capacity, deriving load functions, and calculating the optimal mix of model variants. See [Architecture Overview](#).

Artifacts

This page lists the llm-d release artifacts and dependencies:

1. **CRDs** — the Kubernetes Custom Resource Definitions used by llm-d
2. **llm-d Router** — the Helm chart and container images for the routing layer
3. **Model Servers and Extensions** — the inference engine images and extensions for advanced functionality
4. **Well-Lit Path Guides** — deployment manifests and benchmark scripts for key user stories
5. **Gateway Recipes** — optional recipes for installing Gateways and integrating them with llm-d
6. **Async Processor** — the Helm chart and container image for asynchronous, queue-based request processing

[!IMPORTANT] llm-d follows a modular deployment pattern, enabling gradual feature adoption. Users seeking a single CRD-driven deployment pattern should consider KServe's `LLMInferenceService`.

1. GAIE CRDs

llm-d uses the APIs defined in the Gateway API Inference Extension (GAIE) project:

CRD	Purpose	Provider
InferencePool	Defines a pool of inference endpoints (model servers) and configures the EPP and proxy for LLM-aware routing.	kubernetes-sigs/gateway-api-inference-extension
InferenceObjective	Defines performance goals (priority, latency) for specific model workloads within a pool.	llm-d/llm-d-router
InferenceModelRewrite	Specifies rules for rewriting model names in request bodies, enabling traffic splitting and canary rollouts.	llm-d/llm-d-router

The versions for the GAIE CRDs are derived from the `env.sh` file. This makes upgrades easy as they are stored in a common location across guides.

```
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
source ${REPO_ROOT}/guides/env.sh
```

Manifests are published at `kubernetes-sigs/gateway-api-inference-extension/config/crd` and can be installed like:

```
# GAIE_URL is automatically calculated from GAIE_VERSION at ${REPO_ROOT}/guides/env.sh
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api-inference-extension/${GAIE_URL}/v1-manifests.yaml
```

2. llm-d Router

llm-d Router is deployed via Helm. We offer a chart both Standalone and Gateway Mode:

Chart	Version	OCI Registry	Description
Standalone Mode	v0	oci://ghcr.io/llm-d/charts/llm-d-router-standalone	Deploys an InferencePool and EPP with a standalone sidecar proxy (Envoy by default, agentgateway also supported via router.proxy.proxyType=agentgateway) in the EPP pod
Gateway Mode	v0	oci://ghcr.io/llm-d/charts/llm-d-router-gateway	Deploys an InferencePool and EPP for use with an existing Kubernetes Gateway (e.g. Istio, AgentGateway, Envoy AI Gateway, GKE)

The charts are currently published by the Gateway API Inference Extension (GAIE) project (see standalone mode source and gateway mode source). Each well-lit path guides provides values files on top of the chart defaults to enable the functionality implemented in EPP.

[!NOTE] In a future release, the Helm Charts will be published from the llm-d project rather than from GAIE.

Images

llm-d releases the core EPP image as well as additional sidecar images for advanced functionality:

Image	Description	Version
ghcr.io/llm-d/llm-d-router-endpoint-picker	Core EPP image	main
ghcr.io/llm-d/llm-d-router-disagg-sidecar	Optional sidecar for model servers, enabling KV cache transfer for P/D	v0.10.0
registry.k8s.io/gateway-api-inference-extension/latency-training-server	Optional sidecar for EPP, for predicted-latency model training	v1.5.0
registry.k8s.io/gateway-api-inference-extension/latency-prediction-server	Optional sidecar for EPP, for predicted-latency scheduling	v1.5.0

[!IMPORTANT] The EPP image was renamed from ghcr.io/llm-d/llm-d-inference-scheduler in llm-d v0.8.0. The old package has been removed, so deployments must update the image repository as well as the tag.

[!NOTE] In a future release, the latency server images will be released from the llm-d/llm-d-latency-predictor repo.

3. Model Servers and Extensions

The llm-d stack supports vLLM and SGLang.

[!IMPORTANT] llm-d validates each released guide against specific versions of each model server, but llm-d Router communicates with model servers over the OpenAI-compatible HTTP API and standard inference engine metrics, so any recent release should work.

Upstream Images

We recommend using the upstream images for most guides:

Engine	Image	Tag
vLLM	docker.io/vllm/vllm-openai	v0.26.0
vLLM Omni	docker.io/vllm/vllm-omni	v0.26.0
vLLM TPU	docker.io/vllm/vllm-tpu	v0.26.0
vLLM XPU	docker.io/vllm/vllm-openai-xpu	v0.26.0
vLLM ROCM	docker.io/vllm/vllm-openai-rocm	v0.26.0
vLLM ROCM Omni	docker.io/vllm/vllm-omni-rocm	v0.24.1
vLLM CPU	docker.io/vllm/vllm-openai-cpu	v0.26.0
SGLang	docker.io/lmsysorg/sglang	v0.5.16
TRTLLM	nvcr.io/nvidia/tensorrt-llm/release	1.3.0rc23

Custom Images

In addition to the upstream images, llm-d also builds and releases vLLM images with features not yet merged into vLLM upstream such as:

- GKE IB networking patches
- DeepEP patches for GB200 support
- RIXL support on AMD ROCm

Image	Tag	Accelerator	Base OS	Architectures
ghcr.io/llm-d/llm-d-cuda	v0.9.0	NVIDIA GPU	RHEL UBI9	amd64, arm64
ghcr.io/llm-d/llm-d-aws (deprecated)	v0.9.0	NVIDIA GPU + EFA	RHEL UBI9	amd64, arm64
ghcr.io/llm-d/llm-d-rocm	v0.9.0	AMD ROCm	RHEL UBI9	amd64
ghcr.io/llm-d/llm-d-xpu	v0.9.0	Intel XPU	Ubuntu 24.04	amd64
ghcr.io/llm-d/llm-d-xpu-sglang	v0.9.0	Intel XPU	Ubuntu 24.04	amd64
ghcr.io/llm-d/llm-d-cpu	v0.9.0	CPU	RHEL UBI9	amd64

4. Well-Lit Path Guides

Well-Lit Paths are tested, benchmarked deployment recipes that show off llm-d's key user stories. Each guide lives under `guides/<path>/` and contains:

- **EPP Configurations** - Helm values files with EPP configurations for usage with the charts for llm-d Router.
- **Model Server Manifests** - Kustomize manifests for model server with labels and flags needed for usage with llm-d Router.

[!IMPORTANT] For some guides, we provide cloud provider specific settings. This is especially important for guides requiring IB and RoCE networking, which is not yet standardized. Users can adapt the examples to other platforms as needed.

See the full list of guides for more details.

5. Gateways

llm-d Router supports optional integration with Kubernetes Gateways. These are the versions we test against for the `v0.10.0` release:

Dependency	Tested Versions	Notes
Gateway API CRDs	v1.5.x	Kubernetes SIG (required if using a Gateway)
Istio	1.29.x	Default gateway provider
AgentGateway	v1.4.x	Preferred for new deployments

Install instructions live under `guides/recipes/gateway/`.

6. Async Processor

The Async Processor is an optional component that pulls inference requests from a message queue, gates dispatch on pool capacity, and forwards them to llm-d Router. It is deployed via Helm — see the asynchronous processing guide and the operations guide.

Chart	Version	OCI Registry	Description
Async Processor	v0.10.0	oci://ghcr.io/llm-d/charts/llm-d-async	Deploys the async processor with its queue backend (GCP Pub/Sub or Redis), worker pools, and dispatch gates

Images

Image	Description	Version
ghcr.io/llm-d/llm-d-async	Asynchronous dispatch processor for latency-insensitive traffic	v0.10.0

Clients that publish requests or consume results can import the Go modules released alongside the image — `github.com/llm-d/llm-d-async/api` , `/pipeline` , and `/producer` , each tagged `v0.10.0` .

[!NOTE] The chart was renamed from `async-processor` to `llm-d-async` in v0.8.0, and chart versions now carry a leading `v` . A Deployment's selector is immutable, so moving from an existing `async-processor` install requires `uninstall` and `install` rather than `helm upgrade` .

Source Repositories

Core Libraries

Repository	Language	Description
llm-d/llm-d	—	Main repo: docs, Dockerfiles, guides, CI
llm-d/llm-d-router	Go	EPP routing engine and P/D sidecar
llm-d/llm-d-latency-predictor	Python	XGBoost training and prediction server
llm-d/llm-d-kv-cache	Go, Python, CPP	KV-cache block locality indexer, FS offloading
llm-d/llm-d-workload-variant-autoscaler	Go	SLO-aware workload autoscaler
llm-d/llm-d-async	Go	Asynchronous request processor for latency insensitive traffic
llm-d/llm-d-batch-gateway	Go	OpenAI-compatible API for submitting, tracking, and managing batch inference jobs.

Supporting Libraries

Repository	Language	Description
llm-d/llm-d-benchmark	Python	Benchmarking framework
llm-d/llm-d-inference-sim	Go	GPU-free vLLM simulator

Disaggregated Serving

Functionality

Disaggregated serving separates the **prefill** and **decode** stages of LLM inference onto different model server instances, enabling:

- **Specialization of P and D** - LLM inference is composed of two distinct phases of inference - prefill (FLOPs-bound) and decode (memory bandwidth-bound). Disaggregation enables specialization, e.g. using a larger TP for the memory-bound decoding phase while a smaller TP for the computation-bound prefill phase.
- **Avoidance of Request Interference** - For long context requests, prefills can slow down processing of existing requests in the decode phase. Separating the prefill phase of these long requests into dedicated prefill instances allows the ongoing decoding requests to be efficiently processed without being blocked by these long prefills, improving quality-of-service.
- **Compatibility with DP/EP** - For DP/EP deployments of Mixture of Experts models, disaggregated serving is essential to avoid pipeline bubbles and leveraging the specialized "MaskedGEMM" format for decode.

An implementation of disaggregated serving requires two key components:

- **Request Flow Orchestration** - select and route the requests to the correct prefill and decode pods
- **Efficient KV Transfer** - transfer the KV cache from the P instance to the D instance, typically over RDMA

[!NOTE] Disaggregated Serving requires high performance (RDMA) interconnects between nodes for efficient KV transfer. Without RDMA, NIXL falls back to TCP for transfer which is not efficient and should only be used for testing and development.

Request Flow Orchestration

llm-d's EPP supports the concept of P/D disaggregation by selecting a prefill and decode worker pair, with the following request flow:

sequenceDiagram
participant Client
participant Proxy
participant EPP
participant DSidecar as Routing Sidecar
participant DWorker as Decode Worker
Note over DSidecar,DWorker: Same Pod
participant PWorker as Prefill Worker

Client->>Proxy: Request
Proxy-->>EPP: Run EPP protocol
EPP-->>Proxy: Selects P Worker and D Worker
Proxy->>DSidecar: Request
DSidecar->>PWorker: Request with max_tokens=1, do_remote_decode=True
PWorker-->>PWorker: Run prefill
PWorker->>DSidecar: Response with KVTransferParams
DSidecar->>DWorker: Request with KVTransferParams and do_remote_prefill=True
DWorker-->>PWorker: Pull KV Cache (NIXL RDMA)
PWorker-->>PWorker: Release KV Cache
DWorker-->>DWorker: Run decodes
DWorker->>DSidecar: Response
DSidecar->>Proxy: Response

Next we will discuss the design of the EPP and Routing Sidecar for disaggregated serving.

EPP

The llm-d EPP supports disaggregation via the `disagg-profile-handler`.

[!NOTE] Rather than hardcoding a single scheduling algorithm, the EPP delegates execution to one or more Profile Handlers, each of which represents a complete scheduling strategy. They can be thought of as "the dispatcher", which maps each incoming inference request to the right scheduling strategy before the scorers and pickers do their work of selecting the actual endpoint. By default, llm-d uses the `single-profile-handler` for simple aggregated serving.

When configured with `disagg-profile-handler`, the EPP processes requests in the following steps:

- `proxy` forwards request metadata to the EPP.
- `disagg-profile-handler` runs the `decode-profile`, which executes the `filter`, `score`, `pick` scheduler profile to select D endpoint.
- `disagg-profile-handler` consults the `decider` — given how much of the prompt is cached on D, should this request run disagg?
- If `no`: `disagg-profile-handler` returns only the D endpoint to the `proxy`
- If `yes` (large uncached suffix), `disagg-profile-handler` also runs the `prefill-profile`, which executes the `filter`, `score`, `pick` scheduler profile to select the P endpoint and returns both the P and D endpoints to the proxy.

The flow looks like this:

```

sequenceDiagram
    participant Proxy
    box EPP
        participant Disagg-Profile-Handler
        participant Decider
        participant Decode-Profile
        participant Prefill-Profile
    end

    Proxy-->>Disagg-Profile-Handler: Request
    Disagg-Profile-Handler-->>Decode-Profile: Request
    Decode-Profile-->>Decode-Profile: Filter, Score, Pick
    Decode-Profile-->>Disagg-Profile-Handler: D Endpoint
    Disagg-Profile-Handler-->>Decider: Num uncached tokens on D
    Decider-->>Disagg-Profile-Handler: [do-pd] BOOL
    opt do-pd=FALSE
        Disagg-Profile-Handler-->>Proxy: D Endpoint
    end

    Disagg-Profile-Handler-->>Prefill-Profile: Request
    Prefill-Profile-->>Prefill-Profile: Filter, Score, Pick
    Prefill-Profile-->>Disagg-Profile-Handler: P Endpoint
    Disagg-Profile-Handler-->>Proxy: D, P Endpoint

```

In this way, disaggregated serving functionality composes with the existing set of scheduling functionality, enabling use of the existing set of scorers for prefix and load aware routing in the disaggregated setting.

Note that both the prefill and decode endpoints are part of one `InferencePool`. The `decode-profile` and `prefill-profile` are responsible for selecting only D workers or P workers in the `filter` step. `llm-d` uses the label `llm-d.ai/role` with the following values to filter:

- `prefill` → prefill-only pods
- `decode` → decode-capable pods
- `prefill-decode` → pods capable of both prefill and decode

[!NOTE] To use a different label key, replace the role-specific filters with named `label-selector-filter` plugins in the `prefill` and `decode` profiles. Apply the same labels to the model-server Pods and keep the profile names `prefill` and `decode`; the `disagg-profile-handler` uses those names when it selects the profile.

Routing Proxy Sidecar

The Routing Proxy is deployed as a sidecar in each decode pod, with a two-fold role:

- Facilitate the multi-step inference request
- Mutate the requests to follow each model server's KV transfer protocol

Request Flow

When a request arrives, the sidecar inspects a routing header set by the proxy:

Header	Purpose
x-prefiller-host-port	One or more prefill pod addresses (comma-separated or multi-value)

Based on which headers are present, the sidecar selects one of two execution paths:

1. **Prefiller → Decoder (P/D)** — The standard disaggregated path. The request is sent to a remote prefiller, KV transfer metadata is collected, and the enriched request is forwarded to the local decoder.
2. **Decoder-only** — No routing headers present; the request is proxied directly to the local model server instance.

All non-completion routes (`GET /health` , and any other path) pass through to the decoder unchanged.

KV Transfer Protocol

vLLM and SGLang use slightly different protocols for KV Transfer between the P and D instance, inserting additional parameters in the body of the requests to facilitate the transfer:

- **vLLM** (`nixlv2` , default) — A two-phase sequential protocol. The sidecar sends a prefill request with `kv_transfer_params` containing remote-decode metadata, and `max_tokens=1` to suppress output. It captures the KV transfer parameters from the prefiller's response and injects them into the decode request before forwarding it to the local decoder. If the prefiller returns a server error, the sidecar falls back to decoder-only mode (client errors are not retried).
- **SGLang** (`sglang`) — Uses a concurrent prefill/decode model. Instead of waiting for prefill to complete, the sidecar injects bootstrap coordination parameters (`bootstrap_host` , `bootstrap_port` , `bootstrap_room`) into both requests, fires the prefill asynchronously in a goroutine (with `context.WithoutCancel` to prevent premature cancellation), and immediately sends the decode request synchronously. The decoder and prefiller coordinate KV transfer out-of-band via the bootstrap room.

Efficient KV Transfer

In addition to the **Request Flow Orchestration** which coordinates the metadata and RPC calls to each instance, efficient **KV Transfer** (which moves the KVs from the P instance to the D instance) is critical to a high performance disaggregated deployment.

vLLM and SGLang both support multiple KV transfer engines - in llm-d we currently focus on NIXL.

NIXL Integration

NVIDIA Inference Transfer Library (NIXL) accelerates point to point communications in AI inference frameworks.

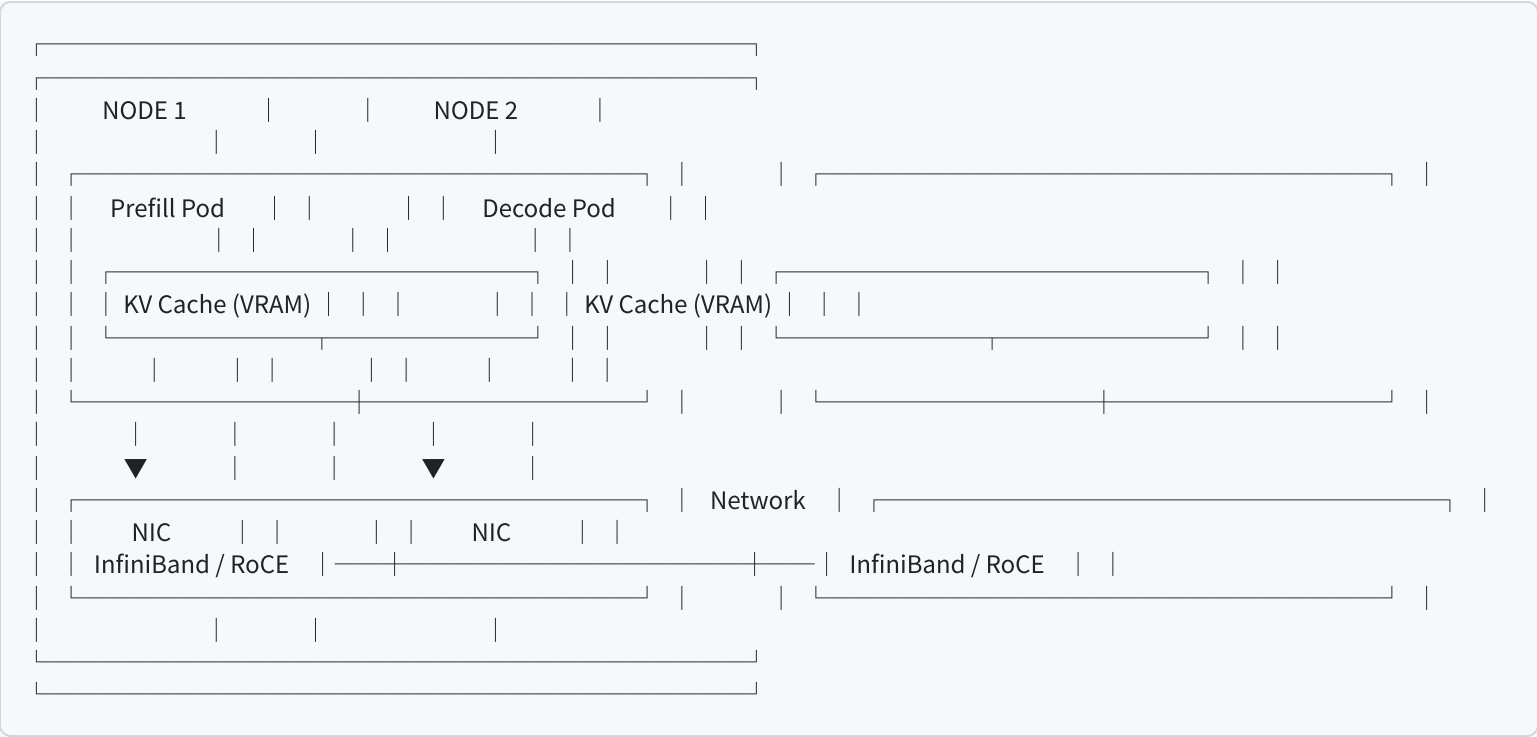
NIXL provides a standardized API for transferring memory between remote instances offering multiple backends (e.g. UCX, UCCL, libfabric) each of which support multiple underlying transports (NVLink, RoCE, IB, EFA, etc).

	IB	RoCE	TCP-X	EFA
UCX	✓	✓		
UCCL	✓	✓	✓	✓
libfabric				✓

[!NOTE] UCX also supports transfer via TCP and NVLINK. TCP is extremely slow and is targeted for local development. NVLINK transfer is used for local development and cannot be used in Kubernetes deployments as it cannot cross pod boundaries.

Direct KV Cache Transfer

vLLM and SGLang both reserve RAM ahead of time for KV cache memory. NIXL directly registers this KV cache memory and transfers the data directly between the KV caches. This avoids the need for additional buffers and memory management. With GPUDirect RDMA enabled (GPU memory registered with NIC), the transfers bypass the CPU, enabling high throughput, low latency transfers.



Operations

Disaggregated serving introduces operational considerations that are specific to each model server's KV transfer mechanism: dynamic connections, request cancellation, fault tolerance, and rollouts. See the engine-specific operations guides:

- [Disaggregated Serving: Operations \(vLLM\)](#)
- [Disaggregated Serving: Operations \(SGLang\)](#)

Inference Payload Processor (IPP)

Functionality

The **Inference Payload Processor (IPP)** is a pluggable framework for processing inference request and response payloads before and after routing decisions.

IPP enables:

- **Request Processing** — Inspect and modify request headers, body, or trailers before routing decisions are made.
- **Response Processing** — Inspect and modify response headers, body, or trailers after the model server responds.

Common use cases include model-aware routing (extracting model names from request bodies), guardrails, content filtering, payload transformation, and observability hooks.

Integration

IPP integrates with the Proxy via Envoy's External Processing (ext-proc) protocol.

Both IPP and EPP contribute to routing decisions:

Component	Routing Scope	Decision
IPP	Pool-level	Which InferencePool?
EPP	Endpoint-level	Which pod within the pool?

Request Flow

![IPP Request Flow](../../assets/ipp-request-flow.svg)

1. **Client** sends an inference request to the Proxy.
2. **Proxy** invokes IPP via ext-proc.
3. **IPP** processes the request through its configured plugin pipeline and returns any mutations (headers, body modifications).
4. **Proxy** applies mutations and routes to the appropriate InferencePool.
5. **EPP** selects the optimal pod within the pool.
6. **Model Server** processes the request and returns the response.

Plugin Architecture

IPP uses a profile-based plugin architecture:

- **Plugins** — Modular units that perform specific processing tasks (e.g., extract fields, set headers, filter models).
- **Profiles** — Named sets of plugins that define processing pipelines for different request types.
- **Profile Picker** — Selects which profile to execute based on request properties.

The processing pipeline executes in this order:

```
PreProcessing → ProfilePicker → Profile Request Plugins → [Model Server] → Profile Response Plugins → PostProcessing
```

For detailed plugin documentation and configuration, see the [IPP repository](#).

Operations

Deployment

IPP is deployed as a standalone Kubernetes Deployment. The Helm chart automatically configures the appropriate Proxy integration based on your environment:

- **Istio** — Creates an EnvoyFilter to insert ext_proc into the Gateway
- **GKE** — Creates a GCPRoutingExtension resource

Monitoring

IPP exposes Prometheus-compatible metrics on port 9090 at `/metrics`, providing visibility into request processing and plugin execution latency.

Further Reading

- [IPP Repository](#) — Source code, configuration reference, and plugin documentation
- [Multi-Model Routing](#) — Using IPP for model-aware routing

Kustomize Dry-Run CI

The `CI Guide Dry Run` workflow (`.github/workflows/ci-kustomize-dry-run.yaml`) validates that every deployable guide in the `guides/` directory produces valid Kubernetes manifests. It runs on every pull request that touches `guides/**` or the workflow itself.

What it does

For each included guide overlay it:

1. Runs `kustomize build` to verify the overlay resolves without errors
2. Runs `kubectrl apply --dry-run=server` against a temporary kind cluster to verify the rendered manifests are accepted by the Kubernetes API

A GitHub job summary is written at the end showing a pass/fail tree of every overlay tested.

Excluding a guide

Exclusions are controlled by `guides/.ci-dry-run-exclusions` . Each non-blank, non-comment line is a path relative to `guides/` that will be skipped. Both top-level directories and subpaths are supported:

```
# Skip an entire guide
no-kubernetes-deployment

# Skip a subdirectory within a guide
recipes/modelserver
```

To add a new guide to CI coverage, remove its entry from the file. To exclude one, add it with a comment explaining why.

Adding CRDs

The workflow installs CRDs for every API group used by the guides before running dry-runs. If you add a guide that uses a new CRD group, add a corresponding install script under `.github/scripts/install-<name>-crds.sh` and reference it in the workflow.

llm-d Infrastructure Providers

This directory contains documentation specific to each Kubernetes provider for deploying llm-d, as well as troubleshooting and known issues.

Other providers

To add a new infrastructure provider to our well-lit paths, we request the following support:

- Documentation on configuring the platform to support one or more well-lit path guides
- The appropriate configuration contributed to the guide to deal with provider-specific variations
- An automated test environment that validates the supported guides
- At least one documented platform maintainer who responds to GitHub issues and is available for regular discussion in the llm-d slack channel [#sig-installation](#) .

Deploying llm-d on Azure Kubernetes Service

Status and scope

This guide describes the AKS infrastructure that supports llm-d on RDMA-capable Azure ND-series GPU nodes. In the Azure VM SKU naming convention, the lower-case `r` in the capability part of the name identifies RDMA support. Examples of this capability part include `asr`, `amsr`, and `isr`.

Examples include:

- `Standard_ND96asr_v4` ;
- `Standard_ND96amsr_A100_v4` ;
- `Standard_ND96isr_H100_v5` ; and
- `Standard_ND96isr_H200_v5` .

Always confirm that the selected SKU supports InfiniBand in the target region. Do not select a general-purpose GPU SKU only because it has NVIDIA GPUs.

Use these support levels:

SKU class	Status in this guide
RDMA-capable ND-series SKU with the <code>r</code> marker	Target deployment class
<code>Standard_ND96isr_H100_v5</code>	Fully qualified reference
Other A100, H100, or H200 ND-series RDMA SKU	Use the procedure, then run all qualification gates
GB200 or GB300 rack-scale SKU	Use the procedure plus the supported ICG/ICB placement path; qualify the SKU-specific image and topology

The same llm-d deployment pattern used by the H100 example also applies to supported GB-series GPU nodes. Use AKS-managed GPU support, DRANET device claims, `IPC_LOCK`, and the NCCL and NIXL validation gates. For GB-series deployments, also use the available rack-aware placement controls.

The infrastructure procedure was qualified with this configuration:

Component	Qualified configuration
AKS	Kubernetes 1.34 with resource.k8s.io/v1 DRA APIs
Validated GPU node	Standard_ND96isr_H100_v5 , eight H100 GPUs
Node OS	Ubuntu 24.04
Minimum node image	AKSUBuntu-2404gen2containerd-202608.14.0
GPU support	AKS-managed NVIDIA driver and GPU resource registration
Host RDMA	Ubuntu Azure kernel inbox mlx5 and RDMA modules
GPUDirect mechanism	DMA-BUF
NIC allocation	DRANET v1.3.0 with Kubernetes Dynamic Resource Allocation
Workload memory locking	IPC_LOCK capability

This procedure prepares AKS for llm-d. The infrastructure tests include raw RDMA, NCCL, and NIXL. A persistent llm-d serving deployment and an inference request are separate application tests.

The deployment model applies to RDMA-capable ND-series GPU SKUs. The recorded test results are specific to the qualified H100 configuration. Run the same qualification gates for each GPU SKU, AKS node image, and driver version.

Use AKSUBuntu-2404gen2containerd-202608.14.0 or a later Ubuntu 24.04 AKS node image. Later node images should work with this procedure. Run the qualification gates after each node-image update before you deploy llm-d.

Prerequisites

You need:

- an AKS cluster;
- quota and capacity for the selected RDMA-capable ND-series GPU SKU;
- two GPU nodes for inter-node RDMA and prefill/decode tests;
- az , kubectl , helm , and jq ;
- cluster administrator access; and
- a Kubernetes API server that exposes resource.k8s.io/v1 .

Set the deployment values:

```
export AZURE_SUBSCRIPTION_ID='<subscription-id>'
export AZURE_RESOURCE_GROUP='<resource-group>'
export AKS_CLUSTER_NAME='<cluster-name>'
export GPU_NODE_POOL='<node-pool-name>'
export GPU_VM_SIZE='Standard_ND96isr_H100_v5'

az account set --subscription "${AZURE_SUBSCRIPTION_ID}"
az aks get-credentials \
  --resource-group "${AZURE_RESOURCE_GROUP}" \
  --name "${AKS_CLUSTER_NAME}"

kubectl config current-context
```

Confirm the DRA API before you install DRANET:

```
kubectl api-resources --api-group=resource.k8s.io
```

The output must include these resources at `v1` :

```
deviceclasses
resourceclaims
resourceclaimtemplates
resourceslices
```

1. Create the RDMA-capable ND-series node pool

Use Ubuntu 24.04 and the AKS-managed GPU stack. The following command is a template. Review availability zones, disk settings, autoscaling, and capacity requirements for your environment before you run it.

```
az aks nodepool add \
  --subscription "${AZURE_SUBSCRIPTION_ID}" \
  --resource-group "${AZURE_RESOURCE_GROUP}" \
  --cluster-name "${AKS_CLUSTER_NAME}" \
  --name "${GPU_NODE_POOL}" \
  --node-count 2 \
  --node-vm-size "${GPU_VM_SIZE}" \
  --os-sku Ubuntu2404 \
  --enable-managed-gpu true \
  --node-taints sku=gpu:NoSchedule
```

`--enable-managed-gpu=true` selects the full AKS-managed GPU experience. AKS then manages the NVIDIA driver, Kubernetes device plugin, DCGM metrics exporter, and GPU health monitoring. See [Create an AKS-managed GPU node pool](#).

The `sku=gpu:NoSchedule` taint is recommended to keep general workloads off the GPU nodes. It is not an RDMA requirement. GPU workloads must include the matching toleration.

The managed GPU experience is currently a preview feature and can require the AKS preview extension:

```
az extension add --upgrade --name aks-preview
```

2. Validated host and GPU baseline

The H100 qualification confirmed the required baseline on `Standard_ND96isr_H100_v5` nodes. The final conformance run used the retained cross-pool node pair. The detailed host inspection used two nodes with the same VM size, node image, kernel, and GPU driver.

Item	Confirmed result
GPU inventory	Eight NVIDIA H100 80 GB GPUs per node
Kubernetes GPU resources	Eight allocatable <code>nvidia.com/gpu</code> resources per node
Node OS	Ubuntu 24.04.4 LTS
Kernel	<code>6.8.0-1064-azure</code>
NVIDIA driver	AKS-managed NVIDIA open kernel driver <code>580.159.04</code>
RDMA modules	<code>mlx5_core</code> , <code>mlx5_ib</code> , <code>ib_core</code> , and <code>ib_uverbs</code> loaded from <code>linux-modules-6.8.0-1064-azure</code>
InfiniBand devices	Eight workload devices, <code>mlx5_0</code> through <code>mlx5_7</code> , Active and LinkUp at 400 Gb/s NDR
DMA-BUF	NCCL reported <code>DMA-BUF</code> is available and used <code>GDRDMA</code>
Containerd NRI	Enabled with <code>disable = false</code> ; <code>/var/run/nri/nri.sock</code> present

The Kubernetes node result was:

NAME	READY	GPU	OS	KERNEL
aks-manageddh100-48873085-vmss000007	True	8	Ubuntu 24.04.4 LTS	<code>6.8.0-1064-azure</code>
aks-h100xpg-32240380-vmss000001	True	8	Ubuntu 24.04.4 LTS	<code>6.8.0-1064-azure</code>

The detailed host inspection recorded this module state on both inspected H100 hosts:

<code>mlx5_core</code> loaded=yes package=linux-modules-6.8.0-1064-azure
<code>mlx5_ib</code> loaded=yes package=linux-modules-6.8.0-1064-azure
<code>ib_core</code> loaded=yes package=linux-modules-6.8.0-1064-azure
<code>ib_uverbs</code> loaded=yes package=linux-modules-6.8.0-1064-azure

The eight workload InfiniBand ports reported this pattern:

<code>mlx5_0</code> state=ACTIVE physical_state=LinkUp link_layer=InfiniBand rate=400 Gb/sec
...
<code>mlx5_7</code> state=ACTIVE physical_state=LinkUp link_layer=InfiniBand rate=400 Gb/sec

NCCL confirmed the GPU-direct path:

DMA-BUF is available on GPU device 0
GPU Direct RDMA Enabled for HCA 0 'mlx5_0'
Channel 00/0 ... via NET/IB/0/GDRDMA

The effective containerd configuration confirmed NRI support:

```
[plugins.'io.containerd.nri.v1.nri']
  disable = false
  socket_path = '/var/run/nri/nri.sock'

/var/run/nri/nri.sock: socket, owner=root:root, mode=755
```

3. Install DRANET

DRANET provides Kubernetes DRA allocation and isolation for the InfiniBand NICs. It does not install the host NIC driver.

AKS-managed DRANET support is work in progress. AKS is expected to provide a managed DRANET solution in the future. Until that service is available, you must install and operate DRANET in the cluster.

Create the workload namespace:

```
kubectl create namespace llm-d-infra \
--dry-run=client -o yaml | kubectl apply -f -
```

Create `dranet-values.yaml` :

```
image:
  repository: registry.k8s.io/networking/dranet
  tag: v1.3.0
  pullPolicy: IfNotPresent

nodeSelector:
  kubernetes.azure.com/agentpool: <node-pool-name>

args:
  cloudProviderHint: AZURE
  moveIBInterfaces: false

resources:
  requests:
    cpu: 100m
    memory: 50Mi
  limits:
    memory: 256Mi
```

`moveIBInterfaces: false` keeps the IP over InfiniBand interface on the host and lets DRANET inject the allocated RDMA character devices.

The guide uses the AKS-provided `kubernetes.azure.com/agentpool=<node-pool-name>` label. Custom `llm-d.ai/*` labels are not required. If one DRANET deployment must cover multiple RDMA node pools, an operator-managed common label is an optional way to select all of those pools.

Install the pinned chart:

```
helm upgrade --install dranet \
oci://registry.k8s.io/networking/charts/dranet \
--version v1.3.0 \
--namespace kube-system \
--values dranet-values.yaml \
--wait \
--timeout 10m
```

Verify that DRANET runs only on the intended RDMA nodes:

```
kubectl get daemonset dranet -n kube-system -o wide
kubectl get pods -n kube-system \
--selector=app.kubernetes.io/name=dranet \
-o wide
```

4. Create the RDMA DeviceClass

The `DeviceClass` accepts only RDMA-capable devices that the `dra.net` driver publishes:

```
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: dranet.net
spec:
  selectors:
  - cel:
      expression: >-
        device.driver == "dra.net" &&
        device.attributes["dra.net"].rdma == true
```

Apply the object:

```
kubectl apply -f dranet-device-class.yaml
```

DRANET then publishes one `ResourceSlice` for each selected ND-series node. The number of devices is SKU-specific. Discover and record the expected count before you create a full-node claim. The H100 reference has eight devices.

```
kubectl get resourceslices.resource.k8s.io -o json | jq -r '
.items[]
| select(.spec.driver == "dra.net")
| [
  .spec.nodeName,
  (.spec.devices | length),
  .spec.devices[0].attributes["azure.dra.net/vmSize"].string,
  .spec.devices[0].attributes["azure.dra.net/placementGroupId"].string
]
| @tsv'
```

Expected shape:

```
<node-1> 8 Standard_ND96isr_H100_v5 <placement-group-id>
<node-2> 8 Standard_ND96isr_H100_v5 <placement-group-id>
```

For another SKU, set the claim count to the RDMA-device count that DRANET publishes for that SKU.

5. Create a full-node NIC claim template

Create one template for each VM size. This H100 example allocates all eight InfiniBand NICs to one pod. Change the name, count, and VM-size selector for a different RDMA-capable ND SKU:

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaimTemplate
metadata:
  name: nd-rdma-full-node-h100
  namespace: llm-d-infra
spec:
  spec:
    devices:
      requests:
        - name: rdma-nics
          exactly:
            deviceClassName: dranet.net
            count: 8
            selectors:
              - cel:
                  expression: >-
                    device.attributes["azure.dra.net"]["vmSize"] ==
                    "Standard_ND96isr_H100_v5"
```

Apply it:

```
kubectl apply -f nd-rdma-full-node-h100.yaml
```

Use a smaller claim count for a partial-node experiment. Partial-node GPU and NIC placement needs additional PCI and NUMA topology validation.

6. Configure llm-d model-server pods

Each full-node model-server pod requests the GPU and NIC counts for its SKU. The following H100 example requests eight GPUs and the eight-NIC claim. It also receives `IPC_LOCK` :

```

spec:
  nodeSelector:
    kubernetes.azure.com/agentpool: <node-pool-name>
  tolerations:
    - key: sku
      operator: Equal
      value: gpu
      effect: NoSchedule
  resourceClaims:
    - name: rdma-nics
      resourceClaimTemplateName: nd-rdma-full-node-h100
  containers:
    - name: modelserver
      securityContext:
        allowPrivilegeEscalation: false
      capabilities:
        add:
          - IPC_LOCK
        drop:
          - ALL
      env:
        - name: UCX_TLS
          value: rc_x,cuda_copy,cuda_ipc,self
      resources:
        requests:
          nvidia.com/gpu: 8
        limits:
          nvidia.com/gpu: 8
        claims:
          - name: rdma-nics

```

Do not add `privileged: true`. Do not set an explicit 64 GiB memory-lock limit. The qualified test kept the normal 8 MiB `RLIMIT_MEMLOCK` value and used `IPC_LOCK` to lock 64 MiB successfully.

For prefill/decode disaggregation, use one full GPU node for prefill and one full GPU node for decode. Set the tensor-parallel size and resource counts for the selected SKU. Add required pod anti-affinity so the two engines run on different nodes.

7. Validate before you deploy the serving stack

GPU and ResourceSlice publication alone does not make the infrastructure ready. Run active data-path tests on the selected nodes.

Required gates:

1. A one-NIC DRA isolation pod sees exactly one allocated `uverbs` device.
2. A full-node claim exposes the expected number of `uverbs` devices.
3. Two-node `ib_write_bw` uses InfiniBand RC transport without TCP fallback.
4. NCCL reports DMA-BUF and `GDRDMA` and completes with zero wrong values.
5. NIXL selects UCX `rc_mlx5` and transfers GPU memory with zero wrong values.
6. A full-node NCCL test uses all expected GPUs and RDMA rails.

We validated this data path on two `Standard_ND96isr_H100_v5` nodes. All required tests passed:

Test	Validated result
DRA device isolation	An unprivileged pod saw only its allocated <code>uverbs</code> device
Full-node DRA claim	Eight RDMA devices were visible in the pod
Raw InfiniBand	<code>ib_write_bw</code> used RC transport and reached 375.97 Gb/s on one rail
NCCL	Sixteen GPUs used all eight GDRDMA rails with zero wrong values
NIXL	UCX <code>rc_mlx5</code> completed a verified GPU-memory transfer with zero wrong values
Cross-placement-group path	Raw InfiniBand, NCCL, and NIXL passed between different placement groups

Placement guidance

Azure provides finer-grained topology scheduling for supported GB-series rack-scale SKUs through Interconnect Groups, Interconnect Blocks, and per-rack interconnect subgroups. Use this model when an llm-d workload requires explicit rack-aware placement. For current ND-series deployments, `placementGroupId` is diagnostic VMSS metadata. It is not proof that two nodes are in the same physical rack.

llm-d on DigitalOcean Kubernetes Service (DOKS)

This document covers configuring DOKS clusters for running high performance LLM inference with llm-d.

For deployment instructions, see the well-lit path guides.

Prerequisites

llm-d on DOKS is tested with the following configurations:

- GPU types: NVIDIA H100, NVIDIA RTX 6000 Ada, NVIDIA RTX 4000 Ada, NVIDIA L40S
- Versions: DOKS 1.33.1-do.3
- Networking: VPC-native clusters (required)

Cluster Configuration

The DOKS cluster should be configured with the following settings:

- GPU-enabled node pools with at least 2 GPU nodes
- VPC-native networking (default for new clusters)
- kubectl configured for cluster access

GPU Driver Management

DigitalOcean automatically installs and manages GPU drivers on DOKS clusters:

- **NVIDIA Device Plugin:** Automatic installation for GPU discovery and scheduling
- **Driver Updates:** Managed alongside cluster updates
- **GPU Monitoring:** Built-in metrics collection via DCGM Exporter

Verify automatic GPU setup:

```
kubectl get pods -n nvidia-device-plugin-system
kubectl get nodes -o custom-columns="NAME:.metadata.name,GPU:.status.allocatable.nvidia\.com/gpu"
```

GPU Node Taints

DOKS GPU nodes use taints to prevent non-GPU workloads from scheduling. Add the following toleration to model server deployments:

tolerations:
- **key:** "nvidia.com/gpu"
operator: "Exists"
effect: "NoSchedule"

Deploying llm-d

Follow the well-lit path guides to deploy llm-d workloads. Each guide includes DigitalOcean-specific steps where applicable.

Troubleshooting

LoadBalancer Pending or API Errors

Error: LoadBalancer stuck in `<pending>` state with API errors

Cause: DigitalOcean API rate limiting or concurrent LoadBalancer operations

Solution:

```
# Check LoadBalancer status
kubectl describe svc <service-name> -n <namespace>

# Wait for API operations to complete (typically 2-3 minutes)
# Sequential deployments avoid conflicts
```

Pods Fail to Schedule on GPU Nodes

Error: `untolerated taint {nvidia.com/gpu}`

Cause: DOKS GPU nodes have automatic taints to prevent non-GPU workloads

Solution: Ensure your deployment includes the `nvidia.com/gpu` toleration. Verify it is applied:

```
kubectl describe pod <pod-name> -n <namespace> | grep Tolerations
# Should show: nvidia.com/gpu:NoSchedule op=Exists
```

Gateway Not Programmed

Error: Gateway shows `PROGRAMMED: False`

Solution: Verify Istio is running and the LoadBalancer IP has been assigned:

```
kubectl get pods -n istio-system
kubectl get gateway -n <namespace>
```

llm-d on Google Kubernetes Engine (GKE)

This document covers configuring GKE clusters for running high performance LLM inference with llm-d.

Prerequisites

llm-d on GKE is tested with the following configurations:

- Machine types: A3, A4, A4X, ct5p, ct5lp, ct6e
- Versions: GKE 1.33.4+

Cluster Configuration

The GCP cluster should be configured with the following settings:

- All prerequisites for GKE Inference Gateway enablement

GPUs

For A3 machines, deploy a cluster and configure high performance networking with TCPX if you plan to leverage Prefill/Decode disaggregation.

For A3 Ultra, A4, and A4X machines, follow the steps for creating an AI-optimized GKE cluster with GPUs and enable GPUDirect RDMA.

GPU Dynamic Resource Allocation (DRA) and DRANET (RoCE) on GKE

This section provides specific instructions for deploying P/D (Prefill/Decode) disaggregation on GKE using **Dynamic Resource Allocation (DRA)** for NVIDIA GPUs and RDMA (RoCE) networking (e.g. NVIDIA H200 GPUs on A3 Ultra).

[!NOTE] Follow the official GCP documentation for the latest updates and detailed instructions:

- GKE AI Hypercomputer Custom Provisioning
- Set up GPU Dynamic Resource Allocation (DRA)
- Allocate network resources by using GKE managed DRANET

Prerequisites

Set up the environment variables:

```
export PROJECT="<your GCP project>"
export LOCATION="<your GCP location>"
export CLUSTER_NAME="<your cluster name>"
export NAMESPACE="llm-d-pd-disaggregation"
```

1. Create Cluster (Managed DRANET)

GKE network DRA (**DRANET**) supports managed networking out of the box, so you do not need to install custom networking drivers. However, you **must** enable **Dataplane V2** when creating your cluster:

```
gcloud container clusters create "${CLUSTER_NAME}" \
  --enable-dataplane-v2 \
  --managed-otel-scope=COLLECTION_AND_INSTRUMENTATION_COMPONENTS \
  --location="${LOCATION}" \
  --project="${PROJECT}"
```

[!NOTE] **Deploying on an Existing GKE Cluster:** If you are deploying on an existing cluster instead of creating a new one, your configuration path depends on whether Dataplane V2 is enabled:

- **Existing Cluster with Dataplane V2 Enabled:** You can skip Step 1 (creating the cluster) and proceed directly to Step 2 (creating the node pool) using the automated GKE-managed DRANet profile (`--accelerator-network-profile=auto`).
- **Existing Cluster without Dataplane V2 Enabled:** GKE-managed automated DRANet is not supported. You **CANNOT** directly use the node pool creation command in **Step 2** below. Instead, you must **manually** configure and manage the multi-networking for the additional network interfaces (NICs) by manually creating subnetworks and mapping node pool network interfaces to them, as detailed in GKE's [Set up multi-network support for pods](#) documentation. Once the subnets and node pool are manually provisioned, proceed directly to **Step 3**.

2. Create Node Pool (Enable DRANET & Disable Default GPU Driver)

GPU DRA is not yet fully managed by GKE. Therefore, you must disable the automated GPU driver and the default GPU device plugin via `--accelerator-gpu-driver-version=disabled` .

GKE managed **DRANET** is enabled by configuring `--accelerator-network-profile=auto` and adding the `cloud.google.com/gke-networking-dra-driver=true` node label:

```
gcloud beta container node-pools create a3u-dra-pool-1 \
  --project="${PROJECT}" \
  --location="${LOCATION}" \
  --cluster="${CLUSTER_NAME}" \
  --accelerator type=nvidia-h200-141gb,count=8,gpu-driver-version=disabled \
  --machine-type=a3-ultragpu-8g \
  --num-nodes=2 \
  --spot \
  --accelerator-network-profile=auto \
  --node-labels="cloud.google.com/gke-networking-dra-driver=true,goog-gke-accelerator-type=nvidia-h200-141gb,nvidia.com/gpu.present=true,cloud.google.com/gke-nvidia-gpu-dra-driver=true,gke-no-default-nvidia-gpu-device-plugin=true"
```

3. Install NVIDIA & GPU DRA Drivers

Since automated GPU driver management is disabled, you must install the NVIDIA Driver and the NVIDIA GPU DRA Driver manually.

3.1 Install NVIDIA Driver

Apply the preloaded COS NVIDIA driver DaemonSet:

```
kubectl apply -f https://raw.githubusercontent.com/GoogleCloudPlatform/container-engine-accelerators/master/nvidia-driver-installer/cos/daemonset-preloaded.yaml
```

3.2 Install NVIDIA GPU DRA Driver

Install the official NVIDIA DRA Driver:

```
helm install nvidia-dra-driver-gpu nvidia/nvidia-dra-driver-gpu \
--version="25.8.0" --create-namespace --namespace=nvidia-dra-driver-gpu \
--set nvidiaDriverRoot="/home/kubernetes/bin/nvidia/" \
--set gpuResourcesEnabledOverride=true \
--set resources.computeDomains.enabled=false \
--set kubeletPlugin.priorityClassName="" \
--set 'kubeletPlugin.tolerations[0].key=nvidia.com/gpu' \
--set 'kubeletPlugin.tolerations[0].operator=Exists' \
--set 'kubeletPlugin.tolerations[0].effect=NoSchedule'
```

3.3 Verify DRA Device Readiness

Verify that the DRA driver pods are running and `ResourceSlices` are successfully populated with `gpu.nvidia.com` and `mrdma.google.com` devices:

```
kubectl get pods -n nvidia-dra-driver-gpu
```

NAME	READY	STATUS	RESTARTS	AGE
nvidia-dra-driver-gpu-kubelet-plugin-52cdm	1/1	Running	0	46s

```
kubectl get resourceslices -o yaml
```

► **Click to view expected output**

GKE A4X (GB200) Setup

For GKE A4X (NVIDIA GB200) clusters, deploy model servers using the `gke/a4x` overlay path (`modelserver/gpu/vllm/gke/a4x`) which inherits from `gke/base` . This configuration enables native InfiniBand RDMA networking via DRANet (`mrdma.google.com`) and GPUDirect hardware acceleration:

- **Hardware Interconnect:** Configures `UCX_TLS=rc_mlx5,rc,cuda_copy,cuda_ipc,sm,shm,self` for hardware-accelerated InfiniBand transport.
- **DRANet Resource Claims:** Uses `gke-rdma-nic-template` to request DRANet (`mrdma.google.com`) interfaces while managing GPUs via NVIDIA Device Plugin (`nvidia.com/gpu`).

Mitigating Hugging Face Model Download Rate Limiting

When deploying large-scale model inference on GKE clusters (such as multi-replica prefill and decode deployments), concurrent model weight downloads across multiple pods can trigger Hugging Face HTTP 429 Rate Limiting errors, leading to container startup timeouts.

To ensure resilient model loading across pods, consider the following strategies:

1. Node-Level Host Caching

Mount a local host directory to `/root/.cache/huggingface` inside workload pods so that multiple pods scheduled on the same node share cached weights without re-downloading across pod restarts.

```
env:
- name: HF_HOME
  value: /root/.cache/huggingface
# ...
volumeMounts:
- mountPath: /root/.cache/huggingface
  name: huggingface-cache
volumes:
- name: huggingface-cache
  hostPath:
    path: /var/cache/huggingface
    type: DirectoryOrCreate
```

[!NOTE] The host path `/var/cache/huggingface` in the manifest snippet above serves as an illustrative reference. Users can configure any suitable host directory based on their node disk allocation and storage policies.

2. Hub Timeouts & Startup Probe Tuning

Configure download retry and timeout environment variables, and increase `startupProbe.failureThreshold` (e.g., to 240) to give pods sufficient headroom to complete initial model weight downloads under network constraints:

```
env:
- name: HF_HUB_DOWNLOAD_TIMEOUT
  value: "60"
- name: HF_HUB_ETAG_TIMEOUT
  value: "60"
- name: HF_HUB_DISABLE_XET
  value: "1"
startupProbe:
  failureThreshold: 240
  periodSeconds: 30
```

3. Google Cloud Storage Integration (Recommended for Production)

For large production workloads, store model weights in a Google Cloud Storage (GCS) bucket and mount the weights directly to pods. This completely eliminates external Hugging Face network dependencies during pod scaling.

For step-by-step instructions, see the [GKE Hugging Face GCS Transfer Guide](#).

TPUs

For all TPU machines, follow the TPUs in GKE documentation.

For Ironwood (TPU7x), GKE supports dynamic slicing: TPU capacity is provisioned as fixed sub-blocks and partitioned into workload-shaped sub-slices (e.g. 2x2x1 , 2x2x2) at scheduling time, improving recovery time, startup latency, and fleet utilization. See TPU Dynamic Slicing on GKE for cluster setup and the corresponding llm-d model server recipes.

Monitoring

We recommend enabling Google Managed Prometheus and automatic application monitoring to enable automatic metrics collection and dashboards for vLLM deployed on the cluster, and Managed OpenTelemetry to collect telemetry data in OTLP format like traces for the llm-d stack.

When creating your cluster, the `--managed-otel-scope=COLLECTION_AND_INSTRUMENTATION_COMPONENTS` flag enables Managed OpenTelemetry on the cluster. Setting the scope to `COLLECTION_AND_INSTRUMENTATION_COMPONENTS` tells GKE to deploy both the OpenTelemetry Collector (in the `gke-managed-otel` namespace) and the Instrumentation Custom Resource Definition (CRD), which is used to automatically inject OpenTelemetry configurations into your workload pods.

Workload Configuration

GPUs

Configuring support for RDMA on GKE workload pods

GCP provides CX-7 support on A3 Ultra+ GPU hosts via RoCE.

Model servers that need to use fast internode networking for P/D disaggregation or wide expert parallelism will need to request RDMA resources on your workload pods. The cluster creation guide describes the required changes to a pod to access the RDMA devices (e.g. for A3 Ultra / A4).

In addition, expert parallel deployments leveraging DeepEP with NVIDIA NVSHMEM will need to run their pods with `privileged: true` in order to perform GPU-initiated RDMA connections, or enable `PeerMappingOverride=1` in your NVIDIA kernel settings with a manual GPU driver installation.

NOTE: While GDRCopy allows CPU-initiated RDMA connections, at the current time we have not measured a benefit to this configuration and instead recommend the default GPU-initiated setting. You can disable the GDRCopy warning in NVSHMEM initialization by setting the `NVSHMEM_DISABLE_GDRCOPY=1` environment variable on your container.

Ensuring network topology aware scheduling of pod replicas with RDMA

Select appropriate node selectors to ensure multi-host replicas are all colocated on the same network fabric. For many deployments, your A3 Ultra or newer reservation will be within a single zone. This ensures reachability but may not achieve the desired aggregate throughput.

On RDMA-enabled GKE nodes the `cloud.google.com/gce-topology-block` label identifies machines within the same fast network and can be used as the primitive for higher level orchestration to group the pods within a multi-host replica into the same RDMA network.

GKE recommends using Topology Aware Scheduling with Kueue and LeaderWorkerSet for multi-host training and inference workloads.

For smaller scale expert parallel deployments (2 or 4 node replicas) we have not observed significant wins in requiring all nodes in the replica to be within the same `cloud.google.com/gce-topology-subblock`. We do recommend setting a pod affinity rule to place all pods within the same `cloud.google.com/gce-topology-block`:

```
affinity:
podAffinity:
  # Subblock affinity cannot guarantee all pods in the replica
  # are in the same subblock, but is better than random spreading
  preferredDuringSchedulingIgnoredDuringExecution:
  - weight: 2
    podAffinityTerm:
      labelSelector:
        matchLabels:
          app: vllm-deepseek-ep
      matchLabelKeys:
      - component
      topologyKey: cloud.google.com/gce-topology-block
  - weight: 1
    podAffinityTerm:
      labelSelector:
        matchLabels:
          app: vllm-deepseek-ep
      matchLabelKeys:
      - component
      topologyKey: cloud.google.com/gce-topology-subblock
```

Known Issues

Undetected platform on vLLM 0.10.0 on GKE

The GKE managed GPU driver automatically mounts the configured node CUDA driver at `/usr/local/nvidia`. CUDA applications like vLLM must have `/usr/local/nvidia` in their `LD_LIBRARY_PATH` or they will not be able to locate the necessary CUDA libraries.

In vLLM, this causes startup to fail with the following logging:

```
INFO 05-28 14:02:21 [__init__.py:247] No platform detected, vLLM is running on UnspecifiedPlatform
...
INFO 05-28 14:02:26 [config.py:1909] Disabled the custom all-reduce kernel because it is not supported on current platform.
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/usr/local/lib/python3.12/dist-packages/vllm/entrypoints/openai/api_server.py", line 1372, in <module>
    parser = make_arg_parser(parser)
  File "/usr/local/lib/python3.12/dist-packages/vllm/entrypoints/openai/cli_args.py", line 246, in make_arg_parser
    parser = AsyncEngineArgs.add_cli_args(parser)
  File "/usr/local/lib/python3.12/dist-packages/vllm/engine/arg_utils.py", line 1565, in add_cli_args
    parser = EngineArgs.add_cli_args(parser)
  File "/usr/local/lib/python3.12/dist-packages/vllm/engine/arg_utils.py", line 825, in add_cli_args
    vllm_kwargs = get_kwargs(VllmConfig)
  File "/usr/local/lib/python3.12/dist-packages/vllm/engine/arg_utils.py", line 174, in get_kwargs
    default = field.default_factory()
  File "<string>", line 4, in __init__
  File "/usr/local/lib/python3.12/dist-packages/vllm/config.py", line 2245, in __post_init__
    raise RuntimeError(
RuntimeError: Failed to infer device type, please set the environment variable `VLLM_LOGGING_LEVEL=DEBUG` to turn on verbose logging to help debug the issue.
```

The root cause of this issue is that the CUDA 12.8 and 12.9 NVIDIA Docker images that are used as a base for vLLM container images changed the location of the installed CUDA drivers from `/usr/local/nvidia` to `/usr/local/cuda` and altered `LD_LIBRARY_PATH`. As a result, the libraries from the driver mount are not found by vLLM.

A workaround exists in llm-d container images and vLLM after commit 5546acb463243ce. Users who customize their vLLM image will need to ensure their `LD_LIBRARY_PATH` in their vLLM image includes `/usr/local/nvidia/lib64`.

Google InfiniBand 1.10 required for vLLM 0.11.0 (gIB)

vLLM v0.11.0 and newer require NCCL 2.27, which is supported in gIB 1.10+. See the appropriate section in cluster configuration for installing the RDMA binary and configuring NCCL (e.g. for A3 Ultra / A4). To get 1.10, use at least the version of the RDMA installer DaemonSet described in this 1.10 pull request.

NVSHMEM reports Unable to create ah. on initialization for DeepEP

Some versions (3.3.20 to 3.4.5) of NVSHMEM contain a bug where the `ibv_ah_attr` struct passed to device initialization was not zeroed out in code. Versions of the Linux kernel that validate the value of the `static_rate` field could fail to start due to an `EINVAL` and reporting `Unable to create ah.` when using the DeepEP kernels for wide expert parallelism.

To work around this issue llm-d applies a patch to NVSHMEM that invokes `memset(..., 0, ...)` on the struct before it is passed to the kernel.

NVSHMEM fails to initialize IBGDA transport due to no active IB device that supports GPU-initiated communication for DeepEP

When starting wide expert parallel deployments using the DeepEP kernels, container images (especially Ubuntu) that are not compiled against the Mellanox OFED drivers as recommended by NVIDIA may fail to start with the following error:

```
/tmp/nvshmem_src/src/modules/transport/ibgda/ibgda.cpp 3888 no active IB device that supports GPU-initiated communication is found, exiting...
```

```
/tmp/nvshmem_src/src/host/transport/transport.cpp:nvshmemi_transport_init:282: init failed for transport: IBGDA
```

The default llm-d images based on RHEL UBI are not impacted. Issue 412 tracks updating our Ubuntu based images.

To resolve this issue in custom built images add the Mellanox OFED apt repository

```
wget -qO - https://www.mellanox.com/downloads/ofed/RPM-GPG-KEY-Mellanox | apt-key add -  
cd /etc/apt/sources.list.d/ && wget https://linux.mellanox.com/public/repo/mlnx_ofed/24.10-  
0.7.0.0/ubuntu22.04/mellanox_mlnx_ofed.list
```

before installing `libibverbs-dev` or other `rdma-core-devel` packages.

TPU Dynamic Slicing on GKE

This document covers the llm-d-specific configuration for serving model servers on GKE TPU7x dynamic sub-slices. Cluster preparation is documented by Google Cloud and linked below rather than repeated here. It is the infrastructure prerequisite for the dynamic-slice recipes in the well-lit path guides:

- Optimized Baseline on TPU sub-slices
- P/D Disaggregation on TPU sub-slices

Overview

Dynamic slicing decouples TPU provisioning from slice allocation. Ironwood (TPU7x) capacity is provisioned as fixed 4x4x4 sub-blocks (16 tpu7x-standard-4t nodes, 64 chips) with no active Inter-Chip Interconnect, and a GKE-managed slice controller forms slices at scheduling time from Slice custom resources. **Dynamic sub-slicing** partitions a sub-block into independent 2x2x1, 2x2x2, 2x2x4, or 2x4x4 slices, which lets aggregated replicas, prefill workers, and decode workers of different shapes share the same pre-provisioned capacity.

GKE supports two consumption models: a custom scheduler that manages Slice resources directly, or Kueue with Topology-Aware Scheduling (TAS), where a Kueue admission check creates and manages Slice resources automatically. The llm-d recipes use the **Kueue TAS** path.

Prerequisites

Prepare the cluster by following Use dynamic slicing in GKE with Kueue:

1. Requirements: GKE Standard on the Rapid channel, TPU7x, an All Capacity mode reservation, and the minimum Kueue, JobSet, and LeaderWorkerSet (LWS) versions for sub-slicing. LWS is required: the llm-d recipes deploy every model server replica as a LeaderWorkerSet group.
2. Enable the slice controller
3. Install Kueue, JobSet, and LWS
4. Create node pools with incremental provisioning: a provision_only workload policy, then one 16-node pool per reservation sub-block.
5. Install the Kueue slice controller
6. Verify the status of the nodes and the partitions: every node should carry cloud.google.com/gke-tpu-partition-<shape>-id and -state labels for each sub-slice shape.

Kueue Resources for llm-d

The Kueue resources published in the GCP guide target super-slicing. Sub-slicing needs a Topology that enumerates the full partition hierarchy of a sub-block, from cloud.google.com/gce-topology-block down through each cloud.google.com/gke-tpu-partition-<shape>-id label to kubernetes.io/hostname . kueue-tas.yaml provides that Topology together with a ResourceFlavor , an AdmissionCheck delegating slice formation to the slice controller (accelerator.gke.io/slice), and a ClusterQueue covering google.com/tpu , cpu , and memory . Apply it once per cluster:

```
kubectl apply -f kueue-tas.yaml
```

Then create the LocalQueue in every namespace that runs dynamic-slice model servers, e.g. for the P/D disaggregation guide:

```
kubectl apply -n llm-d-pd-disaggregation -f kueue-localqueue.yaml
```

Workload Requirements

The llm-d dynamic-slice recipes set the following on every model server pod; they are listed for users adapting their own manifests:

Field	Value
Workload label	kueue.x-k8s.io/queue-name: <LocalQueue name>
Pod annotation	cloud.google.com/gke-tpu-slice-topology: "<shape>" (e.g. 2x2x2)
Pod nodeSelector	cloud.google.com/gke-tpu-accelerator: tpu7x
Pod nodeSelector (health)	cloud.google.com/gke-tpu-partition-<shape>-state: "HEALTHY" (see partition health selection)
Toleration	key google.com/tpu , effect NoSchedule
Resources	google.com/tpu: 4 per pod (requests and limits)

Do **not** set the static cloud.google.com/gke-tpu-topology nodeSelector used by conventional TPU node pools; slice placement is resolved by Kueue and the slice controller.

Each tpu7x-standard-4t node has 4 chips and each TPU7x chip has 2 cores, so a shape AxBxC maps to (A*B*C)/4 pods per slice and supports --tensor-parallel-size up to A*B*C*2 :

Shape	Chips	Pods per slice (LWS size)	Cores (max TP)
2x2x1	4	1	8
2x2x2	8	2	16
2x2x4	16	4	32
2x4x4	32	8	64

Slice names are limited to 49 characters and Kueue derives them from the namespace, workload name, and replica index, so keep namespace plus `LeaderWorkerSet` names short.

Operations

Slice status (`ACTIVATING` , `ACTIVE` , `FAILED` , `INCOMPLETE`) is visible with `kubectrl get slices -A` ; see [Monitor the slice](#) for status details and Cloud Monitoring metrics. When tearing down, delete the `LeaderWorkerSet` resources first so Kueue removes the `Slice` resources it created; active slices block node pool deletion. See [Clean up](#).

llm-d on minikube

Prerequisites

Platform Setup

This can be run on a minimum ec2 node type g6e.12xlarge (4xL40S 48GB but only 2 are used by default) to infer the model meta-llama/Llama-3.2-3B-Instruct that will get spun up.

Verify you have properly installed the container toolkit with the runtime of your choice.

Podman

```
podman run --rm --security-opt=label=disable --device=nvidia.com/gpu=all ubuntu nvidia-smi
```

Docker

```
sudo docker run --rm --runtime=nvidia --gpus all ubuntu nvidia-smi
```

llm-d-infra Installation

TBD

llm-d on OpenShift

This document covers configuring OpenShift clusters for running high performance LLM inference with llm-d.

For deployment instructions, see the well-lit path guides.

Prerequisites

llm-d on OpenShift is tested with the following configurations:

- Versions: OpenShift 4.19, 4.20, 4.21
- Ensure no ServiceMesh(OSSM) or Istio installations exist on the cluster — included CRDs may conflict with the llm-d gateway component
- Cluster administrator privileges are required to install cluster-scoped resources

Cluster Configuration

GPU Setup

Install the NFD (Node Feature Discovery) and NVIDIA GPU Operators before deploying llm-d workloads.

The ocp-gpu-setup repo provides guided scripts for provisioning GPU nodes and deploying the required operators on AWS:

```
git clone https://github.com/rh-aishervices-bu/ocp-gpu-setup.git
cd ocp-gpu-setup
```

```
# Configure GPU MachineSet
./machine-set/gpu-machineset.sh
```

```
# Deploy NFD Operator
oc apply -f ./nfd
```

```
# Deploy NVIDIA GPU Operator
oc apply -f ./gpu-operator
```

```
# Apply supporting CRs
oc apply -f ./crs
```

GPU Node Taints

GPU nodes on OpenShift may have taints applied (e.g. `nvidia.com/gpu: NVIDIA-L40S-PRIVATE`). If model server pods are stuck in `Pending`, add the appropriate toleration to the deployment:


```
oc patch deployment <deployment-name> \
-p '{"spec":{"template":{"spec":{"tolerations":[{"key":"nvidia.com/gpu","operator":"Equal","value":"NVIDIA-L40S-PRIVATE","effect":"NoSchedule"}]}]}}'}
```

Deploying llm-d

Follow the well-lit path guides to deploy llm-d workloads. Each guide includes OpenShift-specific steps where applicable.

Use `oc` in place of `kubectl` for OpenShift CLI commands, or configure `kubectl` to use your OpenShift cluster credentials via `oc login` .

llm-d on Red Hat OpenShift on AWS

This document serves as a reference guide, pointing you to the official Red Hat and AWS documentation, as well as NVIDIA documentation, for setting up a Red Hat OpenShift Service on AWS (ROSA) cluster with NVIDIA GPU support.

Prerequisites

Before you begin, ensure you meet the prerequisites outlined in the official documentation for ROSA:

- An AWS account with necessary permissions for ROSA.
- A Red Hat account and access to the Red Hat OpenShift Cluster Manager.
- Required tools (like the rosa CLI) and familiarity with ROSA concepts.

Refer to the official ROSA documentation for detailed prerequisites:

<https://docs.aws.amazon.com/rosa/latest/userguide/prerequisites.html>

Step 1: Create a Red Hat OpenShift Service on AWS (ROSA) Cluster

The primary method for creating a ROSA cluster is through the Red Hat OpenShift Cluster Manager or using the ROSA CLI.

For detailed instructions on creating your ROSA cluster, refer to the official documentation:

- **Creating a ROSA cluster:** <https://docs.aws.amazon.com/rosa/latest/userguide/create-cluster.html>
- **Getting started with ROSA:** <https://docs.aws.amazon.com/rosa/latest/userguide/getting-started.html>

Step 2: Add a Machineset with NVIDIA GPU Instances

To add worker nodes with GPU capabilities to your ROSA cluster, you will create a new machineset configured to use AWS instance types that include NVIDIA GPUs. This is done through the OpenShift console or oc CLI after the ROSA cluster is provisioned.

Learn how to manage machinesets and configure them for specific instance types in the official documentation:

- **Managing machinesets in OpenShift (applies to ROSA worker nodes):** https://docs.openshift.com/container-platform/latest/machine_management/creating-machinesets.html
- **Adding machinesets with specific instance types (referencing AWS GPU instances):** Consult the AWS documentation for available GPU instance types (e.g., P, G, and Inf instances) and use the OpenShift machineset documentation to configure your machineset accordingly. <https://aws.amazon.com/ec2/instance-types/>

Step 3: Enable GPU support on OpenShift with the NFD and GPU Operators

To enable GPU acceleration on your GPU-enabled nodes, you need to install and configure both the Node Feature Discovery (NFD) Operator and the NVIDIA GPU Operator. The NVIDIA documentation provides comprehensive steps for this process on OpenShift.

Refer to the NVIDIA documentation for detailed instructions on installing and configuring both operators:

- **Enabling GPU Support on OpenShift (covers NFD and GPU Operator installation):**
<https://docs.nvidia.com/datacenter/cloud-native/openshift/latest/steps-overview.html>

Next Steps

Once the operators are installed and configured, you can proceed with deploying llm-d using the llm-d quick start.

Try out the well-lit path guides for more information.

Remote Direct Memory Access (RDMA) and Networking Configuration

Why Networking Matters

In prefill/decode disaggregation, the Key-Value (KV) Cache must transfer from prefill to decode workers before the first token can be generated. This transfer time lands directly on Time to First Token (TTFT) — and the cost grows with context length and model size.

For Wide Expert Parallelism, all-to-all GPU communication across nodes is on the critical path for every token generated.

Networking is a first-order concern for distributed inference latency.

The Networking Stack

llm-d uses a layered networking stack for KV Cache transfers and inter-node communication:



NIXL

NIXL (NVIDIA Inference Xfer Library) is the transfer library used by vLLM to move KV Cache between GPUs. It abstracts the underlying transport behind a unified API, so vLLM can initiate transfers without knowledge of the network fabric.

NIXL operates in a **pull-based model**: the decode pod fetches KV Cache blocks directly from the prefill pod's GPU memory using one-sided RDMA reads (direct memory access without involving the remote CPU), without requiring active participation from the prefill pod. This reduces synchronization overhead.

Key capabilities:

- Works across InfiniBand, RDMA over Converged Ethernet (RoCE), Elastic Fabric Adapter (EFA), and TCP
- Supports GPU memory (VRAM), CPU Dynamic RAM (DRAM), and storage backends
- Plugin architecture for adding new transport backends
- Supports Tensor Parallel (TP) heterogeneity (prefill and decode can use different tensor-parallel sizes)

UCX

UCX (Unified Communication X) is the default transport backend for NIXL. It is a mature, open-source communication framework with broad adoption across High-Performance Computing (HPC) clusters. UCX abstracts RDMA transports (InfiniBand, RoCE), shared memory, and TCP behind a single API.

UCX is a good default: it is battle-tested, widely supported, and works across most hardware. However, it was designed for HPC workloads and carries complexity that can make it harder to tune for AI inference traffic patterns.

UCCL

UCCL (Unified Cloud Communication Library) is a newer transport backend integrated into NIXL as of llm-d v0.5. It implements a CPU-managed software transport stack — managing transport logic on the CPU rather than relying solely on network interface card (NIC) hardware offload. This enables fine-grained flow splitting and adaptive congestion control.

UCCL currently supports:

- Native RDMA (InfiniBand/RoCE)
- GPUDirect TCP-X (Google Cloud)
- TCP
- EFA (AWS)

UCCL uses RDMA transport by default. To switch to other transport types set `UCCL_P2P_TRANSPORT=ib|efa|nccl|tcp|tcp_x` at runtime. UCCL automatically discovers network interface cards (NICs) based on PCIe proximity during memory registration, removing the need for manual NIC-to-GPU mapping in most cases. UCCL also supports intra-node transfers using IPC mechanisms.

libfabric

On AWS, NIXL uses libfabric as the transport backend. EFA (Elastic Fabric Adapter) requires OpenFabrics Interfaces (OFI) — UCX does not support EFA natively. The libfabric plugin provides multi-rail RDMA (using multiple network paths simultaneously for higher bandwidth) with topology-aware GPU-to-EFA mapping via hwloc.

Choosing a Transport Backend

Environment	Backend	Rationale
On-premise InfiniBand / RoCE	UCX	Mature, battle-tested on HPC fabrics with dedicated, uncongested paths
Cloud with RoCE (GKE, Azure, etc.)	UCCL	Software packet spraying avoids single-path congestion on shared fabric
GKE with GPUDirect TCP-X	UCCL	Native support for Google's GPU-initiated TCP transport
AWS with EFA	libfabric/UCCL	EFA requires OFI/libfabric; UCX doesn't support EFA
TCP-only (XPU, CPU)	UCX/UCCL	Simplest configuration for non-RDMA environments

The core tradeoff:

- **UCX** offloads transport to NIC hardware — works best when the network fabric has dedicated, uncongested paths, typical in on-premise High-Performance Computing (HPC) clusters with InfiniBand.

- **UCCL** manages transport in software on the CPU — it splits traffic across up to 256 network paths with adaptive congestion control. This matters in cloud environments where network paths are shared and individual paths may be congested.
- **libfabric** is the default option for AWS EFA. UCCL also supports EFA by setting `UCCL_P2P_TRANSPORT=efa` during runtime. UCX does not support EFA.

NIXL selects the backend based on what is available and the memory types involved. You control which backends are loaded at agent creation time.

Configuration

vLLM KV Transfer

Enable NIXL-based KV Cache transfer via the `--kv-transfer-config` flag:

```
vllm serve <model> \  
--kv-transfer-config '{"kv_connector":"NixlConnector",  
"kv_role":"kv_both",  
"kv_buffer_device":"cuda",  
"kv_connector_extra_config":{"backends":["UCX"]}]'
```

The `kv_role` is `kv_both` for both prefill and decode pods — each pod can both send and receive KV Cache.

For XPU devices where KV transfer happens via CPU memory, add:

```
--kv-transfer-config '{"kv_connector":"NixlConnector","kv_role":"kv_both","kv_buffer_device":"cpu"}'
```

Backend Selection

NIXL uses UCX backend by default. The GPU example above selects it explicitly using `kv_connector_extra_config`.

To configure NIXL with UCCL backend:

```
vllm serve <model> \  
--kv-transfer-config '{"kv_connector":"NixlConnector",  
"kv_role":"kv_both",  
"kv_connector_extra_config":  
{"backends":["UCCL"]}]'
```

NIXL Side Channel

NIXL uses a side channel for metadata exchange between pods. Configure with:

Variable	Description	Default
VLLM_NIXL_SIDE_CHANNEL_HOST	Pod IP (use <code>status.podIP</code> fieldRef)	Required
VLLM_NIXL_SIDE_CHANNEL_PORT	Metadata exchange port	5557

UCX Transport Selection

UCX transport is configured via environment variables:

Variable	Description	Example
UCX_TLS	Transport layers (TLS) to use	sm,cuda_ipc,cuda_copy,rc,tcp
UCX_SOCKADDR_TLS_PRIORITY	Priority for socket-based transport layers	tcp
UCX_PROTO_INFO	Check transport selection	y
UCX_NET_DEVICES	Network devices to use for transport	e.g. mlx5_0:1, mlx5_1:1

For RDMA-capable clusters, UCX will automatically use RDMA verbs when available. For TCP-only clusters (XPU), set `UCX_TLS=tcp`.

Verify the Selected UCX Transport

Enable UCX transport logging temporarily on both prefill and decode pods:

```
- name: UCX_LOG_LEVEL
  value: info
- name: UCX_PROTO_INFO
  value: "y"
```

Send a representative P/D request and inspect the UCX protocol table for the large GPU-memory operation. Confirm the selected data lane: `cuda_ipc/cuda` identifies CUDA IPC, while a lane such as `rc_mlx5` identifies RDMA. Backend initialization, an `intra-node` endpoint, or `cuda_ipc/cuda` appearing among the available endpoint lanes does not prove that the KV Cache payload used CUDA IPC.

In a standard Kubernetes deployment, separate prefill and decode pods receive separate GPU allocations. Same-node placement therefore must not be assumed to enable CUDA IPC; same-node and cross-node payloads can both select RDMA. CUDA IPC requires both processes to have access to the participating peer GPUs, and `hostIPC` alone does not provide that access. See NVIDIA Dynamo's Kubernetes disaggregated-communication guidance for the cross-pod GPU-isolation constraints and recommended RDMA configuration.

The NIXL side channel uses the regular network for metadata exchange, so the presence of TCP in a log is not by itself evidence that KV Cache payloads use TCP. Disable `UCX_PROTO_INFO` after validation because its protocol tables are verbose.

RDMA Resources and Capabilities

RDMA requires device resources and elevated capabilities in the pod spec:

```
resources:
  limits:
    rdma/roce_gdr: "2"
  requests:
    rdma/roce_gdr: "2"
```

securityContext:

capabilities:

add:

- IPC_LOCK
- SYS_RAWIO
- NET_ADMIN
- NET_RAW

NIC Selection

Use `NCCL_EXCLUDE_IB_HCA` to exclude specific Host Channel Adapters (HCAs) from NVIDIA Collective Communications Library (NCCL) traffic (e.g., management NICs):

```
- name: NCCL_EXCLUDE_IB_HCA  
value: "mlx5_0,mlx5_2,mlx5_4,mlx5_8"
```

For Wide Expert Parallelism, map GPUs to specific HCAs for optimal topology:

```
- name: DEEP_EP_DEVICE_TO_HCA_MAPPING  
value: "0:mlx5_0:1,1:mlx5_1:1,2:mlx5_2:1,3:mlx5_3:1,4:mlx5_4:1,5:mlx5_5:1,6:mlx5_6:1,7:mlx5_7:1"
```

Platform-Specific Notes

GKE

- Use GKE multi-NIC annotations for RDMA interfaces:

```
yaml annotations: networking.gke.io/default-interface: eth0 networking.gke.io/interfaces: '["interfaceName":"eth0","network":"default",  
...]'
```

- Source `set_nccl_env.sh` from `/usr/local/gib/scripts/` at container startup
- Set `NVSHMEM_DISABLE_GDRCOPY=1` (GKE recommendation)
- Use pod affinity on `cloud.google.com/gce-topology-block` for topology-aware placement
- GPU-initiated RDMA requires `privileged: true` in the security context

OpenShift / OCP

- Use Multus CNI for secondary RDMA networks:

```
yaml annotations: k8s.v1.cni.cncf.io/networks: "multi-nic-compute"
```

- Request `rdma/roce_gdr` device resources as shown above

AWS (EFA) (deprecated - use upstream AWS images)

- NIXL uses the `libfabric` backend (not UCX) — see Choosing a Transport Backend
- Requires `libfabric v1.21.0+` (or latest AWS EFA installer)
- The `libfabric` plugin auto-discovers GPU-to-EFA topology via `hwloc` for optimal multi-rail placement

- UCCL backend also supports EFA, however, it requires compiling with the `USE_EFA` option — see UCCL

Verifying Network Performance

After deploying model servers, verify two things:

1. GPU Topology

Confirm GPUs within each pod are optimally connected:

```
# NVIDIA
nvidia-smi topo -m      # Look for NV/PIX, not SYS or PHB
nvidia-smi nvlink --status # Verify NVLink is active

# AMD
rocm-smi --showtopo     # Confirm Infinity Fabric connectivity
```

GPUs showing `SYS` or `PHB` topology are communicating over PCIe across Non-Uniform Memory Access (NUMA) nodes — this adds latency, especially for collective operations.

2. Inter-Pod Network

Verify RDMA connectivity and bandwidth between pods:

```
# Check RDMA devices are available
ibv_devinfo
```

Run NIXL benchmark between prefill and decode pods. nixlbench requires an etcd server for peer coordination when using network backends.

Start a standalone etcd (e.g., in one of the pods or as a separate pod):

```
etcd --listen-client-urls http://0.0.0.0:2379 \
--advertise-client-urls http://$(hostname -i):2379 \
--listen-peer-urls http://0.0.0.0:2380 \
--initial-advertise-peer-urls http://$(hostname -i):2380 \
--initial-cluster "default=http://$(hostname -i):2380" &
```

From the prefill/decode pods, run:

```
nixlbench --etcd_endpoints http://<ETCD_SERVER_IP>:2379 --backend <UCX/UCCL/LIBFABRIC> --op_type=READ --check-consistency -
--start_batch_size=100 --max_batch_size=100 --max-block-size=85899340
```

The above test runs nixl benchmarks with the specified backend for message sizes of 1GB - 8GB, and reports the throughput, latency, etc.

If throughput is significantly below the expected line rate for your fabric, check NIC affinity, Maximum Transmission Unit (MTU) settings, and whether traffic is falling back to TCP (by setting `UCX_PROTO_INFO=y` for UCX backend).

If vLLM is already running, GPU memory may be insufficient for in-pod benchmarks. Add a pre-start script that runs tests before vLLM launches and blocks until a condition is met (e.g., removal of a sentinel file). In the future, this diagnostic will be automated as runtime scripts.

Further Reading

- [NIXL repository](#)
- [UCCL repository](#)
- [P/D Disaggregation Well-Lit Path](#) — deployment patterns using NIXL
- [Wide Expert-Parallelism Well-Lit Path](#) — multi-node deployment with DeepEP networking
- [Model Servers](#) — vLLM/SGLang configuration including KV transfer flags